React for Beginners

A Friendly Guide

Learn React Step by Step — From Zero to Building Real Apps

Functional Components • Hooks • Modern Patterns

2025 Edition • 32 Chapters

Table of Contents

Chapter 1 — What Is React
Chapter 2 — Why React
Chapter 3 — How React Works
Chapter 4 — Setting Up
Chapter 5 — Project Structure
Chapter 6 — JSX
Chapter 7 — Components
Chapter 8 — Props
Chapter 9 — State
Chapter 10 — Events
Chapter 11 — Conditional Rendering
Chapter 12 — Lists and Keys
Chapter 13 — Forms
Chapter 14 — useState
Chapter 15 — useEffect
Chapter 16 — API Calling
Chapter 17 — Component Reuse
Chapter 18 — Lifting State Up
Chapter 19 — Basic Routing
Chapter 20 — Simple Project
Chapter 21 — Common Mistakes
Chapter 22 — Best Practices
Chapter 23 — Interview Questions
Chapter 24 — Final Project
Chapter 25 — Thinking in React
Chapter 26 — Keeping Components Pure
Chapter 27 — How React Renders

Chapter 28 — Refs
Chapter 29 — useReducer
Chapter 30 — Context
Chapter 31 — Custom Hooks
Chapter 32 — TypeScript Basics

Chapter 1: What Is React?

What You Will Learn

- What React is, explained in plain words
- Where React came from
- What React is used to build
- The difference between a website and a web application
- Why React is one of the most popular tools for building websites today

Why This Chapter Matters

Before you start learning any tool, it helps to understand what it is and what it does. Think of it this way: before you learn to drive a car, you should know what a car is and what it is used for. This chapter gives you that basic understanding of React. No code yet. Just clear ideas.

Let Us Start from the Very Beginning

What Is a Website?

You use websites every day. When you open Google to search for something, you are using a website. When you scroll through Instagram, watch videos on YouTube, or read the news online, you are using websites.

A website is simply a page (or a collection of pages) that you can view in a browser. A browser is the app you use to visit websites. Google Chrome, Safari, Firefox, and Microsoft Edge are all browsers.

Every website is built using code. The code tells the browser what to show and how to show it.

What Are Web Applications?

Some websites are simple. A blog that shows articles is simple. You read the article, and that is it.

But some websites are more complex. Think about Gmail. You can read emails, write emails, search through emails, delete emails, and organize them into folders. Gmail is not just a page you read. It is more like an app you use.

When a website behaves like an app, we call it a web application (or web app for short). Web apps are interactive. They respond to what you do. They update without reloading the entire page.

Here are some examples of web applications:

- Gmail - You read, write, and manage emails
- Facebook - You post, like, comment, and chat
- Google Maps - You search, zoom, and get directions
- Netflix - You browse, search, and watch videos
- Spotify - You search, play, and create playlists

Building these kinds of web apps is complex. There is a lot of code to write, a lot of things to keep track of, and a lot of things that can go wrong. This is where React helps.

So, What Is React?

React is a tool that helps you build web applications. That is it. That is the simplest explanation.

More specifically, React is a JavaScript library for building user interfaces.

Let us break that down into plain language:

- JavaScript is a programming language. It is the language that makes websites interactive. When you click a button and something happens, that is JavaScript at work.
- Library means a collection of pre-written code that you can use. Instead of writing everything from scratch, you use the code that React gives you. Think of a library like a toolbox. You do not build your own hammer. You take the hammer from the toolbox and use it.

- User interface (often shortened to UI) is everything the user sees and interacts with on the screen. Buttons, text, images, forms, menus, pop-ups, lists. All of that is the user interface.

So, putting it all together:

React is a collection of pre-written JavaScript code that helps you build the things users see and interact with on a website.

A Real-Life Analogy

Imagine you want to build a house. You could cut down trees, make your own bricks, forge your own nails, and do everything from scratch. That would take forever.

Or you could go to a hardware store, buy pre-made bricks, nails, doors, and windows, and put them together. Much faster.

React is like that hardware store. It gives you ready-made pieces (called components, which we will learn about later) that you can put together to build your website.

Where Did React Come From?

React was created by Facebook (now called Meta) in 2013. Facebook had a problem. Their website was getting very big and very complex. Millions of people were using it at the same time. The code was becoming hard to manage.

Facebook's engineers needed a better way to build and update the user interface. So they created React. It worked so well that they shared it with the world for free. Today, React is open source, which means anyone can use it, study it, and contribute to it without paying anything.

Since 2013, React has grown to become one of the most popular tools for building websites. Thousands of companies use it, including:

- Facebook and Instagram
- Netflix
- Airbnb
- Uber
- Twitter (now X)
- WhatsApp Web
- Dropbox

What Can You Build with React?

You can build almost any kind of web application with React:

- Social media apps (like Facebook or Twitter)
- Online stores (like Amazon or Shopify)
- Dashboards (admin panels with charts and tables)
- Chat applications (like WhatsApp Web)
- Video streaming apps (like Netflix)
- Blogs and news sites
- Portfolio websites
- To-do lists and note-taking apps
- Weather apps
- Calculator apps

In this book, we will build some of these together, step by step.

React Is Not the Whole Website

This is an important thing to understand early on. React only handles the user interface (what people see and click on). It does not handle things like:

- Storing data in a database
- Sending emails
- Processing payments
- User login on the server

Those things are handled by other tools and technologies on the back end (the part of the application that runs on a server, not in the browser).

React runs in the front end (the part that runs in the browser, the part users see).

```
What the user sees:          What happens behind the scenes:
+-------------------+        +----------------------------+
|                   |        |                            |
|  React handles    |        |  Other tools handle this   |
```

```
|  this part        |  <-->  |  (servers, databases,      |
|  (the screen)     |        |   payment systems, etc.)   |
|                   |        |                            |
+-------------------+        +----------------------------+
    Front End                        Back End
    (Browser)                        (Server)
```

You do not need to know anything about the back end to learn React. We will focus entirely on what happens in the browser.

How Is React Different from HTML and CSS?

If you have heard of HTML and CSS, you might wonder: "Do I still need them if I use React?"

Yes! React works with HTML and CSS, not instead of them. Let us understand what each one does:

- HTML (HyperText Markup Language) defines the structure of a page. It says "here is a heading, here is a paragraph, here is an image, here is a button."
- CSS (Cascading Style Sheets) defines the look of a page. It says "this heading should be blue, this button should be round, this text should be large."
- JavaScript defines the behavior of a page. It says "when the user clicks this button, show a message."
- React is built on top of JavaScript. It gives you a smarter, easier way to combine HTML, CSS, and JavaScript together to build interactive pages.

```
Building a Website:
+-----------------------------------------------+
|                                               |
|  HTML  = The skeleton (structure)             |
|  CSS   = The clothes (appearance)             |
|  JS    = The brain (behavior)                 |
|  React = A smarter brain (better behavior)    |
|                                               |
+-----------------------------------------------+
```

You do not need to be an expert in HTML, CSS, or JavaScript before learning React. But knowing the basics will help. If you know what a heading is, what a button is, and what it means to "click something," you are ready.

A Very Quick Peek at React Code

Do not worry about understanding this code right now. We will explain every piece later. This is just to give you a tiny preview of what React code looks like:

```jsx
function Welcome() {
  return <h1>Hello! Welcome to React.</h1>;
}
```

That is a React component. It is a small piece of code that shows a heading on the screen that says "Hello! Welcome to React."

See how short it is? Just three lines. That is one of the nice things about React. You can build small pieces like this and then combine them to build bigger things.

We will learn exactly how this works in the coming chapters. For now, just notice that it looks almost like a mix of JavaScript and HTML. That is by design. React lets you write HTML-like code inside JavaScript. This makes it easy to describe what should appear on the screen.

Key Words You Will Hear Often

Here are some words that will come up a lot in this book. Do not memorize them now. Just read them so they feel familiar later:

Word	Simple Meaning
Component	A small, reusable piece of a website (like a button, a card, or a menu)
JSX	The special way React lets you write HTML-like code inside JavaScript
Props	Information you pass to a component (like giving instructions to someone)
State	Information that a component remembers and can change (like a counter that goes up when you click)
Hook	A special React function that lets you add features to your components
Render	The process of showing something on the screen

We will learn each of these in detail in later chapters. For now, just know they exist.

Quick Summary

Let us recap what we learned in this chapter:

- A website is a page you view in a browser. A web application is a website that works like an app.
- React is a JavaScript library that helps you build user interfaces (the things users see and interact with).
- React was created by Facebook in 2013 and is free to use.
- React handles only the front end (what users see), not the back end (servers and databases).
- React works with HTML and CSS, not instead of them.
- React lets you build small pieces called components and combine them to build complete web pages.

Key Points to Remember

1. React is a tool for building what users see on screen.
2. React is a JavaScript library, not a programming language.
3. React is free, open source, and used by thousands of companies.
4. You write React code using a mix of JavaScript and HTML-like syntax.
5. React focuses on the front end only.

Practice Questions

1. In your own words, what is React?
2. What is the difference between a website and a web application? Give one example of each.
3. React handles the front end. What does "front end" mean?
4. Name three web applications that are built with React.
5. What is a "component" in simple terms?

Exercises

1. Open your browser and visit three websites. For each one, decide: is it a simple website or a web application? Write down why you think so.
2. Look at the screen of any app you use daily (Instagram, YouTube, Gmail, etc.). Try to identify different parts of the screen: the header, the buttons, the text areas, the images. Each of those could be a React component. Write down at least five parts you notice.
3. In a notebook or document, write one paragraph explaining React to a friend who has never heard of it. Use your own words. Keep it simple.

What Is Coming Next?

In the next chapter, we will answer the question: Why do people use React instead of just writing plain JavaScript? You will understand the specific problems React solves and why it has become so popular. This will give you motivation to keep learning!

Chapter 2: Why Do People Use React?

What You Will Learn

In this chapter, you will learn:

- Why building big websites with plain HTML, CSS, and JavaScript gets hard
- How React makes building websites easier
- What "reusable components" means and why they save time
- Famous websites that use React
- What the word "library" means in programming
- When React is a good choice and when you do not need it

Why This Chapter Matters

Before you learn any tool, it helps to know why that tool exists. Imagine someone hands you a power drill. If you do not know what problems a power drill solves, you will not know when to use it. This chapter shows you the problems that web developers faced before React. Once you see those problems, React will make a lot more sense.

The Problem: Building Big Websites Is Hard

Let us start with something you already know. You have visited websites like Facebook, Instagram, or Netflix. These websites have thousands of buttons, menus, images, and moving parts. Everything updates instantly. You click a "like" button and the number changes right away. You type in a search box and results appear as you type.

Now imagine building all of that with just HTML, CSS, and JavaScript. Let us see why that gets difficult.

Problem 1: Repeating Yourself Over and Over

Think about Instagram. Every post on your feed looks the same. It has a user photo, a username, an image, a like button, and a comment section. If you have 100 posts on the screen, do you write the same HTML 100 times?

With plain HTML, you would have to copy and paste the same block of code again and again. If you later decide to change the design of a post (maybe you want to add a "share" button), you would need to find and update all 100 copies. That is slow, boring, and easy to mess up.

Problem 2: Keeping the Page Up to Date

Imagine you click the "like" button on a post. The number should go from 42 to 43. With plain JavaScript, you have to:

1. Find the right number on the page
2. Read its current value
3. Add one to it
4. Put the new number back on the page

That sounds simple for one button. But what if clicking that button also changes a notification counter at the top of the page? And updates a "liked posts" list in the sidebar? Now you have to find and update three different places. On a big website, one action might need to update ten or twenty different parts of the page. Keeping track of all those updates is where most mistakes happen.

Problem 3: Speed

Every time you change something on a web page, the browser has to do a lot of work. It has to recalculate where everything goes, repaint the screen, and check if anything else moved. If you change many things one at a time, the browser does all that work again and again. This makes the page feel slow.

How React Solves These Problems

React was created by a team at Facebook in 2013. They were facing exactly the problems above. Their website was getting bigger and harder to manage. So they built a tool to help. That tool is React.

Let us see how React fixes each problem.

Solution 1: Reusable Pieces (Components)

React lets you build your website from small, reusable pieces. These pieces are called components. A component is like a Lego block.

Think about Lego blocks. Each block is small and simple on its own. But you can snap them together to build a house, a car, or a spaceship. If you want to change the wheels on your Lego car, you just swap out the wheel pieces. You do not rebuild the whole car.

React works the same way. You build a small piece once (like a "Post" component) and then use it as many times as you want. If you need 100 posts, you use the same component 100 times. If you want to add a "share" button to every post, you change the component in one place, and every post on the page updates automatically.

Here is what that looks like as an idea (do not worry about the code yet):

```
Post component:
  - User photo
  - Username
  - Image
  - Like button
  - Comment section

Page:
  - Post (Alice's photo)
  - Post (Bob's photo)
  - Post (Carol's photo)
  ... reuse the same Post block 100 times
```

One component. Used many times. Changed in one place.

Solution 2: Automatic Updates

With React, you do not tell the browser how to update the page. Instead, you tell React what the page should look like. React figures out how to make it happen.

Think of it like ordering food at a restaurant. With plain JavaScript, you are going into the kitchen yourself. You find the pan, turn on the stove, cook the food, and put it on the plate. With React, you just tell the waiter what you want. The kitchen handles the rest.

When data changes (like a "like" count going from 42 to 43), React automatically finds every place on the page that uses that data and updates it. You do not have to track anything yourself. This means fewer mistakes and less stress.

Solution 3: Speed (The Virtual DOM)

React uses a clever trick to keep things fast. It does not change the real web page every time something updates. Instead, it keeps a lightweight copy of the page in memory. This copy is called the Virtual DOM (we will explain this in detail in the next chapter).

When something changes, React first updates the lightweight copy. Then it compares the copy to the real page. It finds the smallest number of changes needed and applies only those changes. This is much faster than updating the whole page.

Think of it like editing a document. Instead of rewriting the entire document every time you fix a typo, you just fix the one word that changed. React does the same thing with your web page.

Real Websites That Use React

React is not just a learning exercise. Some of the biggest websites in the world use it every day:

- Facebook — React was born here. The entire Facebook website is built with React.
- Instagram — The web version of Instagram uses React to show your feed, stories, and messages.
- Netflix — The Netflix website uses React to show you movie and TV show listings.
- Airbnb — The Airbnb website uses React for searching listings, viewing photos, and booking stays.
- WhatsApp Web — The browser version of WhatsApp uses React.
- Dropbox — The Dropbox file management interface uses React.

These are websites used by billions of people. The fact that they trust React tells you it is a reliable and powerful tool.

Lego Blocks vs. Carving from Wood

Let us compare two ways of building things.

Approach 1: Carving from One Piece of Wood

Imagine you want to make a toy house. You take a large block of wood and start carving. You carve the walls, the roof, the windows, and the door — all from one piece. It looks beautiful.

But what if you want to change the door? You cannot just pull it off. It is part of the whole block. You might have to carve the entire house again.

This is what building a big website with plain HTML and JavaScript feels like. Everything is connected. Changing one part can break other parts.

Approach 2: Building with Lego Blocks

Now imagine building that same toy house with Lego blocks. The walls are separate pieces. The roof is a separate piece. The door is a separate piece.

Want to change the door? Pull it off and snap on a new one. Want to add a second floor? Stack more blocks on top. Want to build a second house? You already have all the block designs. Just assemble another one.

This is how React works. Each component is a separate block. You can change, remove, or add blocks without breaking the rest of the house.

Feature	Carving (Plain HTML/JS)	Lego Blocks (React)
Making changes	Hard — might break other parts	Easy — change one block
Reusing designs	Start from scratch each time	Reuse the same block
Finding problems	Hard — everything is tangled	Easy — check one block
Working in a team	Difficult — people get in each other's way	Easy — each person works on their own blocks

---

What Is a "Library"?

You might hear people call React a library. What does that word mean?

In everyday life, a library is a place where you borrow books. You pick the books you need and leave the rest on the shelves. You are in control. You decide which books to read and

when.

A programming library works the same way. It is a collection of tools that someone else wrote. You pick the tools you need and use them in your project. You are still in control of your project. You decide when and where to use the library.

React is a library. It gives you tools for building user interfaces (the part of a website that people see and interact with). You decide how to use those tools. You can use React for your entire website, or just for one small part of it.

Library vs. Framework

You might also hear the word framework. A framework is different from a library. Think of it this way:

- Library: You are the chef. The library is a set of kitchen tools (a blender, a mixer, a knife set). You decide what to cook and which tools to use.
- Framework: The framework is a meal kit delivery service. It gives you a recipe, the ingredients, and step-by-step instructions. You follow the plan it gives you.

A framework controls the flow of your project. A library lets you control the flow yourself.

React is a library, not a framework. This means you have more freedom in how you use it. But it also means you sometimes need to pick additional tools yourself (for example, a tool for navigating between pages). We will talk about those additional tools later in the book.

When Is React a Good Choice?

React is great for many projects, but it is not always the best choice. Let us look at when to use it and when not to.

Use React When:

- Your website has many interactive parts. If users click buttons, fill forms, see live updates, or interact with the page a lot, React is a great fit.
- Your website reuses similar pieces. If your page shows a list of products, a feed of posts, or a grid of cards, React components save you time.
- Your website is big and will keep growing. React helps you organize large projects so they do not become a tangled mess.

- You work in a team. Different team members can work on different components without stepping on each other's toes.

You Might Not Need React When:

- Your website is simple and mostly static. If you are building a personal blog or a single landing page with no interactive features, plain HTML and CSS might be all you need.
- Your website is a small project with one or two pages. The setup time for React might be more than the time you save.
- You just want to display information. If your page does not change after it loads and users do not interact with it much, React adds complexity you do not need.

Think of it this way: you would not use a power drill to hang a single picture frame. A hammer and a nail are enough. But if you are building a bookshelf with 50 screws, the power drill saves you a lot of time and effort.

Quick Summary

In this chapter, you learned that building big websites with plain HTML, CSS, and JavaScript is hard because you repeat yourself, tracking updates is tricky, and performance can suffer. React solves these problems by giving you reusable components (like Lego blocks), automatic updates (tell React what you want, not how to do it), and fast performance through the Virtual DOM.

React is a library, which means it is a set of tools you control. It is used by some of the biggest websites in the world, including Facebook, Instagram, Netflix, and Airbnb. React is a great choice for interactive, growing websites, but simple static pages may not need it.

Key Points to Remember

1. Reusable components save time. Build once, use many times, update in one place.
2. Automatic updates reduce mistakes. You describe what the page should look like, and React handles the rest.
3. The Virtual DOM makes React fast by only changing what actually needs to change.
4. React is a library, not a framework. You are in control.

5. React is best for interactive websites that have many moving parts.
6. Simple, static websites do not need React.
7. Major companies like Facebook, Instagram, Netflix, and Airbnb trust React for their websites.

Practice Questions

1. Name two problems that developers face when building big websites with plain HTML and JavaScript.
2. What is a React component? Describe it using your own analogy (you can use Lego blocks or any other analogy you like).
3. What is the difference between a library and a framework? Which one is React?
4. Give an example of a website where React would be a good choice. Explain why.
5. Give an example of a website where React would NOT be needed. Explain why.

Exercises

Exercise 1: Spot the Repetition

Go to any social media website (like Twitter, Instagram, or Reddit). Look at the feed. Find one piece of the design that repeats over and over (for example, each post or each comment). Write down all the parts that piece contains (like profile picture, username, text, like button, etc.). This is what a React component would represent.

Exercise 2: Think Like a Component Designer

Imagine you are building an online store. The store shows a list of products. Each product has:

- A product image
- A product name
- A price
- An "Add to Cart" button

Draw a simple sketch (on paper or in a notes app) of what one product card would look like. Now imagine you have 20 products. How many times would you reuse your product card

component? If you wanted to add a "star rating" to every product, how many places would you need to change?

Exercise 3: Library or Framework?

For each description below, decide if it sounds more like a library or a framework:

a) "Here is a recipe. Follow steps 1 through 10 in order." b) "Here is a set of paintbrushes. Paint whatever you like." c) "Put your code in these specific folders and name your files this way."

What Is Next?

Now you know why React exists and what problems it solves. But how does it actually work behind the scenes? In the next chapter, we will look at how your browser shows web pages, what the DOM is, and how React's Virtual DOM makes everything fast. Do not worry — there will be no code yet. Just clear explanations and simple analogies. Let us keep going!

Chapter 3: How React Works in Simple Terms

What You Will Learn

In this chapter, you will learn:

- How your browser turns code into the web pages you see
- What the DOM is and how it works (explained like a family tree)
- Why updating the whole page is slow
- What the Virtual DOM is (explained with a restaurant menu analogy)
- How React figures out the smallest number of changes to make
- What components are and how they fit together like building blocks

Why This Chapter Matters

Before you start writing React code, it helps to understand what is happening behind the scenes. This is like learning how a car engine works before you start driving. You do not need to become a mechanic, but knowing the basics will help you understand why things work the way they do. When something goes wrong later, you will have a better idea of where to look.

This chapter has no code. Just ideas, analogies, and simple pictures described in words. Relax and enjoy the ride.

How Your Browser Shows a Web Page

Every time you visit a website, your browser (like Chrome, Firefox, or Safari) does a lot of work behind the scenes. Let us walk through what happens step by step.

Step 1: You Type a Web Address

You type something like `www.example.com` into the address bar and press Enter.

Step 2: The Browser Gets the Files

Your browser sends a request over the internet to a computer called a server. A server is just a computer that stores website files and sends them to anyone who asks. Think of it like calling a pizza place and asking them to deliver.

The server sends back files. These files are usually:

- HTML — The content and structure of the page (the words, headings, images, buttons)
- CSS — The styling (colors, sizes, spacing, fonts)
- JavaScript — The behavior (what happens when you click a button or type in a box)

Step 3: The Browser Reads the HTML and Builds a Tree

This is the important part. When the browser receives the HTML file, it reads it line by line. As it reads, it builds a structure in its memory. This structure is called the DOM.

What Is the DOM?

DOM stands for Document Object Model. That is a fancy name, but the idea is simple.

The DOM Is Like a Family Tree

Think about a family tree. At the top, you have the grandparents. Below them, their children. Below those children, the grandchildren. Each person in the tree is connected to the people above and below them.

The DOM works the same way. At the very top is the web page itself (the "document"). Below it are big sections like the `<head>` (information about the page) and the `<body>` (what you actually see). Inside the body, there are smaller pieces: headings, paragraphs, images, buttons, and so on.

Here is what a simple family tree for a web page might look like:

```
Document (the whole page)
  |
  +-- html
        |
        +-- head
        |    |
        |    +-- title ("My Website")
        |
        +-- body
             |
             +-- h1 ("Welcome!")
             |
             +-- p ("This is my first web page.")
             |
             +-- button ("Click Me")
```

Each item in this tree is called a node. A node is just one piece of the page. The `h1` is a node. The `button` is a node. Even the text "Welcome!" is a node.

Why Does the Browser Build This Tree?

The browser needs this tree so it can:

1. Know what to show on the screen. Each node tells the browser what to display.
2. Know how things are related. The button is inside the body, which is inside the html. This relationship matters for styling and behavior.
3. Allow JavaScript to make changes. When JavaScript wants to change something on the page (like updating a counter), it finds the right node in the tree and changes it.

Think of the DOM as a map of your web page. JavaScript uses this map to find things and change them.

Why Updating the Page Is Slow

Now let us talk about what happens when something on the page needs to change.

What Happens When You Change One Thing

Imagine you have a page with 1,000 items on it. You click a button, and one number needs to change from 42 to 43. Here is what the browser does:

1. Find the node in the DOM tree that holds the number.
2. Change the value from 42 to 43.

3. Recalculate the layout. The browser checks if the new number is wider or taller than the old one. If it is, it might push other things around. The browser recalculates where every item on the page should go. This step is called reflow.
4. Repaint the screen. The browser redraws the affected parts of the page on your screen. This step is called repaint.

For one small change, this is fast. You will not notice anything.

What Happens When You Change Many Things

Now imagine you are on a social media feed. You scroll down, and 20 new posts load. Each post has a profile picture, a username, text, images, buttons, and a comment count. That is hundreds of new nodes being added to the DOM tree.

If the browser does a reflow and repaint after every single change, it has to do all that work hundreds of times. Each round of recalculating and repainting takes time. When you add them all together, the page starts to feel sluggish. Buttons take a moment to respond. Scrolling feels choppy.

The Real-World Analogy

Imagine you are rearranging furniture in your house. Every time you move one piece of furniture, you invite all your friends over to look at the whole house and give their opinion. Move one chair? Everyone comes over. Move a lamp? Everyone comes over again. Move a table? Another visit.

That is exhausting and slow. It would be much smarter to move all the furniture first, then invite everyone over once to see the final result.

This is exactly the problem React solves.

The Virtual DOM: React's Clever Solution

The Virtual DOM is one of React's most important ideas. Let us explain it with a story.

The Restaurant Menu Analogy

Imagine you own a restaurant. You have a big chalkboard menu on the wall. This chalkboard is what your customers see. Changing the chalkboard takes time. You have to erase things carefully, rewrite them neatly, and make sure everything still looks good.

One day, you decide to make some changes to the menu. You could:

Option A (The Slow Way): Walk up to the chalkboard and start erasing and rewriting directly. Every time you change one item, customers see a messy, half-finished menu.

Option B (The Smart Way): Keep a small notepad at your desk. Write the new menu on the notepad first. Compare your notepad to the chalkboard. Circle only the things that are different. Then walk up to the chalkboard and make only those specific changes.

React uses Option B.

How the Virtual DOM Works

Here is the process, step by step:

Step 1: React keeps a copy. React creates a lightweight copy of the DOM in the computer's memory. This copy is the Virtual DOM. It is like the notepad in our restaurant story. It is not shown on the screen. It lives only in memory.

Step 2: Something changes. A user clicks a button, types in a box, or new data arrives. React knows something has changed.

Step 3: React creates a new copy. React creates a new Virtual DOM that shows what the page should look like after the change. Now React has two copies:

- The old Virtual DOM (how the page looked before)
- The new Virtual DOM (how the page should look now)

Step 4: React compares the two copies. React looks at both copies side by side and finds the differences. This process is called diffing (short for "finding the differences"). It is like comparing your notepad to the chalkboard.

Step 5: React updates only what changed. React takes the list of differences and applies only those changes to the real DOM (the actual page the user sees). This is called reconciliation (a big word that just means "making things match").

Why This Is Faster

Instead of updating the real page after every single change, React:

1. Makes all the changes to the lightweight copy first (this is very fast because it is just memory, not the screen)
2. Figures out the minimum number of changes needed
3. Applies those changes to the real page in one batch

Going back to our furniture analogy: React moves all the furniture first on a blueprint (the Virtual DOM), then makes the real changes all at once. One visit from the friends, not a hundred.

A Visual Picture: Real DOM vs. Virtual DOM

Let us imagine a simple page with a heading, a counter, and a button. The counter shows the number 5.

The Real DOM (What the Browser Has)

```
body
  |
  +-- h1 ("My Counter App")
  |
  +-- p ("Count: 5")
  |
  +-- button ("Add One")
```

Now the user clicks the button. The count should change from 5 to 6.

What React Does Behind the Scenes

```
Old Virtual DOM:              New Virtual DOM:

body                          body
  |                             |
  +-- h1 ("My Counter App")     +-- h1 ("My Counter App")   <- Same
  |                             |
  +-- p ("Count: 5")            +-- p ("Count: 6")          <- DIFFERENT!
  |                             |
  +-- button ("Add One")        +-- button ("Add One")      <- Same
```

React compares the two trees. It sees that only the paragraph text changed from "Count: 5" to "Count: 6". Everything else is the same.

So React tells the browser: "Just change the text in that one paragraph. Do not touch the heading. Do not touch the button."

The browser makes that one small change. Fast and efficient.

Components: The Building Blocks

We mentioned components in the last chapter. Now let us look at them a little more closely (still no code — just the concept).

What Is a Component?

A component is a self-contained piece of your web page. It has its own structure, its own appearance, and its own behavior. It is like a single Lego block that knows what it looks like and how it works.

Examples of Components

Think about a typical website. You can break it down into components:

- Header component — The bar at the top with the logo and navigation links
- Search bar component — The box where you type to search
- Product card component — One card showing a product's image, name, and price
- Footer component — The bar at the bottom with copyright information and links

Each of these is a separate component. Each one handles its own job.

Components Inside Components

Components can contain other components. This is called nesting (putting one thing inside another, like stacking boxes).

Think about a "Post" component on a social media website:

```
Post component
  |
  +-- UserInfo component (shows profile picture and username)
  |
  +-- PostImage component (shows the photo)
  |
  +-- LikeButton component (the heart or thumbs-up button)
  |
  +-- CommentSection component
       |
       +-- Comment component (one individual comment)
       +-- Comment component (another comment)
       +-- Comment component (another comment)
```

The Post component is made up of smaller components. The CommentSection component contains multiple Comment components. Each piece is small and focused. Each piece can be built and tested on its own.

The Component Tree

When you put all the components of a website together, they form a tree shape. This is called the component tree. It looks similar to the DOM tree, but it is organized by your components, not by HTML tags.

```
App (the whole application)
  |
  +-- Header
  |    |
  |    +-- Logo
  |    +-- Navigation
  |
  +-- MainContent
  |    |
  |    +-- SearchBar
  |    +-- ProductList
  |         |
  |         +-- ProductCard
  |         +-- ProductCard
  |         +-- ProductCard
  |
  +-- Footer
```

At the top is the `App` component. It is the parent of everything. Below it are the main sections: Header, MainContent, and Footer. Each of those contains smaller components.

This tree structure is what makes React projects organized and easy to understand. When something goes wrong with a ProductCard, you know exactly where to look. You do not have to search through thousands of lines of mixed-together code.

Putting It All Together

Let us review the whole picture:

1. Your browser shows web pages by building a DOM tree from HTML files.
2. Updating the DOM directly is slow when you have many changes because the browser recalculates and repaints the screen each time.
3. React creates a Virtual DOM — a lightweight copy of the real DOM that lives in memory.

4. When something changes, React creates a new Virtual DOM, compares it to the old one, finds the differences, and updates only the parts of the real DOM that actually changed.
5. Components are the building blocks of a React application. Each component handles one piece of the page.
6. Components can contain other components, forming a component tree that keeps your project organized.

Quick Summary

Your browser builds a tree structure called the DOM to represent your web page. Updating this tree directly is slow because the browser has to recalculate and repaint the screen with each change. React solves this by keeping a Virtual DOM in memory. When something changes, React compares the old and new Virtual DOMs, finds the differences, and updates only the parts that changed. This makes React fast. Components are the building blocks of React applications. They are small, self-contained pieces that can be combined into larger pieces, forming a component tree.

Key Points to Remember

1. The DOM is a tree structure that represents your web page. It stands for Document Object Model.
2. Each item in the DOM tree is called a node. Headings, paragraphs, buttons, and even text are all nodes.
3. Updating the real DOM is slow because the browser has to recalculate layout (reflow) and redraw the screen (repaint).
4. The Virtual DOM is a lightweight copy of the real DOM that React keeps in memory.
5. Diffing is when React compares the old Virtual DOM with the new one to find differences.
6. Reconciliation is when React applies only the necessary changes to the real DOM.
7. Components are self-contained building blocks that handle one part of your page.
8. Components can contain other components, creating a tree structure called the component tree.

Practice Questions

1. In your own words, what is the DOM? Use an analogy to explain it.
2. Why is updating the real DOM directly considered slow?
3. Explain the Virtual DOM using the restaurant menu analogy. What is the chalkboard? What is the notepad?
4. What does "diffing" mean in React?
5. What is a component tree? Draw a simple one for a website you use often (for example, a news site or a shopping site).

Exercises

Exercise 1: Build a DOM Tree on Paper

Take a simple web page idea (like a recipe page with a title, an ingredient list, and step-by-step instructions). Draw its DOM tree on paper. Start with the `body` at the top and add branches for each section. How many nodes does your tree have?

Exercise 2: Spot the Difference

Imagine a to-do list app. The page shows:

- A heading: "My To-Do List"
- Three items: "Buy milk", "Walk the dog", "Read a book"
- Each item has a checkbox

The user checks the checkbox next to "Walk the dog". It should now appear with a line through it.

Draw two small trees: the "before" tree and the "after" tree. Circle the part that changed. This is exactly what React's diffing process does.

Exercise 3: Break a Page into Components

Visit any website you like. Look at the page and try to break it into components. Write down:

- The name you would give each component (for example, "HeaderBar", "SearchBox", "ArticleCard")
- Which components are inside other components
- Draw the component tree

Remember: there is no single right answer. Different people might break the same page into components in slightly different ways, and that is perfectly fine.

What Is Next?

Now you understand how React works behind the scenes: the DOM, the Virtual DOM, components, and the component tree. You have a solid mental picture of what React does and why it is fast.

In the next chapter, we will get our hands dirty. You will install the tools you need, create your very first React project, and see it running in your browser. It is time to move from ideas to action. Let us go!

Chapter 4: Setting Up a React Project

What You Will Learn

In this chapter, you will learn:

- What tools you need to start building with React
- How to install Node.js on your computer
- What npm is and why you need it
- How to create your first React project using Vite
- How to start your project and see it in the browser
- What each file and folder in your new project does
- How to fix common errors that beginners run into

Why This Chapter Matters

Imagine you just bought a new toolkit. Before you can build anything, you need to open the box, lay out the tools, and learn what each one does. This chapter is that moment. We are opening the box, setting everything up, and making sure it all works. By the end, you will have a working React project running on your computer. That is a real achievement, and everything you learn from this point forward will build on it.

What You Need: Two Essential Tools

To work with React, you need two things installed on your computer:

1. Node.js — A program that lets you run JavaScript outside of a web browser
2. A code editor — A program where you write your code (we recommend Visual Studio Code)

Let us look at each one.

What Is Node.js?

In the early days of the web, JavaScript could only run inside a browser. You could not use it anywhere else. Node.js changed that. It lets you run JavaScript on your computer, outside the browser.

Think of it this way. JavaScript is like a fish. In the old days, the fish could only live in one pond (the browser). Node.js is a second pond. Now the fish can live in both places.

You might wonder: "If React runs in the browser, why do I need Node.js?" Good question. You need Node.js because the tools that help you build React projects run on your computer, not in the browser. These tools turn your code into files that the browser can understand. Node.js also comes with a tool called npm, which we will talk about shortly.

What Is a Code Editor?

A code editor is a program for writing code. You could write code in Notepad (Windows) or TextEdit (Mac), but a proper code editor gives you helpful features:

- Syntax highlighting — Different parts of your code show in different colors, making it easier to read.
- Auto-complete — The editor suggests code as you type, like predictive text on your phone.
- Error detection — The editor underlines mistakes before you even run your code.
- Built-in terminal — You can type commands without leaving the editor.

We recommend Visual Studio Code (often called "VS Code"). It is free, works on Windows, Mac, and Linux, and almost every React developer uses it.

Step 1: Install Node.js

On Windows

1. Open your web browser.

2. Go to the website: https://nodejs.org
3. You will see two big green download buttons. Click the one that says "LTS" (this stands for Long Term Support — it is the most stable version).
4. A file will download. It will be called something like `node-v20.x.x-x64.msi`.
5. Double-click the downloaded file.
6. A setup wizard will appear. Click "Next" on each screen. You can keep all the default settings.
7. Click "Install" when you reach the final screen.
8. Wait for the installation to finish, then click "Finish".

On Mac

1. Open your web browser.
2. Go to the website: https://nodejs.org
3. Click the "LTS" download button.
4. A file will download. It will be called something like `node-v20.x.x.pkg`.
5. Double-click the downloaded file.
6. Follow the steps in the installer. Click "Continue" and "Agree" until the installation completes.

On Linux

Open a terminal and type:

```bash
sudo apt update
sudo apt install nodejs npm
```

Type your password when asked, and press Enter.

How to Check If Node.js Is Installed Correctly

Now let us make sure it worked. This is an important step. Do not skip it.

1. Open a terminal (also called "command prompt" or "command line"):
    - Windows: Press the Windows key, type "cmd", and press Enter.
    - Mac: Press Command + Space, type "Terminal", and press Enter.
    - Linux: Press Ctrl + Alt + T.
2. Type this command and press Enter:

```bash
node --version
```

What you should see:

```
v20.11.0
```

The exact number might be different. That is fine. As long as you see a version number starting with `v`, Node.js is installed correctly.

3. Now type this command and press Enter:

```bash
npm --version
```

What you should see:

```
10.2.4
```

Again, the exact number might be different. As long as you see a number, npm is installed.

If you see an error like "command not found" or "'node' is not recognized", Node.js did not install correctly. Try these steps:

- Close your terminal and open a new one (sometimes the terminal needs a fresh start).
- If that does not work, restart your computer and try again.
- If it still does not work, go back to the Node.js website and reinstall.

Step 2: Install Visual Studio Code

1. Go to https://code.visualstudio.com in your browser.
2. Click the big download button. The website will detect your operating system and give you the right version.
3. Run the downloaded file and follow the installation steps.
4. Open VS Code after installation.

What you should see: A welcome screen with a dark (or light) background. There will be tabs like "Get Started" or "Welcome". You are ready to go.

VS Code is your new workspace. You will spend a lot of time here, so take a moment to look around. Do not worry about all the buttons and menus. We will only use a few of them.

What Is npm?

We mentioned npm earlier. Let us explain it properly.

npm stands for Node Package Manager. Think of it as an app store for code.

When you install an app on your phone, you go to the App Store or Google Play, find the app, and tap "Install". npm works the same way, but for code libraries. Want to use React? npm installs it for you. Want a tool that helps you format dates? npm installs that too. Want a library that makes animations easier? npm has it.

There are over one million packages (that is what code libraries are called in npm) available for free. When you install Node.js, npm comes with it automatically. That is why we checked for both `node --version` and `npm --version` earlier.

Here is a quick comparison:

Feature	Phone App Store	npm
What it installs	Apps for your phone	Code libraries for your projects
How you use it	Tap a button	Type a command in the terminal
Cost	Some free, some paid	Almost everything is free
Number of items	Millions of apps	Over one million packages

Step 3: Create Your First React Project

Now for the exciting part. We are going to create a real React project. We will use a tool called Vite (pronounced "veet", like the French word for "fast"). Vite sets up everything you need to start a React project in just a few seconds.

Open Your Terminal

You can use the terminal inside VS Code (which is very convenient) or a separate terminal. To open the terminal inside VS Code:

1. Open VS Code.
2. Click on "Terminal" in the top menu.
3. Click "New Terminal".

A panel will appear at the bottom of VS Code. This is your terminal.

Navigate to Where You Want Your Project

Before creating the project, decide where on your computer you want to store it. We suggest creating a folder for all your coding projects. Let us use the Desktop for now (you can choose anywhere you like).

Type this command and press Enter:

On Mac or Linux:

```bash
cd ~/Desktop
```

On Windows:

```bash
cd %USERPROFILE%\Desktop
```

The `cd` command stands for "change directory". It moves you to a different folder. You just moved to your Desktop.

Create the Project

Now type this command and press Enter:

```bash
npm create vite@latest my-first-app -- --template react
```

Let us break down what each part of this command means:

Part	What It Does
`npm`	Use the npm tool
`create`	Tell npm to create something new
`vite@latest`	Use the latest version of the Vite tool
`my-first-app`	The name of your project (also the folder name)

`--`	Separates npm's options from Vite's options
`--template react`	Tell Vite to set up a React project (not a plain JavaScript project)

What you should see:

The terminal will show some output. It might ask you to install the `create-vite` package. If it does, type `y` and press Enter to confirm.

After a few seconds, you will see a message like this:

```
Scaffolding project in /Users/yourname/Desktop/my-first-app...

Done. Now run:

  cd my-first-app
  npm install
  npm run dev
```

This message tells you three things to do next. Let us do them one at a time.

Move into the Project Folder

```bash
cd my-first-app
```

This moves you into the folder that Vite just created for your project.

Install the Dependencies

```bash
npm install
```

This command tells npm to download all the code libraries your project needs. React is one of them. npm reads a file called `package.json` inside your project folder to figure out what to download.

What you should see:

```
added 152 packages in 8s
```

The exact number of packages and time might be different. This is normal. npm just downloaded React and all the other tools your project needs.

What if you see warnings? You might see yellow warning messages that say things like "deprecated" or "WARN". In most cases, you can safely ignore these. They are not errors. They are just notes about packages that have newer versions available.

Start the Development Server

```bash
npm run dev
```

This command starts a local development server. A development server is a small program that runs on your computer and serves your website to your browser. It is like having a tiny web server right on your machine.

What you should see:

```
  VITE v5.0.0  ready in 300 ms

  ➜  Local:   http://localhost:5173/
  ➜  Network: use --host to expose
  ➜  press h + enter to show help
```

The important line is: http://localhost:5173/

This is the address where your website is running. Let us open it.

Step 4: See Your App in the Browser

1. Open your web browser (Chrome, Firefox, Safari, or Edge).
2. In the address bar, type: http://localhost:5173/
3. Press Enter.

What you should see:

A page with the Vite and React logos. The logos might be spinning. There is a heading that says "Vite + React". Below that, there is a button that says "count is 0". If you click the button, the number goes up: "count is 1", "count is 2", and so on.

There is also text that says "Edit src/App.jsx and save to test HMR".

Congratulations! You are looking at your first React application. It is alive and running on your computer.

What Is "localhost"?

The word localhost means "this computer". When you type `http://localhost:5173/` in your browser, you are telling the browser: "Show me the website that is being served from my own computer, on port 5173."

A port is like a door number on a building. Your computer has many ports (numbered from 0 to 65535). The development server picked port 5173 as its door. Your browser knocks on that door and the development server answers.

How to Stop the Server

When you are done working, go back to your terminal and press Ctrl + C (on both Windows and Mac). This stops the development server. The website will no longer be available in your browser until you start the server again with `npm run dev`.

What Is Inside Your Project?

Let us look at the files and folders that Vite created. Open the `my-first-app` folder in VS Code:

1. In VS Code, click "File" in the top menu.
2. Click "Open Folder".
3. Navigate to your Desktop, select the `my-first-app` folder, and click "Open".

You will see something like this in the file explorer on the left side:

```
my-first-app/
  |-- node_modules/       (a big folder - do not touch it)
  |-- public/              (files that go directly to the browser)
  |-- src/                 (your code lives here)
  |   |-- assets/          (images and other files)
  |   |-- App.css          (styling for the App component)
  |   |-- App.jsx          (the main component - this is important!)
  |   |-- index.css        (global styles)
  |   |-- main.jsx         (the starting point of your app)
  |-- .gitignore           (tells Git which files to ignore)
  |-- index.html           (the one HTML file for your app)
  |-- package.json         (lists your project's dependencies)
  |-- package-lock.json    (locks dependency versions)
  |-- vite.config.js       (settings for Vite)
```

Let us explain the important ones:

node_modules/

This folder contains all the code libraries that npm downloaded when you ran `npm install`. It has thousands of files. You should never edit anything in this folder. If you delete it by accident, just run `npm install` again and it will come back.

src/ (Source)

This is where your code lives. When you write React components, you will put them here. The most important file right now is `App.jsx`.

App.jsx

This is the main component of your application. The `.jsx` extension stands for "JavaScript XML". It is a special type of JavaScript file that lets you write something that looks like HTML inside your JavaScript code. We will learn all about this in a later chapter.

main.jsx

This is the starting point of your React application. It is the first file that runs. It loads the `App` component and puts it on the page.

index.html

This is the one and only HTML file in your project. Yes, just one. React is a "single-page application" tool. It uses JavaScript to change what you see on the page instead of loading separate HTML files.

package.json

This file lists all the code libraries your project depends on and includes scripts (commands) you can run. When you type `npm run dev`, npm looks in this file to find out what that command means.

Let Us Make a Small Change

Let us prove that everything is connected. We will change one line of code and see the result in the browser instantly.

1. Make sure your development server is running (run `npm run dev` if it is not).
2. Open the file `src/App.jsx` in VS Code.
3. Find the line that says:

```jsx
<h1>Vite + React</h1>
```

4. Change it to:

```jsx
<h1>Hello World</h1>
```

5. Save the file (Ctrl + S on Windows, Command + S on Mac).

What you should see: Go back to your browser. The page will update automatically, without you pressing refresh. The heading will now say "Hello World" instead of "Vite + React".

This automatic updating feature is called HMR, which stands for Hot Module Replacement. It means that when you save a file, the browser shows the change instantly. No manual refreshing needed. This makes development fast and enjoyable.

Understanding the Code in App.jsx

Let us look at the code inside `App.jsx` step by step. Do not worry if it looks unfamiliar. We will explain every line.

```jsx
import { useState } from 'react'
import reactLogo from './assets/react.svg'
import viteLogo from '/vite.svg'
import './App.css'

function App() {
  const [count, setCount] = useState(0)

  return (
    <>
      <div>
        <a href="https://vite.dev" target="_blank">
          <img src={viteLogo} className="logo" alt="Vite logo" />
        </a>
        <a href="https://react.dev" target="_blank">
          <img src={reactLogo} className="logo react" alt="React logo" />
        </a>
      </div>
```

```
      <h1>Vite + React</h1>
      <div className="card">
        <button onClick={() => setCount((count) => count + 1)}>
          count is {count}
        </button>
        <p>
          Edit <code>src/App.jsx</code> and save to test HMR
        </p>
      </div>
      <p class="read-the-docs">
        Click on the Vite and React logos to learn more
      </p>
    </>
  )
}

export default App
```

Here is a line-by-line explanation:

Line 1: `import { useState } from 'react'` This loads a tool called `useState` from the React library. We will learn about `useState` in a later chapter. For now, just know it lets your component remember things (like the current count).

Line 2: `import reactLogo from './assets/react.svg'` This loads the React logo image so we can show it on the page.

Line 3: `import viteLogo from '/vite.svg'` This loads the Vite logo image.

Line 4: `import './App.css'` This loads the CSS styling file for this component.

Line 6: `function App() {` This creates a component called `App`. In React, a component is just a JavaScript function that returns what should appear on the screen.

Line 7: `const [count, setCount] = useState(0)` This creates a piece of data called `count` that starts at 0. `setCount` is a function that changes the count. We will explain this fully in a later chapter.

Line 9: `return (` Everything inside the parentheses after `return` is what will appear on the page. This is called JSX — it looks like HTML but it is written inside JavaScript.

Lines 10-28: The JSX code that describes the page. It includes logos, a heading, a button, and some text. Notice how it looks like HTML with tags like `<div>`, `<h1>`, `<button>`, and `<p>`.

Line 19: `<button onClick={() => setCount((count) => count + 1)}>` When the user clicks this button, the count goes up by one. `onClick` is how React handles button clicks.

Line 20: `count is {count}` The curly braces `{}` let you put JavaScript values inside JSX. Here it shows the current value of `count`.

Line 31: `export default App` This makes the `App` component available to other files. The `main.jsx` file imports `App` and puts it on the page.

Common Errors and How to Fix Them

Here are problems that beginners often run into, along with solutions.

Error: "command not found: node" or "'node' is not recognized"

What it means: Node.js is not installed, or your terminal cannot find it.

How to fix it:

- Close your terminal and open a new one.
- If that does not work, restart your computer.
- If it still does not work, reinstall Node.js from https://nodejs.org.

Error: "npm ERR! code EACCES" (Permission denied)

What it means: Your computer is not letting npm write files.

How to fix it (Mac/Linux):

- Add `sudo` before the command: `sudo npm create vite@latest my-first-app --
--template react`
- Type your password when asked.

How to fix it (Windows):

- Right-click on your terminal program and select "Run as Administrator".

Error: "ENOENT: no such file or directory"

What it means: You are in the wrong folder.

How to fix it:

- Make sure you ran `cd my-first-app` after creating the project.
- Use `ls` (Mac/Linux) or `dir` (Windows) to see what files are in your current folder.

Error: "Port 5173 is already in use"

What it means: Another program is already using port 5173, or you have another development server still running.

How to fix it:

- Check if you have another terminal running `npm run dev`. If so, stop it with Ctrl + C.
- If no other server is running, Vite will usually pick the next available port (like 5174). Look at the terminal output for the correct URL.

The Page Is Blank

What it means: There might be an error in your code.

How to fix it:

- Open your browser's developer tools (press F12 or right-click the page and select "Inspect").
- Click on the "Console" tab.
- Look for red error messages. They will usually tell you which file and line number has the problem.
- Go back to VS Code and check that file.

Nothing Happens When I Save

What it means: The development server might have stopped, or there is a syntax error in your code.

How to fix it:

- Check your terminal. If the server stopped, run `npm run dev` again.
- Check for syntax errors. A common one is forgetting to close a tag. Every opening tag like `<div>` needs a closing tag like `</div>`.

Quick Summary

In this chapter, you installed Node.js and Visual Studio Code, the two essential tools for React development. You learned that npm is like an app store for code. You created your first React project using the command `npm create vite@latest my-first-app -- --template react`. You installed the project's dependencies with `npm install`, started the development server with `npm run dev`, and saw your app running in the browser. You made a small change to `App.jsx` and watched the browser update automatically. You also learned what each file and folder in

your project does and how to fix common errors.

Key Points to Remember

1. Node.js lets you run JavaScript outside the browser. You need it for the tools that build your React project.
2. npm is the Node Package Manager. It downloads and manages code libraries for your projects.
3. VS Code is the recommended code editor. It is free and has many helpful features.
4. The command `npm create vite@latest my-first-app -- --template react` creates a new React project.
5. `npm install` downloads all the code libraries your project needs.
6. `npm run dev` starts a local development server so you can see your app in the browser.
7. Your code lives in the `src/` folder. The main file is `App.jsx`.
8. HMR (Hot Module Replacement) automatically updates the browser when you save a file.
9. `localhost` means "this computer". Your development server runs locally.
10. If something goes wrong, check the terminal and browser console for error messages.

Practice Questions

1. What is Node.js, and why do you need it for React development?
2. What does npm stand for, and what does it do? Use the "app store" analogy to explain it.
3. What does the command `npm run dev` do?
4. What is the purpose of the `node_modules` folder? Should you edit files inside it?
5. What is HMR, and why is it useful when developing a React app?

Exercises

Exercise 1: Personalize Your App

Open `src/App.jsx` and make these changes:

- Change the heading from "Vite + React" to your name (for example, "Sarah's First App").
- Change the button text from "count is {count}" to "Clicked {count} times".
- Save the file and check that the changes appear in your browser.

Exercise 2: Explore the Terminal

Without looking at the instructions above, try to answer these questions by typing commands in your terminal:

- What version of Node.js do you have? (Hint: `node --version`)
- What version of npm do you have? (Hint: `npm --version`)
- What files are in your project folder? (Hint: `ls` on Mac/Linux or `dir` on Windows)

Exercise 3: Break It and Fix It

This exercise is about learning from mistakes. In `src/App.jsx`:

1. Delete one closing tag (for example, delete a `</div>`).
2. Save the file.
3. Look at your terminal and browser. What error messages do you see?
4. Now fix the error by adding the closing tag back. Save and check that the app works again.

Making mistakes and fixing them is one of the best ways to learn. Do not be afraid to break things. You can always undo your changes.

What Is Next?

Your development environment is ready. You have a working React project. You have already changed some code and seen the result. You are officially a React developer (a beginner one, but that still counts!).

In the next chapter, we will take a closer look at JSX, the special syntax that lets you write HTML-like code inside JavaScript. You will learn why React uses it, what the rules are, and how to write your own JSX from scratch. Things are about to get really fun. Let us keep going!

Chapter 5: Understanding Project Structure

What You Will Learn

In this chapter, you will learn:

- What each file and folder in your React project does
- Why each file exists and when you will use it
- Which files you will edit and which ones you should leave alone
- How all the pieces fit together to make your app work

Why This Chapter Matters

When you open your React project for the first time, you see many files and folders. It can feel overwhelming. You might think, "What are all these files? Do I need to understand all of them?"

The good news is: you only need to work with a few of them. But knowing what each file does will help you feel confident. It is like moving into a new house. You want to know where the kitchen is, where the bathroom is, and where the light switches are. You do not need to understand the plumbing on day one, but knowing the layout makes everything easier.

This chapter is your tour of the house.

The Big Picture: Your Project Folder

When you created your React project with Vite in the previous chapter, it made a folder with several files inside. Let us look at the full structure.

Here is what your project looks like:

```
my-first-react-app/
├── node_modules/        <-- The library shelf (never touch this)
├── public/              <-- Static files (things that do not change)
│   └── vite.svg
├── src/                 <-- YOUR code lives here (you work here most)
│   ├── assets/
│   │   └── react.svg
│   ├── App.css
│   ├── App.jsx
│   ├── index.css
│   └── main.jsx
├── .eslintrc.cjs        <-- Code quality checker settings
├── .gitignore           <-- Tells Git which files to ignore
├── index.html           <-- The one and only HTML page
├── package.json         <-- The recipe card for your project
├── package-lock.json    <-- Exact version list (auto-generated)
├── README.md            <-- Notes about your project
└── vite.config.js       <-- Vite settings
```

Do not worry. We will go through each one, step by step.

The `src` Folder — Where You Do Your Work

The `src` folder is the most important folder. The name `src` is short for "source." This is where your source code lives. Source code means the code that YOU write.

Think of it this way:

- The `src` folder is your kitchen. This is where you cook (write code).
- Everything else in the project is like the pantry, the recipe book, or the delivery truck. They help, but you do your main work in the kitchen.

When you are building your app, you will spend almost all of your time inside the `src` folder.

`main.jsx` — Where React Starts

This file is the front door of your app. When your app starts, this is the very first file that runs.

Let us look at what is inside:

```jsx
import React from 'react'
import ReactDOM from 'react-dom/client'
import App from './App.jsx'
import './index.css'

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>,
)
```

Let us go through this line by line.

Line 1: `import React from 'react'` This line brings in the React library. It is like saying, "I need my toolbox."

Line 2: `import ReactDOM from 'react-dom/client'` This line brings in ReactDOM. "DOM" stands for Document Object Model. It is how the browser organizes everything on a web page. ReactDOM connects React to the browser's web page.

Line 3: `import App from './App.jsx'` This line brings in your App component. A component is a piece of your page. We will learn about components in Chapter 7. For now, think of it as the main content of your app.

Line 4: `import './index.css'` This line brings in your main CSS file. CSS controls how things look — colors, sizes, spacing.

Lines 6-9: These lines tell React: "Find the HTML element with the id of 'root', and put the App component inside it."

Think of it like this: React finds an empty box on the page (the `root` element) and fills it with your app.

You will rarely need to change this file. It is set up once and then left alone.

`App.jsx` — Your First Component

This is the main component of your app. When you first open it, it has some starter code:

```jsx
import './App.css'

function App() {
  return (
    <div>
      <h1>Hello React!</h1>
    </div>
```

```
  )
}

export default App
```

Let us go through this line by line.

Line 1: `import './App.css'` This brings in the styles (colors, sizes, etc.) for this component.

Line 3: `function App() {` This creates a function called `App`. In React, a component is just a function. This function describes what should appear on the screen.

Lines 4-8: The `return` part tells React what to show. Here, it shows a heading that says "Hello React!" inside a `div`. A `div` is like a container or a box that holds other things.

Line 11: `export default App` This line makes the `App` component available to other files. Remember how `main.jsx` imported `App`? This is the line that allows that to happen. It is like putting a label on a box so someone can find it and use it.

This is the file you will edit the most when you are starting out.

`index.css` and `App.css` — Styling Files

These are CSS files. CSS stands for Cascading Style Sheets. CSS controls how things look on your page.

- `index.css` has styles that apply to your entire app. Things like the background color of the page or the default font.
- `App.css` has styles that are specific to the App component.

Think of it this way:

- `index.css` is like painting the walls of your entire house.
- `App.css` is like decorating one specific room.

`assets` Folder

This folder holds images, icons, and other media files that your app uses. Right now, it just has `react.svg`, which is the React logo.

When you want to add images to your app, you can put them here.

`index.html` — The One HTML Page

React apps are called "single-page applications" (often shortened to SPA). This means your entire app runs inside one single HTML page.

Here is what `index.html` looks like:

```html
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml" href="/vite.svg" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Vite + React</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

The most important line is this one:

```html
<div id="root"></div>
```

This is the empty box we talked about. React takes this empty box and fills it with your entire app. Everything you build in React will appear inside this one `div`.

The other important line is:

```html
<script type="module" src="/src/main.jsx"></script>
```

This tells the browser to run `main.jsx`, which starts your React app.

Here is the flow:

```
Browser opens index.html
        |
        v
index.html loads main.jsx
        |
        v
main.jsx loads App component
        |
        v
```

```
App component shows on the page
```

You will almost never need to edit `index.html`. Maybe you will change the title of your app, but that is about it.

`package.json` — The Recipe Card

Think of `package.json` as a recipe card for your project. A recipe card tells you:

- What the dish is called
- What ingredients you need
- What steps to follow

Similarly, `package.json` tells your computer:

- What your project is called
- What libraries (ingredients) it needs
- What commands (steps) are available

Here is what it looks like:

```json
{
  "name": "my-first-react-app",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview"
  },
  "dependencies": {
    "react": "^18.2.0",
    "react-dom": "^18.2.0"
  },
  "devDependencies": {
    "vite": "^5.0.0"
  }
}
```

Let us break this down:

`"name"` — The name of your project. Like the title on a recipe card.

`"scripts"` — These are commands you can run. Remember `npm run dev`? That works because `"dev": "vite"` is listed here. It is a shortcut. Instead of typing a long command, you just type `npm run dev`.

Here is what each script does:

- `"dev"` — Starts your app for development (what you use while building)
- `"build"` — Creates a final version of your app ready for the real internet
- `"preview"` — Lets you preview the final version before putting it online

`"dependencies"` — These are the libraries your app needs to run. Right now, you need `react` and `react-dom`. These are your main ingredients.

`"devDependencies"` — These are tools you need only while building your app, not when it is running on the internet. `vite` is here because it is a development tool. Think of it like an oven. You need an oven to bake a cake, but you do not deliver the oven with the cake.

`node_modules` — The Library Shelf

The `node_modules` folder is a big folder that contains all the libraries your project uses. When you ran `npm install`, this folder was created automatically.

Think of it as a library shelf. When your recipe (`package.json`) says you need React, the computer goes and gets React and puts it on this shelf.

Important Rules About `node_modules`

1. Never edit files inside `node_modules`. These files are managed by npm. If you change them, things will break.
2. Never share this folder. It is very large (sometimes thousands of files). When you share your project, you share the recipe (`package.json`), not the ingredients. Other people can run `npm install` to fill their own shelf.
3. You can delete it and get it back. If something goes wrong, you can delete the entire `node_modules` folder and run `npm install` again. It will rebuild everything.

```
You edit these:          You do NOT edit these:
--------------           ----------------------
src/App.jsx              node_modules/ (anything)
src/main.jsx             package-lock.json
src/index.css
src/App.css
```

`public` Folder — Static Files

The `public` folder holds files that do not change and do not need to be processed by Vite. Right now, it just has `vite.svg` (the Vite logo that shows as the browser tab icon).

Things you might put in the `public` folder:

- A favicon (the small icon in the browser tab)
- Images that you want to reference directly
- A `robots.txt` file (tells search engines about your site)

Most of the time, you will put your images in `src/assets/` instead. But the `public` folder is there when you need it.

Other Files

`.gitignore`

Git is a tool that tracks changes to your code (we will talk about it later). The `.gitignore` file tells Git which files to ignore. For example, it tells Git to ignore the `node_modules` folder because that folder is too large to track.

`.eslintrc.cjs`

ESLint is a tool that checks your code for common mistakes. This file tells ESLint what rules to follow. You do not need to worry about this file right now.

`package-lock.json`

This file is auto-generated. It records the exact version of every library installed. You never edit this file directly.

`vite.config.js`

This file has settings for Vite. The default settings work fine for beginners. You do not need to change it.

`README.md`

This is a notes file about your project. The `.md` stands for Markdown, which is a simple way to format text. You can write notes about your project here.

Which Files Will You Edit?

Here is a simple guide:

```
+-----------------------------------------------------+
|              Files You Will Edit Often              |
+-----------------------------------------------------+
| src/App.jsx       Your main component               |
| src/App.css       Styles for your main component   |
| src/index.css     Global styles                    |
| Files you create inside src/                       |
+-----------------------------------------------------+

+-----------------------------------------------------+
|           Files You Will Edit Sometimes             |
+-----------------------------------------------------+
| index.html        Maybe change the page title      |
| package.json      When adding new libraries        |
+-----------------------------------------------------+

+-----------------------------------------------------+
|             Files You Will NOT Edit                 |
+-----------------------------------------------------+
| node_modules/     Managed by npm                   |
| package-lock.json Auto-generated                   |
| vite.config.js    Default settings are fine        |
+-----------------------------------------------------+
```

How Everything Connects

Let us see the full picture of how your app starts:

```
1. You type: npm run dev

2. Vite starts a development server
```

```
3. Your browser opens index.html

4. index.html has <div id="root"></div>
   and loads main.jsx

5. main.jsx tells React:
   "Put the App component inside the root div"

6. App.jsx returns JSX
   (the content to show on the page)

7. You see your app in the browser!
```

Here is another way to see it:

```
index.html  -->  main.jsx  -->  App.jsx  -->  What you see
(the page)      (the start)    (the content)  (in browser)
```

Every React app follows this same pattern. Once you understand it, you will feel at home in any React project.

Quick Summary

- Your React project has a clear structure. Each file has a job.
- The `src` folder is where you do most of your work.
- `main.jsx` is the starting point. It connects React to the page.
- `App.jsx` is your main component. You will edit this the most.
- `index.html` is the one HTML page. React fills it with content.
- `package.json` is like a recipe card. It lists what your project needs.
- `node_modules` is the library shelf. Never edit it.
- The `public` folder holds static files that do not change.

Key Points to Remember

1. You work inside the `src` folder almost all the time.

2. Never edit files in `node_modules`. If something goes wrong, delete the folder and run `npm install`.
3. `package.json` is your recipe card. It lists dependencies (ingredients) and scripts (cooking steps).
4. React is a single-page app. Everything runs inside one HTML page with a `<div id="root">`.
5. The flow is: `index.html` loads `main.jsx`, which loads `App.jsx`. This chain is how your app starts.

Practice Questions

1. What is the purpose of the `src` folder? Why is it called `src`?
2. You want to change what your app shows on the screen. Which file do you edit?
3. A friend wants to try your project on their computer. Should you send them the `node_modules` folder? Why or why not?
4. What does `package.json` do? What real-life thing is it similar to?
5. Your app is not starting. You think a library might be broken. What can you try? (Hint: it involves `node_modules` and `npm install`.)

Exercises

Exercise 1: Change the Page Title

Open `index.html` and find this line:

```html
<title>Vite + React</title>
```

Change it to:

```html
<title>My First React App</title>
```

Save the file. Look at your browser tab. The title should change.

Exercise 2: Edit the App Component

Open `src/App.jsx`. Replace everything inside the `return` statement with your own content:

```jsx
function App() {
  return (
    <div>
      <h1>Welcome to My App</h1>
      <p>I am learning React!</p>
    </div>
  )
}

export default App
```

Save the file. Your browser should update automatically and show your new content.

Exercise 3: Explore `package.json`

Open `package.json` and answer these questions:

1. What is the name of your project?
2. How many dependencies does it have? What are they?
3. What command does the `"dev"` script run?

Write your answers down. Checking your own understanding is a great way to learn.

What Is Next?

You now know your way around a React project. You know where to find things and which files to edit. You have a map of the house.

In the next chapter, we will learn about JSX — the special way React lets you write what your app looks like. It looks a lot like HTML, but it has some important differences. JSX is one of the things that makes React special and fun to use. Let us go learn about it!

Chapter 6: JSX Explained Simply

What You Will Learn

In this chapter, you will learn:

- What JSX is and why React uses it
- The rules you must follow when writing JSX
- How to put JavaScript values inside your JSX using curly braces `{}`
- Common mistakes beginners make with JSX and how to fix them
- The key differences between JSX and HTML

Why This Chapter Matters

JSX is one of the first things you see when you open a React file. It looks like HTML, but it is not exactly HTML. If you try to write regular HTML in React, you will run into errors.

Think of JSX like a different dialect of a language. If you speak English, you can mostly understand Australian English. But some words are different. "Boot" means the trunk of a car. "Arvo" means afternoon. You need to learn the differences, or you will get confused.

JSX is React's dialect. Once you learn its rules, everything will feel natural.

What Is JSX?

JSX stands for JavaScript XML. XML is a way of writing structured data using tags, similar to HTML.

In simple terms: JSX lets you write HTML-like code inside your JavaScript files.

Here is a simple example:

```jsx
function Greeting() {
  return <h1>Hello, World!</h1>
}
```

That `<h1>Hello, World!</h1>` part is JSX. It looks like HTML, but it is actually inside a JavaScript function.

Why Does React Use JSX?

In traditional web development, you keep your HTML, CSS, and JavaScript in separate files:

```
Traditional way:
  index.html    (structure — what is on the page)
  styles.css    (appearance — how it looks)
  script.js     (behavior — what it does)
```

React does things differently. It puts the structure and behavior together in the same file using JSX.

Why? Because in modern apps, the structure and behavior of a piece of the page are closely connected. If you have a button, the way it looks and what it does when you click it are related. Keeping them in the same file makes it easier to understand and change.

Think of it like a recipe. Imagine if someone gave you a recipe where the ingredients were in one book, the cooking steps were in another book, and the serving instructions were in a third book. That would be hard to follow. It is much easier when everything for one recipe is on one card.

JSX is like putting the recipe on one card.

Your First JSX

Let us write a simple component with JSX:

```jsx
function Welcome() {
  return (
    <div>
```

```
      <h1>Welcome to React</h1>
      <p>This is my first JSX code.</p>
    </div>
  )
}
```

Line 1: We create a function called `Welcome`. This is a React component.

Line 2: The `return` keyword tells React what to show on the screen.

Lines 3-6: This is JSX. It describes what the page should look like. It has a `div` container with a heading and a paragraph inside it.

Expected output in the browser:

```
Welcome to React
This is my first JSX code.
```

The JSX looks like HTML, but React converts it into real HTML behind the scenes.

The Rules of JSX

JSX has rules. If you break them, your app will show an error. Let us learn each rule with examples.

Rule 1: You Must Return One Parent Element

In JSX, everything you return must be wrapped inside one single parent element. You cannot return two elements side by side without a wrapper.

This is WRONG:

```jsx
// This will cause an error!
function Broken() {
  return (
    <h1>Hello</h1>
    <p>World</p>
  )
}
```

React will show an error: "Adjacent JSX elements must be wrapped in an enclosing tag."

This is CORRECT:

```jsx
function Fixed() {
  return (
    <div>
      <h1>Hello</h1>
      <p>World</p>
    </div>
  )
}
```

We wrapped both elements inside a `<div>`. Now there is one parent element.

Another correct way — using a Fragment:

```jsx
function AlsoFixed() {
  return (
    <>
      <h1>Hello</h1>
      <p>World</p>
    </>
  )
}
```

The `<>` and `</>` are called a Fragment. A Fragment is an invisible wrapper. It groups elements together without adding an extra `div` to your page. Think of it like a clear plastic bag. It holds things together, but you cannot see it.

Rule 2: Close All Tags

In HTML, some tags do not need a closing tag. For example, `<br>` and `<img>` can stand alone in HTML.

In JSX, every tag must be closed. No exceptions.

This is WRONG:

```jsx
// These will cause errors in JSX!
<br>
<img src="photo.jpg">
<input type="text">
```

This is CORRECT:

```jsx
// Self-closing tags — note the / before >
<br />
<img src="photo.jpg" />
<input type="text" />
```

The `/` before the `>` closes the tag. These are called self-closing tags.

Rule 3: Use `className` Instead of `class`

In HTML, you use `class` to give an element a CSS class:

```html
<!-- HTML way -->
<div class="container">Hello</div>
```

In JSX, you use `className` instead:

```jsx
// JSX way
<div className="container">Hello</div>
```

Why? Because `class` is already a reserved word in JavaScript. It means something else in JavaScript (it is used to create objects). To avoid confusion, React uses `className`.

Think of it like this: in a school, "class" might mean a group of students or a classroom. To avoid confusion, you might call one "class-group" and the other "classroom." React does a similar thing.

Rule 4: Use camelCase for Attributes

HTML uses lowercase attribute names:

```html
<!-- HTML -->
<div onclick="doSomething()" tabindex="1">Click me</div>
```

JSX uses camelCase for attribute names. CamelCase means the first word is lowercase, and every next word starts with a capital letter, like the humps of a camel:

```jsx
// JSX
<div onClick={doSomething} tabIndex="1">Click me</div>
```

Here are some common examples:

```
HTML attribute     JSX attribute
---------------    -------------
onclick            onClick
onchange           onChange
tabindex           tabIndex
maxlength          maxLength
for                htmlFor
```

Notice that `for` becomes `htmlFor`. Just like `class`, the word `for` already means something in JavaScript (it is used in loops). So JSX uses `htmlFor`.

Rule 5: JavaScript Expressions Use Curly Braces

When you want to put JavaScript inside your JSX, you wrap it in curly braces `{}`. We will cover this in detail in the next section.

Curly Braces: Putting JavaScript Inside JSX

This is one of the most useful features of JSX. Curly braces `{}` let you insert JavaScript values into your JSX.

Think of curly braces like fill-in-the-blank spaces on a form. The JSX is the form, and the curly braces are the blanks where you fill in dynamic values.

Showing a Variable

```jsx
function Hello() {
  const name = "Sarah"

  return <h1>Hello, {name}!</h1>
}
```

Line 2: We create a variable called `name` and give it the value `"Sarah"`. A variable is like a labeled box that holds a value.

Line 4: Inside the JSX, `{name}` gets replaced with the value of the variable.

Expected output:

```
Hello, Sarah!
```

Doing Math

```jsx
function MathExample() {
  const price = 10
  const quantity = 3

  return <p>Total: ${price * quantity}</p>
}
```

Line 2-3: We create two variables.

Line 5: Inside `{}`, we multiply `price` times `quantity`. React calculates `10 * 3` and shows `30`.

Expected output:

```
Total: $30
```

Showing Today's Date

```jsx
function Today() {
  const today = new Date()
  const day = today.toLocaleDateString()

  return <p>Today is: {day}</p>
}
```

Line 2: `new Date()` creates an object that represents the current date and time.

Line 3: `.toLocaleDateString()` converts it to a readable format like "3/25/2026".

Line 5: We show the date inside the JSX.

Expected output:

```
Today is: 3/25/2026
```

Calling a Function

```jsx
function Shout() {
  const message = "hello"

  return <p>{message.toUpperCase()}</p>
}
```

Line 4: `.toUpperCase()` converts text to all capital letters. We can call functions inside curly braces.

Expected output:

```
HELLO
```

What You Can Put Inside Curly Braces

You can put any JavaScript expression inside curly braces. An expression is anything that produces a value.

```
Can use inside {}:          Cannot use inside {}:
-------------------         ----------------------
Variables:    {name}        if/else statements
Math:         {2 + 2}      for loops
Functions:    {getName()}   Variable declarations
Strings:      {"hello"}      (const x = 5)
```

If you need to use if/else logic, you can use a ternary expression (a short way to write if/else):

```jsx
function Weather() {
  const isSunny = true

  return (
    <p>
      The weather is {isSunny ? "sunny" : "cloudy"}.
    </p>
  )
}
```

Line 6: The `?` and `:` create a ternary expression. It reads like this: "Is `isSunny` true? If yes, use 'sunny'. If no, use 'cloudy'."

Expected output:

```
The weather is sunny.
```

JSX Is NOT HTML: Key Differences

Here is a quick reference table of the main differences:

```
+-------------------------------------------+
|        HTML              JSX              |
+-------------------------------------------+
| class="..."         className="..."        |
| for="..."           htmlFor="..."         |
| onclick="..."       onClick={...}         |
| tabindex="..."      tabIndex="..."        |
| <br>                <br />                |
| <img ...>           <img ... />           |
| style="color: red"  style={{color: "red"}} |
| <!-- comment -->    {/* comment */}       |
+-------------------------------------------+
```

Notice two special differences:

Inline styles use double curly braces and camelCase property names:

```jsx
// HTML way:
// <p style="background-color: blue; font-size: 20px">Hello</p>

// JSX way:
<p style={{ backgroundColor: "blue", fontSize: "20px" }}>Hello</p>
```

The outer `{}` means "here comes JavaScript." The inner `{}` creates a JavaScript object (a collection of key-value pairs). Style property names use camelCase: `background-color` becomes `backgroundColor`.

Comments in JSX use a different format:

```jsx
function Example() {
  return (
    <div>
      {/* This is a comment in JSX */}
      <p>Hello</p>
    </div>
  )
}
```

HTML comments `<!-- -->` do not work in JSX. You must use `{/* */}`.

Common Mistakes and How to Fix Them

Mistake 1: Forgetting to Close a Tag

```jsx
// WRONG
<img src="cat.jpg">

// RIGHT
<img src="cat.jpg" />
```

Error you might see: "Expected corresponding JSX closing tag."

Mistake 2: Using `class` Instead of `className`

```jsx
// WRONG
<div class="box">Hello</div>

// RIGHT
<div className="box">Hello</div>
```

What happens: React will show a warning in the console: "Invalid DOM property 'class'. Did you mean 'className'?"

Mistake 3: Returning Two Elements Without a Wrapper

```jsx
// WRONG
return (
  <h1>Title</h1>
  <p>Paragraph</p>
)

// RIGHT — use a div or Fragment
return (
  <>
    <h1>Title</h1>
    <p>Paragraph</p>
  </>
)
```

Error you might see: "Adjacent JSX elements must be wrapped in an enclosing tag."

Mistake 4: Forgetting Curly Braces for JavaScript

```jsx
const name = "Alex"

// WRONG — this shows the text "name" literally
<p>Hello, name!</p>

// RIGHT — curly braces tell React to use the variable
<p>Hello, {name}!</p>
```

What happens: Without curly braces, React shows the literal word "name" instead of the value "Alex."

Mistake 5: Using Quotes Around Curly Braces

```jsx
const name = "Alex"

// WRONG — this shows the text "{name}" literally
<p>Hello, "{name}"!</p>

// RIGHT
<p>Hello, {name}!</p>
```

Do not wrap curly braces in quotes. The curly braces ARE the signal to React. Quotes around them turn everything into plain text.

A Complete Example

Let us put it all together in one component:

```jsx
function ProfileCard() {
  const name = "Maria"
  const age = 28
  const hobby = "painting"
  const isMember = true

  return (
    <div className="card">
      <h2>{name}</h2>
      <p>Age: {age}</p>
```

```
      <p>Hobby: {hobby}</p>
      <p>Member: {isMember ? "Yes" : "No"}</p>
      <p>Birth year: roughly {2026 - age}</p>
      {/* This line will not show on the page */}
    </div>
  )
}
```

Expected output:

```
Maria
Age: 28
Hobby: painting
Member: Yes
Birth year: roughly 1998
```

Line by line:

- Lines 2-5: We create variables that hold information about a person.
- Line 9: `{name}` shows "Maria."
- Line 10: `{age}` shows 28.
- Line 11: `{hobby}` shows "painting."
- Line 12: The ternary expression checks if `isMember` is true. It is, so it shows "Yes."
- Line 13: `{2026 - age}` does math and shows 1998.
- Line 14: This is a JSX comment. It does not show on the page.

Quick Summary

- JSX is HTML-like code that you write inside JavaScript files.
- React uses JSX so you can keep your structure and behavior together.
- JSX has rules: one parent element, close all tags, use `className`, use camelCase.
- Curly braces `{}` let you put JavaScript values inside JSX, like fill-in-the-blank spaces.
- JSX is not HTML. There are important differences in syntax.
- Common mistakes are easy to fix once you know the rules.

Key Points to Remember

1. JSX must have one parent element. Wrap everything in a `<div>` or a Fragment `<>`.
2. Close every tag. Self-closing tags need a slash: `<br />`, `<img />`.
3. Use `className` instead of `class`. This is one of the most common JSX rules.
4. Curly braces `{}` insert JavaScript into JSX. Variables, math, functions — all go inside curly braces.
5. JSX looks like HTML but is actually JavaScript. React converts your JSX into real HTML behind the scenes.

Practice Questions

1. What does JSX stand for? In your own words, what is JSX?
2. Why does React use `className` instead of `class`?
3. What are curly braces `{}` used for in JSX? Give an example.
4. What is a Fragment? When would you use one instead of a `div`?
5. You write `<p>Hello, name!</p>` but you want it to show the value of a variable called `name`. What is wrong, and how do you fix it?

Exercises

Exercise 1: Create a Personal Info Card

Create a component called `AboutMe` that shows your name, your age, and your favorite food. Use variables and curly braces.

```jsx
function AboutMe() {
  const name = "Your Name"
  const age = 25
  const food = "pizza"

  return (
    <div>
      <h1>About Me</h1>
      <p>Name: {name}</p>
      <p>Age: {age}</p>
      <p>Favorite food: {food}</p>
    </div>
```

```
  )
}
```

Try changing the variable values and see the output update.

Exercise 2: Build a Simple Calculator Display

Create a component that shows the results of some math:

```jsx
function Calculator() {
  const a = 15
  const b = 4

  return (
    <div>
      <h2>Calculator</h2>
      <p>{a} + {b} = {a + b}</p>
      <p>{a} - {b} = {a - b}</p>
      <p>{a} x {b} = {a * b}</p>
    </div>
  )
}
```

Change the values of `a` and `b` and check that the results update correctly.

Exercise 3: Fix the Broken JSX

This code has several mistakes. Find and fix all of them:

```jsx
function Broken() {
  const city = "Tokyo"

  return (
    <h1>My City</h1>
    <p class="info">I live in city.</p>
    <img src="tokyo.jpg">
  )
}
```

Hint: There are at least three mistakes. Look at the rules you learned in this chapter.

What Is Next?

You now know how to write JSX — the language React uses to describe your pages. You know the rules, you know how to put JavaScript inside it, and you know how to avoid common mistakes.

In the next chapter, we will learn about Components — the building blocks of every React app. Components are like Lego bricks. Each one is a small, reusable piece. You combine them to build something amazing. Let us go build some!

Chapter 7: Components

What You Will Learn

In this chapter, you will learn:

- What a component is and why components are the heart of React
- How to create your own component
- Why component names must start with a capital letter
- How to use one component inside another
- How to put multiple components on one page
- How to organize components in separate files
- How a component tree works

Why This Chapter Matters

Components are the most important idea in React. Everything in React is a component. The header of a website is a component. A button is a component. A search bar is a component. Even the entire page is a component.

Think of components like Lego bricks. Each brick is small and simple on its own. But when you connect many bricks together, you can build a house, a car, or a spaceship. In React, you build small components and connect them together to create your full app.

Once you understand components, you understand React. This chapter is the foundation for everything that comes after it.

What Is a Component?

A component is a reusable piece of your page. It is a small, independent unit that shows something on the screen.

Here are some examples of components on a typical website:

```
+-----------------------------------------------------+
|  [ Logo ]     [ Home ] [ About ] [ Contact ]     |  <-- Header component
+-----------------------------------------------------+
|                                                     |
|   +-------------------------------------------+  |
|   |  Welcome to My Website!                   |  |  <-- Hero component
|   |  We make great things.                    |  |
|   +-------------------------------------------+  |
|                                                     |
|   +----------+  +----------+  +----------+       |
|   |  Card 1  |  |  Card 2  |  |  Card 3  |       |  <-- Card components
|   |          |  |          |  |          |       |
|   +----------+  +----------+  +----------+       |
|                                                     |
+-----------------------------------------------------+
|  Copyright 2026 | Privacy | Terms                 |  <-- Footer component
+-----------------------------------------------------+
```

Each section is a component. The three cards are all the same component used three times with different content. That is the power of reusability.

Creating Your First Component

A component in React is simply a function that returns JSX. That is it. Let us create one.

```jsx
function Greeting() {
  return <h1>Hello, welcome to my app!</h1>
}
```

Line 1: We create a function called `Greeting`. This is our component.

Line 2: The function returns JSX — the content that will appear on the screen.

That is a complete component. Just a function that returns what to show.

Using Your Component

To use a component, you write it like an HTML tag:

```jsx
function App() {
  return (
    <div>
      <Greeting />
    </div>
  )
}
```

Line 4: `<Greeting />` tells React to run the `Greeting` function and show its result here.

Expected output:

```
Hello, welcome to my app!
```

Notice how we write `<Greeting />` with a capital G and a self-closing slash. This is how you "call" a component in JSX.

The Capital Letter Rule

Component names must start with a capital letter. This is not optional. It is a strict rule.

```jsx
// CORRECT — starts with a capital letter
function Greeting() {
  return <h1>Hello!</h1>
}

// WRONG — starts with a lowercase letter
function greeting() {
  return <h1>Hello!</h1>
}
```

Why does this rule exist? Because React uses the first letter to tell the difference between a component and a regular HTML tag.

```jsx
<div>       // lowercase = HTML tag (a regular div)
<Greeting>  // uppercase = React component (your custom piece)
```

If you write `<greeting>`, React thinks you mean an HTML tag called "greeting." Since no such HTML tag exists, things will not work correctly.

Think of it like proper nouns in English. Regular words are lowercase: "table," "chair," "door." But names are capitalized: "Sarah," "Tokyo," "Monday." Components are like names. They are special, so they get a capital letter.

A Component with More Content

Components can return as much JSX as you need. Just remember to wrap everything in one parent element.

```jsx
function UserCard() {
  return (
    <div>
      <h2>Jane Smith</h2>
      <p>Age: 30</p>
      <p>Job: Designer</p>
    </div>
  )
}
```

Expected output:

```
Jane Smith
Age: 30
Job: Designer
```

Using a Component Inside Another Component

This is where things get exciting. You can put components inside other components. This is called composition. Composition means building something by combining smaller pieces.

```jsx
function Header() {
  return <h1>My Website</h1>
}

function MainContent() {
  return <p>Welcome to the best website ever!</p>
```

```
}

function Footer() {
  return <p>Copyright 2026</p>
}

function App() {
  return (
    <div>
      <Header />
      <MainContent />
      <Footer />
    </div>
  )
}
```

Line 14-20: The `App` component uses three other components: `Header`, `MainContent`, and `Footer`.

Expected output:

```
My Website
Welcome to the best website ever!
Copyright 2026
```

Each component handles its own part of the page. This keeps your code organized. If you want to change the footer, you go to the `Footer` component. You do not have to search through hundreds of lines of code.

Multiple Components on One Page

You can use the same component more than once:

```jsx
function Divider() {
  return <hr />
}

function App() {
  return (
    <div>
      <h1>My Page</h1>
      <Divider />
```

```
      <p>First section of content.</p>
      <Divider />
      <p>Second section of content.</p>
      <Divider />
      <p>Third section of content.</p>
    </div>
  )
}
```

We used the `Divider` component three times. The `<hr />` tag creates a horizontal line. Each `<Divider />` shows a line on the page.

Expected output:

```
My Page
___________________
First section of content.
___________________
Second section of content.
___________________
Third section of content.
```

Write once, use many times. That is the beauty of components.

The Component Tree

When you put components inside other components, you create a component tree. It looks like a family tree. The top component is the parent, and the components inside it are the children.

Here is the tree for our earlier example:

```
          App
         / | \
        /  |  \
   Header  |  Footer
           |
      MainContent
```

`App` is the parent. It has three children: `Header`, `MainContent`, and `Footer`.

Here is a more complex example:

```
              App
             / | \
            /  |  \
      Header  Main  Footer
       / \      |
      /   \     |
   Logo  Nav  ArticleList
              /    \
             /      \
        Article   Article
```

The tree can go as deep as you need. Each level is a component that contains smaller components. This is how real React apps are built.

React always starts from the top of the tree (the `App` component) and works its way down, rendering each component.

Organizing Components in Separate Files

So far, we have written all our components in one file. In real projects, each component gets its own file. This keeps things organized.

Here is how you do it:

Step 1: Create a New File

Create a file called `Header.jsx` inside your `src` folder:

```
src/
├── App.jsx
├── Header.jsx      <-- new file
├── Footer.jsx      <-- new file
├── main.jsx
└── index.css
```

Step 2: Write the Component and Export It

In `Header.jsx`:

```jsx
function Header() {
  return (
    <header>
      <h1>My Website</h1>
      <p>The best website on the internet</p>
    </header>
  )
}

export default Header
```

Line 10: `export default Header` makes this component available to other files. Without this line, other files cannot use it.

Think of `export` like putting a label on a package. The label says, "This is the Header. Other files can request it."

In `Footer.jsx`:

```jsx
function Footer() {
  return (
    <footer>
      <p>Copyright 2026. All rights reserved.</p>
    </footer>
  )
}

export default Footer
```

Step 3: Import the Component Where You Need It

In `App.jsx`:

```jsx
import Header from './Header.jsx'
import Footer from './Footer.jsx'

function App() {
  return (
    <div>
      <Header />
      <p>This is the main content of my app.</p>
      <Footer />
    </div>
  )
}

export default App
```

Line 1: `import Header from './Header.jsx'` brings the Header component into this file. The `./` means "in the same folder."

Line 2: Same thing for Footer.

Lines 7-9: Now we use them just like before.

Expected output:

```
My Website
The best website on the internet
This is the main content of my app.
Copyright 2026. All rights reserved.
```

Understanding Import and Export

Think of import and export like a mail system:

- `export` is like putting a package in the mailbox. You are sending it out.
- `import` is like receiving a package. You are bringing it in.

A file can only export something if it has something to share. A file can import something only if the other file has exported it.

```
Header.jsx                        App.jsx
+--------------------+            +--------------------+
|                    |            |                    |
| function Header()  |  export   | import Header      |
| { ... }           | -------->  | from './Header'    |
|                    |            |                    |
| export default     |            | <Header />         |
|   Header           |            |                    |
+--------------------+            +--------------------+
```

Putting It All Together: A Mini Website

Let us build a small website with multiple components in separate files.

`Header.jsx`

```jsx
function Header() {
  return (
    <header>
      <h1>Book Store</h1>
      <nav>
        <a href="#">Home</a> | <a href="#">Books</a> | <a href="#">Contact</a>
      </nav>
    </header>
  )
}

export default Header
```

Main.jsx

```jsx
function Main() {
  return (
    <main>
      <h2>Featured Books</h2>
      <p>Check out our top picks for this month!</p>
    </main>
  )
}

export default Main
```

Footer.jsx

```jsx
function Footer() {
  return (
    <footer>
      <p>Book Store - Helping you find great reads since 2024</p>
    </footer>
  )
}

export default Footer
```

App.jsx

```jsx
import Header from './Header.jsx'
import Main from './Main.jsx'
```

```
import Footer from './Footer.jsx'

function App() {
  return (
    <div>
      <Header />
      <Main />
      <Footer />
    </div>
  )
}

export default App
```

Expected output:

```
Book Store
Home | Books | Contact

Featured Books
Check out our top picks for this month!

Book Store – Helping you find great reads since 2024
```

The component tree for this app:

```
        App
       / | \
      /  |  \
 Header Main Footer
```

Each component has one job. The `Header` shows the top of the page. `Main` shows the content. `Footer` shows the bottom. If you want to change the navigation links, you only edit `Header.jsx`. Everything else stays the same.

Quick Summary

- A component is a function that returns JSX. It is a reusable piece of your page.
- Component names must start with a capital letter.
- You use a component by writing it like a tag: `<MyComponent />`.
- Components can be placed inside other components. This is called composition.
- You can use the same component multiple times.

- In real projects, each component goes in its own file.
- `export default` makes a component available. `import` brings it into another file.
- Components form a tree, with `App` at the top.

Key Points to Remember

1. A component is just a function that returns JSX. Nothing more, nothing less.
2. Capital first letter is required. `<Header />` is a component. `<header>` is an HTML tag.
3. One component, one job. Each component should do one thing well.
4. export and import connect components across files. Export sends it out. Import brings it in.
5. The component tree starts at `App` and branches downward. React renders from top to bottom.

Practice Questions

1. What is a component in React? Describe it in your own words.
2. Why must component names start with a capital letter? What happens if they do not?
3. What is the difference between `export default` and `import`? Use the mail analogy to explain.
4. Can you use the same component more than once on a page? Give an example of when that would be useful.
5. What is a component tree? Draw the component tree for an app that has an `App` component containing a `Sidebar`, a `Content` area, and a `Footer`.

Exercises

Exercise 1: Create a Greeting Component

Create a component called `Greeting` that shows a friendly welcome message. Use it inside the `App` component.

Your component should display:

```
Welcome, Friend!
We are glad you are here.
```

Exercise 2: Build a Page with Three Components

Create three separate components:

1. `Header` — shows the title "My Blog"
2. `Article` — shows a paragraph of text about any topic you like
3. `Footer` — shows "Thanks for reading!"

Use all three inside the `App` component. If you feel comfortable, put each one in its own file.

Exercise 3: Reuse a Component

Create a `Star` component that simply shows the text " *" (five stars). Use this component three times inside your `App` to create a rating display:

```
Food Quality:
* * * * *
Service:
* * * * *
Atmosphere:
* * * * *
```

What Is Next?

You now know how to create and use components. But right now, our components always show the same thing. A `Greeting` component always says the same message. A `UserCard` always shows the same person.

What if you could make a component that shows different content depending on what you tell it? What if you could pass information to a component, like handing it a note that says "show this name" or "use this color"?

That is exactly what Props are. In the next chapter, you will learn how to pass data to your components and make them truly flexible and reusable. Let us go!

Chapter 8: Props

What You Will Learn

In this chapter, you will learn:

- What props are and why they are essential
- How to pass different types of data to a component
- How to read props inside a component
- The shortcut way to read props (destructuring)
- Why props are read-only and cannot be changed
- How to set default values for props
- How to pass props to multiple components
- What the special `children` prop is

Why This Chapter Matters

In the last chapter, you learned how to create components. But those components were static. They always showed the same thing. A `Greeting` component always said "Hello, welcome!" no matter what.

In real apps, components need to be flexible. A name tag component should show different names. A product card should show different products. A button should show different text.

Props make this possible. Props are how you pass information to a component so it can show different things each time you use it.

Think of a component like a greeting card template at a store. The template has spaces for a name and a message. When you buy the card, you fill in the blanks. Props are the blanks you fill in.

What Are Props?

Props is short for properties. Props are pieces of information that you pass to a component from the outside.

Think of props like passing a note to someone. You write information on the note and hand it over. The person reads the note and uses that information.

```
You (parent component)          Greeting (child component)
+------------------+           +------------------------+
|                  |           |                        |
| "Hey Greeting,   |  props   | "I received the note.  |
|  the name is     | -------> |  The name is Sarah.    |
|  Sarah."         |           |  I will show it."      |
|                  |           |                        |
+------------------+           +------------------------+
```

The parent component passes data. The child component receives and uses it.

Passing Your First Prop

Let us start with a simple example. We will make a `Greeting` component that shows a personalized message.

Step 1: Pass the Prop

When you use a component, you pass props like HTML attributes:

```jsx
function App() {
  return (
    <div>
      <Greeting name="Sarah" />
      <Greeting name="James" />
      <Greeting name="Maria" />
    </div>
  )
}
```

Lines 4-6: We use the `Greeting` component three times. Each time, we pass a different `name` prop. It looks just like an HTML attribute.

Step 2: Receive the Prop

The component receives all props as a single object (a collection of key-value pairs). This object is passed as the first parameter of the function:

```jsx
function Greeting(props) {
  return <h1>Hello, {props.name}!</h1>
}
```

Line 1: The function takes one parameter called `props`. This is an object that contains all the props that were passed.

Line 2: We access the name by writing `props.name`.

The Complete Example

```jsx
function Greeting(props) {
  return <h1>Hello, {props.name}!</h1>
}

function App() {
  return (
    <div>
      <Greeting name="Sarah" />
      <Greeting name="James" />
      <Greeting name="Maria" />
    </div>
  )
}
```

Expected output:

```
Hello, Sarah!
Hello, James!
Hello, Maria!
```

One component, three different results. That is the power of props.

Passing Different Types of Data

You can pass many types of data as props. Let us see each one.

Strings

For text, use quotes:

```jsx
<UserCard name="Alice" city="Tokyo" />
```

Numbers

For numbers, use curly braces:

```jsx
<UserCard age={25} score={98.5} />
```

Why curly braces? Because in JSX, curly braces mean "this is a JavaScript value." The number `25` is a JavaScript value, not text.

If you write `age="25"`, it becomes the text string "25," not the number 25. This matters if you want to do math with it later.

Booleans

A boolean is a value that is either `true` or `false`. Think of it like a light switch: on or off.

```jsx
<UserCard isOnline={true} isPremium={false} />
```

There is a shortcut for `true`. If you write just the prop name without a value, it defaults to `true`:

```jsx
// These two are the same:
<UserCard isOnline={true} />
<UserCard isOnline />
```

A Complete Example with Multiple Types

```jsx
function ProductCard(props) {
  return (
    <div>
      <h2>{props.name}</h2>
      <p>Price: ${props.price}</p>
      <p>{props.inStock ? "In Stock" : "Sold Out"}</p>
```

```
    </div>
  )
}

function App() {
  return (
    <div>
      <ProductCard name="Notebook" price={4.99} inStock={true} />
      <ProductCard name="Pen Set" price={12.50} inStock={false} />
    </div>
  )
}
```

Expected output:

```
Notebook
Price: $4.99
In Stock

Pen Set
Price: $12.5
Sold Out
```

Line 6: The ternary expression `props.inStock ? "In Stock" : "Sold Out"` checks if the product is in stock. If `true`, it shows "In Stock." If `false`, it shows "Sold Out."

Destructuring Props (The Shortcut Way)

Writing `props.name`, `props.age`, and `props.city` everywhere can feel repetitive. There is a shortcut called destructuring. Destructuring means pulling out specific values from an object and giving them their own names.

Without Destructuring

```jsx
function Greeting(props) {
  return <h1>Hello, {props.name}! You are {props.age} years old.</h1>
}
```

With Destructuring

```jsx
function Greeting({ name, age }) {
  return <h1>Hello, {name}! You are {age} years old.</h1>
}
```

Line 1: Instead of writing `props`, we write `{ name, age }`. This pulls `name` and `age` directly out of the props object.

Line 2: Now we can write `{name}` instead of `{props.name}`. Shorter and cleaner.

Both ways do exactly the same thing. Destructuring is just a shortcut. Most React developers prefer it because it is easier to read.

Think of it like receiving a package. Without destructuring, you get the whole box and reach inside for each item. With destructuring, the items are unpacked and laid out on the table for you.

```
Without destructuring:        With destructuring:
+------------------+          name = "Sarah"
| props            |   -->    age = 25
|   name: "Sarah"  |          city = "Paris"
|   age: 25        |
|   city: "Paris"  |
+------------------+
```

Props Are Read-Only

This is a very important rule: you cannot change props inside a component.

Props are like a letter you received. You can read the letter. You can use the information in the letter. But you cannot change what the letter says. The sender wrote it, and you just receive it.

```jsx
// WRONG — This will cause an error!
function Greeting({ name }) {
  name = "Someone Else"  // DO NOT do this!
  return <h1>Hello, {name}!</h1>
}
```

React will show an error or unexpected behavior if you try to change props. This rule exists for a good reason: it keeps your app predictable. If a parent passes `name="Sarah"`, the component should always show "Sarah." If components could change their own props, it would be very

hard to understand what your app is doing.

If you need a value that can change, you will learn about state in a later chapter. State is like a whiteboard inside your component: you can write on it, erase it, and write something new.

Default Props

Sometimes a prop is optional. If the parent does not pass it, you want a fallback value. This is called a default prop.

You set default values using the `=` sign in the destructuring:

```jsx
function Greeting({ name = "Friend", color = "blue" }) {
  return (
    <h1 style={{ color: color }}>
      Hello, {name}!
    </h1>
  )
}
```

Line 1: If `name` is not provided, it will be `"Friend"`. If `color` is not provided, it will be `"blue"`.

Now you can use the component with or without those props:

```jsx
function App() {
  return (
    <div>
      <Greeting name="Sarah" color="red" />
      <Greeting name="James" />
      <Greeting />
    </div>
  )
}
```

Expected output:

```
Hello, Sarah!       (in red)
Hello, James!       (in blue — default color)
Hello, Friend!      (in blue — both defaults used)
```

- Line 4: Both props are provided. Shows "Sarah" in red.
- Line 5: Only `name` is provided. `color` uses the default value "blue."

- Line 6: No props are provided. Both default values are used.

Default props are like the pre-printed text on a form. If you do not fill in the blank, the default text is used.

Passing Props to Multiple Components

In real apps, you often pass props through several layers of components. The data flows downward, from parent to child to grandchild.

```jsx
function App() {
  return <UserProfile name="Sarah" age={28} />
}

function UserProfile({ name, age }) {
  return (
    <div>
      <UserName name={name} />
      <UserAge age={age} />
    </div>
  )
}

function UserName({ name }) {
  return <h2>{name}</h2>
}

function UserAge({ age }) {
  return <p>Age: {age}</p>
}
```

Expected output:

```
Sarah
Age: 28
```

Here is how the data flows:

```
App
 |  passes name="Sarah" and age={28}
 v
UserProfile
 |  passes name to UserName
 |  passes age to UserAge
```

```
 v              v
UserName      UserAge
shows "Sarah" shows "Age: 28"
```

Data in React flows in one direction: from parent to child. This is called one-way data flow. It makes your app easier to understand because you always know where the data comes from.

The Children Prop

There is a special prop called `children`. It represents whatever you put between the opening and closing tags of a component.

Regular Props (Self-Closing)

```jsx
<Greeting name="Sarah" />
```

Using Children (Opening and Closing Tags)

```jsx
<Card>
  <h2>Hello!</h2>
  <p>This is inside the card.</p>
</Card>
```

Everything between `<Card>` and `</Card>` becomes the `children` prop.

How to Use Children Inside a Component

```jsx
function Card({ children }) {
  return (
    <div style={{
      border: "1px solid gray",
      padding: "16px",
      borderRadius: "8px"
    }}>
      {children}
    </div>
  )
}
```

Line 1: We destructure `children` from the props.

Line 8: We place `{children}` where we want the content to appear.

Using the Card Component

```jsx
function App() {
  return (
    <div>
      <Card>
        <h2>Welcome</h2>
        <p>This is my first card.</p>
      </Card>

      <Card>
        <h2>About Me</h2>
        <p>I am learning React!</p>
      </Card>
    </div>
  )
}
```

Expected output:

```
+---------------------------+
| Welcome                   |
| This is my first card.    |
+---------------------------+

+---------------------------+
| About Me                  |
| I am learning React!      |
+---------------------------+
```

Each `Card` shows different content, but they all have the same border and padding. The `children` prop lets you create wrapper components — components that provide a common look while accepting any content inside.

Think of `children` like a picture frame. The frame is always the same (the `Card` component with its border and padding). But you can put any picture inside it (the `children`).

Mini Project: Greeting Card Component

Let us put everything together and build a greeting card component that accepts a name and a message.

```jsx
function GreetingCard({ name, message, color = "black" }) {
  return (
    <div style={{
      border: "2px solid " + color,
      padding: "20px",
      margin: "10px",
      borderRadius: "10px"
    }}>
      <h2 style={{ color: color }}>Dear {name},</h2>
      <p>{message}</p>
      <p style={{ fontStyle: "italic" }}>With love</p>
    </div>
  )
}

function App() {
  return (
    <div>
      <h1>Greeting Cards</h1>

      <GreetingCard
        name="Mom"
        message="Thank you for everything you do!"
        color="red"
      />

      <GreetingCard
        name="Best Friend"
        message="You always make me laugh. Never change!"
        color="blue"
      />

      <GreetingCard
        name="Teacher"
        message="Thank you for your patience and wisdom."
      />
    </div>
  )
}
```

Expected output:

```
Greeting Cards

+--- red border -------------------+
| Dear Mom,                        |
| Thank you for everything you do! |
| With love                        |
+----------------------------------+
```

```
+--- blue border ------------------+
| Dear Best Friend,                |
| You always make me laugh.        |
| Never change!                    |
| With love                        |
+----------------------------------+

+--- black border (default) -------+
| Dear Teacher,                    |
| Thank you for your patience      |
| and wisdom.                      |
| With love                        |
+----------------------------------+
```

Let us break down the `GreetingCard` component:

Line 1: It accepts three props: `name`, `message`, and `color` (with a default of `"black"`).

Line 3-8: The outer `div` creates a bordered card. The border color comes from the `color` prop.

Line 9: The greeting uses the `name` prop.

Line 10: The body uses the `message` prop.

Line 11: Every card has "With love" in italics.

Notice how the third card does not pass a `color` prop. It uses the default value `"black"`.

Quick Summary

- Props are how you pass data from a parent component to a child component.
- Props are passed like HTML attributes: `<Greeting name="Sarah" />`.
- Inside the component, you read props from the function parameter.
- Destructuring `{ name, age }` is a shortcut to access props directly.
- Props are read-only. A component cannot change its own props.
- Default props provide fallback values when a prop is not passed.
- The `children` prop represents content placed between a component's opening and closing tags.
- Data flows one way in React: from parent to child.

Key Points to Remember

1. Props are like notes passed to a component. They carry information the component needs to do its job.
2. Use curly braces for non-string values. Strings get quotes. Numbers, booleans, and expressions get curly braces.
3. Destructuring makes your code cleaner. Write `{ name }` instead of `props` and then `props.name`.
4. Never change props. They are read-only. If you need something that changes, you will use state (coming in a future chapter).
5. The `children` prop is special. It lets you create wrapper components that accept any content inside them.

Practice Questions

1. What are props? Explain them using a real-life analogy.
2. What is the difference between passing `age="25"` and `age={25}`? Why does it matter?
3. Why are props read-only? What should you use instead if you need a value that can change?
4. What is destructuring? Write a component that uses destructuring to receive `title` and `author` props.
5. What is the `children` prop? When would you use it?

Exercises

Exercise 1: Create a Book Component

Create a `Book` component that accepts `title`, `author`, and `pages` as props. Use it three times in `App` to show three different books.

```jsx
function Book({ title, author, pages }) {
  return (
    <div>
```

```
      <h3>{title}</h3>
      <p>by {author}</p>
      <p>{pages} pages</p>
    </div>
  )
}

function App() {
  return (
    <div>
      <h1>My Book Shelf</h1>
      <Book title="The Great Gatsby" author="F. Scott Fitzgerald" pages={180} />
      <Book title="To Kill a Mockingbird" author="Harper Lee" pages={281} />
      <Book title="1984" author="George Orwell" pages={328} />
    </div>
  )
}
```

Try adding a fourth book of your choice.

Exercise 2: Add Default Props

Create a `Button` component that accepts a `label` and a `color` prop. Give `color` a default value of `"gray"`. Test it with and without the `color` prop.

```jsx
function Button({ label, color = "gray" }) {
  return (
    <button style={{ backgroundColor: color, padding: "10px 20px" }}>
      {label}
    </button>
  )
}

function App() {
  return (
    <div>
      <Button label="Submit" color="green" />
      <Button label="Cancel" color="red" />
      <Button label="Info" />
    </div>
  )
}
```

What color does the "Info" button have?

Exercise 3: Use the Children Prop

Create a `Container` component that wraps its children in a styled box. Then use it to wrap different content.

```jsx
function Container({ title, children }) {
  return (
    <div style={{
      border: "2px solid navy",
      padding: "15px",
      margin: "10px"
    }}>
      <h2>{title}</h2>
      {children}
    </div>
  )
}

function App() {
  return (
    <div>
      <Container title="Section 1">
        <p>This is the first section.</p>
        <p>It has two paragraphs.</p>
      </Container>

      <Container title="Section 2">
        <p>This section has different content.</p>
        <ul>
          <li>Item one</li>
          <li>Item two</li>
        </ul>
      </Container>
    </div>
  )
}
```

Notice how each `Container` has the same style but different content inside it. That is the `children` prop at work.

What Is Next?

You now know how to create components and pass data to them with props. Your components can show different content depending on what information they receive. This is a huge step!

But what if you want your component to remember things and change over time? What if you want a counter that goes up when you click a button? What if you want to show or hide something? Props cannot do this because props are read-only.

For that, you need State — the component's own personal memory. In the next chapter, you will learn how state works and how to make your components interactive and dynamic. Get ready for things to come alive!

Chapter 9: State — Remembering Things

What You Will Learn

- What "state" means in React
- How state is different from props
- How to use the `useState` tool to give your component a memory
- How to build a simple counter that goes up when you click a button
- Why React updates the screen automatically when state changes
- Why regular variables do not work for remembering things in React

Why This Chapter Matters

So far, your components have been like printed posters. They show information, but they cannot change. A poster on the wall always says the same thing. You cannot tap it and watch the text update.

But real apps change all the time. You click a heart button and the like count goes up. You type in a search box and results appear. You press "Add to Cart" and the cart number increases.

To make any of this happen, your components need a memory — a way to remember things and update the screen when those things change. In React, this memory is called state.

What Is State?

State is a component's personal memory. It is information that the component keeps track of and can change over time.

Think of state like a whiteboard in a classroom.

```
+---------------------------+
|                           |
|   Score: 5                |  <-- You can erase and rewrite this
|                           |
+---------------------------+
        WHITEBOARD
```

- A teacher writes "Score: 5" on the whiteboard.
- A student answers a question correctly.
- The teacher erases "5" and writes "6".
- Everyone in the room sees the new score instantly.

State works the same way in React:

1. Your component has a piece of information (like a score).
2. Something happens (like a button click).
3. The information changes (score goes from 5 to 6).
4. React automatically updates the screen to show the new information.

The key word here is automatically. You do not have to tell React to update the screen. When state changes, React does it for you.

Props vs State

You already know about props from earlier chapters. Let us compare props and state so you can see how they are different.

Props are like a gift someone gives you. You receive them, but you cannot change them. If someone gives you a red shirt, you cannot magically turn it into a blue shirt.

State is like your own notebook. You own it. You can write in it, erase things, and change whatever you want.

Here is a simple comparison:

```
+------------------+-----------------------------------+
|                  |  Props            |  State         |
+------------------+-----------------------------------+
| Who controls it? |  The parent       |  The component |
|                  |  component        |  itself        |
+------------------+-------------------+----------------+
| Can it change?   |  No (read-only)   |  Yes           |
+------------------+-------------------+----------------+
| Where does it    |  Passed in from   |  Created       |
| come from?       |  outside          |  inside        |
+------------------+-------------------+----------------+
```

Think of it this way:

- Props = information given TO you (you cannot change it)
- State = information YOU control (you can change it)

A component can have both props and state at the same time. For example, a `LikeButton` component might receive a `username` prop (who posted the content) and have a `likeCount` state (how many likes it has).

Introduction to useState

To give a component state, React provides a special tool called `useState`.

The word `useState` literally means "use state." It is a hook — a special function that React provides so your components can do extra things. We will learn more about hooks in Chapter 14. For now, just think of `useState` as a tool that gives your component a memory.

Here is how you use it:

```jsx
const [count, setCount] = useState(0);
```

This one line does a lot. Let us break it down piece by piece.

Breaking Down useState

```
const [count, setCount] = useState(0);
       -----  --------           -
         |       |                |
         |       |                +-- Starting value (begin at 0)
```

```
          |       |
          |       +-- Function to UPDATE the value
          |
          +-- The CURRENT value
```

- `count` — This is the current value. It starts at `0` because we wrote `useState(0)`. You can read this value anytime.
- `setCount` — This is a special function that lets you change the value. When you call `setCount(5)`, the count becomes `5` and React updates the screen.
- `useState(0)` — The `0` inside the parentheses is the starting value (also called the initial value). This is what `count` will be when the component first appears on the screen.

The square brackets `[ ]` around `count, setCount` are called array destructuring. That is a fancy term. It just means "take the two things that `useState` gives back and put them into two separate variables." Do not worry about this term too much right now.

Naming Convention

You can name the two parts anything you want, but there is a common pattern:

- The first name describes what the value is: `count`, `name`, `isOpen`
- The second name starts with `set` followed by the same word: `setCount`, `setName`, `setIsOpen`

This pattern makes your code easy to read. When you see `setCount`, you immediately know it changes the `count`.

A Simple Counter

Let us build a counter. When you click a button, the number goes up by one.

```jsx
import { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0);

  function handleClick() {
    setCount(count + 1);
  }

  return (
```

```
    <div>
      <p>You clicked {count} times</p>
      <button onClick={handleClick}>Click me</button>
    </div>
  );
}

export default Counter;
```

Expected Output

When the page first loads:

```
You clicked 0 times
[ Click me ]
```

After clicking the button three times:

```
You clicked 3 times
[ Click me ]
```

Line-by-Line Explanation

Let us walk through every line of this code.

Line 1: `import { useState } from 'react';` We import the `useState` tool from React. We need this to give our component a memory. The curly braces `{ }` are needed because `useState` is a named export from the React library.

Line 3: `function Counter() {` We create a component called `Counter`. Remember, components are just functions that return what should appear on the screen.

Line 4: `const [count, setCount] = useState(0);` We create a piece of state. `count` starts at `0`. We get `setCount` to change it later.

Line 6: `function handleClick() {` We create a regular function called `handleClick`. This function describes what should happen when the button is clicked.

Line 7: `setCount(count + 1);` When called, this takes the current count, adds 1 to it, and sets that as the new count. If `count` is `3`, then `count + 1` is `4`, so the count becomes `4`.

Line 10-15: The `return` block. This is what appears on the screen:

- A paragraph showing `You clicked {count} times`. The `{count}` part will be replaced with the actual number.
- A button that calls `handleClick` when clicked.

Line 18: `export default Counter;` We export the component so other files can use it.

How React Re-Renders When State Changes

When you call `setCount`, something magical happens behind the scenes. Let us follow the steps:

```
Step 1: You click the button
         |
         v
Step 2: handleClick() runs
         |
         v
Step 3: setCount(count + 1) is called
         |
         v
Step 4: React sees the state changed
         |
         v
Step 5: React calls the Counter function again (re-render)
         |
         v
Step 6: The new count value is used
         |
         v
Step 7: The screen updates with the new number
```

This process is called a re-render. The word "render" means "to draw on the screen." A re-render means React draws the component on the screen again with the new information.

The important thing to understand is: you do not update the screen yourself. You just change the state, and React takes care of updating the screen. This is one of the biggest reasons React is so popular. It handles all the screen-updating work for you.

Why Regular Variables Do Not Work

You might wonder: "Why do I need `useState`? Why not just use a regular variable?"

Great question. Let us try it and see what happens.

```jsx
function BrokenCounter() {
  let count = 0;  // regular variable

  function handleClick() {
    count = count + 1;
    console.log(count);  // This WILL show the new number in the console
  }

  return (
    <div>
      <p>You clicked {count} times</p>
      <button onClick={handleClick}>Click me</button>
    </div>
  );
}
```

What Happens

If you click the button, the screen will always say "You clicked 0 times." It never changes. But if you look at the console (the developer tool in your browser), you will see the number going up: 1, 2, 3...

Why?

Two reasons:

1. Regular variables do not trigger a re-render. When you change a regular variable, React does not know anything changed. It does not update the screen. The `useState` function is special because it tells React: "Hey, something changed! Please update the screen."
2. Regular variables reset every render. Even if React did re-render, the line `let count = 0` runs every time the component function runs. So `count` would go back to `0` on every render.

Think of it this way:

```
Regular variable = Writing on a piece of paper, then throwing
                   the paper away and starting with a new one
                   every time.

State variable   = Writing on a whiteboard. It stays there
                   until you erase it and write something new.
```

This is why we need `useState`. It is the only way to:

- Tell React to update the screen
- Keep the value between renders

More State Examples

Example: Toggle a Message

State does not have to be a number. It can be a boolean (true or false), a string (text), or many other things.

Here is an example that shows or hides a message:

```jsx
import { useState } from 'react';

function ToggleMessage() {
  const [isVisible, setIsVisible] = useState(false);

  function handleClick() {
    setIsVisible(!isVisible);
  }

  return (
    <div>
      <button onClick={handleClick}>
        {isVisible ? 'Hide' : 'Show'} Message
      </button>
      {isVisible && <p>Hello! Now you can see me!</p>}
    </div>
  );
}
```

What is happening here:

- `isVisible` starts as `false` (the message is hidden).
- When you click the button, `!isVisible` flips the value. If it was `false`, it becomes `true`. If it was `true`, it becomes `false`. The `!` symbol means "the opposite of."
- When `isVisible` is `true`, the paragraph appears. When it is `false`, it disappears.
- The button text also changes: it says "Show Message" when hidden and "Hide Message" when visible.

Do not worry about the `? :` and `&&` syntax too much right now. We will cover those in Chapter 11 on conditional rendering.

Example: Name Display

Here is an example with a text string as state:

```jsx
import { useState } from 'react';

function NameDisplay() {
  const [name, setName] = useState('World');

  function changeName() {
    setName('React Learner');
  }

  return (
    <div>
      <p>Hello, {name}!</p>
      <button onClick={changeName}>Change Name</button>
    </div>
  );
}
```

Expected output when the page loads:

```
Hello, World!
[ Change Name ]
```

After clicking the button:

```
Hello, React Learner!
[ Change Name ]
```

Mini Project: Like Button Counter

Let us build a simple like button, similar to what you see on social media apps. When you click the heart, the like count goes up.

```jsx
import { useState } from 'react';

function LikeButton() {
  const [likes, setLikes] = useState(0);
```

```
  function handleLike() {
    setLikes(likes + 1);
  }

  return (
    <div>
      <button onClick={handleLike}>
        ♥ Like
      </button>
      <p>{likes} {likes === 1 ? 'like' : 'likes'}</p>
    </div>
  );
}

export default LikeButton;
```

Expected Output

When the page loads:

```
[ ♥ Like ]
0 likes
```

After clicking three times:

```
[ ♥ Like ]
3 likes
```

After clicking once:

```
[ ♥ Like ]
1 like     <-- Notice: "like" not "likes" (singular)
```

Line-by-Line Explanation

Line 1: `import { useState } from 'react';` Import the `useState` tool from React.

Line 3: `function LikeButton() {` Create a component called `LikeButton`.

Line 4: `const [likes, setLikes] = useState(0);` Create state called `likes`, starting at `0`. The function `setLikes` will let us change the number.

Line 6-8: `function handleLike() { setLikes(likes + 1); }` This function adds 1 to the current like count.

Line 12: `<button onClick={handleLike}>` When the button is clicked, run the `handleLike` function.

Line 13: `♥ Like` The text on the button. The ♥ is a heart symbol.

Line 15: `<p>{likes} {likes === 1 ? 'like' : 'likes'}</p>` Show the number of likes. The `likes === 1 ? 'like' : 'likes'` part checks: "Is the number exactly 1?" If yes, show "like" (singular). If no, show "likes" (plural). The `===` symbol means "is exactly equal to."

Quick Summary

- State is a component's memory. It stores information that can change.
- Props are given to a component from outside. State is created and controlled inside the component.
- `useState` is a React tool (hook) that gives your component state.
- `useState` returns two things: the current value and a function to change it.
- When you call the set function (like `setCount`), React re-renders the component — it updates the screen automatically.
- Regular variables do not work because they do not trigger re-renders and they reset on every render.

Key Points to Remember

1. Always import `useState` from React: `import { useState } from 'react';`
2. The starting value goes inside the parentheses: `useState(0)`, `useState('')`, `useState(false)`
3. Never change state directly. Always use the set function. Write `setCount(5)`, not `count = 5`.
4. State can hold numbers, strings, booleans, arrays, objects — almost anything.
5. Each component has its own separate state. If you put two `Counter` components on the page, each one has its own count.
6. When state changes, React automatically updates what you see on the screen.

Practice Questions

1. In your own words, what is the difference between props and state?
2. Look at this code. What is the starting value of `temperature`?

```jsx
const [temperature, setTemperature] = useState(72);
```

3. Why does this code NOT update the screen when you click the button?

```jsx
function Counter() {
  let count = 0;
  function handleClick() {
    count = count + 1;
  }
  return <button onClick={handleClick}>{count}</button>;
}
```

4. What will the screen show after clicking the button twice in the counter example from this chapter?
5. What does "re-render" mean?

Practice Exercises

Exercise 1: Score Keeper

Build a component called `ScoreKeeper` that:

- Shows "Score: 0" when the page loads
- Has a button that says "Score!"
- Each click adds 1 to the score

Hint: This is very similar to the counter example. Just change the variable names and the text.

Exercise 2: Mood Changer

Build a component called `MoodChanger` that:

- Shows a mood emoji (start with "☐")

- Has a button that says "Change Mood"
- When clicked, the emoji changes to "□"

Hint: Use `useState` with a string value.

Exercise 3: Step Counter

Build a component called `StepCounter` that:

- Shows "Steps today: 0"
- Has a button that says "Take a Step"
- Each click adds 1 to the step count
- Also shows a message: if steps are 0, show "Let's get moving!" If steps are more than 0, show "Keep going!"

Hint: You can use the same `count` state and check if it is 0 or not.

What Is Next?

Now your components have a memory. They can remember things and change. But right now, the only way to change state is with a button click. In the next chapter, Chapter 10: Events — Making Things Happen, you will learn about all the different ways users can interact with your app — clicking, typing, hovering, submitting forms, and more. Events are what make your app come alive!

Chapter 10: Events — Making Things Happen

What You Will Learn

- What events are and why they matter
- How to use `onClick` to respond to button clicks
- How to write event handler functions
- Why we write `onClick={handleClick}` and NOT `onClick={handleClick()}`
- How to pass information with events
- How to use `onChange` for input fields
- How to use `onSubmit` for forms
- The most common events in React

Why This Chapter Matters

In the last chapter, you learned how to give your components memory with state. But memory alone is not enough. You need a way to trigger changes. You need something to happen.

Think about your phone. It has lots of information stored inside it. But nothing happens until you do something — you tap the screen, you swipe, you type. These actions are called events.

Events are the bridge between the user and your app. Without events, your app would just sit there, doing nothing. Events are what make your app interactive and alive.

What Are Events?

An event is something that happens. That is it. It is really that simple.

In everyday life, events happen all the time:

- You knock on a door (event: knock)
- The door opens (response to the event)

In a web app:

- A user clicks a button (event: click)
- The app shows a message (response to the event)

Here are some common events in web apps:

```
+------------------+-----------------------------------+
| Event            | When It Happens                   |
+------------------+-----------------------------------+
| Click            | User clicks on something          |
| Change           | User types in an input field      |
| Submit           | User submits a form               |
| Mouse Enter      | User moves the mouse over something|
| Key Down         | User presses a key on the keyboard |
+------------------+-----------------------------------+
```

In React, we respond to events by attaching event handlers to our elements. An event handler is just a function that runs when the event happens.

---

onClick — The Most Common Event

The most common event is the click. You already used it in the last chapter. Let us look at it more closely.

```jsx
function ClickExample() {
  function handleClick() {
    alert('You clicked the button!');
  }

  return (
    <button onClick={handleClick}>Click me</button>
  );
}
```

Expected Output

When you click the button, a pop-up box appears that says: "You clicked the button!"

Line-by-Line Explanation

Line 2-4: `function handleClick() { alert('You clicked the button!'); }` We create a function called `handleClick`. This is the event handler. It contains the code that should run when the button is clicked. The `alert()` function shows a pop-up message in the browser.

Line 7: `<button onClick={handleClick}>Click me</button>` We attach the `handleClick` function to the button using `onClick`. Notice:

- `onClick` starts with a lowercase `on` and then a capital `C` for `Click`. This style is called camelCase (like a camel's humps: small, big, small, big).
- We put the function name inside curly braces `{ }`.
- We write `handleClick` — just the name, no parentheses after it.

Writing Event Handler Functions

An event handler is a function that runs when an event happens. You have already seen one: `handleClick`. There is a common pattern for naming them:

```
handle + EventName = handler function name

handleClick    → runs when something is clicked
handleChange   → runs when an input value changes
handleSubmit   → runs when a form is submitted
```

You can define event handlers in two common ways.

Way 1: A Named Function (Before the Return)

```jsx
function Greeting() {
  function handleClick() {
    alert('Hello there!');
  }

  return <button onClick={handleClick}>Say Hello</button>;
}
```

Way 2: An Arrow Function (Inline, Inside the JSX)

```jsx
function Greeting() {
  return (
    <button onClick={() => alert('Hello there!')}>
      Say Hello
    </button>
  );
}
```

Both ways work exactly the same. Way 1 is easier to read when the handler does many things. Way 2 is nice for short, simple actions.

The `() =>` is called an arrow function. It is a short way to write a function. Think of the arrow `=>` as saying "do this." So `() => alert('Hello!')` means "do this: show an alert that says Hello."

Why We Write onClick={handleClick} and NOT onClick={handleClick()}

This is a very common mistake for beginners. Let us understand why it matters.

```jsx
// CORRECT - passes the function (React will call it later)
<button onClick={handleClick}>Click me</button>

// WRONG - calls the function immediately when the page loads!
<button onClick={handleClick()}>Click me</button>
```

See the difference? The second version has `()` after `handleClick`. Those parentheses tell JavaScript: "Run this function right now!"

Here is an analogy:

```
CORRECT: onClick={handleClick}
→ "Hey React, here is the function. Call it when someone clicks."
→ Like giving someone your phone number: "Call me when you need me."

WRONG: onClick={handleClick()}
→ "Run this function RIGHT NOW, even though nobody clicked anything."
→ Like calling someone before they even ask you to.
```

With the wrong version:

- The function runs immediately when the page loads.
- The button click does nothing.
- Your app might behave in unexpected ways.

Remember: Give React the function. Do not call the function yourself.

```
handleClick   → "Here is the function" (CORRECT)
handleClick() → "Run the function NOW" (WRONG)
```

When You NEED Parentheses

Sometimes you want to pass information to your handler function. In that case, you wrap it in an arrow function:

```jsx
// Passing information — this is correct!
<button onClick={() => handleClick('hello')}>Click me</button>
```

This creates a small function that says: "When clicked, call `handleClick` with the word 'hello'." The outer arrow function `() =>` waits for the click. The inner `handleClick('hello')` runs only when that click happens.

Passing Information with Events

Sometimes you need your event handler to know some extra information. For example, which button was clicked?

```jsx
import { useState } from 'react';

function FruitPicker() {
  const [fruit, setFruit] = useState('none');

  function handlePick(fruitName) {
    setFruit(fruitName);
  }

  return (
    <div>
      <p>You picked: {fruit}</p>
      <button onClick={() => handlePick('Apple')}>Apple</button>
```

```
      <button onClick={() => handlePick('Banana')}>Banana</button>
      <button onClick={() => handlePick('Cherry')}>Cherry</button>
    </div>
  );
}
```

Expected Output

When the page loads:

```
You picked: none
[ Apple ] [ Banana ] [ Cherry ]
```

After clicking "Banana":

```
You picked: Banana
[ Apple ] [ Banana ] [ Cherry ]
```

How It Works

Each button uses an arrow function: `() => handlePick('Apple')`. When you click the Apple button:

1. The arrow function runs.
2. It calls `handlePick('Apple')`.
3. `handlePick` receives `'Apple'` as the `fruitName`.
4. `setFruit('Apple')` updates the state.
5. React re-renders the component.
6. The screen now shows "You picked: Apple."

---

onChange for Input Fields

When a user types in an input field, the `onChange` event fires with every keystroke. This is how you capture what the user is typing.

```jsx
import { useState } from 'react';

function NameInput() {
  const [name, setName] = useState('');

  function handleChange(event) {
    setName(event.target.value);
  }

  return (
    <div>
      <input
        type="text"
        onChange={handleChange}
        placeholder="Type your name"
      />
      <p>Hello, {name}!</p>
    </div>
  );
}
```

Expected Output

When the page loads:

```
[Type your name          ]
Hello, !
```

As you type "Sam":

```
[Sam                     ]
Hello, Sam!
```

Line-by-Line Explanation

Line 4: `const [name, setName] = useState('');` We create state called `name` with an empty string `''` as the starting value.

Line 6: `function handleChange(event) {` Our handler function receives an event object. When React calls your event handler, it automatically passes this object. It contains information about what happened.

Line 7: `setName(event.target.value);` This line has three parts:

- `event` — the event object that React gives us

- `event.target` — the element where the event happened (the input field)
- `event.target.value` — the current text inside the input field

So if the user types "Sam", `event.target.value` is `"Sam"`. We use that to update our state.

Line 12-15: The `<input>` element.

- `type="text"` — this is a text input field.
- `onChange={handleChange}` — every time the user types a character, call `handleChange`.
- `placeholder="Type your name"` — gray hint text that appears when the field is empty.

The Event Object

When any event happens, React creates an event object and passes it to your handler function. This object has useful information. You can name it anything, but most people call it `event` or `e` for short.

```
event object
├── event.target       → the element that triggered the event
├── event.target.value → the current value (for inputs)
├── event.type         → what kind of event ("click", "change", etc.)
└── (many more properties)
```

You do not need to memorize all the properties. Just remember `event.target.value` for inputs — you will use it a lot.

---

onSubmit for Forms

When a user fills out a form and clicks a submit button, the `onSubmit` event fires. Here is an example:

```jsx
import { useState } from 'react';

function SimpleForm() {
  const [name, setName] = useState('');
  const [submitted, setSubmitted] = useState(false);

  function handleSubmit(event) {
    event.preventDefault();
    setSubmitted(true);
  }
```

```
  function handleChange(event) {
    setName(event.target.value);
  }

  return (
    <div>
      {submitted ? (
        <p>Thanks, {name}! Form submitted.</p>
      ) : (
        <form onSubmit={handleSubmit}>
          <input
            type="text"
            onChange={handleChange}
            placeholder="Enter your name"
          />
          <button type="submit">Submit</button>
        </form>
      )}
    </div>
  );
}
```

Expected Output

When the page loads:

```
[Enter your name        ] [ Submit ]
```

After typing "Alex" and clicking Submit:

```
Thanks, Alex! Form submitted.
```

Important: event.preventDefault()

Line 8: `event.preventDefault();`

This line is very important. By default, when you submit a form in a web browser, the browser refreshes the entire page. This is old behavior from the early days of the internet. In React, we do not want the page to refresh — we want React to handle everything.

`event.preventDefault()` means: "Stop the browser from doing what it normally does." It prevents the default behavior (page refresh).

Think of it like this: the browser says, "A form was submitted! I should refresh the page!" And `preventDefault` says, "No, do not do that. We will handle it ourselves."

You will almost always use `event.preventDefault()` inside `onSubmit` handlers.

Common Events List

Here are the events you will use most often in React:

```
+--------------------+----------------------------+---------------------------+
| React Event        | When It Fires              | Common Use                |
+--------------------+----------------------------+---------------------------+
| onClick            | User clicks an element     | Buttons, links, cards     |
| onChange           | Input value changes        | Text inputs, checkboxes   |
| onSubmit           | Form is submitted          | Login forms, search bars  |
| onMouseEnter       | Mouse moves over element   | Showing tooltips          |
| onMouseLeave       | Mouse leaves element       | Hiding tooltips           |
| onKeyDown          | User presses a key         | Keyboard shortcuts        |
| onFocus            | Input field is selected    | Showing hints             |
| onBlur             | Input field loses focus    | Validation                |
+--------------------+----------------------------+---------------------------+
```

You do not need to memorize all of these. You will learn them as you need them. The three most important ones are:

1. `onClick` — for buttons and clickable things
2. `onChange` — for input fields
3. `onSubmit` — for forms

Mini Examples

Example 1: Button That Shows an Alert

```jsx
function AlertButton() {
  function handleClick() {
    alert('Button was clicked!');
  }
```

```
  return <button onClick={handleClick}>Show Alert</button>;
}
```

This is the simplest possible event example. Click the button, see a pop-up.

Example 2: Button That Changes Text

```jsx
import { useState } from 'react';

function TextChanger() {
  const [message, setMessage] = useState('Hello!');

  function handleClick() {
    setMessage('You changed the text!');
  }

  return (
    <div>
      <p>{message}</p>
      <button onClick={handleClick}>Change Text</button>
    </div>
  );
}
```

Before clicking:

```
Hello!
[ Change Text ]
```

After clicking:

```
You changed the text!
[ Change Text ]
```

Example 3: Input That Shows What You Type

```jsx
import { useState } from 'react';

function LiveTyping() {
  const [text, setText] = useState('');
```

```
  return (
    <div>
      <input
        type="text"
        onChange={(e) => setText(e.target.value)}
        placeholder="Type something..."
      />
      <p>You typed: {text}</p>
    </div>
  );
}
```

As you type "React is fun":

```
[React is fun            ]
You typed: React is fun
```

Notice that this example uses an inline arrow function: `(e) => setText(e.target.value)`. The `e` is short for `event`. This is a shorter way to write the handler when it only does one thing.

Quick Summary

- Events are things that happen — clicks, typing, form submissions, mouse movements.
- You respond to events by writing event handler functions.
- Use `onClick` for button clicks, `onChange` for input fields, and `onSubmit` for forms.
- Write `onClick={handleClick}` (no parentheses). Do not write `onClick={handleClick()}`.
- Use arrow functions when you need to pass information: `onClick={() => handleClick('data')}`.
- Event handlers receive an event object with information about what happened.
- Use `event.target.value` to get the current value from an input field.
- Use `event.preventDefault()` in form submissions to stop the page from refreshing.

Key Points to Remember

1. Event names in React use camelCase: `onClick`, `onChange`, `onSubmit` (not `onclick` or `on-click`).

2. Pass the function, do not call it: `onClick={handleClick}` not `onClick={handleClick()}`.
3. The event object is automatically passed to your handler function by React.
4. `event.target.value` gives you the text inside an input field.
5. Always use `event.preventDefault()` inside `onSubmit` handlers.
6. You can use named functions (defined before the return) or inline arrow functions — both work.

Practice Questions

1. What is an event? Give two examples from everyday life and two from web apps.
2. What is wrong with this code?

```jsx
<button onClick={handleClick()}>Click me</button>
```

3. How do you get the text that a user typed into an input field inside your event handler?
4. Why do we call `event.preventDefault()` in form submit handlers?
5. What is the difference between `onClick={handleClick}` and `onClick={() => handleClick('hello')}`?

Practice Exercises

Exercise 1: Compliment Button

Build a component called `ComplimentButton` that:

- Has a button that says "Get a Compliment"
- When clicked, shows a nice message below the button (like "You are amazing!")
- Use `useState` to store the message

Exercise 2: Color Picker

Build a component called `ColorPicker` that:

- Shows the text "Current color: none"
- Has three buttons: "Red", "Green", "Blue"

- When you click a button, the text updates to show that color (for example, "Current color: Red")
- Hint: This is similar to the FruitPicker example

Exercise 3: Character Counter

Build a component called `CharacterCounter` that:

- Has an input field where you can type text
- Below the input, shows "Characters: 0"
- As you type, the character count updates in real time
- Hint: A string has a `.length` property. If `text` is `"hello"`, then `text.length` is `5`.

What Is Next?

You now know how to make things happen in your app. Users can click buttons, type in fields, and submit forms. But what if you want to show different things depending on a condition? What if you want to show a "Welcome" message to logged-in users and a "Please log in" message to everyone else? In the next chapter, Chapter 11: Conditional Rendering, you will learn how to show different content based on conditions — like a traffic light that shows different colors.

Chapter 11: Conditional Rendering — Showing Different Things

What You Will Learn

- What conditional rendering means
- How to use `if/else` outside JSX to pick what to show
- How to use the ternary operator `? :` inside JSX
- How to use `&&` to show something only when a condition is true
- How to show and hide components
- How to build a Login/Logout button
- How to display loading messages while waiting for data

Why This Chapter Matters

Imagine a store with a sign on the door. When the store is open, the sign says "OPEN." When the store is closed, it says "CLOSED." The sign changes based on a condition (is the store open or not?).

```
+------------------+        +------------------+
|                  |        |                  |
|      OPEN        |        |     CLOSED       |
|                  |        |                  |
+------------------+        +------------------+
   Store is open               Store is closed
```

Your React components need to do the same thing. Sometimes you want to show one thing, and other times you want to show something else. A "Log In" button should appear when the user is not logged in. A "Log Out" button should appear when they are logged in. A loading message should appear while data is being fetched.

This is called conditional rendering — showing different things based on a condition. It is one of the most useful skills in React.

What Is Conditional Rendering?

Let us break down the term:

- Conditional means "based on a condition." A condition is something that is either true or false. "Is it raining?" is a condition. The answer is either yes (true) or no (false).
- Rendering means "showing on the screen."

So conditional rendering means: showing different things on the screen based on whether something is true or false.

You use conditional rendering all the time in real life:

- If it is raining, bring an umbrella. Otherwise, leave it at home.
- If you have a ticket, enter the movie theater. Otherwise, buy a ticket first.
- If the battery is low, show a warning. Otherwise, show the battery percentage.

React gives you three main ways to do conditional rendering:

1. `if/else` statements (outside JSX)
2. The ternary operator `? :` (inside JSX)
3. The `&&` operator (inside JSX)

Let us learn each one.

Method 1: Using if/else Outside JSX

The simplest way to conditionally render is to use a regular `if/else` statement before the `return`.

```jsx
function Greeting({ isLoggedIn }) {
  if (isLoggedIn) {
    return <h1>Welcome back!</h1>;
  } else {
    return <h1>Please log in.</h1>;
```

```
  }
}
```

Expected Output

If `isLoggedIn` is `true`:

```
Welcome back!
```

If `isLoggedIn` is `false`:

```
Please log in.
```

Line-by-Line Explanation

Line 1: `function Greeting({ isLoggedIn }) {` The component receives a prop called `isLoggedIn`. This prop is a boolean — it is either `true` or `false`.

Line 2: `if (isLoggedIn) {` We check the condition. If `isLoggedIn` is `true`, we go into this block.

Line 3: `return <h1>Welcome back!</h1>;` If logged in, return a heading that says "Welcome back!" The `return` statement ends the function here. Nothing below this line runs.

Line 4-5: `} else {` If `isLoggedIn` is `false`, we go into the `else` block.

Line 5: `return <h1>Please log in.</h1>;` If not logged in, return a heading that says "Please log in."

When to Use if/else

Use `if/else` when:

- The two options look very different (completely different layouts)
- You need to do some extra work before returning (like calculations)
- You want to return early from a component

Method 2: The Ternary Operator Inside JSX

The ternary operator is a shortcut for `if/else` that you can use right inside your JSX. It has three parts (that is why it is called "ternary" — it means "three parts"):

```
condition ? showIfTrue : showIfFalse
```

Read it like this: "Is the condition true? Then show this. Otherwise, show that."

Here is an example:

```jsx
function Greeting({ isLoggedIn }) {
  return (
    <h1>
      {isLoggedIn ? 'Welcome back!' : 'Please log in.'}
    </h1>
  );
}
```

This does exactly the same thing as the `if/else` example above, but in fewer lines.

How to Read the Ternary Operator

```
{isLoggedIn ? 'Welcome back!' : 'Please log in.'}
 _________   ________________   ________________
     |              |                  |
 condition     if TRUE            if FALSE
               show this          show this
```

The `?` means "then" and the `:` means "otherwise."

More Ternary Examples

Showing different buttons:

```jsx
function ActionButton({ isSaved }) {
  return (
    <button>
      {isSaved ? 'Edit' : 'Save'}
    </button>
  );
}
```

If `isSaved` is true, the button says "Edit." If false, it says "Save."

Showing different components:

```jsx
function StatusMessage({ isOnline }) {
  return (
    <div>
      {isOnline ? (
        <p style={{ color: 'green' }}>User is online</p>
      ) : (
        <p style={{ color: 'gray' }}>User is offline</p>
      )}
    </div>
  );
}
```

When the parts after `?` and `:` are longer, put them on separate lines and wrap them in parentheses `( )` for readability.

When to Use the Ternary Operator

Use the ternary operator when:

- You want to choose between two options inside JSX
- Both options are short and simple
- You want compact, readable code

Method 3: Using && for "Show Only If True"

Sometimes you do not have an "otherwise." You just want to show something when a condition is true, and show nothing when it is false.

The `&&` operator (called "and") is perfect for this:

```jsx
function Notification({ hasMessages }) {
  return (
    <div>
      <h1>Dashboard</h1>
      {hasMessages && <p>You have new messages!</p>}
    </div>
  );
}
```

Expected Output

If `hasMessages` is `true`:

```
Dashboard
You have new messages!
```

If `hasMessages` is `false`:

```
Dashboard
```

The paragraph simply does not appear. No error, no blank space — it is just not there.

How && Works

Think of `&&` like a bouncer at a club door:

```
{hasMessages && <p>You have new messages!</p>}

→ "Is hasMessages true? If yes, let the paragraph through.
   If no, show nothing."
```

The rule is simple:

- If the left side is true, show what is on the right side.
- If the left side is false, show nothing.

More && Examples

Show a badge only if there are items:

```jsx
function CartIcon({ itemCount }) {
  return (
    <div>
      Cart
      {itemCount > 0 && <span> ({itemCount})</span>}
    </div>
  );
}
```

If `itemCount` is `3`, it shows: `Cart (3)` If `itemCount` is `0`, it shows: `Cart`

Show a warning only if needed:

```jsx
function PasswordField({ password }) {
  return (
    <div>
      <input type="password" />
      {password.length < 8 && (
        <p style={{ color: 'red' }}>
          Password must be at least 8 characters
        </p>
      )}
    </div>
  );
}
```

A Common Trap with &&

Be careful with numbers. Look at this code:

```jsx
// CAREFUL with this!
{count && <p>You have {count} items</p>}
```

If `count` is `0`, you might expect nothing to appear. But instead, React will show the number `0` on the screen. This is because `0` is a "falsy" value in JavaScript, and React renders `0` as text.

The fix is to use a comparison that gives a true boolean:

```jsx
// BETTER – use a comparison
{count > 0 && <p>You have {count} items</p>}
```

Now React checks "Is count greater than 0?" The answer is `true` or `false` — and `false` shows nothing.

When to Use &&

Use `&&` when:

- You want to show something OR show nothing
- There is no "else" case
- The condition is simple

Showing and Hiding Components

A very common pattern is showing and hiding parts of your app. Let us build a component that shows and hides a message when you click a button.

```jsx
import { useState } from 'react';

function ToggleDetails() {
  const [showDetails, setShowDetails] = useState(false);

  function handleClick() {
    setShowDetails(!showDetails);
  }

  return (
    <div>
      <h2>Product</h2>
      <p>A wonderful widget.</p>
      <button onClick={handleClick}>
        {showDetails ? 'Hide Details' : 'Show Details'}
      </button>
      {showDetails && (
        <div>
          <p>Weight: 2kg</p>
          <p>Color: Blue</p>
          <p>Material: Aluminum</p>
        </div>
      )}
    </div>
  );
}
```

Expected Output

When the page loads:

```
Product
A wonderful widget.
[ Show Details ]
```

After clicking "Show Details":

```
Product
A wonderful widget.
[ Hide Details ]
Weight: 2kg
Color: Blue
Material: Aluminum
```

After clicking "Hide Details":

```
Product
A wonderful widget.
[ Show Details ]
```

How It Works

- `showDetails` starts as `false`.
- The button text uses a ternary: if `showDetails` is true, say "Hide Details." Otherwise, say "Show Details."
- The details section uses `&&`: only show the details div when `showDetails` is true.
- `!showDetails` flips the value. If it is `false`, it becomes `true`. If it is `true`, it becomes `false`. The `!` symbol means "not" or "the opposite."

Login/Logout Button Example

Let us build a practical example: a button that switches between "Log In" and "Log Out."

```jsx
import { useState } from 'react';

function LoginControl() {
  const [isLoggedIn, setIsLoggedIn] = useState(false);

  function handleLogin() {
    setIsLoggedIn(true);
  }

  function handleLogout() {
    setIsLoggedIn(false);
  }

  return (
    <div>
      {isLoggedIn ? (
        <div>
          <p>Welcome! You are logged in.</p>
          <button onClick={handleLogout}>Log Out</button>
        </div>
      ) : (
        <div>
          <p>You are not logged in.</p>
          <button onClick={handleLogin}>Log In</button>
        </div>
```

```
      )}
    </div>
  );
}
```

Expected Output

When the page loads (not logged in):

```
You are not logged in.
[ Log In ]
```

After clicking "Log In":

```
Welcome! You are logged in.
[ Log Out ]
```

After clicking "Log Out":

```
You are not logged in.
[ Log In ]
```

What Is Happening

The ternary operator checks `isLoggedIn`:

- If `true`: show the welcome message and a Log Out button.
- If `false`: show the "not logged in" message and a Log In button.

The Log In button sets `isLoggedIn` to `true`. The Log Out button sets it to `false`. Each change triggers a re-render, and the correct content appears.

Displaying Loading Messages

When your app waits for data (like loading information from the internet), you should show a loading message so the user knows something is happening.

```jsx
import { useState } from 'react';

function DataLoader() {
  const [isLoading, setIsLoading] = useState(false);
  const [data, setData] = useState(null);

  function handleLoad() {
    setIsLoading(true);

    // Simulate waiting 2 seconds for data to load
    setTimeout(() => {
      setData('Here is your data!');
      setIsLoading(false);
    }, 2000);
  }

  if (isLoading) {
    return <p>Loading... Please wait.</p>;
  }

  if (data) {
    return <p>{data}</p>;
  }

  return <button onClick={handleLoad}>Load Data</button>;
}
```

Expected Output

When the page loads:

```
[ Load Data ]
```

After clicking (for 2 seconds):

```
Loading... Please wait.
```

After the data loads:

```
Here is your data!
```

What Is Happening

This example uses multiple `if` statements with early returns:

1. First, check if we are loading. If yes, show "Loading..."
2. Then, check if we have data. If yes, show the data.
3. If neither, show the "Load Data" button.

The `setTimeout` function waits for a certain number of milliseconds before running the code inside it. 2000 milliseconds = 2 seconds. This simulates waiting for data from the internet. In real apps, you would use a real data-fetching tool, but the loading pattern is the same.

Mini Project: Toggle Dark Mode / Light Mode Message

Let us build a fun component that switches between a "dark mode" and "light mode" message, changing the background and text colors.

```jsx
import { useState } from 'react';

function ThemeToggle() {
  const [isDark, setIsDark] = useState(false);

  function handleToggle() {
    setIsDark(!isDark);
  }

  const style = {
    backgroundColor: isDark ? '#333333' : '#ffffff',
    color: isDark ? '#ffffff' : '#333333',
    padding: '20px',
    textAlign: 'center',
    minHeight: '150px',
  };

  return (
    <div style={style}>
      <h2>{isDark ? 'Dark Mode' : 'Light Mode'}</h2>
      <p>
        {isDark
          ? 'Easy on the eyes at night.'
          : 'Bright and clear for daytime.'}
      </p>
      <button onClick={handleToggle}>
        Switch to {isDark ? 'Light' : 'Dark'} Mode
      </button>
    </div>
  );
```

```
}

export default ThemeToggle;
```

Expected Output

When the page loads (light mode):

```
+-------------------------------------------+
|                                           |
|           Light Mode                      |
|   Bright and clear for daytime.           |
|                                           |
|     [ Switch to Dark Mode ]               |
|                                           |
+-------------------------------------------+
  (white background, dark text)
```

After clicking the button (dark mode):

```
+-------------------------------------------+
|///////////////////////////////////////////|
|///////////////////////////////////////////|
|///////////  Dark Mode  ////////////////////|
|///  Easy on the eyes at night.  //////////|
|///////////////////////////////////////////|
|/////  [ Switch to Light Mode ]  //////////|
|///////////////////////////////////////////|
+-------------------------------------------+
  (dark background, white text)
```

Line-by-Line Explanation

Line 4: `const [isDark, setIsDark] = useState(false);` We start in light mode (`false` means "not dark").

Line 6-8: `function handleToggle() { setIsDark(!isDark); }` Flip between dark and light. `!isDark` gives us the opposite value.

Line 10-16: The `style` object. We create a style object with different colors depending on `isDark`:

- `backgroundColor`: dark gray for dark mode, white for light mode
- `color`: white text for dark mode, dark text for light mode

- Other properties control spacing and size

Line 20: `<h2>{isDark ? 'Dark Mode' : 'Light Mode'}</h2>` Show the current mode name.

Line 22-25: The description text changes based on the mode.

Line 27: `Switch to {isDark ? 'Light' : 'Dark'} Mode` The button text tells you what mode you will switch TO, not what mode you are currently in.

Quick Summary

- Conditional rendering means showing different things based on a condition.
- Use `if/else` outside the return when the two options are very different.
- Use the ternary operator `? :` inside JSX when you need to choose between two things.
- Use `&&` inside JSX when you want to show something or nothing.
- Be careful with `&&` and the number `0` — use `count > 0 &&` instead of `count &&`.
- These three methods are the building blocks for making your app respond to different situations.

Key Points to Remember

1. The ternary operator has three parts: `condition ? ifTrue : ifFalse`
2. The `&&` operator shows the right side only if the left side is true.
3. You can use `if/else` with multiple `return` statements — whichever `return` runs first wins.
4. The `!` symbol flips a boolean: `!true` is `false`, `!false` is `true`.
5. Always show loading messages when your app is waiting for something.
6. Conditional rendering is just a fancy way of saying "show different things depending on the situation."

Practice Questions

1. What does "conditional rendering" mean in simple words?

2. What is the difference between the ternary operator and the `&&` operator? When would you use each one?
3. What will this code display if `age` is `20`?

```jsx
<p>{age >= 18 ? 'Adult' : 'Minor'}</p>
```

4. What will this code display if `hasNotification` is `false`?

```jsx
{hasNotification && <p>You have a notification!</p>}
```

5. Why should you write `count > 0 && <p>Items</p>` instead of `count && <p>Items</p>`?

Practice Exercises

Exercise 1: Weather Message

Build a component called `WeatherMessage` that:

- Has a state variable `isRaining` that starts as `false`
- Shows "Bring an umbrella!" when `isRaining` is true
- Shows "Enjoy the sunshine!" when `isRaining` is false
- Has a button to toggle between the two states

Exercise 2: Age Checker

Build a component called `AgeChecker` that:

- Has an input field where the user types their age
- Below the input, shows "You can vote!" if the age is 18 or more
- Shows "You are too young to vote." if the age is less than 18
- Shows nothing if the input is empty

Hint: Use `onChange` to capture the input and `parseInt()` to convert the text to a number.

Exercise 3: Password Strength

Build a component called `PasswordChecker` that:

- Has an input field for a password

- Shows "Weak" (in red) if the password is less than 6 characters
- Shows "Medium" (in orange) if the password is 6-9 characters
- Shows "Strong" (in green) if the password is 10 or more characters

Hint: Use `if/else` to check `password.length` and set a message and color.

What Is Next?

You can now show different things based on conditions. But what about showing many similar things? What if you have a list of 10 products and want to show them all? You would not write 10 separate components by hand. In the next chapter, Chapter 12: Lists and Keys, you will learn how to take a list of data and turn it into a list of components — automatically.

Chapter 12: Lists and Keys — Showing Many Items

What You Will Learn

- Why lists are important in apps
- How the `map()` function works
- How to render a list of items in React
- What keys are and why React needs them
- The rules for choosing good keys
- The common mistake of forgetting keys (and how to fix the warning)
- How to display a list of objects
- How to build a simple shopping list display

Why This Chapter Matters

Think about the apps you use every day. Almost all of them show lists:

- A to-do app shows a list of tasks
- An email app shows a list of messages
- A store shows a list of products
- A social media app shows a list of posts

```
+---------------------+
| My Shopping List    |
|---------------------|
| - Milk              |
| - Eggs              |
| - Bread             |
| - Butter            |
| - Apples            |
```

```
+---------------------+
```

You could write each item by hand:

```jsx
<li>Milk</li>
<li>Eggs</li>
<li>Bread</li>
<li>Butter</li>
<li>Apples</li>
```

But what if you have 100 items? Or what if the items come from a database and you do not know how many there will be? You cannot write them all by hand. You need a way to say: "Take this list of data and create one component for each item." That is what this chapter teaches you.

The map() Function Explained Simply

Before we use lists in React, you need to understand one JavaScript function: `map()`.

`map()` means: do the same thing to every item in a list and give me a new list with the results.

Think of it like a factory assembly line:

```
Raw materials go in:     [apple, banana, cherry]
                            |       |       |
                            v       v       v
Machine does the      (add "juice" to each name)
same thing to each:
                            |       |       |
                            v       v       v
Finished products:   [apple juice, banana juice, cherry juice]
```

Here is a simple JavaScript example:

```jsx
const numbers = [1, 2, 3, 4, 5];
const doubled = numbers.map(function(number) {
  return number * 2;
});
// doubled is now [2, 4, 6, 8, 10]
```

Let us break this down:

- We have a list of numbers: `[1, 2, 3, 4, 5]`.
- We call `.map()` on the list.
- Inside `map()`, we write a function that takes one item (we call it `number`) and does something with it (multiply by 2).
- `map()` runs this function once for each item: `1*2=2`, `2*2=4`, `3*2=6`, `4*2=8`, `5*2=10`.
- We get a new list with the results: `[2, 4, 6, 8, 10]`.

The original list is not changed. `map()` always creates a new list.

Shorter Way with Arrow Functions

Most React developers write `map()` with an arrow function:

```jsx
const numbers = [1, 2, 3, 4, 5];
const doubled = numbers.map(number => number * 2);
// doubled is now [2, 4, 6, 8, 10]
```

This does exactly the same thing, just in one line. The arrow `=>` means "do this." So `number => number * 2` means "take a number, multiply it by 2, and return the result."

Rendering a List of Items

Now let us use `map()` in React to show a list on the screen.

```jsx
function FruitList() {
  const fruits = ['Apple', 'Banana', 'Cherry', 'Date', 'Elderberry'];

  return (
    <ul>
      {fruits.map(fruit => (
        <li key={fruit}>{fruit}</li>
      ))}
    </ul>
  );
}
```

Expected Output

```
- Apple
- Banana
- Cherry
- Date
- Elderberry
```

Line-by-Line Explanation

Line 2: `const fruits = ['Apple', 'Banana', 'Cherry', 'Date', 'Elderberry'];` We create an array (a list) of fruit names.

Line 5: `<ul>` We start an unordered list. This is a regular HTML element that shows bullet points.

Line 6: `{fruits.map(fruit => (` Inside curly braces `{ }` (because we are writing JavaScript inside JSX), we call `map()` on the `fruits` array. For each fruit in the array, the function runs.

Line 7: `<li key={fruit}>{fruit}</li>` For each fruit, we create a list item (`<li>`) that shows the fruit's name. The `key={fruit}` part is something new — we will explain it in the next section.

Line 8: `))}` Close the arrow function, close the `map()`, and close the curly braces.

Line 9: `</ul>` Close the unordered list.

What map() Does Here

```
fruits array:        ['Apple', 'Banana', 'Cherry']
                        |         |         |
                        v         v         v
map() creates:    <li>Apple</li>  <li>Banana</li>  <li>Cherry</li>
                        |         |         |
                        v         v         v
React shows:         - Apple
                     - Banana
                     - Cherry
```

Each string in the array becomes an `<li>` element on the screen. If you add more fruits to the array, more list items appear automatically.

What Are Keys and Why React Needs Them

You probably noticed the `key={fruit}` in the previous example. Let us explain what that is.

A key is a special label that helps React tell items apart. Think of keys like name tags at a party.

```
Party without name tags:           Party with name tags:
+--------+--------+--------+      +--------+--------+--------+
| Person | Person | Person |      |  ALEX  |  BETH  | CHRIS  |
|   ?    |   ?    |   ?    |      |        |        |        |
+--------+--------+--------+      +--------+--------+--------+

"Who left? Who's new?              "Alex left, Dana joined,
 I have no idea!"                   Beth and Chris stayed."
```

Without name tags, if someone leaves and someone new arrives, you cannot tell what happened. With name tags, you can track exactly who is who.

React works the same way. When your list changes (items added, removed, or moved), React uses keys to figure out:

- Which items are new?
- Which items were removed?
- Which items just moved to a different position?

Without keys, React has to guess, and it might make mistakes that cause bugs or slow down your app.

How to Add Keys

You add a `key` to each item inside `map()`:

```jsx
{fruits.map(fruit => (
  <li key={fruit}>{fruit}</li>
))}
```

The `key` goes on the outermost element that `map()` returns.

Rules for Keys

Not any value works well as a key. Here are the rules:

Rule 1: Keys Must Be Unique

Each key must be different from all other keys in the same list. No two items should have the same key.

```jsx
// GOOD - each key is unique
{['Apple', 'Banana', 'Cherry'].map(fruit => (
  <li key={fruit}>{fruit}</li>
))}

// BAD - what if you had two 'Apple' entries?
// Two items would have the same key!
```

Rule 2: Keys Should Be Stable

A "stable" key means it does not change over time. The same item should always have the same key.

```jsx
// GOOD - IDs from your data are stable
{users.map(user => (
  <li key={user.id}>{user.name}</li>
))}

// BAD - Math.random() gives a different key every render!
{users.map(user => (
  <li key={Math.random()}>{user.name}</li>
))}
```

Rule 3: Avoid Using Index If the List Can Change

The index is the position number of an item in the array (0, 1, 2, 3...). You can use it as a key, but it has problems when items are added, removed, or reordered.

```jsx
// OK for lists that NEVER change
{fruits.map((fruit, index) => (
  <li key={index}>{fruit}</li>
))}

// BAD for lists that can change order or have items added/removed
```

Why is index bad for changing lists? Because the index is based on position, not identity:

```
Before removing "Banana":
  index 0 = Apple      key=0
  index 1 = Banana     key=1
  index 2 = Cherry     key=2

After removing "Banana":
  index 0 = Apple      key=0
  index 1 = Cherry     key=1  ← Cherry now has Banana's old key!
```

React sees that `key=1` changed from "Banana" to "Cherry" and thinks the item was edited, not that "Banana" was removed. This can cause confusing bugs.

What Makes a Good Key?

The best keys come from your data:

- Database IDs (like `user.id` or `product.id`) — these are perfect
- Unique strings (like usernames or email addresses)
- Any value that is unique and does not change

Common Mistake: Forgetting Keys

If you forget to add keys, React will show a warning in the browser console (the developer tools area):

```
Warning: Each child in a list should have a unique "key" prop.
```

Here is the code that causes this warning:

```jsx
// MISSING KEYS - React will show a warning
function BadList() {
  const items = ['First', 'Second', 'Third'];

  return (
    <ul>
      {items.map(item => (
        <li>{item}</li>  // No key! React warns you.
      ))}
    </ul>
  );
}
```

How to Fix It

Add a `key` prop to the element inside `map()`:

```jsx
// FIXED - keys added
function GoodList() {
  const items = ['First', 'Second', 'Third'];

  return (
    <ul>
      {items.map(item => (
        <li key={item}>{item}</li>  // Key added!
      ))}
    </ul>
  );
}
```

Remember: The `key` does not appear on the screen. It is only used by React internally to keep track of items. Users never see the keys.

Displaying a List of Objects

So far, we have worked with simple lists of strings. But in real apps, your data often comes as a list of objects — items that have multiple pieces of information like a name, an age, or a price.

```jsx
function StudentList() {
  const students = [
    { id: 1, name: 'Alice', age: 20 },
    { id: 2, name: 'Bob', age: 22 },
    { id: 3, name: 'Charlie', age: 19 },
  ];

  return (
    <div>
      <h2>Students</h2>
      <ul>
        {students.map(student => (
          <li key={student.id}>
            {student.name} - Age: {student.age}
          </li>
        ))}
      </ul>
```

```
    </div>
  );
}
```

Expected Output

```
Students
- Alice - Age: 20
- Bob - Age: 22
- Charlie - Age: 19
```

Line-by-Line Explanation

Line 2-6: We create an array of objects. Each object has three properties: `id`, `name`, and `age`. The `id` is a unique number for each student.

Line 12: `{students.map(student => (` We use `map()` on the `students` array. Each `student` is one object from the array.

Line 13: `<li key={student.id}>` We use `student.id` as the key. This is the ideal key because each student has a unique ID that never changes.

Line 14: `{student.name} - Age: {student.age}` We access the properties of each student object using dot notation: `student.name` gives us the name, `student.age` gives us the age.

Extracting a List Item Into Its Own Component

When each item in your list gets more complex, it is a good idea to create a separate component for it. This keeps your code clean and organized.

```jsx
function StudentCard({ student }) {
  return (
    <div style={{
      border: '1px solid gray',
      padding: '10px',
      margin: '5px',
    }}>
      <h3>{student.name}</h3>
```

```
      <p>Age: {student.age}</p>
    </div>
  );
}

function StudentList() {
  const students = [
    { id: 1, name: 'Alice', age: 20 },
    { id: 2, name: 'Bob', age: 22 },
    { id: 3, name: 'Charlie', age: 19 },
  ];

  return (
    <div>
      <h2>Students</h2>
      {students.map(student => (
        <StudentCard key={student.id} student={student} />
      ))}
    </div>
  );
}
```

Key Placement

Notice that the `key` goes on `<StudentCard>`, not inside the `StudentCard` component. The key always goes on the element that `map()` directly returns.

```
CORRECT:
  {students.map(student => (
    <StudentCard key={student.id} student={student} />
  ))}                 ↑
                      key goes HERE (on the component in map)

WRONG:
  // Inside StudentCard:
  <div key={student.id}>   ← key should NOT be inside the component
```

Mini Project: Simple Shopping List Display

Let us build a shopping list that displays items with their names, quantities, and a "purchased" status.

```jsx
import { useState } from 'react';

function ShoppingItem({ item }) {
  const style = {
    textDecoration: item.purchased ? 'line-through' : 'none',
    color: item.purchased ? 'gray' : 'black',
    padding: '5px 0',
  };

  return (
    <li style={style}>
      {item.name} — Qty: {item.quantity}
      {item.purchased && ' (Purchased)'}
    </li>
  );
}

function ShoppingList() {
  const [items] = useState([
    { id: 1, name: 'Milk', quantity: 2, purchased: false },
    { id: 2, name: 'Eggs', quantity: 12, purchased: true },
    { id: 3, name: 'Bread', quantity: 1, purchased: false },
    { id: 4, name: 'Butter', quantity: 1, purchased: true },
    { id: 5, name: 'Apples', quantity: 6, purchased: false },
  ]);

  const purchasedCount = items.filter(item => item.purchased).length;

  return (
    <div>
      <h2>Shopping List</h2>
      <p>
        {purchasedCount} of {items.length} items purchased
      </p>
      <ul>
        {items.map(item => (
          <ShoppingItem key={item.id} item={item} />
        ))}
      </ul>
    </div>
  );
}

export default ShoppingList;
```

Expected Output

```
Shopping List
2 of 5 items purchased

- Milk — Qty: 2
```

```
- Eggs — Qty: 12 (Purchased)     ← this text has a line through it
- Bread — Qty: 1
- Butter — Qty: 1 (Purchased)    ← this text has a line through it
- Apples — Qty: 6
```

Line-by-Line Explanation

ShoppingItem Component (Lines 3-16):

Line 3: `function ShoppingItem({ item }) {` This component receives one shopping item as a prop.

Line 4-8: The `style` object.

- `textDecoration`: if the item is purchased, show a line through the text. Otherwise, show normal text. "Line-through" is the CSS property that draws a line through text.
- `color`: purchased items appear in gray, unpurchased items in black.

Line 12: `{item.name} — Qty: {item.quantity}` Show the item name and quantity.

Line 13: `{item.purchased && ' (Purchased)'}` If the item is purchased, show the text "(Purchased)." If not, show nothing. This uses the `&&` operator you learned in Chapter 11.

ShoppingList Component (Lines 18-43):

Line 19-26: We create a state variable with an array of shopping items. Each item has an `id`, `name`, `quantity`, and `purchased` status. We write `const [items]` without `setItems` because we are not changing the list in this example. (We will learn how to add and remove items from lists in a later chapter.)

Line 28: `const purchasedCount = items.filter(item => item.purchased).length;` This line counts how many items are purchased. The `filter()` function creates a new list containing only items where `item.purchased` is `true`. Then `.length` gives us how many items are in that new list.

Line 37-39: We use `map()` to create a `ShoppingItem` component for each item. Each one gets a `key` (the item's `id`) and the item data as a prop.

Quick Summary

- Use `map()` to create a list of elements from an array of data.
- Every item in a list needs a `key` prop so React can track it.

- Keys should be unique and stable (they should not change).
- Use IDs from your data as keys when possible.
- Avoid using array index as a key if the list can change.
- When list items get complex, extract them into their own component.
- The `key` goes on the element that `map()` directly returns, not inside the child component.

Key Points to Remember

1. `map()` takes an array and creates a new array by running a function on each item.
2. Always add `key` to list items — React will warn you if you forget.
3. Good keys: database IDs, unique strings. Bad keys: `Math.random()`, array index (for dynamic lists).
4. Keys are not shown on screen — React uses them internally.
5. The `key` prop goes on the outermost element returned by `map()`.
6. The `filter()` function creates a new array with only the items that pass a test.

Practice Questions

1. In your own words, what does the `map()` function do?
2. What is the purpose of the `key` prop in React lists?
3. Why is using `Math.random()` as a key a bad idea?
4. What warning will React show if you forget to add keys to list items?
5. Given this array, write the `map()` call to create a `<p>` element for each color:

```jsx
const colors = ['Red', 'Green', 'Blue'];
```

Practice Exercises

Exercise 1: Number List

Build a component called `NumberList` that:

- Has an array of numbers: `[10, 20, 30, 40, 50]`
- Shows each number in a list item
- Adds the text "is even" or "is odd" after each number

Expected output:

```
- 10 is even
- 20 is even
- 30 is even
- 40 is even
- 50 is even
```

Hint: You can check if a number is even with `number % 2 === 0`. The `%` symbol gives the remainder when dividing. If the remainder is 0, the number is even.

Exercise 2: Contact List

Build a component called `ContactList` that:

- Has an array of contact objects, each with `id`, `name`, and `email`
- Shows each contact's name and email in a styled list
- Uses the `id` as the key

Exercise 3: Movie List with Ratings

Build a component called `MovieList` that:

- Has an array of movie objects with `id`, `title`, and `rating` (a number from 1 to 5)
- Shows each movie title and its rating
- Movies with a rating of 4 or higher should have their text in bold
- Show the total number of movies at the top

Hint: Use the `fontWeight` CSS property: `fontWeight: movie.rating >= 4 ? 'bold' : 'normal'`.

What Is Next?

You now know how to display lists of items, which is one of the most common things you will do in React apps. You have learned about state, events, conditional rendering, and lists — the

four core skills for building interactive user interfaces. In the next chapter, you will learn about forms in more detail, combining everything you have learned so far into practical, real-world patterns.

Chapter 13: Forms

What You Will Learn

- How forms work in React (and how they are different from regular HTML forms)
- What a controlled component is (React controls the input value)
- How to handle text inputs, textareas, select dropdowns, checkboxes, and radio buttons
- How to handle form submission and prevent the page from refreshing
- How to do simple form validation (checking if a field is empty)
- How to collect multiple form fields into one state object
- How to build a simple contact form

Why This Chapter Matters

Forms are everywhere on the internet. Every time you sign up for an account, log in, search for something, or fill out a survey, you are using a form. If you want to build real applications, you need to know how forms work in React.

React handles forms differently from plain HTML. In React, your component stays in control of what the user types. This might sound strange at first, but it gives you a lot of power. You can check what the user types as they type it. You can show error messages immediately. You can format their input on the fly.

Let us learn how.

How Forms Work in Regular HTML

Before we look at React, let us quickly understand how forms work in plain HTML.

In regular HTML, the browser keeps track of what the user types. When the user clicks the submit button, the browser collects all the form data and sends it to a server. The page usually refreshes.

```html
<form>
  <input type="text" name="username" />
  <button type="submit">Submit</button>
</form>
```

The browser controls the input. The browser remembers what the user typed.

How Forms Work in React — Controlled Components

In React, we do things differently. Instead of letting the browser control the input, React controls the input. We use state to store the value of the input. When the user types something, we update the state. The input always shows whatever is in the state.

This is called a controlled component. The word "controlled" means React is in charge. React tells the input what to display.

Think of it like this:

- Regular HTML form: The input is a whiteboard. The user writes on it directly. The whiteboard remembers what was written.
- React controlled form: The input is a TV screen. The user types on a keyboard, but the TV only shows what React tells it to show. React is the one deciding what appears on the screen.

Why would we want this? Because when React controls the input, we always know exactly what the user has typed. We can check it, change it, or validate it at any moment.

Your First Controlled Input

Let us start with a simple text input.

```jsx
import { useState } from 'react';

function NameForm() {
  const [name, setName] = useState('');

  function handleChange(event) {
    setName(event.target.value);
  }

  return (
    <div>
      <label>Your name: </label>
      <input type="text" value={name} onChange={handleChange} />
      <p>Hello, {name}!</p>
    </div>
  );
}

export default NameForm;
```

What you will see on the screen:

```
Your name: [___________]
Hello, !
```

As you type "Sara" into the input, you will see:

```
Your name: [Sara_______]
Hello, Sara!
```

Line-by-Line Explanation

```jsx
const [name, setName] = useState('');
```

We create a state variable called `name`. It starts as an empty string (`''`). This state will hold whatever the user types.

```jsx
function handleChange(event) {
  setName(event.target.value);
}
```

This function runs every time the user types a character. The `event` is an object that React gives us. It contains information about what happened. `event.target` is the input element.

`event.target.value` is whatever text is currently in the input. We take that text and put it into our state using `setName`.

```jsx
<input type="text" value={name} onChange={handleChange} />
```

Two important things here:

- `value={name}` — This tells the input to show whatever is in our `name` state. This is what makes it a controlled component. The input does not decide what to show. React decides.
- `onChange={handleChange}` — Every time the user types, call the `handleChange` function.

```jsx
<p>Hello, {name}!</p>
```

This paragraph displays the name in real time. As the state changes, React re-renders, and the greeting updates.

The Flow of a Controlled Input

Here is what happens step by step when the user types the letter "S":

```
1. User presses "S" on the keyboard
2. React calls the handleChange function
3. handleChange reads event.target.value (which is "S")
4. handleChange calls setName("S")
5. React updates the state to "S"
6. React re-renders the component
7. The input now shows "S" (because value={name} and name is now "S")
8. The paragraph shows "Hello, S!"
```

This happens for every single character the user types. It is very fast. You will not notice any delay.

Handling Form Submission

When a user fills out a form, they usually click a submit button. In React, we need to handle this properly.

```jsx
import { useState } from 'react';

function GreetingForm() {
  const [name, setName] = useState('');
  const [submitted, setSubmitted] = useState(false);

  function handleSubmit(event) {
    event.preventDefault();
    setSubmitted(true);
  }

  return (
    <form onSubmit={handleSubmit}>
      <label>Enter your name: </label>
      <input
        type="text"
        value={name}
        onChange={(event) => setName(event.target.value)}
      />
      <button type="submit">Submit</button>

      {submitted && <p>Welcome, {name}!</p>}
    </form>
  );
}

export default GreetingForm;
```

What you will see:

Before submitting:

```
Enter your name: [Sara_______]  [Submit]
```

After clicking Submit:

```
Enter your name: [Sara_______]  [Submit]
Welcome, Sara!
```

What Is event.preventDefault()?

This is very important. Let us understand it clearly.

When you submit a form in a browser, the browser's default behavior is to refresh the entire page and send the data to a server. That is how forms worked in the old days of the internet.

But in React, we do not want the page to refresh. We want to stay on the same page and handle the data ourselves with JavaScript.

`event.preventDefault()` tells the browser: "Stop! Do not do what you normally do. Do not refresh the page. I will handle this myself."

Think of it like this: Imagine you are at a restaurant. When you finish eating, the waiter normally brings you the bill. But you tell the waiter, "Wait, do not bring the bill yet. I want to order dessert first." You are preventing the default action.

If you forget `event.preventDefault()`, the page will refresh, your state will be lost, and your React app will start over from scratch. Always include it when handling form submissions.

A Shorter Way to Write onChange

Notice that in this example, instead of writing a separate `handleChange` function, we wrote the function directly:

```jsx
onChange={(event) => setName(event.target.value)}
```

This is called an inline function. It does the same thing. It is just shorter. Use whichever style you find easier to read.

Textarea in React

In regular HTML, a textarea works like this:

```html
<textarea>Some text here</textarea>
```

The text goes between the opening and closing tags.

In React, a textarea works more like an input. You use the `value` attribute:

```jsx
import { useState } from 'react';

function MessageForm() {
  const [message, setMessage] = useState('');

  return (
    <div>
      <label>Your message:</label>
      <br />
```

```
      <textarea
        value={message}
        onChange={(event) => setMessage(event.target.value)}
        rows={4}
        cols={40}
      />
      <p>You typed {message.length} characters.</p>
    </div>
  );
}

export default MessageForm;
```

What you will see:

```
Your message:
+--------------------------------------+
| Hello, this is my message            |
|                                      |
|                                      |
|                                      |
+--------------------------------------+
You typed 27 characters.
```

The textarea works exactly like a text input. You use `value` to control it and `onChange` to update the state. The character count updates in real time as you type.

Select Dropdown in React

A select dropdown lets the user choose one option from a list. In React, we control it the same way as inputs.

```jsx
import { useState } from 'react';

function ColorPicker() {
  const [color, setColor] = useState('green');

  return (
    <div>
      <label>Pick your favorite color: </label>
      <select value={color} onChange={(event) => setColor(event.target.value)}>
        <option value="red">Red</option>
        <option value="green">Green</option>
```

```
        <option value="blue">Blue</option>
        <option value="yellow">Yellow</option>
      </select>
      <p>You picked: {color}</p>
    </div>
  );
}

export default ColorPicker;
```

What you will see:

```
Pick your favorite color: [Green ▼]
You picked: green
```

How It Works

- `value={color}` — The dropdown shows whichever option matches the state.
- `onChange` — When the user picks a different option, we update the state.
- `useState('green')` — We start with "green" as the default selection.

The pattern is always the same: state controls the value, onChange updates the state.

Checkboxes in React

Checkboxes are a little different. Instead of `value`, we use `checked`. A checkbox is either checked (true) or not checked (false). That is a boolean.

```jsx
import { useState } from 'react';

function AgreementForm() {
  const [agreed, setAgreed] = useState(false);

  return (
    <div>
      <label>
        <input
          type="checkbox"
          checked={agreed}
          onChange={(event) => setAgreed(event.target.checked)}
        />
```

```
        I agree to the terms and conditions
      </label>

      <br />
      <button disabled={!agreed}>Sign Up</button>

      {agreed && <p>Thank you for agreeing!</p>}
    </div>
  );
}

export default AgreementForm;
```

What you will see (before checking):

```
[ ] I agree to the terms and conditions
[Sign Up] (button is grayed out)
```

After checking the box:

```
[✓] I agree to the terms and conditions
[Sign Up] (button is now active)
Thank you for agreeing!
```

Key Differences for Checkboxes

- We use `checked={agreed}` instead of `value={agreed}`.
- We read `event.target.checked` instead of `event.target.value`.
- The state is a boolean (`true` or `false`), not a string.
- `disabled={!agreed}` means the button is disabled when `agreed` is false. The `!` flips true to false and false to true.

Radio Buttons in React

Radio buttons let the user pick one option from a group. Only one radio button in a group can be selected at a time. Think of them like the buttons on an old car radio. You push one in, and the others pop out.

```jsx
import { useState } from 'react';

function SizeSelector() {
  const [size, setSize] = useState('medium');

  return (
    <div>
      <p>Choose your t-shirt size:</p>

      <label>
        <input
          type="radio"
          value="small"
          checked={size === 'small'}
          onChange={(event) => setSize(event.target.value)}
        />
        Small
      </label>
      <br />

      <label>
        <input
          type="radio"
          value="medium"
          checked={size === 'medium'}
          onChange={(event) => setSize(event.target.value)}
        />
        Medium
      </label>
      <br />

      <label>
        <input
          type="radio"
          value="large"
          checked={size === 'large'}
          onChange={(event) => setSize(event.target.value)}
        />
        Large
      </label>

      <p>You selected: {size}</p>
    </div>
  );
}

export default SizeSelector;
```

What you will see:

```
Choose your t-shirt size:

( ) Small
```

```
(●) Medium
( ) Large

You selected: medium
```

How Radio Buttons Work

- All the radio buttons share the same state variable (`size`).
- Each radio button has a different `value` ("small", "medium", or "large").
- `checked={size === 'small'}` means this radio button is selected only when the state equals "small".
- When you click a radio button, `onChange` updates the state to that button's value. React re-renders, and only the matching radio button appears selected.

Simple Form Validation

Form validation means checking if the user filled out the form correctly before accepting it. The simplest check is: did the user leave a field empty?

```jsx
import { useState } from 'react';

function LoginForm() {
  const [username, setUsername] = useState('');
  const [error, setError] = useState('');

  function handleSubmit(event) {
    event.preventDefault();

    if (username.trim() === '') {
      setError('Username cannot be empty!');
      return;
    }

    setError('');
    alert('Welcome, ' + username + '!');
  }

  return (
    <form onSubmit={handleSubmit}>
      <label>Username: </label>
      <input
        type="text"
```

```
        value={username}
        onChange={(event) => setUsername(event.target.value)}
      />
      <button type="submit">Log In</button>

      {error && <p style={{ color: 'red' }}>{error}</p>}
    </form>
  );
}

export default LoginForm;
```

What you will see if you click Submit with an empty input:

```
Username: [___________]  [Log In]
Username cannot be empty!
```

What you will see if you type "Sara" and click Submit:

```
A popup appears: "Welcome, Sara!"
```

Line-by-Line Explanation of the Validation

```jsx
if (username.trim() === '') {
```

`trim()` removes any spaces from the beginning and end of the text. So if the user typed just spaces, `trim()` turns it into an empty string. We check if the result is an empty string (`''`).

```jsx
setError('Username cannot be empty!');
return;
```

If the username is empty, we set an error message in state. Then we `return`, which means "stop here, do not continue with the rest of the function."

```jsx
setError('');
```

If the username is valid, we clear any previous error message by setting it to an empty string.

```jsx
{error && <p style={{ color: 'red' }}>{error}</p>}
```

This only shows the error message paragraph when `error` is not an empty string. Remember, an empty string is falsy in JavaScript, so if `error` is `''`, the paragraph will not show up.

Collecting Multiple Fields into One State Object

When your form has many fields, creating a separate `useState` for each one can get tiring. Instead, you can store all the form data in one object.

Think of it like a paper form. Instead of having separate sticky notes for each answer, you have one form sheet with all the answers on it.

```jsx
import { useState } from 'react';

function RegistrationForm() {
  const [formData, setFormData] = useState({
    firstName: '',
    lastName: '',
    email: '',
  });

  function handleChange(event) {
    const { name, value } = event.target;
    setFormData({
      ...formData,
      [name]: value,
    });
  }

  function handleSubmit(event) {
    event.preventDefault();
    console.log('Form submitted:', formData);
  }

  return (
    <form onSubmit={handleSubmit}>
      <div>
        <label>First Name: </label>
        <input
          type="text"
          name="firstName"
          value={formData.firstName}
          onChange={handleChange}
        />
      </div>

      <div>
```

```
        <label>Last Name: </label>
        <input
          type="text"
          name="lastName"
          value={formData.lastName}
          onChange={handleChange}
        />
      </div>

      <div>
        <label>Email: </label>
        <input
          type="email"
          name="email"
          value={formData.email}
          onChange={handleChange}
        />
      </div>

      <button type="submit">Register</button>
    </form>
  );
}

export default RegistrationForm;
```

The Key Parts Explained

```jsx
const [formData, setFormData] = useState({
  firstName: '',
  lastName: '',
  email: '',
});
```

Instead of three separate `useState` calls, we have one state that holds an object with three properties.

```jsx
const { name, value } = event.target;
```

This pulls out two things from the input element: its `name` attribute and its current `value`. This is called destructuring. It is a shortcut for writing:

```jsx
const name = event.target.name;
const value = event.target.value;
```

```jsx
setFormData({
  ...formData,
  [name]: value,
});
```

This is the important part. Let us break it down:

- `...formData` — This copies all the current form data. The three dots (`...`) are called the spread operator. Think of it like photocopying a form. You get a copy of everything.
- `[name]: value` — This updates just the one field that changed. The square brackets around `name` let us use a variable as the property name.

So if the user types in the "Email" field, this becomes:

```jsx
setFormData({
  ...formData,        // keep firstName and lastName as they are
  email: 'whatever the user typed',  // update only email
});
```

Each input has a `name` attribute that matches the property name in our state object. That is how React knows which field to update.

Mini Project: Simple Contact Form

Let us build a complete contact form that collects a name, email, and message, validates the fields, and displays the submitted data.

```jsx
import { useState } from 'react';

function ContactForm() {
  const [formData, setFormData] = useState({
    name: '',
    email: '',
    message: '',
  });

  const [errors, setErrors] = useState({});
  const [submittedData, setSubmittedData] = useState(null);

  function handleChange(event) {
    const { name, value } = event.target;
    setFormData({
```

```
      ...formData,
      [name]: value,
    });
  }

  function validate() {
    const newErrors = {};

    if (formData.name.trim() === '') {
      newErrors.name = 'Name is required.';
    }

    if (formData.email.trim() === '') {
      newErrors.email = 'Email is required.';
    }

    if (formData.message.trim() === '') {
      newErrors.message = 'Message is required.';
    }

    return newErrors;
  }

  function handleSubmit(event) {
    event.preventDefault();

    const validationErrors = validate();

    if (Object.keys(validationErrors).length > 0) {
      setErrors(validationErrors);
      setSubmittedData(null);
      return;
    }

    setErrors({});
    setSubmittedData(formData);
    setFormData({ name: '', email: '', message: '' });
  }

  return (
    <div>
      <h2>Contact Us</h2>

      <form onSubmit={handleSubmit}>
        <div>
          <label>Name: </label>
          <input
            type="text"
            name="name"
            value={formData.name}
            onChange={handleChange}
          />
          {errors.name && <p style={{ color: 'red' }}>{errors.name}</p>}
        </div>
```

```
        <div>
          <label>Email: </label>
          <input
            type="email"
            name="email"
            value={formData.email}
            onChange={handleChange}
          />
          {errors.email && <p style={{ color: 'red' }}>{errors.email}</p>}
        </div>

        <div>
          <label>Message: </label>
          <br />
          <textarea
            name="message"
            value={formData.message}
            onChange={handleChange}
            rows={4}
            cols={40}
          />
          {errors.message && <p style={{ color: 'red' }}>{errors.message}</p>}
        </div>

        <button type="submit">Send Message</button>
      </form>

      {submittedData && (
        <div style={{ marginTop: '20px', padding: '10px', backgroundColor: '#f0f0f0' }}>
          <h3>Message Sent!</h3>
          <p><strong>Name:</strong> {submittedData.name}</p>
          <p><strong>Email:</strong> {submittedData.email}</p>
          <p><strong>Message:</strong> {submittedData.message}</p>
        </div>
      )}
    </div>
  );
}

export default ContactForm;
```

What you will see if you submit with empty fields:

```
Contact Us

Name: [___________]
Name is required.

Email: [___________]
Email is required.

Message:
+-------------------------------------+
```

```
|                                       |
+---------------------------------------+
Message is required.

[Send Message]
```

What you will see after filling out and submitting the form:

```
Contact Us

Name: [___________]
Email: [___________]
Message:
+---------------------------------------+
|                                       |
+---------------------------------------+
[Send Message]

Message Sent!
Name: Sara
Email: sara@email.com
Message: Hello, I love your website!
```

How the Validation Works

```jsx
function validate() {
  const newErrors = {};

  if (formData.name.trim() === '') {
    newErrors.name = 'Name is required.';
  }
  // ... more checks ...

  return newErrors;
}
```

The `validate` function creates an empty object. For each field that is empty, it adds an error message to the object. If everything is filled in, the object stays empty.

```jsx
if (Object.keys(validationErrors).length > 0) {
```

`Object.keys()` gives us an array of all the property names in the object. If the array has any items (length is greater than 0), that means there are errors. We stop and show the errors. We do not submit the form.

After a successful submission, we clear the form by setting the form data back to empty strings and show the submitted data in a summary box.

Quick Summary

Concept	How It Works
Controlled component	React controls the input value through state
Text input	`value={state}` and `onChange={handler}`
Textarea	Same as text input, uses `value` attribute
Select dropdown	Same pattern, `value` on the `<select>` element
Checkbox	Uses `checked={state}` and `event.target.checked`
Radio buttons	Same `checked` pattern, shared state variable
Form submission	`onSubmit` on the form, `event.preventDefault()` inside
Validation	Check state values before accepting the submission
Multiple fields	Store all fields in one state object, use spread operator

---

Key Points to Remember

1. In React, the state controls the input. The input shows whatever is in the state. This is called a controlled component.
2. Always use `event.preventDefault()` in your form submission handler. Without it, the browser will refresh the page and your React app will lose its state.
3. For checkboxes, use `checked` instead of `value`. Read `event.target.checked` instead of `event.target.value`.
4. The spread operator (`...`) copies an object. When updating one field in a form object, spread the existing data first, then override the field that changed.

5. Give each input a `name` attribute that matches the property name in your state object. This lets you use one `handleChange` function for all inputs.
6. Validation is just checking state values. Before accepting a submission, check if the fields are empty or invalid.

Practice Questions

1. What is a controlled component in React? Why do we call it "controlled"?
2. What does `event.preventDefault()` do, and why is it important in React forms?
3. How is handling a checkbox different from handling a text input? What attribute and property do we use instead of `value`?
4. If you have a form with five text fields, what are the two ways you can manage their state? Which one is better for many fields?
5. In the expression `setFormData({ ...formData, [name]: value })`, what does each part do?

Exercises

Exercise 1: Favorite Food Form

Create a form with:

- A text input for the user's favorite food
- A submit button
- After submission, show "Your favorite food is: [food]!" below the form
- If the input is empty, show an error message in red

Exercise 2: Pizza Order Form

Create a form where the user can:

- Enter their name (text input)
- Choose a pizza size (radio buttons: Small, Medium, Large)
- Check a box for extra cheese (checkbox)
- Click "Place Order"

- After submission, show a summary: "Order for [name]: [size] pizza, extra cheese: yes/no"

Exercise 3: Feedback Form

Create a feedback form with:

- A text input for the user's name
- A select dropdown to rate their experience (Excellent, Good, Average, Poor)
- A textarea for comments
- Validate that name and comments are not empty
- After submission, display all the feedback in a summary box and clear the form

---

What Is Next?

You now know how to build forms in React. You can collect user input, validate it, and display it. Forms use `useState` to control their values, and you have seen how powerful that is.

In the next chapter, we will take a much deeper look at `useState`. You have been using it for simple strings and booleans, but state can hold arrays, objects, and more. We will learn how to update arrays and objects correctly, avoid common mistakes, and build a to-do list application. Get ready to level up your state skills.

Chapter 14: useState In Depth

What You Will Learn

- A quick recap of how useState works
- How to use state with different types of data: strings, numbers, booleans, arrays, and objects
- How to update arrays in state correctly (adding, removing, and filtering items)
- How to update objects in state correctly using the spread operator
- How to use multiple useState calls in one component
- What the updater function form is and why it matters
- Common mistakes beginners make with state
- How to build a to-do list application

Why This Chapter Matters

You have been using `useState` since Chapter 9. You know how to create a piece of state and update it. But so far, you have mostly used it with simple values like strings and numbers.

Real applications need more. A to-do list is an array. A user profile is an object. A shopping cart is an array of objects. To build real things, you need to know how to work with these more complex types of state.

This chapter will also help you avoid bugs that trip up many beginners. Some of these bugs are sneaky. Your code might look correct but behave in unexpected ways. By the end of this chapter, you will understand why, and you will know how to avoid these problems.

Quick Recap: How useState Works

Let us quickly review what you already know.

```jsx
import { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>Add 1</button>
    </div>
  );
}
```

Here is what is happening:

1. `useState(0)` creates a state variable with an initial value of `0`.
2. It gives us two things: `count` (the current value) and `setCount` (a function to update it).
3. When we call `setCount(count + 1)`, React updates the state and re-renders the component.
4. On re-render, `count` has the new value.

That is the foundation. Now let us build on it.

State with Different Data Types

State can hold any type of data. Let us look at examples of each type.

String State

You have seen this many times. Perfect for text inputs.

```jsx
const [name, setName] = useState('');
const [greeting, setGreeting] = useState('Hello, world!');
```

Number State

Great for counters, scores, quantities, and prices.

```jsx
const [score, setScore] = useState(0);
const [temperature, setTemperature] = useState(72);
```

Boolean State

Perfect for on/off, show/hide, yes/no situations.

```jsx
const [isVisible, setIsVisible] = useState(false);
const [isDarkMode, setIsDarkMode] = useState(true);
```

To toggle a boolean (flip it from true to false or false to true):

```jsx
<button onClick={() => setIsDarkMode(!isDarkMode)}>
  Toggle Dark Mode
</button>
```

The `!` operator flips the value. If `isDarkMode` is `true`, `!isDarkMode` is `false`, and vice versa.

Array State

For lists of items: to-do lists, shopping carts, search results.

```jsx
const [fruits, setFruits] = useState(['Apple', 'Banana', 'Cherry']);
```

Object State

For grouped related data: a user profile, a form, settings.

```jsx
const [user, setUser] = useState({
  name: 'Sara',
  age: 28,
  city: 'New York',
});
```

Updating Arrays in State

This is where things get interesting and where beginners often make mistakes. There is one very important rule:

Never change (mutate) state directly. Always create a new array.

Think of it like this: your state array is a signed document. You should never erase and rewrite parts of the signed document. Instead, you should make a photocopy, make your changes on the copy, and then replace the original with the new copy.

Why? Because React needs to know that something changed. When you create a new array, React can compare the old one and the new one and say, "These are different. I need to re-render." If you just change the old array, React does not notice the change.

Adding an Item to an Array

```jsx
import { useState } from 'react';

function FruitList() {
  const [fruits, setFruits] = useState(['Apple', 'Banana']);
  const [newFruit, setNewFruit] = useState('');

  function addFruit() {
    if (newFruit.trim() === '') return;
    setFruits([...fruits, newFruit]);
    setNewFruit('');
  }

  return (
    <div>
      <input
        type="text"
        value={newFruit}
        onChange={(event) => setNewFruit(event.target.value)}
        placeholder="Enter a fruit"
      />
      <button onClick={addFruit}>Add Fruit</button>

      <ul>
        {fruits.map((fruit, index) => (
          <li key={index}>{fruit}</li>
        ))}
      </ul>
    </div>
  );
}

export default FruitList;
```

What you will see after adding "Cherry":

```
[Cherry____]  [Add Fruit]

- Apple
- Banana
- Cherry
```

How Adding Works

```jsx
setFruits([...fruits, newFruit]);
```

This creates a new array. Let us break it down:

- `[...]` — We are creating a new array.
- `...fruits` — The spread operator copies all the items from the old `fruits` array into the new one.
- `newFruit` — We add the new fruit at the end.

So if `fruits` is `['Apple', 'Banana']` and `newFruit` is `'Cherry'`, the result is `['Apple', 'Banana', 'Cherry']`.

Wrong way (do not do this):

```jsx
// DO NOT DO THIS
fruits.push(newFruit);
setFruits(fruits);
```

This modifies the original array with `push` and then passes the same array to `setFruits`. React sees the same array reference and might not re-render. Always create a new array.

Removing an Item from an Array

To remove an item, we use `filter`. The `filter` method creates a new array that only includes items that pass a test.

```jsx
import { useState } from 'react';

function FruitList() {
  const [fruits, setFruits] = useState(['Apple', 'Banana', 'Cherry', 'Date']);
```

```
  function removeFruit(indexToRemove) {
    setFruits(fruits.filter((fruit, index) => index !== indexToRemove));
  }

  return (
    <ul>
      {fruits.map((fruit, index) => (
        <li key={index}>
          {fruit}
          <button onClick={() => removeFruit(index)}> Remove</button>
        </li>
      ))}
    </ul>
  );
}

export default FruitList;
```

What you will see:

```
- Apple [Remove]
- Banana [Remove]
- Cherry [Remove]
- Date [Remove]
```

After clicking Remove next to "Cherry":

```
- Apple [Remove]
- Banana [Remove]
- Date [Remove]
```

How Removing Works

```jsx
fruits.filter((fruit, index) => index !== indexToRemove)
```

`filter` goes through each item in the array. For each item, it asks: "Is this item's index different from the one we want to remove?" If yes, keep it. If no, leave it out.

So if we want to remove index 2 (Cherry):

- Index 0 (Apple): `0 !== 2` is true. Keep it.
- Index 1 (Banana): `1 !== 2` is true. Keep it.
- Index 2 (Cherry): `2 !== 2` is false. Remove it.
- Index 3 (Date): `3 !== 2` is true. Keep it.

Result: `['Apple', 'Banana', 'Date']`. A new array without Cherry.

Updating an Item in an Array

To change one item in an array, we use `map`. The `map` method creates a new array where each item is transformed.

```jsx
function toggleFruit(indexToToggle) {
  setFruits(
    fruits.map((fruit, index) => {
      if (index === indexToToggle) {
        return fruit.toUpperCase();
      }
      return fruit;
    })
  );
}
```

This finds the item at the given index and converts it to uppercase. All other items stay the same.

Updating Objects in State

Just like arrays, you should never change an object directly. Always create a new object.

The spread operator (`...`) is your best friend here.

The Spread Operator with Objects — A Simple Analogy

Imagine you have a form filled out with your personal information:

```
Name: Sara
Age: 28
City: New York
```

Now you move to a new city. You do not erase "New York" and write "Boston" on the same form. Instead, you:

1. Make a photocopy of the form (everything stays the same).
2. On the copy, cross out "New York" and write "Boston."
3. Throw away the old form and keep the new one.

That is exactly what the spread operator does.

```jsx
import { useState } from 'react';

function UserProfile() {
  const [user, setUser] = useState({
    name: 'Sara',
    age: 28,
    city: 'New York',
  });

  function updateCity() {
    setUser({
      ...user,
      city: 'Boston',
    });
  }

  return (
    <div>
      <p>Name: {user.name}</p>
      <p>Age: {user.age}</p>
      <p>City: {user.city}</p>
      <button onClick={updateCity}>Move to Boston</button>
    </div>
  );
}

export default UserProfile;
```

Before clicking the button:

```
Name: Sara
Age: 28
City: New York
[Move to Boston]
```

After clicking the button:

```
Name: Sara
Age: 28
City: Boston
[Move to Boston]
```

How It Works

```jsx
setUser({
  ...user,       // copy everything: name, age, city
  city: 'Boston', // then override city with the new value
});
```

The spread operator copies all properties from the `user` object. Then `city: 'Boston'` overrides the `city` property. The result is a brand new object with the updated city.

Wrong way (do not do this):

```jsx
// DO NOT DO THIS
user.city = 'Boston';
setUser(user);
```

This changes the original object directly (mutates it) and passes the same object reference. React might not notice the change.

Multiple useState Calls in One Component

You can use `useState` as many times as you want in a single component. Each call creates a separate, independent piece of state.

```jsx
import { useState } from 'react';

function PlayerCard() {
  const [name, setName] = useState('Hero');
  const [health, setHealth] = useState(100);
  const [level, setLevel] = useState(1);
  const [isAlive, setIsAlive] = useState(true);

  function takeDamage() {
    const newHealth = health - 20;
    if (newHealth <= 0) {
      setHealth(0);
      setIsAlive(false);
    } else {
      setHealth(newHealth);
    }
  }

  function heal() {
    if (isAlive) {
```

```
      setHealth(health + 10);
    }
  }

  function levelUp() {
    if (isAlive) {
      setLevel(level + 1);
    }
  }

  return (
    <div>
      <h2>{name}</h2>
      <p>Health: {health}</p>
      <p>Level: {level}</p>
      <p>Status: {isAlive ? 'Alive' : 'Defeated'}</p>

      <button onClick={takeDamage}>Take Damage</button>
      <button onClick={heal}>Heal</button>
      <button onClick={levelUp}>Level Up</button>
    </div>
  );
}

export default PlayerCard;
```

Each piece of state is independent. Changing `health` does not affect `level`. Changing `level` does not affect `name`. They are like separate drawers in a dresser. Opening one drawer does not change what is in the other drawers.

When to Use One Object vs. Multiple useState Calls

Use multiple `useState` calls when the pieces of state are independent. Name, health, and level do not depend on each other.

Use one state object when the pieces of state are related and change together. A form where all fields belong to the same form is a good example. We saw this in the previous chapter.

There is no strict rule. Use whatever feels clearer and easier to manage for your situation.

The Updater Function Form

This is an important concept that catches many beginners off guard. Let us look at a problem first.

The Problem

What happens if you call `setCount` multiple times in the same function?

```jsx
function handleClick() {
  setCount(count + 1);
  setCount(count + 1);
  setCount(count + 1);
}
```

You might expect the count to go up by 3. But it only goes up by 1. Why?

Because all three lines use the same value of `count`. When `handleClick` runs, `count` might be `0`. All three lines read `count` as `0`:

```
setCount(0 + 1);  // sets count to 1
setCount(0 + 1);  // sets count to 1 again
setCount(0 + 1);  // sets count to 1 yet again
```

React batches these updates together. The final result is `count = 1`, not `count = 3`.

The Solution: The Updater Function

Instead of passing a value to `setCount`, you can pass a function. This function receives the previous state as its argument and returns the new state.

```jsx
function handleClick() {
  setCount(prev => prev + 1);
  setCount(prev => prev + 1);
  setCount(prev => prev + 1);
}
```

Now each call uses the result of the previous one:

```
setCount(prev => prev + 1);  // prev is 0, returns 1
setCount(prev => prev + 1);  // prev is 1, returns 2
setCount(prev => prev + 1);  // prev is 2, returns 3
```

The count goes up by 3. Each call gets the most up-to-date value.

When to Use the Updater Function

Use the updater function form when your new state depends on the old state. Here are good times to use it:

```jsx
// Incrementing a counter
setCount(prev => prev + 1);

// Toggling a boolean
setIsOpen(prev => !prev);

// Adding to an array
setItems(prev => [...prev, newItem]);

// Removing from an array
setItems(prev => prev.filter(item => item.id !== idToRemove));
```

The word `prev` is just a name. You can call it anything: `previous`, `old`, `current`, or even just `c`. The name `prev` is a common convention because it clearly says "the previous value."

A Complete Example

```jsx
import { useState } from 'react';

function BetterCounter() {
  const [count, setCount] = useState(0);

  function addThree() {
    setCount(prev => prev + 1);
    setCount(prev => prev + 1);
    setCount(prev => prev + 1);
  }

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(prev => prev + 1)}>Add 1</button>
      <button onClick={addThree}>Add 3</button>
      <button onClick={() => setCount(0)}>Reset</button>
    </div>
  );
}

export default BetterCounter;
```

What you will see after clicking "Add 3":

```
Count: 3
[Add 1]  [Add 3]  [Reset]
```

Common Mistakes with State

Let us look at the mistakes beginners make most often, so you can avoid them.

Mistake 1: Mutating State Directly

```jsx
// WRONG - Mutating the array
const [items, setItems] = useState(['A', 'B', 'C']);

function removeFirst() {
  items.shift();      // This changes the original array!
  setItems(items);    // React may not re-render!
}
```

```jsx
// CORRECT - Creating a new array
function removeFirst() {
  setItems(items.slice(1));  // slice creates a new array
}
```

Methods that mutate arrays (do not use these with state):

- `push()` — adds to the end
- `pop()` — removes from the end
- `shift()` — removes from the beginning
- `unshift()` — adds to the beginning
- `splice()` — adds or removes in the middle
- `sort()` — sorts in place
- `reverse()` — reverses in place

Methods that create new arrays (safe to use with state):

- `map()` — transforms each item
- `filter()` — keeps items that pass a test
- `slice()` — copies a portion
- `concat()` — joins arrays together
- `[...array]` — spread into a new array

Mistake 2: Using State Right After Setting It

```jsx
// WRONG - count is still the old value here
function handleClick() {
  setCount(count + 1);
  console.log(count);  // Still shows the old value!
}
```

State does not update instantly. When you call `setCount`, React schedules an update. The new value is available on the next render, not immediately on the next line.

If you need to use the new value, calculate it first:

```jsx
// CORRECT
function handleClick() {
  const newCount = count + 1;
  setCount(newCount);
  console.log(newCount);  // Shows the new value
}
```

Mistake 3: Forgetting to Spread When Updating Objects

```jsx
// WRONG - Loses name and age!
setUser({ city: 'Boston' });
```

This replaces the entire object with just `{ city: 'Boston' }`. The name and age are gone.

```jsx
// CORRECT - Keeps name and age, updates city
setUser({ ...user, city: 'Boston' });
```

Always spread the existing object first, then override the property you want to change.

Mini Project: To-Do List

Let us put everything together and build a to-do list app. You can add tasks, mark them as complete, and delete them.

```jsx
import { useState } from 'react';
```

```
function TodoApp() {
  const [todos, setTodos] = useState([]);
  const [inputText, setInputText] = useState('');

  function addTodo(event) {
    event.preventDefault();

    if (inputText.trim() === '') return;

    const newTodo = {
      id: Date.now(),
      text: inputText,
      completed: false,
    };

    setTodos(prev => [...prev, newTodo]);
    setInputText('');
  }

  function toggleTodo(idToToggle) {
    setTodos(prev =>
      prev.map(todo => {
        if (todo.id === idToToggle) {
          return { ...todo, completed: !todo.completed };
        }
        return todo;
      })
    );
  }

  function deleteTodo(idToDelete) {
    setTodos(prev => prev.filter(todo => todo.id !== idToDelete));
  }

  return (
    <div>
      <h2>My To-Do List</h2>

      <form onSubmit={addTodo}>
        <input
          type="text"
          value={inputText}
          onChange={(event) => setInputText(event.target.value)}
          placeholder="What do you need to do?"
        />
        <button type="submit">Add</button>
      </form>

      {todos.length === 0 && <p>No tasks yet. Add one above!</p>}

      <ul>
        {todos.map(todo => (
          <li key={todo.id}>
            <span
              onClick={() => toggleTodo(todo.id)}
```

```
              style={{
                textDecoration: todo.completed ? 'line-through' : 'none',
                cursor: 'pointer',
              }}
            >
              {todo.completed ? '✓ ' : '○ '} {todo.text}
            </span>
            <button onClick={() => deleteTodo(todo.id)}> Delete</button>
          </li>
        ))}
      </ul>

      <p>
        Total: {todos.length} | Done: {todos.filter(t => t.completed).length}
      </p>
    </div>
  );
}

export default TodoApp;
```

What you will see after adding some tasks and completing one:

```
My To-Do List

[Buy groceries______]  [Add]

✓  Buy milk (with line-through) [Delete]
○  Clean the house [Delete]
○  Call mom [Delete]

Total: 3 | Done: 1
```

How the To-Do List Works

Adding a task:

```jsx
const newTodo = {
  id: Date.now(),
  text: inputText,
  completed: false,
};
setTodos(prev => [...prev, newTodo]);
```

We create a new object for the task. `Date.now()` gives us a unique number (the current time in milliseconds) to use as an ID. We spread the previous todos and add the new one at the end.

Toggling a task (marking it complete or incomplete):

```jsx
setTodos(prev =>
  prev.map(todo => {
    if (todo.id === idToToggle) {
      return { ...todo, completed: !todo.completed };
    }
    return todo;
  })
);
```

We use `map` to go through every todo. When we find the one we want to toggle, we create a new object with the spread operator and flip the `completed` value. All other todos are returned unchanged.

Deleting a task:

```jsx
setTodos(prev => prev.filter(todo => todo.id !== idToDelete));
```

We use `filter` to create a new array that excludes the todo with the matching ID.

Notice that all three operations use the updater function form (`prev => ...`). This is a good habit. It ensures we always work with the most current state.

Quick Summary

Concept	What to Remember
State types	State can hold strings, numbers, booleans, arrays, and objects
Adding to an array	`setItems(prev => [...prev, newItem])`
Removing from an array	`setItems(prev => prev.filter(...))`
Updating an item in an array	`setItems(prev => prev.map(...))`
Updating an object	`setObj(prev => ({ ...prev, key: newValue }))`
Multiple useState	Each call creates independent state
Updater function	`setState(prev => newValue)` for state that depends on old state

Never mutate	Always create new arrays and objects

Key Points to Remember

1. Never mutate state directly. Do not use `push`, `pop`, `shift`, `splice`, `sort`, or `reverse` on state arrays. Always create a new array with `map`, `filter`, `slice`, or the spread operator.
2. Use the spread operator to update objects. Copy everything first, then override what you want to change: `{ ...oldObject, key: newValue }`.
3. State updates are not instant. The new value is available on the next render, not on the next line of code.
4. Use the updater function (`prev => ...`) when your new state depends on the previous state. This avoids bugs with stale values.
5. Use `Date.now()` or a counter for unique IDs. Every item in a list needs a unique `key`, and using the array index is not ideal when items can be added or removed.

Practice Questions

1. Why should you never use `push()` to add an item to a state array? What should you do instead?
2. What is the difference between `setCount(count + 1)` and `setCount(prev => prev + 1)`? When does the difference matter?
3. If your state is `{ name: 'Sara', age: 28 }` and you call `setUser({ age: 29 })`, what happens to the `name` property? How do you fix this?
4. When should you use multiple `useState` calls versus a single state object? Give an example of each.
5. What are three array methods that are safe to use with state and three that are not safe? Why?

Exercises

Exercise 1: Shopping List

Build a shopping list app where you can:

- Type an item name and click "Add" to add it to the list
- Click "Remove" next to an item to delete it
- Show the total number of items

Exercise 2: Score Tracker

Build a score tracker for two players:

- Show each player's name and score (start at 0)
- Have "+1" and "-1" buttons for each player
- Have a "Reset" button that sets both scores back to 0
- Display who is winning or if it is a tie

Exercise 3: Enhanced To-Do List

Extend the to-do list from this chapter:

- Add a "Clear Completed" button that removes all completed tasks at once
- Add a count showing "3 of 5 tasks completed"
- Add an "Edit" button next to each task that lets you change the task text

---

What Is Next?

You now have a solid understanding of `useState`. You can manage strings, numbers, booleans, arrays, and objects. You know how to add, remove, and update items. You know about the updater function and common mistakes to avoid.

In the next chapter, we will learn about `useEffect`, React's other most important hook. While `useState` lets you remember things, `useEffect` lets you do things at the right time, like fetching data when a page loads, setting up timers, or changing the page title. It is the key to making your components interact with the outside world.

Chapter 15: useEffect — Doing Things at the Right Time

What You Will Learn

- What side effects are in React
- How the useEffect hook works
- How to run code once when a component first appears
- How to run code when specific values change
- How to clean up after your effects (cancel timers, remove listeners)
- How to change the document title based on state
- How to set up a timer with setInterval
- Common mistakes with useEffect and how to avoid them
- How to read the dependency array diagram

Why This Chapter Matters

So far, your components have been doing one thing: rendering what the user sees on the screen. But real applications need to do more than just show things. They need to:

- Fetch data from the internet when a page loads
- Start a timer that counts down
- Change the title of the browser tab
- Listen for keyboard shortcuts
- Save data to the browser's storage

These are all things that happen outside of rendering. React calls them side effects. The `useEffect` hook is how you tell React: "After you finish rendering, also do this other thing." It is one of the most important hooks in React, and understanding it well will make you a much

stronger developer.

What Are Side Effects?

Let us start with a simple idea. A React component has one main job: take some data (state, props) and return some JSX (what to show on the screen). That is rendering.

A side effect is anything else your component does that is not about returning JSX. It is anything that reaches outside of the component to interact with the outside world.

Here are examples of side effects:

- Fetching data from an API (talking to a server on the internet)
- Setting a timer (like `setTimeout` or `setInterval`)
- Changing the document title (the text in the browser tab)
- Writing to the browser's local storage (saving data locally)
- Adding an event listener to the window (like listening for key presses)
- Logging something to the console

A Real-Life Analogy

Think of a waiter at a restaurant.

The waiter's main job is to take your order and bring you food. That is like rendering. Take the data (your order) and show the result (your food).

But the waiter also does other things:

- Turns on the restaurant's music when they arrive (setup)
- Wipes the tables between customers (cleanup)
- Checks the weather to decide if the patio is open (external data)

These are side effects. They are not the main job, but they are necessary.

In React, we use `useEffect` to handle these side effects.

The useEffect Hook

Here is the basic shape of `useEffect`:

```jsx
import { useEffect } from 'react';

useEffect(() => {
  // Your side effect code goes here
});
```

You pass a function to `useEffect`. React will run that function after the component renders. Not before, not during. After.

Think of it as telling React: "Hey React, after you finish showing stuff on the screen, please also do this thing for me."

Your First useEffect

Let us start with a simple example: changing the document title.

```jsx
import { useState, useEffect } from 'react';

function Counter() {
  const [count, setCount] = useState(0);

  useEffect(() => {
    document.title = 'Count: ' + count;
  });

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>Add 1</button>
    </div>
  );
}

export default Counter;
```

What happens:

When you open this page, the browser tab title changes to "Count: 0". Every time you click the button, the tab title updates: "Count: 1", "Count: 2", and so on.

Line-by-Line Explanation

```jsx
import { useState, useEffect } from 'react';
```

We import both `useState` and `useEffect` from React.

```jsx
useEffect(() => {
  document.title = 'Count: ' + count;
});
```

After every render, React runs this function. It changes the browser tab title to show the current count. `document.title` is a built-in browser feature that lets you change the text in the browser tab.

Without the second argument (the dependency array, which we will learn about next), this effect runs after every single render. That is often more than we need.

The Dependency Array

The dependency array is the second argument to `useEffect`. It is an array that tells React: "Only run this effect when these specific values change."

There are three ways to use the dependency array:

1. No Dependency Array — Runs After Every Render

```jsx
useEffect(() => {
  console.log('I run after every render');
});
```

This runs after the first render, and again after every re-render. Use this rarely. It can cause performance problems.

2. Empty Dependency Array `[]` — Runs Once

```jsx
useEffect(() => {
  console.log('I run only once, when the component first appears');
}, []);
```

The empty array `[]` tells React: "There is nothing I depend on. Run me once when the component first appears, and never again."

This is perfect for:

- Fetching data when a page loads
- Setting up something one time
- Logging that a component has appeared

Think of it like setting up a new apartment. You unpack your boxes once when you move in. You do not unpack them again every time you walk through the door.

3. Dependency Array with Values — Runs When Those Values Change

```jsx
useEffect(() => {
  console.log('Count changed to:', count);
}, [count]);
```

This tells React: "Run this effect when `count` changes." It runs once when the component first appears, and then again every time `count` gets a new value. It does NOT run when other state values change.

```jsx
useEffect(() => {
  document.title = 'Count: ' + count;
}, [count]);
```

This is better than our earlier example. The document title only updates when `count` changes, not on every render.

Dependency Array Diagram

Here is a visual summary:

```
useEffect(() => { ... });           No array → Runs after EVERY render
                                     ┌──────────────────────────────┐
                                     │ Render 1: runs               │
                                     │ Render 2: runs               │
                                     │ Render 3: runs               │
                                     │ Every render: runs           │
                                     └──────────────────────────────┘

useEffect(() => { ... }, []);       Empty array → Runs ONCE
                                     ┌──────────────────────────────┐
                                     │ Render 1: runs               │
                                     │ Render 2: does NOT run       │
                                     │ Render 3: does NOT run       │
                                     │ Only first render            │
                                     └──────────────────────────────┘
```

```
useEffect(() => { ... }, [count]);  With values → Runs when value CHANGES
                                    ┌──────────────────────────────────┐
                                    │ Render 1: runs (initial)         │
                                    │ Render 2: runs IF count changed│
                                    │ Render 3: skips if count same    │
                                    │ Only when count changes          │
                                    └──────────────────────────────────┘
```

---

useEffect with an Empty Dependency Array

This is one of the most common patterns. Let us see a clear example.

```jsx
import { useState, useEffect } from 'react';

function WelcomeMessage() {
  const [message, setMessage] = useState('');

  useEffect(() => {
    console.log('Component appeared on the screen!');
    setMessage('Welcome! This page loaded at ' + new Date().toLocaleTimeString());
  }, []);

  return <p>{message}</p>;
}

export default WelcomeMessage;
```

What you will see:

```
Welcome! This page loaded at 2:30:45 PM
```

The time is set once when the component first appears. No matter how many times the component re-renders (because of other state changes), this effect does not run again. The time stays the same.

---

useEffect with Dependencies

Let us build a component that shows a message based on a count, but only recalculates the message when the count changes.

```jsx
import { useState, useEffect } from 'react';

function CountMessage() {
  const [count, setCount] = useState(0);
  const [name, setName] = useState('');
  const [message, setMessage] = useState('');

  useEffect(() => {
    console.log('Count changed! Updating message...');
    if (count === 0) {
      setMessage('You have not clicked yet.');
    } else if (count < 5) {
      setMessage('Keep clicking!');
    } else if (count < 10) {
      setMessage('You are doing great!');
    } else {
      setMessage('You are a clicking champion!');
    }
  }, [count]);

  return (
    <div>
      <input
        type="text"
        value={name}
        onChange={(e) => setName(e.target.value)}
        placeholder="Type your name"
      />
      <p>Hello, {name}!</p>

      <p>Count: {count}</p>
      <p>{message}</p>
      <button onClick={() => setCount(prev => prev + 1)}>Click Me</button>
    </div>
  );
}

export default CountMessage;
```

What you will see:

```
[Type your name___]
Hello, Sara!

Count: 3
Keep clicking!
[Click Me]
```

The important thing to notice: when you type in the name input, the component re-renders, but the `useEffect` does NOT run. It only runs when `count` changes. You can check the console — you will only see "Count changed!" when you click the button, not when you type.

The Cleanup Function

Sometimes your side effects create things that need to be cleaned up. For example:

- You start a timer. When the component disappears, you should stop the timer.
- You add a keyboard listener. When the component disappears, you should remove it.
- You open a connection. When the component disappears, you should close it.

If you do not clean up, you can have memory leaks (the program keeps using memory for things that are no longer needed) or bugs (old timers still running in the background).

A Real-Life Analogy

Think of throwing a party at your house.

- The effect: You set up decorations, play music, and prepare food.
- The cleanup: After the party, you take down the decorations, stop the music, and clean up the food.

If you never clean up, your house gets messier and messier with each party. The same thing happens in your app if you do not clean up side effects.

How Cleanup Works

To add cleanup, you return a function from your effect. React will call this function before the effect runs again, and also when the component is removed from the screen.

```jsx
useEffect(() => {
  // Setup: do something
  console.log('Effect runs');

  return () => {
    // Cleanup: undo what you did
    console.log('Cleanup runs');
  };
}, []);
```

The flow looks like this:

```
1. Component appears → Effect runs (setup)
2. Component disappears → Cleanup runs
```

For effects with dependencies:

```
1. Component appears → Effect runs (setup)
2. Dependency changes → Cleanup runs (for the old effect), then Effect runs again (new ...
3. Component disappears → Cleanup runs (final cleanup)
```

Example: Changing the Document Title

Let us combine what we have learned into a clean example:

```jsx
import { useState, useEffect } from 'react';

function PageTitle() {
  const [title, setTitle] = useState('My App');

  useEffect(() => {
    document.title = title;
  }, [title]);

  return (
    <div>
      <label>Page title: </label>
      <input
        type="text"
        value={title}
        onChange={(event) => setTitle(event.target.value)}
      />
      <p>Look at the browser tab to see the title change!</p>
    </div>
  );
}

export default PageTitle;
```

What happens:

As you type in the input, the browser tab title updates in real time. If you type "Hello World", the browser tab will say "Hello World".

This is a side effect because changing the document title is something outside of what React renders. React draws the input and the paragraph. The document title is part of the browser,

not part of your component's JSX.

Example: Setting Up a Timer

Timers are a classic use case for `useEffect`. Let us build a simple stopwatch.

```jsx
import { useState, useEffect } from 'react';

function Stopwatch() {
  const [seconds, setSeconds] = useState(0);
  const [isRunning, setIsRunning] = useState(false);

  useEffect(() => {
    if (!isRunning) return;

    const intervalId = setInterval(() => {
      setSeconds(prev => prev + 1);
    }, 1000);

    return () => {
      clearInterval(intervalId);
    };
  }, [isRunning]);

  function reset() {
    setIsRunning(false);
    setSeconds(0);
  }

  return (
    <div>
      <p>Time: {seconds} seconds</p>
      <button onClick={() => setIsRunning(true)}>Start</button>
      <button onClick={() => setIsRunning(false)}>Stop</button>
      <button onClick={reset}>Reset</button>
    </div>
  );
}

export default Stopwatch;
```

What you will see:

```
Time: 7 seconds
[Start]  [Stop]  [Reset]
```

Line-by-Line Explanation

```jsx
useEffect(() => {
  if (!isRunning) return;
```

If the stopwatch is not running, we do nothing. We exit the effect early.

```jsx
const intervalId = setInterval(() => {
  setSeconds(prev => prev + 1);
}, 1000);
```

`setInterval` is a built-in JavaScript function. It runs the given function every 1000 milliseconds (1 second). Each second, it adds 1 to the seconds count. We use the updater function (`prev => prev + 1`) to always get the latest value.

`setInterval` returns an ID number. We store this ID so we can stop the timer later.

```jsx
return () => {
  clearInterval(intervalId);
};
```

This is the cleanup function. `clearInterval` stops the timer. This runs when:

- The component disappears from the screen
- The `isRunning` value changes (React cleans up the old effect before running the new one)

```jsx
}, [isRunning]);
```

This effect depends on `isRunning`. When `isRunning` changes from `false` to `true`, the effect runs and starts the timer. When `isRunning` changes from `true` to `false`, the cleanup runs and stops the timer.

Why Cleanup Is Critical Here

Without the cleanup function, every time `isRunning` changes to `true`, a new timer would start. But the old timer would still be running. You would have two timers, then three, then four. The seconds would increase faster and faster. The cleanup ensures only one timer runs at a time.

Common Mistakes with useEffect

Mistake 1: Infinite Loop (Missing or Wrong Dependencies)

This is the most common mistake. It can freeze your browser.

```jsx
// DANGER — INFINITE LOOP
useEffect(() => {
  setCount(count + 1);
}, [count]);
```

What happens here:

1. Component renders. `count` is 0.
2. Effect runs. Sets `count` to 1.
3. `count` changed, so the component re-renders.
4. Effect runs again. Sets `count` to 2.
5. `count` changed again, so the component re-renders.
6. This never stops!

The effect depends on `count`, and it also changes `count`. That creates an infinite loop.

How to avoid it: Be careful about updating a state variable inside an effect that depends on that same variable. Ask yourself: "Will this update trigger the effect again?" If yes, you probably have a problem.

Mistake 2: Missing Dependencies

```jsx
// WARNING — Stale value
useEffect(() => {
  const intervalId = setInterval(() => {
    console.log('Count is:', count);  // This will always show the old value!
  }, 1000);

  return () => clearInterval(intervalId);
}, []);  // Empty array means this effect never re-runs
```

Because the dependency array is empty, this effect runs once with the initial value of `count`. The interval will always log the initial value, even as `count` changes.

Fix: Either add `count` to the dependency array, or use the updater function form if you are setting state.

Mistake 3: Forgetting Cleanup

```jsx
// BAD — No cleanup!
useEffect(() => {
  setInterval(() => {
    console.log('tick');
  }, 1000);
}, []);
```

This starts a timer but never stops it. If this component appears and disappears multiple times (which happens often in React), you will have multiple timers running, all logging "tick" forever.

Fix: Always return a cleanup function when you set up timers, listeners, or subscriptions.

```jsx
// GOOD — With cleanup
useEffect(() => {
  const id = setInterval(() => {
    console.log('tick');
  }, 1000);

  return () => clearInterval(id);  // Stop the timer when cleaning up
}, []);
```

Mistake 4: Putting Too Much in One Effect

```jsx
// Not ideal — too many responsibilities
useEffect(() => {
  document.title = 'Count: ' + count;
  fetchData();
  setupTimer();
  trackAnalytics();
}, [count]);
```

If only the document title depends on `count`, but the other functions do not, they will all run every time `count` changes unnecessarily.

Fix: Use multiple `useEffect` calls. Each one handles one concern.

```jsx
useEffect(() => {
  document.title = 'Count: ' + count;
}, [count]);

useEffect(() => {
```

```
  fetchData();
}, []);

useEffect(() => {
  const id = setupTimer();
  return () => clearInterval(id);
}, []);
```

It is perfectly fine and even recommended to have multiple `useEffect` calls in one component.

A Complete Example: Window Width Tracker

Here is a practical example that uses setup, cleanup, and dependencies:

```jsx
import { useState, useEffect } from 'react';

function WindowWidth() {
  const [width, setWidth] = useState(window.innerWidth);

  useEffect(() => {
    function handleResize() {
      setWidth(window.innerWidth);
    }

    window.addEventListener('resize', handleResize);

    return () => {
      window.removeEventListener('resize', handleResize);
    };
  }, []);

  return (
    <div>
      <p>Window width: {width}px</p>
      <p>
        {width > 768 ? 'Desktop view' : 'Mobile view'}
      </p>
    </div>
  );
}

export default WindowWidth;
```

What you will see:

```
Window width: 1024px
Desktop view
```

When you resize the browser window, the width updates in real time.

How It Works

- Setup: We add a "resize" event listener to the window. This listener updates our state whenever the window size changes.
- Cleanup: We remove the event listener. This prevents memory leaks if the component disappears.
- Empty dependency array: We only need to set up the listener once. The listener itself will keep working until we remove it.

Quick Summary

Concept	What It Does
Side effect	Anything outside of rendering (fetching data, timers, changing the title)
`useEffect(() => {})`	Runs after every render
`useEffect(() => {}, [])`	Runs once when the component first appears
`useEffect(() => {}, [x])`	Runs when the value `x` changes
Cleanup function	Returned from the effect; runs before next effect and on unmount
`clearInterval`	Stops a timer created by `setInterval`
`removeEventListener`	Removes a listener added by `addEventListener`

Key Points to Remember

1. useEffect runs after rendering. React first updates the screen, then runs your effects. Your effects never block the screen from updating.
2. The dependency array controls when the effect runs. No array means every render. Empty array means once. Array with values means when those values change.
3. Always clean up timers and listeners. Return a cleanup function from your effect to stop timers (`clearInterval`), remove listeners (`removeEventListener`), or cancel any ongoing work.
4. Do not update state that is in your own dependency array. This creates an infinite loop. If you see your app freeze or your browser slow down, check for this mistake.
5. Use multiple effects for separate concerns. It is better to have three small, focused effects than one large effect that does everything.

Practice Questions

1. What is a side effect in React? Give three examples.
2. What is the difference between `useEffect(() => { ... })` with no second argument and `useEffect(() => { ... }, [])` with an empty array?
3. Why do we need a cleanup function? What happens if we forget to clean up a timer?
4. Look at this code: `useEffect(() => { setName(name + '!'); }, [name]);`. What will happen? Why?
5. Can you have multiple `useEffect` calls in one component? Is that a good practice or a bad practice?

Exercises

Exercise 1: Dynamic Page Title

Create a component with a text input. As the user types, the browser tab title should update to show what they typed. When the input is empty, the tab title should say "Untitled Page".

Exercise 2: Countdown Timer

Create a countdown timer that:

- Starts at 10 seconds

- Counts down by 1 every second
- Shows "Time is up!" when it reaches 0 and stops the timer
- Has a "Reset" button that restarts the countdown from 10

Remember to clean up the interval properly.

Exercise 3: Online Status

Create a component that:

- Tracks if the browser is online or offline
- Shows "You are online" (in green) or "You are offline" (in red)
- Uses `window.addEventListener('online', ...)` and `window.addEventListener('offline', ...)`
- Properly cleans up the event listeners

Hint: `navigator.onLine` gives you the initial online/offline status.

What Is Next?

Now you know how to do things at the right time with `useEffect`. You can run code when a component appears, when values change, and clean up when a component disappears.

In the next chapter, we will use `useEffect` for one of its most important jobs: fetching data from the internet. You will learn what APIs are, how to ask them for data, and how to display that data in your React components. This is where your apps start feeling like real applications.

Chapter 16: Simple API Calling

What You Will Learn

- What an API is and why it matters
- What JSON is and how to read it
- How to use the `fetch()` function to ask for data
- What `async` and `await` mean and how to use them
- How to handle loading, success, and error states
- How to fetch data when a component loads using `useEffect`
- How to display API data in a list
- How to handle errors gracefully
- How to build an app that fetches and displays random user profiles

Why This Chapter Matters

Up until now, all the data in your apps has come from you, the developer. You typed the to-do items. You typed the names. You typed the fruits. But real applications get their data from the internet. When you open a weather app, it fetches the weather from a server. When you open Instagram, it fetches posts from a server. When you search on Google, it fetches results from a server.

This chapter teaches you how to connect your React app to the outside world. You will learn how to ask a server for data and display it. This is a huge step. After this chapter, your apps will feel like real applications.

What Is an API?

API stands for Application Programming Interface. That is a big name, but the idea is simple.

The Restaurant Analogy

Think of a restaurant. You (the customer) are sitting at a table. The kitchen is in the back, preparing food. You cannot go into the kitchen and cook your own food. You need someone in between.

That someone is the waiter.

- You look at the menu and tell the waiter what you want.
- The waiter goes to the kitchen and places your order.
- The kitchen prepares your food.
- The waiter brings the food back to you.

An API is like the waiter:

- You are the React app (the front end).
- The kitchen is the server (the back end), where data is stored.
- The waiter is the API. It takes your request to the server and brings back the data.

You do not need to know how the kitchen works. You just need to know how to talk to the waiter (the API). You make a request, and you get a response.

What Kind of Data Do APIs Provide?

APIs can provide all kinds of data:

- Weather information
- News articles
- User profiles
- Product listings
- Movie information
- Stock prices
- Cat pictures (yes, there is an API for that)

There are thousands of free APIs on the internet that you can use to practice.

What Is JSON?

When an API sends you data, it sends it in a specific format. The most common format is JSON, which stands for JavaScript Object Notation.

JSON looks a lot like JavaScript objects and arrays. Here is an example:

```json
{
  "name": "Sara",
  "age": 28,
  "city": "New York",
  "hobbies": ["reading", "coding", "hiking"]
}
```

You can read this easily, right? It has a name, an age, a city, and a list of hobbies. That is the beauty of JSON. It is structured in a way that both humans and computers can understand.

JSON Rules

JSON has a few simple rules:

- Keys must be in double quotes: `"name"`, not `name` or `'name'`.
- Strings must be in double quotes: `"Sara"`, not `'Sara'`.
- Numbers do not have quotes: `28`, not `"28"`.
- Booleans are `true` or `false`: no quotes.
- Arrays use square brackets: `["a", "b", "c"]`.
- Objects use curly braces: `{ "key": "value" }`.

When your React app receives JSON from an API, JavaScript can easily turn it into a regular JavaScript object that you can work with.

The fetch() Function

JavaScript has a built-in function called `fetch()` that lets you make requests to APIs. You give it a URL (the address of the API), and it goes out to the internet, asks for the data, and brings it back.

Here is the simplest possible fetch:

```jsx
fetch('https://jsonplaceholder.typicode.com/users/1')
  .then(response => response.json())
  .then(data => console.log(data));
```

This asks a free practice API for user number 1, converts the response to JSON, and logs it to the console.

But this syntax with `.then()` can get confusing, especially when you have many steps. There is a cleaner way to write it.

---

async and await — A Simpler Way to Fetch

`async` and `await` are JavaScript features that make it easier to work with operations that take time (like fetching data from the internet).

The Analogy

Imagine you order a coffee at a coffee shop.

- You place your order.
- You wait for the barista to make it.
- When it is ready, you take your coffee and move on.

That is exactly how `async/await` works:

- You make a request (fetch).
- You await the response (wait for the server to reply).
- When the response arrives, you use the data.

How It Looks in Code

```jsx
async function getUser() {
  const response = await fetch('https://jsonplaceholder.typicode.com/users/1');
  const data = await response.json();
  console.log(data);
}
```

Line-by-Line Explanation

```jsx
async function getUser() {
```

The `async` keyword before the function tells JavaScript: "This function will do something that takes time. It will use `await` inside."

```jsx
const response = await fetch('https://jsonplaceholder.typicode.com/users/1');
```

`fetch()` goes to the internet and asks the API for data. This takes time (maybe half a second, maybe a few seconds). The `await` keyword says: "Wait here until the data comes back. Do not move to the next line until it is ready."

The `response` is what the server sends back. It contains the data, but we need to extract it.

```jsx
const data = await response.json();
```

`response.json()` reads the response and converts it from raw text into a JavaScript object we can use. This also takes a moment, so we `await` it too.

```jsx
console.log(data);
```

Now `data` is a regular JavaScript object. We can access its properties like `data.name`, `data.email`, and so on.

The Three States of Fetching Data

When your app fetches data, it goes through three stages. Think of ordering a package online:

1. Loading: You placed the order. The package is on its way. You see "Your order is being delivered."
2. Success: The package arrived! You open it and see your items.
3. Error: Something went wrong. The package got lost. You see "Delivery failed."

In your React app, you need to handle all three states:

```
Loading → Show "Loading..." or a spinner
Success → Show the data
Error   → Show an error message
```

Here is how to do this with state:

```jsx
const [data, setData] = useState(null);        // The fetched data
const [isLoading, setIsLoading] = useState(true);  // Are we still waiting?
const [error, setError] = useState(null);      // Did something go wrong?
```

Fetching Data When a Component Loads

This is where `useEffect` meets `fetch`. We want to fetch data as soon as the component appears on the screen. That is a side effect, and we run it with `useEffect` and an empty dependency array.

```jsx
import { useState, useEffect } from 'react';

function UserProfile() {
  const [user, setUser] = useState(null);
  const [isLoading, setIsLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    async function fetchUser() {
      try {
        const response = await fetch(
          'https://jsonplaceholder.typicode.com/users/1'
        );

        if (!response.ok) {
          throw new Error('Something went wrong!');
        }

        const data = await response.json();
        setUser(data);
      } catch (err) {
        setError(err.message);
      } finally {
        setIsLoading(false);
      }
    }

    fetchUser();
  }, []);

  if (isLoading) {
    return <p>Loading...</p>;
  }
```

```
  if (error) {
    return <p>Error: {error}</p>;
  }

  return (
    <div>
      <h2>{user.name}</h2>
      <p>Email: {user.email}</p>
      <p>Phone: {user.phone}</p>
      <p>City: {user.address.city}</p>
    </div>
  );
}

export default UserProfile;
```

What you will see while loading:

```
Loading...
```

What you will see after the data arrives:

```
Leanne Graham
Email: Sincere@april.biz
Phone: 1-770-736-8031 x56442
City: Gwenborough
```

Line-by-Line Explanation

```jsx
useEffect(() => {
  async function fetchUser() {
```

We cannot make the `useEffect` callback itself `async`. React does not allow that. Instead, we create an `async` function inside the effect and then call it.

```jsx
try {
```

`try` means "try to do the following code. If anything goes wrong, jump to the `catch` block." This is how we handle errors.

```jsx
const response = await fetch('https://jsonplaceholder.typicode.com/users/1');
```

We ask the API for user number 1. We wait for the response.

```jsx
if (!response.ok) {
  throw new Error('Something went wrong!');
}
```

`response.ok` is `true` if the server responded successfully (status code 200). If the server returned an error (like 404 Not Found or 500 Server Error), `response.ok` is `false`. In that case, we `throw` an error, which jumps to the `catch` block.

```jsx
const data = await response.json();
setUser(data);
```

We convert the response to JSON and store it in state.

```jsx
} catch (err) {
  setError(err.message);
}
```

If anything went wrong (network error, server error, bad JSON), we land here. We save the error message in state.

```jsx
} finally {
  setIsLoading(false);
}
```

`finally` runs no matter what, whether the fetch succeeded or failed. We set loading to `false` because either way, we are done waiting.

```jsx
fetchUser();
```

We call the function we just defined. This starts the fetch.

```jsx
}, []);
```

Empty dependency array. Fetch once when the component appears.

Why We Check Three States in the Return

```jsx
if (isLoading) return <p>Loading...</p>;
if (error) return <p>Error: {error}</p>;
```

These are called early returns. They check the state before trying to display the data. If we did not check and tried to show `user.name` while `user` is still `null` (because the data has not arrived yet), we would get a crash.

Displaying a List of Data from an API

Let us fetch a list of users and display them.

```jsx
import { useState, useEffect } from 'react';

function UserList() {
  const [users, setUsers] = useState([]);
  const [isLoading, setIsLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    async function fetchUsers() {
      try {
        const response = await fetch(
          'https://jsonplaceholder.typicode.com/users'
        );

        if (!response.ok) {
          throw new Error('Failed to fetch users.');
        }

        const data = await response.json();
        setUsers(data);
      } catch (err) {
        setError(err.message);
      } finally {
        setIsLoading(false);
      }
    }

    fetchUsers();
  }, []);

  if (isLoading) return <p>Loading users...</p>;
  if (error) return <p>Error: {error}</p>;

  return (
```

```
    <div>
      <h2>User Directory</h2>
      <ul>
        {users.map(user => (
          <li key={user.id}>
            <strong>{user.name}</strong> — {user.email}
          </li>
        ))}
      </ul>
      <p>Total users: {users.length}</p>
    </div>
  );
}

export default UserList;
```

What you will see:

```
User Directory

- Leanne Graham — Sincere@april.biz
- Ervin Howell — Shanna@melissa.tv
- Clementine Bauch — Nathan@yesenia.net
  ... (10 users total)

Total users: 10
```

The pattern is the same as fetching a single user. The only difference is that the API returns an array instead of a single object, so we use `map` to render each user as a list item.

Handling Errors Gracefully

Things can go wrong when fetching data. The internet might be slow. The server might be down. The API might have moved. Good applications handle these situations gracefully instead of crashing.

Here is a more user-friendly error handling approach:

```jsx
import { useState, useEffect } from 'react';

function PostViewer() {
  const [post, setPost] = useState(null);
```

```
  const [isLoading, setIsLoading] = useState(true);
  const [error, setError] = useState(null);

  async function fetchPost() {
    setIsLoading(true);
    setError(null);

    try {
      const response = await fetch(
        'https://jsonplaceholder.typicode.com/posts/1'
      );

      if (!response.ok) {
        throw new Error('Could not load the post. Please try again.');
      }

      const data = await response.json();
      setPost(data);
    } catch (err) {
      setError(err.message);
    } finally {
      setIsLoading(false);
    }
  }

  useEffect(() => {
    fetchPost();
  }, []);

  if (isLoading) {
    return <p>Loading post...</p>;
  }

  if (error) {
    return (
      <div>
        <p style={{ color: 'red' }}>Error: {error}</p>
        <button onClick={fetchPost}>Try Again</button>
      </div>
    );
  }

  return (
    <div>
      <h2>{post.title}</h2>
      <p>{post.body}</p>
    </div>
  );
}

export default PostViewer;
```

What you will see if the fetch fails:

```
Error: Could not load the post. Please try again.
[Try Again]
```

The "Try Again" button calls `fetchPost` again. This gives the user a way to retry without refreshing the entire page. Notice that we reset `isLoading` to `true` and `error` to `null` before retrying so the loading state shows again.

Understanding the Full Fetch Pattern

Here is the pattern you will use over and over again. It is worth memorizing:

```
1. Set up three state variables: data, isLoading, error
2. Create an async function that fetches data
3. Inside the function:
   a. try to fetch
   b. check if the response is ok
   c. convert to JSON
   d. store in state
   e. catch any errors
   f. finally, set loading to false
4. Call the function inside useEffect with []
5. In the JSX:
   a. If loading, show a loading message
   b. If error, show an error message
   c. If data is ready, show the data
```

```
┌──────────────────────────────────────────┐
│         Component Renders                │
│                                          │
│  isLoading: true                         │
│  → Shows "Loading..."                   │
└──────────────────────────────────────────┘
                │
                │
                ▼
┌──────────────────────────────────────────┐
│      useEffect runs fetch()              │
│                                          │
│  Asks the API for data...               │
└──────────────────────────────────────────┘
                │
                │
        ┌───────┴───────┐
        ▼               ▼
   ┌─────────┐   ┌──────────┐
   │ Success │   │  Error   │
   │         │   │          │
```

```
    │ setData │   │ setError │
    │ setLoad │   │ setLoad  │
    │  false  │   │  false   │
    └─────────┘   └──────────┘
         │             │
         ▼             ▼
    ┌─────────┐   ┌──────────┐
    │ Shows   │   │ Shows    │
    │ the     │   │ error    │
    │ data    │   │ message  │
    └─────────┘   └──────────┘
```

Mini Project: Random User Profiles

Let us build an app that fetches random user profiles from an API and displays them. You can click a button to load a new random user.

We will use the Random User API (`https://randomuser.me/api/`), which provides randomly generated user profiles including names, photos, locations, and more.

```jsx
import { useState, useEffect } from 'react';

function RandomUser() {
  const [user, setUser] = useState(null);
  const [isLoading, setIsLoading] = useState(true);
  const [error, setError] = useState(null);

  async function fetchRandomUser() {
    setIsLoading(true);
    setError(null);

    try {
      const response = await fetch('https://randomuser.me/api/');

      if (!response.ok) {
        throw new Error('Failed to fetch user.');
      }

      const data = await response.json();
      setUser(data.results[0]);
    } catch (err) {
      setError(err.message);
    } finally {
      setIsLoading(false);
    }
```

```
  }

  useEffect(() => {
    fetchRandomUser();
  }, []);

  if (isLoading) {
    return <p>Loading user profile...</p>;
  }

  if (error) {
    return (
      <div>
        <p style={{ color: 'red' }}>Error: {error}</p>
        <button onClick={fetchRandomUser}>Try Again</button>
      </div>
    );
  }

  return (
    <div style={{ textAlign: 'center', maxWidth: '400px', margin: '0 auto' }}>
      <h2>Random User Profile</h2>

      <img
        src={user.picture.large}
        alt={user.name.first}
        style={{ borderRadius: '50%', width: '150px', height: '150px' }}
      />

      <h3>
        {user.name.first} {user.name.last}
      </h3>

      <p>Email: {user.email}</p>
      <p>
        Location: {user.location.city}, {user.location.country}
      </p>
      <p>Phone: {user.phone}</p>

      <button onClick={fetchRandomUser} style={{ marginTop: '10px' }}>
        Load Another User
      </button>
    </div>
  );
}

export default RandomUser;
```

What you will see while loading:

```
Loading user profile...
```

What you will see after the data arrives:

```
          Random User Profile

          [Round Photo]

          Emma Johnson

Email: emma.johnson@example.com
Location: Dublin, Ireland
Phone: 041-232-5678

    [Load Another User]
```

Clicking "Load Another User" fetches a brand new random user from the API.

How It Works

```jsx
const data = await response.json();
setUser(data.results[0]);
```

The Random User API wraps its data in an object with a `results` array. Even though we asked for one user, it comes inside an array. We take the first (and only) item with `[0]`.

```jsx
<img
  src={user.picture.large}
  alt={user.name.first}
  style={{ borderRadius: '50%', width: '150px', height: '150px' }}
/>
```

The API provides a photo URL in `user.picture.large`. We display it as a round image using `borderRadius: '50%'`.

```jsx
<button onClick={fetchRandomUser}>Load Another User</button>
```

This button calls the same `fetchRandomUser` function. It resets the loading state, makes a new request, and displays the new user. The entire flow (loading, success/error) happens again.

Why We Define fetchRandomUser Outside of useEffect

In the previous examples, we defined the fetch function inside the effect. Here, we defined it outside so that both the `useEffect` and the button's `onClick` can call the same function. This avoids repeating code.

Quick Summary

Concept	What It Is
API	A service that lets your app request data from a server
JSON	A text format for structured data (looks like JavaScript objects)
`fetch()`	A built-in function to make HTTP requests
`async/await`	A way to write asynchronous code that reads like regular code
`response.ok`	`true` if the server responded with a success status
`response.json()`	Converts the raw response into a JavaScript object
`try/catch`	Handles errors gracefully instead of crashing
Loading state	Tells the user data is on its way
Error state	Tells the user something went wrong
`useEffect + fetch`	The standard way to load data when a component appears

Key Points to Remember

1. Always handle three states: loading, success, and error. Never assume the fetch will succeed. The internet is unpredictable.
2. Use `async/await` inside `useEffect` by creating a separate function. You cannot make the `useEffect` callback itself `async`. Define an `async` function inside and call it.
3. Check `response.ok` before reading the data. A `fetch` call does not throw an error for 404 or 500 responses. You need to check manually.
4. Use try/catch/finally for error handling. `try` attempts the fetch. `catch` handles errors. `finally` always runs (good for setting loading to false).

5. Start with `isLoading: true` and set it to `false` when done. This way, the loading message shows while the data is being fetched, and it disappears when the data arrives or an error occurs.

Practice Questions

1. What is an API? Explain it using your own analogy (not the restaurant one from this chapter).
2. What is the difference between `response` and `response.json()`? Why do we need to call `.json()`?
3. Why can we not write `useEffect(async () => { ... })`? What do we do instead?
4. What are the three states of fetching data? Why do we need all three?
5. What happens if you forget to check `response.ok`? What kind of bugs could this cause?

Exercises

Exercise 1: Post Fetcher

Build a component that:

- Fetches a post from `https://jsonplaceholder.typicode.com/posts/1`
- Shows the post's title and body
- Has "Previous" and "Next" buttons that fetch post number 2, 3, 4, etc.
- Shows the current post number
- Handles loading and error states

Hint: Store the post ID in state and use it in the fetch URL. Put the post ID in the dependency array of `useEffect`.

Exercise 2: User Search

Build a component that:

- Fetches all users from `https://jsonplaceholder.typicode.com/users`
- Displays them in a list
- Has a search input that filters the list by name (client-side filtering, no new fetch needed)

- Shows "No users found" if the search does not match anyone

Exercise 3: Multi-Resource Viewer

Build a component with three buttons: "Users", "Posts", and "Todos". When the user clicks a button:

- Fetch the corresponding data from `https://jsonplaceholder.typicode.com/users`, `/posts`, or `/todos`
- Display the first 5 items from the result
- Show a loading state while fetching
- Show which category is currently selected

What Is Next?

Congratulations! You have just learned one of the most important skills in modern web development: fetching data from APIs. Your React apps can now connect to the outside world and display real data.

In the next chapter, we will explore how to reuse components effectively. You have already been building components, but you will learn patterns for making them more flexible and reusable so that one component can serve many different purposes across your application.

Chapter 17: Component Reuse

What You Will Learn

In this chapter, you will learn:

- Why reusing components saves you time and effort
- How to build a reusable Button component with props
- How to build a reusable Card component
- What composition means and how to put components inside other components
- How to use the special `children` prop for flexible content
- When to turn something into its own component
- How to organize reusable components in a folder
- How to build a product card grid using one reusable component

Why This Chapter Matters

Imagine you are baking cookies. You have a cookie cutter shaped like a star. You press it into the dough once, and you get a star cookie. You press it again, and you get another star cookie. You did not have to carve each star by hand. The cookie cutter lets you make the same shape over and over, quickly and perfectly.

Components in React work the same way. You write a component once, and then you use it as many times as you want. Each time, you can change small details (like the text or color), but the basic shape stays the same.

This idea is called reuse. Reuse means using something again instead of building it from scratch. It is one of the most powerful ideas in programming. It saves time. It reduces mistakes. It makes your code easier to fix, because you only need to fix one place instead of many.

This chapter teaches you how to think in reusable pieces. Once you learn this skill, you will build apps much faster.

Why Reuse Matters

Let us say you are building a website. The website has three buttons:

- A "Save" button
- A "Delete" button
- A "Cancel" button

Without reuse, you might write the code for each button separately. That means writing similar code three times. If you later want to change how buttons look, you have to change all three places. That is slow and easy to mess up.

With reuse, you write one Button component. Then you use it three times with different labels. If you want to change how buttons look, you change one file. All three buttons update automatically.

Think of it like a stamp. You create the stamp once. Then you press it onto paper as many times as you like. The stamp stays the same, but you can use different colored ink each time.

The Benefits of Reuse

1. Less code to write. You write the component once.
2. Fewer bugs. One component means one place to fix.
3. Consistent look. All buttons (or cards, or forms) look the same.
4. Faster development. You build new pages by combining existing pieces.

Making a Reusable Button Component

Let us build a simple, reusable Button component. This button will accept props so we can customize it each time we use it.

Step 1: Create the Button Component

```jsx
function Button({ label, onClick, color }) {
  const style = {
    backgroundColor: color,
    color: 'white',
    padding: '10px 20px',
    border: 'none',
    borderRadius: '5px',
    cursor: 'pointer',
    fontSize: '16px'
  };

  return (
    <button style={style} onClick={onClick}>
      {label}
    </button>
  );
}
```

Let us go through this line by line:

- `function Button({ label, onClick, color })` — This creates a component called Button. It receives three props: `label` (the text on the button), `onClick` (what happens when you click it), and `color` (the background color).
- `const style = { ... }` — This creates a style object. It sets the background color, text color, padding (space inside the button), and other visual properties.
- `backgroundColor: color` — This uses the `color` prop to set the background. Each button can have a different color.
- `<button style={style} onClick={onClick}>` — This renders an HTML button with our custom style and click behavior.
- `{label}` — This shows the text that was passed in as a prop.

Step 2: Use the Button Component

```jsx
function App() {
  function handleSave() {
    alert('Saved!');
  }

  function handleDelete() {
    alert('Deleted!');
  }

  function handleCancel() {
    alert('Cancelled!');
  }
```

```
  return (
    <div>
      <h1>My App</h1>
      <Button label="Save" onClick={handleSave} color="green" />
      <Button label="Delete" onClick={handleDelete} color="red" />
      <Button label="Cancel" onClick={handleCancel} color="gray" />
    </div>
  );
}
```

Let us go through the important parts:

- `<Button label="Save" onClick={handleSave} color="green" />` — This uses our Button component. We pass "Save" as the label, the `handleSave` function as the click handler, and "green" as the color.
- We use the same Button component three times, but each one looks and behaves differently because we pass different props.

Expected Output

You will see three buttons on the screen:

- A green button that says "Save"
- A red button that says "Delete"
- A gray button that says "Cancel"

When you click each button, a different alert message appears.

Adding Default Props

What if someone forgets to pass a color? We can set a default value:

```jsx
function Button({ label, onClick, color = 'blue' }) {
  // ... same code as before
}
```

Now if you write `<Button label="Click Me" onClick={handleClick} />` without a color, the button will be blue by default. The `= 'blue'` part sets the default. This is called a default parameter — a value that is used when nothing else is provided.

Making a Reusable Card Component

A card is a common design pattern. It is a box that holds some content, like a title, an image, and a description. You see cards everywhere on the web: product listings, blog posts, user profiles.

Let us build a reusable Card component:

```jsx
function Card({ title, description, imageUrl }) {
  const style = {
    border: '1px solid #ddd',
    borderRadius: '8px',
    padding: '16px',
    maxWidth: '300px',
    boxShadow: '0 2px 4px rgba(0,0,0,0.1)'
  };

  const imageStyle = {
    width: '100%',
    borderRadius: '4px'
  };

  return (
    <div style={style}>
      {imageUrl && <img src={imageUrl} alt={title} style={imageStyle} />}
      <h2>{title}</h2>
      <p>{description}</p>
    </div>
  );
}
```

Line by line:

- `function Card({ title, description, imageUrl })` — Our Card accepts three props: a title, a description, and an optional image URL.
- `const style = { ... }` — This creates a box with a light border, rounded corners, padding, a maximum width, and a subtle shadow.
- `{imageUrl && <img ... />}` — This is a neat trick. It says: "If `imageUrl` exists, show the image. If it does not exist, show nothing." The `&&` operator works like "and" — both sides must be true. If `imageUrl` is empty, React skips the image.
- `<h2>{title}</h2>` — This shows the title in a heading.
- `<p>{description}</p>` — This shows the description in a paragraph.

Using the Card Component

```jsx
function App() {
  return (
```

```
    <div>
      <Card
        title="Beautiful Sunset"
        description="A photo of the sunset over the ocean."
        imageUrl="https://example.com/sunset.jpg"
      />
      <Card
        title="Mountain View"
        description="Snow-covered peaks in the morning light."
        imageUrl="https://example.com/mountain.jpg"
      />
      <Card
        title="Simple Note"
        description="This card has no image, and that is okay!"
      />
    </div>
  );
}
```

Expected Output

You will see three cards stacked on the page:

- The first card shows a sunset image, the title "Beautiful Sunset", and its description.
- The second card shows a mountain image, the title "Mountain View", and its description.
- The third card has no image (because we did not pass `imageUrl`), just the title "Simple Note" and its description.

Composition: Putting Components Inside Other Components

Composition is a big word that means a simple thing: building something by combining smaller parts. Think of building with LEGO bricks. Each brick is simple on its own, but when you put them together, you can build houses, cars, or castles.

In React, composition means putting components inside other components. You have already been doing this! Every time you put a `<Button />` inside an `<App />`, you were using composition.

Let us see a clear example:

```jsx
function Header() {
  return (
    <header>
      <h1>My Website</h1>
      <nav>
        <Button label="Home" onClick={() => {}} color="blue" />
        <Button label="About" onClick={() => {}} color="blue" />
        <Button label="Contact" onClick={() => {}} color="blue" />
      </nav>
    </header>
  );
}

function Footer() {
  return (
    <footer>
      <p>Copyright 2025. All rights reserved.</p>
    </footer>
  );
}

function App() {
  return (
    <div>
      <Header />
      <main>
        <h2>Welcome to my website!</h2>
        <p>This is the main content area.</p>
      </main>
      <Footer />
    </div>
  );
}
```

Here, `App` is composed of `Header`, some content, and `Footer`. The `Header` is composed of a heading and three `Button` components. Each piece is small and easy to understand on its own.

This is the power of composition: small, simple pieces that combine into something bigger.

The `children` Prop for Flexible Content

So far, our components receive data through props like `label`, `title`, and `color`. But what if you want to put anything inside a component? Maybe sometimes you want text inside a card, and other times you want buttons, or images, or even other components.

This is where the special `children` prop comes in.

The `children` prop is whatever you put between the opening and closing tags of a component. It is like a container that can hold anything.

Example: A Container Component

```jsx
function Container({ children }) {
  const style = {
    border: '2px solid blue',
    borderRadius: '8px',
    padding: '20px',
    margin: '10px'
  };

  return <div style={style}>{children}</div>;
}
```

- `function Container({ children })` — The component receives `children` as a prop. You do not pass `children` like `children={...}`. Instead, React automatically collects whatever is between the opening `<Container>` and closing `</Container>` tags.
- `{children}` — This renders whatever was placed inside the component.

Using the Container

```jsx
function App() {
  return (
    <div>
      <Container>
        <h2>Hello!</h2>
        <p>This text is inside the container.</p>
      </Container>

      <Container>
        <Button label="Click Me" onClick={() => alert('Hi!')} color="green" />
      </Container>

      <Container>
        <img src="https://example.com/photo.jpg" alt="A photo" />
        <p>A photo with a caption.</p>
      </Container>
    </div>
  );
}
```

Expected Output

You will see three blue-bordered boxes on the page:

- The first box contains a heading and a paragraph.
- The second box contains a green button.
- The third box contains an image and a caption.

The Container component does not care what is inside it. It just wraps its content in a styled box. This makes it very flexible and reusable.

Combining Props and Children

You can use both regular props and `children` together:

```jsx
function Panel({ title, children }) {
  return (
    <div style={{ border: '1px solid #ccc', padding: '16px', margin: '10px' }}>
      <h3>{title}</h3>
      <div>{children}</div>
    </div>
  );
}

function App() {
  return (
    <div>
      <Panel title="Welcome">
        <p>Thank you for visiting our website.</p>
      </Panel>

      <Panel title="Settings">
        <Button label="Save" onClick={() => {}} color="green" />
        <Button label="Reset" onClick={() => {}} color="gray" />
      </Panel>
    </div>
  );
}
```

The `Panel` component uses the `title` prop for the heading and `children` for everything else inside it. This is a very common and useful pattern.

When to Make Something a Separate Component

A common question beginners ask is: "When should I create a new component?" Here is a simple rule to follow:

The "Used More Than Once" Rule

If you use something more than once, make it a component.

That is the simplest rule. If you find yourself copying and pasting similar code, stop. Turn that code into a component instead.

Other Signs You Should Create a Component

1. The code is getting long. If your component has more than 50-80 lines, consider breaking it into smaller pieces.
2. It has a clear purpose. If a piece of your code does one specific thing (like showing a navigation bar), it can be its own component.
3. You want to test it separately. Smaller components are easier to test.
4. It makes the code easier to read. If naming a piece of code would make the parent component clearer, create a component.

Example: Before and After

Before (everything in one component):

```jsx
function App() {
  return (
    <div>
      <div style={{ backgroundColor: 'blue', color: 'white', padding: '10px' }}>
        <h1>My Store</h1>
      </div>

      <div style={{ border: '1px solid #ddd', padding: '16px', margin: '10px' }}>
        <h2>Product A</h2>
        <p>$10.00</p>
        <button>Add to Cart</button>
      </div>

      <div style={{ border: '1px solid #ddd', padding: '16px', margin: '10px' }}>
        <h2>Product B</h2>
        <p>$20.00</p>
        <button>Add to Cart</button>
      </div>

      <div style={{ border: '1px solid #ddd', padding: '16px', margin: '10px' }}>
        <h2>Product C</h2>
        <p>$15.00</p>
        <button>Add to Cart</button>
      </div>
    </div>
  );
}
```

Notice how the product blocks look almost identical. This is a sign that we should create a component.

After (with reusable components):

```jsx
function StoreHeader() {
  return (
    <div style={{ backgroundColor: 'blue', color: 'white', padding: '10px' }}>
      <h1>My Store</h1>
    </div>
  );
}

function ProductCard({ name, price }) {
  return (
    <div style={{ border: '1px solid #ddd', padding: '16px', margin: '10px' }}>
      <h2>{name}</h2>
      <p>${price}</p>
      <button>Add to Cart</button>
    </div>
  );
}

function App() {
  return (
    <div>
      <StoreHeader />
      <ProductCard name="Product A" price="10.00" />
      <ProductCard name="Product B" price="20.00" />
      <ProductCard name="Product C" price="15.00" />
    </div>
  );
}
```

The "after" version is shorter, cleaner, and easier to change. If you want to redesign the product cards, you change one component and all three cards update.

Organizing Reusable Components in a Folder

As your project grows, you will have many components. It helps to organize them. A common approach is to put reusable components in a `components` folder.

Here is a typical folder structure:

```
src/
  components/
    Button.jsx
    Card.jsx
    Container.jsx
    Panel.jsx
  pages/
    Home.jsx
    About.jsx
  App.jsx
```

- The `components` folder holds reusable pieces that can be used anywhere in your app.
- The `pages` folder holds components that represent full pages.
- `App.jsx` is the main component that puts everything together.

How to Import Components

When your components are in separate files, you import them:

```jsx
// Inside Button.jsx
function Button({ label, onClick, color = 'blue' }) {
  const style = {
    backgroundColor: color,
    color: 'white',
    padding: '10px 20px',
    border: 'none',
    borderRadius: '5px',
    cursor: 'pointer',
    fontSize: '16px'
  };

  return (
    <button style={style} onClick={onClick}>
      {label}
    </button>
  );
}

export default Button;
```

```jsx
// Inside App.jsx
import Button from './components/Button';

function App() {
  return (
    <div>
      <Button label="Save" onClick={() => alert('Saved!')} color="green" />
```

```
    </div>
  );
}

export default App;
```

- `export default Button;` — This makes the Button component available to other files. Think of it like putting a book on a library shelf so others can borrow it.
- `import Button from './components/Button';` — This brings the Button component into App.jsx. Think of it like borrowing that book from the shelf.

Mini Project: Product Card Grid

Let us put everything together and build a product card grid. We will create one reusable `ProductCard` component and use it to display several products.

Step 1: The ProductCard Component

```jsx
function ProductCard({ name, price, description, imageUrl, onAddToCart }) {
  const cardStyle = {
    border: '1px solid #ddd',
    borderRadius: '8px',
    padding: '16px',
    width: '250px',
    boxShadow: '0 2px 4px rgba(0,0,0,0.1)'
  };

  const imageStyle = {
    width: '100%',
    height: '150px',
    objectFit: 'cover',
    borderRadius: '4px'
  };

  const buttonStyle = {
    backgroundColor: '#007bff',
    color: 'white',
    padding: '8px 16px',
    border: 'none',
    borderRadius: '4px',
    cursor: 'pointer',
    width: '100%',
    fontSize: '14px'
  };
```

```
  return (
    <div style={cardStyle}>
      <img src={imageUrl} alt={name} style={imageStyle} />
      <h3>{name}</h3>
      <p>{description}</p>
      <p><strong>${price}</strong></p>
      <button style={buttonStyle} onClick={onAddToCart}>
        Add to Cart
      </button>
    </div>
  );
}
```

Line by line:

- `function ProductCard({ name, price, description, imageUrl, onAddToCart })` — This component accepts five props: the product name, price, description, image URL, and a function to call when the user clicks "Add to Cart."
- `objectFit: 'cover'` — This makes the image fill its space without stretching. It might crop the edges, but the image always looks good.
- `width: '100%'` on the button — This makes the button stretch to fill the full width of the card.
- `onClick={onAddToCart}` — When the button is clicked, it calls the function that was passed in as a prop.

Step 2: The Product Data

```jsx
const products = [
  {
    id: 1,
    name: 'Wireless Headphones',
    price: 49.99,
    description: 'Great sound quality with noise cancellation.',
    imageUrl: 'https://example.com/headphones.jpg'
  },
  {
    id: 2,
    name: 'Coffee Mug',
    price: 12.99,
    description: 'Keeps your coffee warm for hours.',
    imageUrl: 'https://example.com/mug.jpg'
  },
  {
    id: 3,
    name: 'Notebook',
```

```
    price: 8.99,
    description: 'Lined pages, perfect for notes and ideas.',
    imageUrl: 'https://example.com/notebook.jpg'
  },
  {
    id: 4,
    name: 'Desk Lamp',
    price: 29.99,
    description: 'Adjustable brightness for comfortable reading.',
    imageUrl: 'https://example.com/lamp.jpg'
  }
];
```

This is an array (a list) of product objects. Each product has an `id`, `name`, `price`, `description`, and `imageUrl`.

Step 3: The App Component

```jsx
function App() {
  function handleAddToCart(productName) {
    alert(productName + ' added to cart!');
  }

  const gridStyle = {
    display: 'flex',
    flexWrap: 'wrap',
    gap: '20px',
    padding: '20px',
    justifyContent: 'center'
  };

  return (
    <div>
      <h1 style={{ textAlign: 'center' }}>Our Products</h1>
      <div style={gridStyle}>
        {products.map(function (product) {
          return (
            <ProductCard
              key={product.id}
              name={product.name}
              price={product.price}
              description={product.description}
              imageUrl={product.imageUrl}
              onAddToCart={function () {
                handleAddToCart(product.name);
              }}
            />
          );
        })}
      </div>
```

```
    </div>
  );
}
```

Line by line:

- `handleAddToCart(productName)` — A function that shows an alert when a product is added to the cart.
- `display: 'flex'` — This puts the cards in a row. Flex is a layout system that arranges items side by side.
- `flexWrap: 'wrap'` — This tells the browser to wrap cards to the next line if they do not fit.
- `gap: '20px'` — This adds space between the cards.
- `products.map(function (product) { ... })` — This loops through the products array and creates one `ProductCard` for each product.
- `key={product.id}` — React needs a unique key for each item in a list so it can track them properly.
- `onAddToCart={function () { handleAddToCart(product.name); }}` — We wrap the function call so that it passes the right product name when clicked.

Expected Output

You will see a centered heading "Our Products" and four product cards arranged in a grid. Each card shows an image, a product name, a description, a price, and an "Add to Cart" button. Clicking any "Add to Cart" button shows an alert with the product name.

Quick Summary

- Reuse means writing a component once and using it many times.
- Props let you customize each use of a component.
- Composition means building big things from small components.
- The `children` prop lets you put any content inside a component.
- Create a new component when you use something more than once.
- Organize reusable components in a `components` folder.
- Use `map` to create many instances of a component from data.

Key Points to Remember

1. A reusable component is like a cookie cutter. You design it once, then use it many times with small differences.
2. Props make components flexible. The same Button component can be green, red, or gray depending on what props you pass.
3. The `children` prop is special. It automatically receives whatever you put between the opening and closing tags of a component.
4. Composition is how React apps are built. Small components combine into bigger ones, which combine into pages, which combine into apps.
5. The "used more than once" rule is a simple guide: if you copy-paste code, turn it into a component instead.
6. Default props (like `color = 'blue'`) prevent errors when someone forgets to pass a prop.

Practice Questions

1. What does "reuse" mean in the context of React components?
2. How does the `children` prop work? How is it different from other props?
3. You have a website with 10 pages, and each page has the same footer. Should you create a Footer component? Why?
4. What is the difference between passing `label="Save"` as a prop and putting content between `<Component>` and `</Component>`?
5. Why is it a good idea to organize reusable components in a `components` folder?

Exercises

Exercise 1: Reusable Alert Box

Create a reusable `AlertBox` component that shows a colored message box. It should accept a `type` prop that can be `"success"`, `"warning"`, or `"error"`. Each type should have a different background color (green, yellow, red). It should use the `children` prop for the message text.

Use it like this:

```jsx
<AlertBox type="success">Your file was saved!</AlertBox>
<AlertBox type="warning">Your session will expire soon.</AlertBox>
<AlertBox type="error">Something went wrong.</AlertBox>
```

Exercise 2: User Profile Card

Create a reusable `ProfileCard` component that shows a user's name, email, and avatar image. Then display three different profile cards using the same component with different data.

Exercise 3: Reusable Layout

Create a `PageLayout` component that uses `children`. It should include a header at the top (with a title), the children content in the middle, and a footer at the bottom. Use this layout to wrap two different pages of content.

What Is Next?

You now know how to build reusable components and combine them together. But what happens when two separate components need to share the same piece of data? For example, what if a product list and a shopping cart both need to know what items the user has selected?

In the next chapter, you will learn about lifting state up — a technique for sharing data between components by moving the state to their common parent. This is one of the most important patterns in React, and it builds directly on what you learned here about props and composition.

Chapter 18: Lifting State Up

What You Will Learn

In this chapter, you will learn:

- Why two components sometimes need to share the same data
- What "lifting state up" means
- How to move state to a parent component so children can share it
- How to pass state down as props and functions as props
- The parent-holds-state pattern
- How data flows in React (down through props, up through functions)
- When to lift state up vs. when to keep state local
- How to build a simple shopping cart where two components share state

Why This Chapter Matters

Imagine you have two thermometers in your house. One is in the kitchen, and one is in the living room. Now imagine they are both connected to the same heating system. When you turn up the heat, both thermometers should show the same temperature.

But what if each thermometer tracked its own temperature separately? The kitchen might say 72 degrees while the living room says 68. They would be out of sync. That is confusing and wrong.

In React, the same problem happens. Sometimes two components need to show the same data. If each component keeps its own copy of that data, they can get out of sync. The solution is to move the data to a place where both components can access it: their shared parent.

This idea is called lifting state up. It is one of the most important patterns in React. Once you understand it, you will know how to share data between any two components in your app.

The Problem: Two Components Need the Same Data

Let us look at a simple example. Imagine you have two text inputs. One shows a temperature in Celsius, and the other shows the same temperature in Fahrenheit. When you type in the Celsius input, the Fahrenheit input should update automatically, and vice versa.

The Wrong Approach: Each Component Has Its Own State

```jsx
function CelsiusInput() {
  const [celsius, setCelsius] = React.useState('');

  return (
    <div>
      <label>Celsius: </label>
      <input
        value={celsius}
        onChange={function (event) {
          setCelsius(event.target.value);
        }}
      />
    </div>
  );
}

function FahrenheitInput() {
  const [fahrenheit, setFahrenheit] = React.useState('');

  return (
    <div>
      <label>Fahrenheit: </label>
      <input
        value={fahrenheit}
        onChange={function (event) {
          setFahrenheit(event.target.value);
        }}
      />
    </div>
  );
}
```

The problem here is clear: `CelsiusInput` and `FahrenheitInput` each have their own state. They do not know about each other. Typing in one input does not affect the other. They are completely disconnected.

This is like those two thermometers tracking temperature separately. They will never agree.

What "Lifting State Up" Means

Lifting state up means moving the state from a child component to the nearest parent component that both children share.

Think of it this way. You have two children who each have their own piggy bank. They keep arguing about how much money the family has. The solution? Give the money to the parent. Now the parent holds the money and tells each child how much there is. If one child earns money, they give it to the parent, and the parent updates everyone.

In React terms:

1. Remove state from the child components.
2. Add state to the parent component.
3. Pass the state down to children as props.
4. Pass functions down so children can request changes.

The parent becomes the "single source of truth." There is only one copy of the data, and everyone gets it from the same place.

Step-by-Step Example: Temperature Converter

Let us fix our temperature converter by lifting state up.

Step 1: Create the Parent Component with State

```jsx
function TemperatureConverter() {
  const [temperature, setTemperature] = React.useState('');
  const [scale, setScale] = React.useState('celsius');

  function handleCelsiusChange(value) {
    setTemperature(value);
    setScale('celsius');
  }
```

```
  function handleFahrenheitChange(value) {
    setTemperature(value);
    setScale('fahrenheit');
  }

  let celsius;
  let fahrenheit;

  if (scale === 'celsius') {
    celsius = temperature;
    fahrenheit = temperature ? String(Number(temperature) * 9 / 5 + 32) : '';
  } else {
    fahrenheit = temperature;
    celsius = temperature ? String((Number(temperature) - 32) * 5 / 9) : '';
  }

  return (
    <div>
      <h1>Temperature Converter</h1>
      <TemperatureInput
        label="Celsius"
        value={celsius}
        onChange={handleCelsiusChange}
      />
      <TemperatureInput
        label="Fahrenheit"
        value={fahrenheit}
        onChange={handleFahrenheitChange}
      />
    </div>
  );
}
```

Let us go through this line by line:

- `const [temperature, setTemperature] = React.useState('');` — The parent holds the temperature value. This is the state that was "lifted up" from the children.
- `const [scale, setScale] = React.useState('celsius');` — The parent also tracks which input the user typed in last. `scale` remembers whether the user typed in Celsius or Fahrenheit.
- `handleCelsiusChange(value)` — When the user types in the Celsius input, this function saves the value and marks the scale as Celsius.
- `handleFahrenheitChange(value)` — When the user types in the Fahrenheit input, this function saves the value and marks the scale as Fahrenheit.
- The conversion math — If the user typed Celsius, we calculate Fahrenheit. If the user typed Fahrenheit, we calculate Celsius. The formula `C * 9/5 + 32` converts Celsius to Fahrenheit. The formula `(F - 32) * 5/9` converts Fahrenheit to Celsius.
- `temperature ? String(...) : ''` — If the temperature is empty, show an empty string. Otherwise, do the conversion. The `?` and `:` form a ternary operator, which is like a short

if-else statement.

Step 2: Create the Reusable Child Component

```jsx
function TemperatureInput({ label, value, onChange }) {
  return (
    <div style={{ margin: '10px 0' }}>
      <label>{label}: </label>
      <input
        value={value}
        onChange={function (event) {
          onChange(event.target.value);
        }}
      />
    </div>
  );
}
```

Line by line:

- `function TemperatureInput({ label, value, onChange })` — This component does NOT have its own state. It receives everything from its parent through props.
- `value={value}` — The input shows whatever value the parent passed down.
- `onChange(event.target.value)` — When the user types, the component calls the `onChange` function from the parent. It does not update any state itself. Instead, it tells the parent: "Hey, the user typed something new!"

Expected Output

You will see two inputs on the screen: one labeled "Celsius" and one labeled "Fahrenheit." When you type `100` in the Celsius input, the Fahrenheit input automatically shows `212`. When you type `32` in the Fahrenheit input, the Celsius input automatically shows `0`.

Both inputs always stay in sync because they share the same state in the parent.

Passing State Down and Functions Up

This pattern is so important that it deserves its own section. Here is the pattern in plain English:

1. The parent holds the state. It is the single source of truth.

2. The parent passes state down as props. Children receive the data they need to display.
3. The parent passes functions down as props. Children receive functions they can call to request changes.
4. Children call the parent's functions when something happens. For example, when the user types in an input.
5. The parent updates the state. This causes a re-render, and all children receive the updated data.

The Data Flow Diagram

Here is how data flows in our temperature converter:

```
        TemperatureConverter (Parent)
        ┌──────────────────────────────────┐
        │  State: temperature, scale       │
        │                                  │
        │  Calculates celsius & fahrenheit│
        └──────────────────────────────────┘
                │                 │
                │                 │
        Props   │                 │  Props
        (value, │                 │  (value,
        onChange)│                 │  onChange)
                │                 │
                ▼                 ▼
        ┌──────────────┐   ┌──────────────────┐
        │ Celsius      │   │ Fahrenheit       │
        │ Input        │   │ Input            │
        └──────────────┘   └──────────────────┘
              │                   │
              │                   │
              │  User types       │  User types
              │  → calls          │  → calls
              │  onChange()       │  onChange()
              │                   │
              └─────────────────┘
                      │
                      ▼
            Parent updates state
            Both children re-render
```

Data flows down through props (parent to children). Events flow up through functions (children to parent). This is called one-way data flow, and it is a key concept in React.

A Simpler Example: Shared Counter

The temperature converter has some math that might distract from the pattern. Let us see the same pattern with a simpler example: two buttons that share a counter.

```jsx
function CounterDisplay({ count }) {
  return <h2>Count: {count}</h2>;
}

function CounterButtons({ onIncrement, onDecrement }) {
  return (
    <div>
      <button onClick={onDecrement}>- Subtract</button>
      <button onClick={onIncrement}>+ Add</button>
    </div>
  );
}

function Counter() {
  const [count, setCount] = React.useState(0);

  function handleIncrement() {
    setCount(count + 1);
  }

  function handleDecrement() {
    setCount(count - 1);
  }

  return (
    <div>
      <CounterDisplay count={count} />
      <CounterButtons
        onIncrement={handleIncrement}
        onDecrement={handleDecrement}
      />
    </div>
  );
}
```

Line by line:

- `CounterDisplay` only receives `count` as a prop and displays it. It has no state.
- `CounterButtons` only receives two functions as props. It has no state. It just calls the functions when buttons are clicked.
- `Counter` (the parent) holds the state (`count`). It passes the count down to `CounterDisplay` and the functions down to `CounterButtons`.
- When a button is clicked, the function runs in the parent, updates the state, and both children re-render with the new data.

Expected Output

You see the text "Count: 0" and two buttons: "- Subtract" and "+ Add." Clicking "+ Add" changes the display to "Count: 1." Clicking "- Subtract" changes it to "Count: -1." The display and buttons are separate components, but they stay in sync because they share state through the parent.

When to Lift State Up vs. Keep State Local

Not all state needs to be lifted up. Here is a simple guide:

Keep State Local When:

- Only one component uses it. If a search input only needs to track what the user is typing, and no other component cares about that text, keep the state in the search input component.
- It is temporary. For example, whether a dropdown menu is open or closed. Other components do not need to know this.
- It is about the UI, not the data. For example, whether a tooltip is visible.

Lift State Up When:

- Two or more components need the same data. Like our temperature inputs.
- A child needs to change something that affects its sibling. For example, clicking a button in one component should update a display in another component.
- The parent needs to know about the child's state. For example, a form parent needs to know all the input values to submit them.

A Quick Test

Ask yourself: "Does any other component need this piece of data?" If the answer is yes, lift the state up to the nearest shared parent. If the answer is no, keep it local.

Mini Project: Simple Shopping Cart

Let us build a shopping cart where an item list and a cart summary share the same state. This is a real-world example of lifting state up.

The Plan

We need three components:

1. `ProductList` — Shows the products and "Add to Cart" buttons.
2. `CartSummary` — Shows what is in the cart and the total price.
3. `ShoppingApp` — The parent that holds the cart state.

Step 1: The Product Data

```jsx
const products = [
  { id: 1, name: 'Apple', price: 1.50 },
  { id: 2, name: 'Bread', price: 3.00 },
  { id: 3, name: 'Milk', price: 2.50 },
  { id: 4, name: 'Cheese', price: 5.00 }
];
```

Step 2: The ProductList Component

```jsx
function ProductList({ products, onAddToCart }) {
  const listStyle = {
    listStyle: 'none',
    padding: 0
  };

  const itemStyle = {
    display: 'flex',
    justifyContent: 'space-between',
    alignItems: 'center',
    padding: '10px',
    borderBottom: '1px solid #eee'
  };

  const buttonStyle = {
    backgroundColor: '#28a745',
    color: 'white',
    border: 'none',
    padding: '5px 15px',
    borderRadius: '4px',
    cursor: 'pointer'
  };

  return (
    <div>
      <h2>Products</h2>
      <ul style={listStyle}>
```

```
        {products.map(function (product) {
          return (
            <li key={product.id} style={itemStyle}>
              <span>{product.name} - ${product.price.toFixed(2)}</span>
              <button
                style={buttonStyle}
                onClick={function () {
                  onAddToCart(product);
                }}
              >
                Add to Cart
              </button>
            </li>
          );
        })}
      </ul>
    </div>
  );
}
```

Line by line:

- `function ProductList({ products, onAddToCart })` — This component receives the product list and a function to add items to the cart. It does NOT hold any state.
- `products.map(function (product) { ... })` — This loops through each product and creates a list item for it.
- `product.price.toFixed(2)` — This formats the price to always show two decimal places. So `1.5` becomes `1.50`.
- `onClick={function () { onAddToCart(product); }}` — When the "Add to Cart" button is clicked, it calls the parent's function and passes the entire product object.

Step 3: The CartSummary Component

```jsx
function CartSummary({ cartItems, onRemoveFromCart }) {
  const total = cartItems.reduce(function (sum, item) {
    return sum + item.price * item.quantity;
  }, 0);

  if (cartItems.length === 0) {
    return (
      <div>
        <h2>Cart</h2>
        <p>Your cart is empty.</p>
      </div>
    );
  }
```

```
  return (
    <div>
      <h2>Cart</h2>
      <ul style={{ listStyle: 'none', padding: 0 }}>
        {cartItems.map(function (item) {
          return (
            <li key={item.id} style={{ padding: '5px 0' }}>
              {item.name} x {item.quantity} = ${(item.price * item.quantity).toFixed(2)}
              <button
                onClick={function () {
                  onRemoveFromCart(item.id);
                }}
                style={{ marginLeft: '10px', color: 'red', cursor: 'pointer' }}
              >
                Remove
              </button>
            </li>
          );
        })}
      </ul>
      <hr />
      <p><strong>Total: ${total.toFixed(2)}</strong></p>
    </div>
  );
}
```

Line by line:

- `function CartSummary({ cartItems, onRemoveFromCart })` — This component receives the cart items and a function to remove items. Again, no state of its own.
- `cartItems.reduce(...)` — `reduce` is a way to calculate a single value from an array. Here, it adds up the price times quantity of every item to get the total. The `0` at the end is the starting value for the sum.
- `if (cartItems.length === 0)` — If the cart is empty, show a simple message instead of the item list.
- `item.name} x {item.quantity}` — Shows how many of each item are in the cart.
- The "Remove" button — Calls the parent's `onRemoveFromCart` function with the item's ID.

Step 4: The Parent Component (ShoppingApp)

```jsx
function ShoppingApp() {
  const [cartItems, setCartItems] = React.useState([]);

  function handleAddToCart(product) {
    setCartItems(function (currentItems) {
      const existingItem = currentItems.find(function (item) {
```

```
        return item.id === product.id;
      });

      if (existingItem) {
        return currentItems.map(function (item) {
          if (item.id === product.id) {
            return { ...item, quantity: item.quantity + 1 };
          }
          return item;
        });
      }

      return [...currentItems, { ...product, quantity: 1 }];
    });
  }

  function handleRemoveFromCart(productId) {
    setCartItems(function (currentItems) {
      return currentItems.filter(function (item) {
        return item.id !== productId;
      });
    });
  }

  const appStyle = {
    display: 'flex',
    gap: '40px',
    padding: '20px',
    maxWidth: '800px',
    margin: '0 auto'
  };

  return (
    <div>
      <h1 style={{ textAlign: 'center' }}>Simple Shop</h1>
      <div style={appStyle}>
        <div style={{ flex: 1 }}>
          <ProductList products={products} onAddToCart={handleAddToCart} />
        </div>
        <div style={{ flex: 1 }}>
          <CartSummary
            cartItems={cartItems}
            onRemoveFromCart={handleRemoveFromCart}
          />
        </div>
      </div>
    </div>
  );
}
```

Line by line:

- `const [cartItems, setCartItems] = React.useState([]);` — The parent holds the cart state. It starts as an empty array.

- `handleAddToCart(product)` — This function checks if the product is already in the cart. If yes, it increases the quantity. If no, it adds the product with quantity 1.
- `currentItems.find(...)` — `find` looks through the array and returns the first item that matches the condition. Here, it checks if an item with the same ID already exists.
- `{ ...item, quantity: item.quantity + 1 }` — This creates a copy of the item with the quantity increased by 1. The `...item` part copies all existing properties. This is called the spread operator.
- `[...currentItems, { ...product, quantity: 1 }]` — This creates a new array with all current items plus the new product (with quantity set to 1).
- `handleRemoveFromCart(productId)` — This function creates a new array that includes everything except the item with the matching ID. `filter` keeps only items where the condition is true.
- The parent passes `cartItems` and functions to both `ProductList` and `CartSummary`.

Expected Output

You will see a page with two columns. The left column shows a list of products (Apple, Bread, Milk, Cheese) with prices and "Add to Cart" buttons. The right column shows the cart, which starts as "Your cart is empty."

When you click "Add to Cart" on Apple, the cart shows "Apple x 1 = $1.50" and "Total: $1.50." Click it again, and it shows "Apple x 2 = $3.00." Click "Add to Cart" on Bread, and the cart shows both items. Click "Remove" next to an item to remove it from the cart.

The product list and the cart summary are completely separate components, but they always show consistent data because they share state through the parent.

Quick Summary

- Lifting state up means moving state from child components to their shared parent.
- The parent becomes the single source of truth for the shared data.
- Props carry data from parent to children (downward flow).
- Functions passed as props let children communicate changes back to the parent (upward flow).
- This is called one-way data flow: data goes down, events go up.
- Keep state local when only one component needs it. Lift state up when multiple components need the same data.

Key Points to Remember

1. If two components need the same data, neither one should own it. Their closest shared parent should own it.
2. Children never modify props directly. They call functions provided by the parent to request changes.
3. One-way data flow makes your app predictable. You always know where data comes from (the parent) and how it changes (through specific functions).
4. Lifting state up does not mean putting ALL state in the top-level component. Only lift state when it needs to be shared.
5. The pattern is always the same: parent holds state, passes it down as props, and passes updater functions down as props.
6. When state is lifted up, the child components become simpler because they do not manage their own state.

Practice Questions

1. What problem does lifting state up solve?
2. In the temperature converter example, why does the parent hold the temperature state instead of each input holding its own?
3. What is one-way data flow? Describe how data moves in a React app.
4. Give an example of when you should keep state local instead of lifting it up.
5. In the shopping cart example, what would go wrong if both `ProductList` and `CartSummary` each had their own copy of the cart state?

Exercises

Exercise 1: Synced Text Inputs

Create two text inputs. Whatever you type in one input should appear in the other, and vice versa. They should always show the same text. (Hint: Lift the text state up to a parent

component.)

Exercise 2: Color Picker

Create a component with three buttons: "Red," "Green," and "Blue." Create another component that displays a colored box. When you click a color button, the box should change to that color. The two components should share the selected color through a parent.

Exercise 3: Vote Counter

Create a simple voting app. There should be a list of options (like "Pizza," "Tacos," "Sushi") with a "Vote" button next to each. Below the list, show a summary of the vote counts. The vote list and the summary should be separate components that share the vote data through a parent.

What Is Next?

You now know how to share data between components by lifting state up to their parent. This works great for small apps and for components that are close together in the component tree.

But what about navigation? What if you want your app to have multiple "pages" — like a Home page, an About page, and a Contact page — without actually reloading the browser?

In the next chapter, you will learn about routing in React. Routing lets you show different components based on the URL, just like a real multi-page website, but faster and smoother.

Chapter 19: Basic Routing

What You Will Learn

In this chapter, you will learn:

- What routing means and why it matters
- How to install React Router
- How to use `BrowserRouter`, `Routes`, and `Route`
- How to create page components
- How to use `Link` for navigation without page reloads
- How to use `NavLink` for active navigation styling
- How to use `useNavigate` for going to a page with code
- How to handle "Page Not Found" (404) errors
- How to build a simple multi-page website with a navigation bar

Why This Chapter Matters

Think about how you use a website. You click on "Home" and you see the home page. You click on "About" and you see the about page. You click on "Contact" and you see the contact page. Each page has its own address in the browser's address bar.

On a traditional website, every time you click a link, the browser loads a completely new page from the server. This causes a brief flash. The entire page disappears and a new one loads. It works, but it can feel slow.

React apps work differently. A React app is a single-page application (often called an SPA). This means the browser loads one HTML page, and then React swaps out the content without reloading the whole page. It is like changing the picture in a picture frame instead of buying a new frame every time.

But here is the problem: if everything happens on one page, how do you handle different URLs? How do you make the "Back" button work? How do you let users bookmark a specific page?

That is where routing comes in. A router is a tool that watches the URL in the address bar and shows the right content for that URL. It makes a single-page app feel like a multi-page website.

In this chapter, you will learn how to add routing to your React app using a popular library called React Router.

What Routing Means

Routing is the process of deciding what to show based on the URL.

Think of routing like a hotel front desk. When a guest arrives and says "Room 204," the front desk directs them to the second floor, fourth door on the right. The hotel has many rooms, and the front desk knows which guest goes where.

In a React app, routing works the same way:

- The URL is like the room number. For example, `/about` or `/contact`.
- The router is like the front desk. It looks at the URL and decides which component to show.
- The components are like the rooms. Each one has different content.

When the URL changes, the router swaps out the old component and shows the new one. The browser does not reload. The switch is instant and smooth.

Installing React Router

React Router is not built into React. It is a separate library that you need to install. A library is a collection of pre-written code that you can use in your project.

To install it, open your terminal (the command-line tool) and run this command in your project folder:

```
npm install react-router-dom
```

Let us break this command down:

- `npm` — This is the Node Package Manager. It downloads and installs libraries for your project.
- `install` — This tells npm to install something.
- `react-router-dom` — This is the name of the React Router library for web browsers. The "dom" part stands for Document Object Model, which is the browser's way of representing a web page.

After running this command, React Router is ready to use in your project.

BrowserRouter, Routes, and Route

React Router gives you three main building blocks. Let us learn each one.

BrowserRouter

`BrowserRouter` is a wrapper that you put around your entire app. It enables routing. Without it, none of the other routing features work.

Think of `BrowserRouter` as turning on the routing system. It is like flipping the switch that powers the hotel front desk.

```jsx
import { BrowserRouter } from 'react-router-dom';

function App() {
  return (
    <BrowserRouter>
      {/* Your app content goes here */}
    </BrowserRouter>
  );
}
```

You usually only use `BrowserRouter` once, at the very top of your app.

Routes and Route

`Routes` is a container that holds all your individual `Route` components. Each `Route` maps a URL path to a specific component.

```jsx
import { BrowserRouter, Routes, Route } from 'react-router-dom';

function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
        <Route path="/contact" element={<Contact />} />
      </Routes>
    </BrowserRouter>
  );
}
```

Let us go through this:

- `<Routes>` — This component looks at the current URL and finds the matching `Route` to display. Only one `Route` is shown at a time.
- `<Route path="/" element={<Home />} />` — This says: "When the URL is `/` (the root, or home page), show the `Home` component."
- `<Route path="/about" element={<About />} />` — This says: "When the URL is `/about`, show the `About` component."
- `<Route path="/contact" element={<Contact />} />` — This says: "When the URL is `/contact`, show the `Contact` component."

The `path` prop is the URL pattern to match. The `element` prop is the component to display.

Creating Page Components

Each "page" in your app is just a regular React component. There is nothing special about them. Here are simple examples:

```jsx
function Home() {
  return (
    <div>
      <h1>Home</h1>
      <p>Welcome to our website! This is the home page.</p>
    </div>
  );
}

function About() {
```

```
  return (
    <div>
      <h1>About Us</h1>
      <p>We are a small team that loves building things with React.</p>
    </div>
  );
}

function Contact() {
  return (
    <div>
      <h1>Contact</h1>
      <p>Email us at hello@example.com</p>
    </div>
  );
}
```

These are normal components. They return JSX just like any other component you have written. The only difference is that React Router controls when they appear on the screen.

The Link Component

In regular HTML, you use `<a href="/about">About</a>` to create a link. But in a React app with routing, you should NOT use regular `<a>` tags for internal navigation. Why? Because `<a>` tags cause the browser to reload the entire page. That defeats the purpose of a single-page app.

Instead, React Router provides the `Link` component. It looks and works like a link, but it does not reload the page. It just tells the router to show a different component.

```jsx
import { Link } from 'react-router-dom';

function Navigation() {
  return (
    <nav>
      <Link to="/">Home</Link>
      <Link to="/about">About</Link>
      <Link to="/contact">Contact</Link>
    </nav>
  );
}
```

- `import { Link } from 'react-router-dom';` — This brings the `Link` component from React Router.

- `<Link to="/">Home</Link>` — The `to` prop tells the link which URL to go to. It works like `href` in a regular `<a>` tag, but without the page reload.

When you click a `Link`, the URL in the address bar changes, and React Router swaps the content. The page does not reload. It is instant.

Link vs. Regular `<a>` Tag

Feature	`<a href="...">`	`<Link to="...">`
Page reload	Yes, full reload	No reload
Speed	Slower	Instant
Use for	External links	Internal navigation

Use `Link` for navigation within your app. Use regular `<a>` tags only for links to other websites.

NavLink for Active Navigation Styling

`NavLink` is a special version of `Link` that knows whether it is currently active. An active link is one that matches the current URL.

Why is this useful? Because you usually want to highlight the current page in your navigation bar. For example, if you are on the About page, the "About" link should look different from the others.

```jsx
import { NavLink } from 'react-router-dom';

function Navigation() {
  const linkStyle = {
    padding: '10px 15px',
    textDecoration: 'none',
    color: '#333'
  };

  const activeLinkStyle = {
    padding: '10px 15px',
    textDecoration: 'none',
    color: 'white',
    backgroundColor: '#007bff',
    borderRadius: '4px'
```

```
  };

  return (
    <nav style={{ display: 'flex', gap: '5px', padding: '10px' }}>
      <NavLink
        to="/"
        style={function ({ isActive }) {
          return isActive ? activeLinkStyle : linkStyle;
        }}
      >
        Home
      </NavLink>
      <NavLink
        to="/about"
        style={function ({ isActive }) {
          return isActive ? activeLinkStyle : linkStyle;
        }}
      >
        About
      </NavLink>
      <NavLink
        to="/contact"
        style={function ({ isActive }) {
          return isActive ? activeLinkStyle : linkStyle;
        }}
      >
        Contact
      </NavLink>
    </nav>
  );
}
```

Let us go through the key parts:

- `<NavLink to="/">` — Works like `Link`, but it also tracks whether it is the active link.
- `style={function ({ isActive }) { ... }}` — `NavLink` passes an object with an `isActive` property to the style function. This property is `true` if the current URL matches the link's `to` prop.
- `isActive ? activeLinkStyle : linkStyle` — If the link is active, use the active style (blue background, white text). Otherwise, use the normal style.

Expected Output

The navigation bar shows three links: Home, About, and Contact. The link for the current page has a blue background and white text. The other links have dark text and no background. When you click a different link, the highlight moves to the new link.

useNavigate for Programmatic Navigation

Sometimes you want to go to a different page using code, not by clicking a link. For example, after the user submits a form, you might want to redirect them to a "Thank You" page. Or after they log in, you might want to send them to the dashboard.

React Router provides a hook called `useNavigate` for this purpose. A hook, as you learned earlier, is a special function that gives your component extra abilities.

```jsx
import { useNavigate } from 'react-router-dom';

function LoginPage() {
  const navigate = useNavigate();

  function handleLogin() {
    // Imagine we check the username and password here
    alert('Login successful!');
    navigate('/');
  }

  return (
    <div>
      <h1>Login</h1>
      <button onClick={handleLogin}>Log In</button>
    </div>
  );
}
```

Line by line:

- `const navigate = useNavigate();` — This gives you a function called `navigate` that you can use to go to any page.
- `navigate('/');` — This tells the router to go to the home page (`/`). The URL changes, and the Home component appears. No page reload.

Going Back

You can also use `navigate` to go back, like pressing the browser's back button:

```jsx
function SomePage() {
  const navigate = useNavigate();

  return (
    <div>
      <h1>Some Page</h1>
      <button onClick={function () { navigate(-1); }}>
```

```
        Go Back
      </button>
    </div>
  );
}
```

- `navigate(-1)` — The `-1` means "go back one page in history." It is like pressing the browser's back button. You could use `-2` to go back two pages.

Simple 404 / "Page Not Found" Route

What happens when someone types a URL that does not exist? For example, `/pizza`. Without a catch-all route, React Router shows nothing. That is confusing for the user.

A 404 page is a special page that appears when no other route matches. The name "404" comes from the HTTP status code that means "Not Found."

```jsx
function NotFound() {
  return (
    <div style={{ textAlign: 'center', padding: '50px' }}>
      <h1>404</h1>
      <h2>Page Not Found</h2>
      <p>Sorry, the page you are looking for does not exist.</p>
      <Link to="/">Go back to Home</Link>
    </div>
  );
}
```

To use this as a catch-all, add it as the last `Route` with `path="*"`:

```jsx
<Routes>
  <Route path="/" element={<Home />} />
  <Route path="/about" element={<About />} />
  <Route path="/contact" element={<Contact />} />
  <Route path="*" element={<NotFound />} />
</Routes>
```

- `path="*"` — The asterisk (`*`) means "match anything." It catches all URLs that do not match any of the routes above it.
- Order matters. The `*` route should always be last. React Router checks routes from top to bottom and stops at the first match. If `*` were first, it would match everything and no other route would ever work.

Expected Output

If a user goes to `/pizza` or `/xyz` or any other URL that does not have a route, they will see the "404 - Page Not Found" message with a link to go back home.

Putting It All Together

Let us combine everything into a complete example:

```jsx
import { BrowserRouter, Routes, Route, NavLink, Link, useNavigate } from 'react-router-...

function Navigation() {
  const linkStyle = {
    padding: '10px 15px',
    textDecoration: 'none',
    color: '#333'
  };

  const activeLinkStyle = {
    padding: '10px 15px',
    textDecoration: 'none',
    color: 'white',
    backgroundColor: '#007bff',
    borderRadius: '4px'
  };

  function getStyle({ isActive }) {
    return isActive ? activeLinkStyle : linkStyle;
  }

  return (
    <nav style={{
      display: 'flex',
      gap: '5px',
      padding: '15px',
      backgroundColor: '#f8f9fa',
      borderBottom: '1px solid #ddd'
    }}>
      <NavLink to="/" style={getStyle}>Home</NavLink>
      <NavLink to="/about" style={getStyle}>About</NavLink>
      <NavLink to="/contact" style={getStyle}>Contact</NavLink>
    </nav>
  );
}

function Home() {
```

```
  return (
    <div style={{ padding: '20px' }}>
      <h1>Welcome Home</h1>
      <p>This is the home page of our website.</p>
      <p>Use the navigation bar above to explore other pages.</p>
    </div>
  );
}

function About() {
  return (
    <div style={{ padding: '20px' }}>
      <h1>About Us</h1>
      <p>We are learning React and building amazing things.</p>
      <p>This page was created using React Router.</p>
    </div>
  );
}

function Contact() {
  const navigate = useNavigate();

  function handleSubmit() {
    alert('Message sent! Redirecting to home page...');
    navigate('/');
  }

  return (
    <div style={{ padding: '20px' }}>
      <h1>Contact Us</h1>
      <p>We would love to hear from you.</p>
      <button onClick={handleSubmit}>Send Message</button>
    </div>
  );
}

function NotFound() {
  return (
    <div style={{ padding: '20px', textAlign: 'center' }}>
      <h1>404</h1>
      <h2>Page Not Found</h2>
      <p>Sorry, this page does not exist.</p>
      <Link to="/">Go back to Home</Link>
    </div>
  );
}

function App() {
  return (
    <BrowserRouter>
      <Navigation />
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
        <Route path="/contact" element={<Contact />} />
```

```
        <Route path="*" element={<NotFound />} />
      </Routes>
    </BrowserRouter>
  );
}
```

Expected Output

You will see a navigation bar at the top with three links: Home, About, and Contact. The current page's link is highlighted in blue.

- Clicking "Home" shows the welcome message.
- Clicking "About" shows information about the team.
- Clicking "Contact" shows a contact page with a "Send Message" button. Clicking the button shows an alert and redirects you to the home page.
- Typing a random URL like `/pizza` shows the 404 page with a link back to home.

Notice that the navigation bar stays at the top of every page. It is outside the `<Routes>` component, so it is always visible. Only the content below it changes.

Mini Project: Simple Multi-Page Website

Let us build a more complete website with a proper navigation bar, multiple pages, and some styling.

The Complete Code

```jsx
import { BrowserRouter, Routes, Route, NavLink, Link } from 'react-router-dom';

function Navbar() {
  const navStyle = {
    display: 'flex',
    justifyContent: 'space-between',
    alignItems: 'center',
    padding: '10px 30px',
    backgroundColor: '#2c3e50',
    color: 'white'
  };

  const linksStyle = {
```

```
    display: 'flex',
    gap: '10px',
    listStyle: 'none',
    margin: 0,
    padding: 0
  };

  const linkStyle = {
    color: '#bdc3c7',
    textDecoration: 'none',
    padding: '8px 16px',
    borderRadius: '4px'
  };

  const activeLinkStyle = {
    color: 'white',
    textDecoration: 'none',
    padding: '8px 16px',
    borderRadius: '4px',
    backgroundColor: '#3498db'
  };

  function getStyle({ isActive }) {
    return isActive ? activeLinkStyle : linkStyle;
  }

  return (
    <nav style={navStyle}>
      <h2 style={{ margin: 0 }}>MyWebsite</h2>
      <ul style={linksStyle}>
        <li><NavLink to="/" style={getStyle}>Home</NavLink></li>
        <li><NavLink to="/about" style={getStyle}>About</NavLink></li>
        <li><NavLink to="/services" style={getStyle}>Services</NavLink></li>
        <li><NavLink to="/contact" style={getStyle}>Contact</NavLink></li>
      </ul>
    </nav>
  );
}

function HomePage() {
  const heroStyle = {
    textAlign: 'center',
    padding: '60px 20px',
    backgroundColor: '#ecf0f1'
  };

  return (
    <div>
      <div style={heroStyle}>
        <h1>Welcome to MyWebsite</h1>
        <p>We help you learn React, one step at a time.</p>
        <Link
          to="/about"
          style={{
            display: 'inline-block',
```

```
            padding: '12px 24px',
            backgroundColor: '#3498db',
            color: 'white',
            textDecoration: 'none',
            borderRadius: '4px',
            marginTop: '10px'
          }}
        >
          Learn More About Us
        </Link>
      </div>
    </div>
  );
}

function AboutPage() {
  return (
    <div style={{ padding: '40px', maxWidth: '600px', margin: '0 auto' }}>
      <h1>About Us</h1>
      <p>
        We are a team of developers who believe that anyone can learn
        to code. Our mission is to make programming accessible to everyone.
      </p>
      <h2>Our Story</h2>
      <p>
        We started in 2024 with a simple idea: create the most
        beginner-friendly React book in the world.
      </p>
    </div>
  );
}

function ServicesPage() {
  const services = [
    { title: 'Web Development', description: 'We build modern websites using React.' },
    { title: 'Training', description: 'We teach beginners how to code.' },
    { title: 'Consulting', description: 'We help teams adopt React.' }
  ];

  return (
    <div style={{ padding: '40px', maxWidth: '600px', margin: '0 auto' }}>
      <h1>Our Services</h1>
      {services.map(function (service, index) {
        return (
          <div
            key={index}
            style={{
              border: '1px solid #ddd',
              padding: '20px',
              marginBottom: '15px',
              borderRadius: '8px'
            }}
          >
            <h3>{service.title}</h3>
            <p>{service.description}</p>
```

```
          </div>
        );
      })}
    </div>
  );
}

function ContactPage() {
  return (
    <div style={{ padding: '40px', maxWidth: '600px', margin: '0 auto' }}>
      <h1>Contact Us</h1>
      <p>We would love to hear from you! Here is how to reach us:</p>
      <ul>
        <li>Email: hello@mywebsite.com</li>
        <li>Phone: (555) 123-4567</li>
        <li>Address: 123 React Street</li>
      </ul>
    </div>
  );
}

function NotFoundPage() {
  return (
    <div style={{ textAlign: 'center', padding: '60px' }}>
      <h1 style={{ fontSize: '72px', color: '#e74c3c' }}>404</h1>
      <h2>Oops! Page Not Found</h2>
      <p>The page you are looking for does not exist.</p>
      <Link
        to="/"
        style={{
          display: 'inline-block',
          padding: '10px 20px',
          backgroundColor: '#3498db',
          color: 'white',
          textDecoration: 'none',
          borderRadius: '4px',
          marginTop: '15px'
        }}
      >
        Go Back Home
      </Link>
    </div>
  );
}

function App() {
  return (
    <BrowserRouter>
      <Navbar />
      <Routes>
        <Route path="/" element={<HomePage />} />
        <Route path="/about" element={<AboutPage />} />
        <Route path="/services" element={<ServicesPage />} />
        <Route path="/contact" element={<ContactPage />} />
        <Route path="*" element={<NotFoundPage />} />
```

```
      </Routes>
    </BrowserRouter>
  );
}

export default App;
```

Expected Output

You will see a dark navigation bar at the top with "MyWebsite" on the left and four links on the right: Home, About, Services, and Contact. The active link is highlighted in blue.

- Home page: A centered welcome section with a heading, a description, and a "Learn More About Us" button that links to the About page.
- About page: Text about the team and their story.
- Services page: Three service cards listed vertically.
- Contact page: Contact information displayed in a list.
- Any other URL: A large red "404" with a message and a link back home.

The navigation bar is always visible. Only the content below it changes. Every transition is instant with no page reload.

Quick Summary

- Routing shows different content based on the URL without reloading the page.
- React Router is a library you install with `npm install react-router-dom`.
- `BrowserRouter` wraps your app and enables routing.
- `Routes` contains your route definitions. `Route` maps a URL path to a component.
- `Link` navigates without reloading the page. Use it instead of `<a>` for internal links.
- `NavLink` is like `Link` but knows when it is active. Use it for navigation bars.
- `useNavigate` lets you navigate with code (for example, after a form submission).
- `path="*"` catches all unmatched URLs for a 404 page.

Key Points to Remember

1. Always wrap your app with `BrowserRouter` at the top level. Without it, routing does not work.
2. Use `Link` for internal navigation, not `<a>` tags. Regular links cause a full page reload.
3. Use `NavLink` when you need to highlight the active link in a navigation bar.
4. Put the `path="*"` route last. It is a catch-all and should only match when nothing else does.
5. `useNavigate` is for programmatic navigation — going to a page using code instead of a click.
6. Components outside of `<Routes>` (like a navigation bar) appear on every page. Components inside `<Route>` only appear when their URL matches.

Practice Questions

1. What is the difference between a single-page application and a traditional website?
2. Why should you use `Link` instead of `<a>` for internal navigation in a React app?
3. What does `path="*"` do in a `Route` component?
4. What is the difference between `Link` and `NavLink`?
5. When would you use `useNavigate` instead of `Link`?

Exercises

Exercise 1: Add a New Page

Add a "Blog" page to the mini project. Create a `BlogPage` component that shows a list of three blog post titles. Add a "Blog" link to the navigation bar.

Exercise 2: Navigation with useNavigate

Create a simple quiz app with two pages: a "Question" page and a "Result" page. The Question page has a question and two buttons. When the user clicks either button, use `useNavigate` to go to the Result page.

Exercise 3: Active Styling with CSS Classes

Instead of using inline styles with `NavLink`, try using the `className` prop. `NavLink` can accept a function for `className` just like it does for `style`. Create a navigation bar where the active link gets a CSS class called `active-link`.

What Is Next?

You now know how to build multi-page React apps with routing. You can create navigation bars, handle different pages, and even redirect users with code.

In the next chapter, we will combine everything you have learned in this book into one complete project: a Todo App. You will use components, state, props, event handling, lists, conditional rendering, and routing to build a real, working application from scratch. It is time to put all the pieces together.

Chapter 20: Building a Simple Project

What You Will Learn

In this chapter, you will learn:

- How to plan a project before writing any code
- How to build a complete Todo App step by step
- How to add, complete, delete, and filter todo items
- How to show a count of remaining items
- How to structure your project files and folders
- How to add simple CSS styling
- What to do when you get stuck (debugging tips)

Why This Chapter Matters

You have learned many individual skills throughout this book: components, state, props, events, lists, conditional rendering, and routing. Each one is like a single tool in a toolbox. You know what a hammer does. You know what a screwdriver does. You know what a saw does.

But knowing individual tools is not the same as building a house. To build something real, you need to use many tools together. You need a plan. You need to know which tool to use and when.

This chapter is where it all comes together. We will build a complete Todo App from scratch. Along the way, you will see how all the concepts you learned work together to create a real, working application.

A Todo App is the perfect first project because it covers the most important skills:

- Creating and organizing components

- Managing state with `useState`
- Handling user input and events
- Working with lists and keys
- Conditional rendering
- Passing data between components with props

Take your time with this chapter. Type the code yourself instead of copying it. When something does not work, try to figure out why before looking at the solution. That is how real learning happens.

Planning Before Coding

Professional developers do not just start typing code. They plan first. Even for a small project, a few minutes of planning saves hours of confusion later.

Let us plan our Todo App by asking three questions:

Question 1: What Should the App Do?

Our Todo App needs to:

1. Let the user type a new todo and add it to the list.
2. Show all the todos in a list.
3. Let the user mark a todo as complete (done).
4. Let the user delete a todo.
5. Let the user filter todos: show All, only Active (not done), or only Completed (done).
6. Show a count of how many items are left to do.

Question 2: What Components Do We Need?

Let us break the app into pieces:

- `App` — The main component. Holds the state and puts everything together.
- `TodoForm` — The input field and "Add" button for creating new todos.
- `TodoList` — The list of todo items.
- `TodoItem` — A single todo item with a checkbox, text, and delete button.
- `TodoFilter` — The filter buttons (All, Active, Completed) and the item count.

Question 3: What State Do We Need?

We need to track:

- `todos` — An array of todo objects. Each todo has an `id`, `text`, and `completed` status.
- `filter` — A string that says which filter is active: `"all"`, `"active"`, or `"completed"`.

Both pieces of state will live in the `App` component because multiple children need access to them.

The Plan Summary

```
App (holds todos and filter state)
├── TodoForm (adds new todos)
├── TodoFilter (filter buttons + item count)
└── TodoList (displays filtered todos)
    └── TodoItem (single todo with checkbox and delete)
```

Now we know what to build. Let us start coding.

Setting Up the Project

If you already have a React project from earlier chapters, you can use it. Otherwise, create a new one:

```
npx create-react-app todo-app
cd todo-app
npm start
```

Folder Structure

Here is how we will organize our files:

```
src/
  components/
    TodoForm.jsx
    TodoList.jsx
    TodoItem.jsx
    TodoFilter.jsx
  App.jsx
```

```
  App.css
  index.js
```

We put our reusable components in a `components` folder, just like we learned in Chapter 17.

Step 1: Building the TodoItem Component

We start with the smallest piece and work our way up. This is called bottom-up development — building the small parts first, then assembling them.

```jsx
// src/components/TodoItem.jsx

function TodoItem({ todo, onToggle, onDelete }) {
  const itemStyle = {
    display: 'flex',
    alignItems: 'center',
    justifyContent: 'space-between',
    padding: '12px 16px',
    borderBottom: '1px solid #eee',
    backgroundColor: todo.completed ? '#f0f0f0' : 'white'
  };

  const textStyle = {
    textDecoration: todo.completed ? 'line-through' : 'none',
    color: todo.completed ? '#999' : '#333',
    flex: 1,
    marginLeft: '10px'
  };

  const deleteButtonStyle = {
    backgroundColor: '#e74c3c',
    color: 'white',
    border: 'none',
    padding: '4px 10px',
    borderRadius: '4px',
    cursor: 'pointer',
    fontSize: '12px'
  };

  return (
    <div style={itemStyle}>
      <input
        type="checkbox"
        checked={todo.completed}
        onChange={function () {
```

```
          onToggle(todo.id);
        }}
      />
      <span style={textStyle}>{todo.text}</span>
      <button
        style={deleteButtonStyle}
        onClick={function () {
          onDelete(todo.id);
        }}
      >
        Delete
      </button>
    </div>
  );
}

export default TodoItem;
```

Let us go through this line by line:

- `function TodoItem({ todo, onToggle, onDelete })` — This component receives three props: the todo object itself, a function to toggle its completed status, and a function to delete it.
- `backgroundColor: todo.completed ? '#f0f0f0' : 'white'` — If the todo is completed, the background is light gray. Otherwise, it is white. This gives visual feedback to the user.
- `textDecoration: todo.completed ? 'line-through' : 'none'` — If the todo is completed, the text has a line through it. This is the classic "crossed off" look for done items.
- `<input type="checkbox" checked={todo.completed} ...>` — A checkbox that is checked when the todo is completed.
- `onChange={function () { onToggle(todo.id); }}` — When the checkbox changes (checked or unchecked), it calls the `onToggle` function with the todo's ID. The parent will handle the actual state change.
- `onClick={function () { onDelete(todo.id); }}` — When the Delete button is clicked, it calls the `onDelete` function with the todo's ID.

Step 2: Building the TodoList Component

This component receives an array of todos and renders a `TodoItem` for each one.

```jsx
// src/components/TodoList.jsx

import TodoItem from './TodoItem';

function TodoList({ todos, onToggle, onDelete }) {
  if (todos.length === 0) {
    return (
      <p style={{ textAlign: 'center', color: '#999', padding: '20px' }}>
        No todos to show. Add one above!
      </p>
    );
  }

  return (
    <div>
      {todos.map(function (todo) {
        return (
          <TodoItem
            key={todo.id}
            todo={todo}
            onToggle={onToggle}
            onDelete={onDelete}
          />
        );
      })}
    </div>
  );
}

export default TodoList;
```

Line by line:

- `import TodoItem from './TodoItem';` — We import the TodoItem component we just created.
- `if (todos.length === 0)` — If there are no todos to show, display a friendly message instead of an empty list.
- `todos.map(function (todo) { ... })` — This loops through the array of todos and creates one `TodoItem` for each.
- `key={todo.id}` — Each item in a list needs a unique key so React can track it properly.
- We pass `onToggle` and `onDelete` down to each `TodoItem`. These functions came from the parent (`App`) and are just being passed through.

Step 3: Building the TodoForm Component

This component has an input field and a button for adding new todos.

```jsx
// src/components/TodoForm.jsx

import React from 'react';

function TodoForm({ onAdd }) {
  const [text, setText] = React.useState('');

  function handleSubmit(event) {
    event.preventDefault();

    const trimmedText = text.trim();

    if (trimmedText === '') {
      return;
    }

    onAdd(trimmedText);
    setText('');
  }

  const formStyle = {
    display: 'flex',
    gap: '10px',
    padding: '20px',
    borderBottom: '2px solid #eee'
  };

  const inputStyle = {
    flex: 1,
    padding: '10px',
    fontSize: '16px',
    border: '1px solid #ddd',
    borderRadius: '4px'
  };

  const buttonStyle = {
    padding: '10px 20px',
    fontSize: '16px',
    backgroundColor: '#3498db',
    color: 'white',
    border: 'none',
    borderRadius: '4px',
    cursor: 'pointer'
  };

  return (
    <form onSubmit={handleSubmit} style={formStyle}>
      <input
        type="text"
        placeholder="What do you need to do?"
        value={text}
        onChange={function (event) {
          setText(event.target.value);
```

```
        }}
        style={inputStyle}
      />
      <button type="submit" style={buttonStyle}>
        Add
      </button>
    </form>
  );
}

export default TodoForm;
```

Line by line:

- `const [text, setText] = React.useState('');` — This component has its own local state for the text input. This state does NOT need to be lifted up because no other component needs to know what the user is typing before they press "Add."
- `function handleSubmit(event)` — This runs when the form is submitted (either by clicking "Add" or pressing Enter).
- `event.preventDefault();` — This stops the browser from reloading the page when the form is submitted. By default, forms cause a page reload. We prevent that.
- `const trimmedText = text.trim();` — `trim()` removes spaces from the beginning and end of the text. This prevents adding empty or whitespace-only todos.
- `if (trimmedText === '') { return; }` — If the text is empty after trimming, do nothing. Just stop here.
- `onAdd(trimmedText);` — Call the parent's function to add the new todo.
- `setText('');` — Clear the input field so the user can type the next todo.
- `<form onSubmit={handleSubmit}>` — We use a `<form>` element instead of just a button. This lets the user press Enter to add a todo, which is a nicer experience.
- `placeholder="What do you need to do?"` — This shows gray hint text in the input when it is empty.

Step 4: Building the TodoFilter Component

This component shows filter buttons and the count of remaining items.

```jsx
// src/components/TodoFilter.jsx

function TodoFilter({ filter, onFilterChange, remainingCount }) {
```

```
  const containerStyle = {
    display: 'flex',
    justifyContent: 'space-between',
    alignItems: 'center',
    padding: '12px 16px',
    backgroundColor: '#f8f9fa',
    borderTop: '2px solid #eee'
  };

  function getButtonStyle(buttonFilter) {
    const isActive = filter === buttonFilter;

    return {
      padding: '6px 14px',
      border: isActive ? '2px solid #3498db' : '2px solid transparent',
      backgroundColor: isActive ? '#3498db' : 'transparent',
      color: isActive ? 'white' : '#555',
      borderRadius: '4px',
      cursor: 'pointer',
      fontSize: '14px'
    };
  }

  return (
    <div style={containerStyle}>
      <span style={{ color: '#888', fontSize: '14px' }}>
        {remainingCount} {remainingCount === 1 ? 'item' : 'items'} left
      </span>
      <div style={{ display: 'flex', gap: '5px' }}>
        <button
          style={getButtonStyle('all')}
          onClick={function () { onFilterChange('all'); }}
        >
          All
        </button>
        <button
          style={getButtonStyle('active')}
          onClick={function () { onFilterChange('active'); }}
        >
          Active
        </button>
        <button
          style={getButtonStyle('completed')}
          onClick={function () { onFilterChange('completed'); }}
        >
          Completed
        </button>
      </div>
    </div>
  );
}

export default TodoFilter;
```

Line by line:

- `function TodoFilter({ filter, onFilterChange, remainingCount })` — This component receives three props: the current filter value, a function to change the filter, and the number of remaining (not completed) items.
- `function getButtonStyle(buttonFilter)` — This function creates the style for each filter button. It checks if this button's filter matches the current active filter.
- `const isActive = filter === buttonFilter;` — This is `true` if this button represents the currently selected filter.
- `border: isActive ? '2px solid #3498db' : '2px solid transparent'` — The active button has a visible blue border. Other buttons have a transparent border (invisible, but it keeps the size consistent).
- `{remainingCount} {remainingCount === 1 ? 'item' : 'items'} left` — This shows "1 item left" or "3 items left." The ternary operator picks the right word: singular "item" when there is exactly 1, plural "items" otherwise.

Step 5: Building the App Component

This is the main component that holds all the state and brings everything together.

```jsx
// src/App.jsx

import React from 'react';
import TodoForm from './components/TodoForm';
import TodoList from './components/TodoList';
import TodoFilter from './components/TodoFilter';

function App() {
  const [todos, setTodos] = React.useState([
    { id: 1, text: 'Learn React basics', completed: true },
    { id: 2, text: 'Build a todo app', completed: false },
    { id: 3, text: 'Practice every day', completed: false }
  ]);

  const [filter, setFilter] = React.useState('all');

  // --- Adding a todo ---
  function handleAddTodo(text) {
    const newTodo = {
      id: Date.now(),
      text: text,
      completed: false
    };

    setTodos(function (currentTodos) {
```

```
    return [...currentTodos, newTodo];
  });
}

// --- Toggling a todo (complete / not complete) ---
function handleToggleTodo(id) {
  setTodos(function (currentTodos) {
    return currentTodos.map(function (todo) {
      if (todo.id === id) {
        return { ...todo, completed: !todo.completed };
      }
      return todo;
    });
  });
}

// --- Deleting a todo ---
function handleDeleteTodo(id) {
  setTodos(function (currentTodos) {
    return currentTodos.filter(function (todo) {
      return todo.id !== id;
    });
  });
}

// --- Filtering ---
let filteredTodos;

if (filter === 'active') {
  filteredTodos = todos.filter(function (todo) {
    return !todo.completed;
  });
} else if (filter === 'completed') {
  filteredTodos = todos.filter(function (todo) {
    return todo.completed;
  });
} else {
  filteredTodos = todos;
}

// --- Counting remaining items ---
const remainingCount = todos.filter(function (todo) {
  return !todo.completed;
}).length;

// --- Styles ---
const appStyle = {
  maxWidth: '500px',
  margin: '40px auto',
  backgroundColor: 'white',
  borderRadius: '8px',
  boxShadow: '0 2px 10px rgba(0,0,0,0.1)',
  overflow: 'hidden'
};
```

```
  const headerStyle = {
    textAlign: 'center',
    padding: '20px',
    backgroundColor: '#3498db',
    color: 'white',
    margin: 0
  };

  return (
    <div style={{ backgroundColor: '#ecf0f1', minHeight: '100vh', padding: '20px' }}>
      <div style={appStyle}>
        <h1 style={headerStyle}>My Todo List</h1>
        <TodoForm onAdd={handleAddTodo} />
        <TodoList
          todos={filteredTodos}
          onToggle={handleToggleTodo}
          onDelete={handleDeleteTodo}
        />
        <TodoFilter
          filter={filter}
          onFilterChange={setFilter}
          remainingCount={remainingCount}
        />
      </div>
    </div>
  );
}

export default App;
```

Let us go through the most important parts:

The State

- `const [todos, setTodos] = React.useState([...])` — We start with three example todos. Each todo is an object with `id`, `text`, and `completed`. In a real app, this would probably start as an empty array.
- `const [filter, setFilter] = React.useState('all')` — The filter starts as "all," meaning all todos are shown.

Adding a Todo

```jsx
function handleAddTodo(text) {
  const newTodo = {
    id: Date.now(),
    text: text,
    completed: false
```

```
  };

  setTodos(function (currentTodos) {
    return [...currentTodos, newTodo];
  });
}
```

- `id: Date.now()` — We use the current timestamp as a unique ID. `Date.now()` returns the number of milliseconds since January 1, 1970. Since this number is always increasing, each todo gets a unique ID. (In a real app, the server would usually provide IDs.)
- `completed: false` — New todos start as not completed.
- `[...currentTodos, newTodo]` — We create a new array that contains all existing todos plus the new one. The `...` (spread operator) copies all items from the current array.

Toggling a Todo

```jsx
function handleToggleTodo(id) {
  setTodos(function (currentTodos) {
    return currentTodos.map(function (todo) {
      if (todo.id === id) {
        return { ...todo, completed: !todo.completed };
      }
      return todo;
    });
  });
}
```

- `currentTodos.map(...)` — We loop through all todos and create a new array.
- `if (todo.id === id)` — We find the todo that was toggled.
- `{ ...todo, completed: !todo.completed }` — We create a copy of that todo but flip its `completed` status. `!todo.completed` turns `true` into `false` and `false` into `true`. The `!` is the not operator — it flips a boolean value.
- `return todo;` — All other todos stay the same.

Deleting a Todo

```jsx
function handleDeleteTodo(id) {
  setTodos(function (currentTodos) {
    return currentTodos.filter(function (todo) {
      return todo.id !== id;
    });
  });
}
```

- `currentTodos.filter(...)` — `filter` creates a new array that only includes items where the condition is `true`.
- `todo.id !== id` — Keep every todo whose ID does NOT match the one we want to delete. The deleted todo is simply left out of the new array.

Filtering the List

```jsx
let filteredTodos;

if (filter === 'active') {
  filteredTodos = todos.filter(function (todo) {
    return !todo.completed;
  });
} else if (filter === 'completed') {
  filteredTodos = todos.filter(function (todo) {
    return todo.completed;
  });
} else {
  filteredTodos = todos;
}
```

- If the filter is "active," we only show todos that are NOT completed.
- If the filter is "completed," we only show todos that ARE completed.
- Otherwise (filter is "all"), we show all todos.
- We pass `filteredTodos` (not `todos`) to the `TodoList` component. The original `todos` array is never changed by filtering — we just create a new view of it.

Counting Remaining Items

```jsx
const remainingCount = todos.filter(function (todo) {
  return !todo.completed;
}).length;
```

- We filter the todos to get only the ones that are not completed, then count them using `.length`.
- We always count from the full `todos` array, not from `filteredTodos`. This way, the count is always accurate regardless of which filter is active.

Expected Output

When the app loads, you see:

1. A blue header that says "My Todo List."
2. An input field with the placeholder "What do you need to do?" and an "Add" button.
3. Three todo items:
    - "Learn React basics" — checked, with gray text and a line through it.
    - "Build a todo app" — unchecked, with normal black text.
    - "Practice every day" — unchecked, with normal black text.
4. A footer showing "2 items left" and three filter buttons: All, Active, Completed.

You can:

- Type "Read a book" in the input and click "Add" (or press Enter). A new todo appears at the bottom.
- Click the checkbox next to "Build a todo app." It becomes crossed out and gray.
- Click "Delete" next to any todo to remove it.
- Click "Active" to see only uncompleted todos.
- Click "Completed" to see only completed todos.
- Click "All" to see everything again.
- The item count updates automatically as you complete or delete todos.

Adding Simple CSS

So far, we used inline styles (styles written directly in JavaScript). For a small project, inline styles work fine. But as your project grows, a separate CSS file is cleaner.

Here is a CSS file you can use instead of (or alongside) inline styles:

```css
/* src/App.css */

body {
  margin: 0;
  font-family: Arial, sans-serif;
  background-color: #ecf0f1;
}

.app-container {
  max-width: 500px;
  margin: 40px auto;
  background-color: white;
```

```
  border-radius: 8px;
  box-shadow: 0 2px 10px rgba(0, 0, 0, 0.1);
  overflow: hidden;
}

.app-header {
  text-align: center;
  padding: 20px;
  background-color: #3498db;
  color: white;
  margin: 0;
}

.todo-form {
  display: flex;
  gap: 10px;
  padding: 20px;
  border-bottom: 2px solid #eee;
}

.todo-input {
  flex: 1;
  padding: 10px;
  font-size: 16px;
  border: 1px solid #ddd;
  border-radius: 4px;
}

.todo-input:focus {
  outline: none;
  border-color: #3498db;
}

.add-button {
  padding: 10px 20px;
  font-size: 16px;
  background-color: #3498db;
  color: white;
  border: none;
  border-radius: 4px;
  cursor: pointer;
}

.add-button:hover {
  background-color: #2980b9;
}

.todo-item {
  display: flex;
  align-items: center;
  justify-content: space-between;
  padding: 12px 16px;
  border-bottom: 1px solid #eee;
}
```

```
.todo-item.completed {
  background-color: #f0f0f0;
}

.todo-text {
  flex: 1;
  margin-left: 10px;
}

.todo-text.completed {
  text-decoration: line-through;
  color: #999;
}

.delete-button {
  background-color: #e74c3c;
  color: white;
  border: none;
  padding: 4px 10px;
  border-radius: 4px;
  cursor: pointer;
  font-size: 12px;
}

.delete-button:hover {
  background-color: #c0392b;
}

.filter-bar {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 12px 16px;
  background-color: #f8f9fa;
  border-top: 2px solid #eee;
}

.filter-button {
  padding: 6px 14px;
  border: 2px solid transparent;
  background-color: transparent;
  color: #555;
  border-radius: 4px;
  cursor: pointer;
  font-size: 14px;
}

.filter-button.active {
  border-color: #3498db;
  background-color: #3498db;
  color: white;
}

.remaining-count {
  color: #888;
```

```
  font-size: 14px;
}

.empty-message {
  text-align: center;
  color: #999;
  padding: 20px;
}
```

To use this CSS file, import it at the top of your App component:

```jsx
import './App.css';
```

Then replace the `style` props in your JSX with `className` props. For example, change:

```jsx
<div style={appStyle}>
```

to:

```jsx
<div className="app-container">
```

The `className` prop in React is the same as the `class` attribute in HTML. We use `className` instead of `class` because `class` is a reserved word in JavaScript.

The CSS gives you extra benefits like `:hover` effects (the button changes color when you move your mouse over it) and `:focus` effects (the input border changes color when you click into it). These are harder to do with inline styles.

What to Do When You Get Stuck

Every developer gets stuck. It is a normal part of programming. Here are some tips for when things do not work:

Tip 1: Read the Error Message

Error messages look scary, but they are actually trying to help you. Focus on:

- The first line. It usually tells you what went wrong.
- The file name and line number. It tells you where the problem is.

For example: `TypeError: Cannot read properties of undefined (reading 'map')` — This means you tried to use `.map()` on something that is `undefined`. Check that you are passing the right data.

Tip 2: Use console.log

When you are not sure what a variable contains, add `console.log` to check:

```jsx
function handleAddTodo(text) {
  console.log('Adding todo:', text);
  // rest of the function
}
```

Open your browser's developer tools (press F12 or right-click and choose "Inspect," then click the "Console" tab) to see the output.

Tip 3: Check Your Props

A very common bug is passing the wrong prop or forgetting to pass a prop. If a component is not working:

1. Add `console.log(props)` at the top of the component to see what it received.
2. Check the parent component to make sure it is passing the right values.
3. Make sure prop names match exactly (they are case-sensitive).

Tip 4: Start Small

If something is not working, simplify. Remove parts of the code until the basic version works. Then add features back one at a time. This helps you find exactly where the problem is.

Tip 5: Compare with Working Code

If you are stuck on this project, compare your code with the examples in this chapter. Look for small differences: a missing comma, a misspelled variable name, a missing `return` statement.

Tip 6: Take a Break

Sometimes the best debugging tool is walking away for a few minutes. Your brain often solves the problem in the background. Come back with fresh eyes, and the answer might be obvious.

Complete Project Code

Here is the full code for every file in the project. If you got lost at any point, you can refer to this section.

src/components/TodoItem.jsx

```jsx
function TodoItem({ todo, onToggle, onDelete }) {
  const itemStyle = {
    display: 'flex',
    alignItems: 'center',
    justifyContent: 'space-between',
    padding: '12px 16px',
    borderBottom: '1px solid #eee',
    backgroundColor: todo.completed ? '#f0f0f0' : 'white'
  };

  const textStyle = {
    textDecoration: todo.completed ? 'line-through' : 'none',
    color: todo.completed ? '#999' : '#333',
    flex: 1,
    marginLeft: '10px'
  };

  const deleteButtonStyle = {
    backgroundColor: '#e74c3c',
    color: 'white',
    border: 'none',
    padding: '4px 10px',
    borderRadius: '4px',
    cursor: 'pointer',
    fontSize: '12px'
  };

  return (
    <div style={itemStyle}>
      <input
        type="checkbox"
        checked={todo.completed}
        onChange={function () {
          onToggle(todo.id);
        }}
      />
      <span style={textStyle}>{todo.text}</span>
      <button
        style={deleteButtonStyle}
        onClick={function () {
          onDelete(todo.id);
        }}
      >
```

```
        Delete
      </button>
    </div>
  );
}

export default TodoItem;
```

src/components/TodoList.jsx

```jsx
import TodoItem from './TodoItem';

function TodoList({ todos, onToggle, onDelete }) {
  if (todos.length === 0) {
    return (
      <p style={{ textAlign: 'center', color: '#999', padding: '20px' }}>
        No todos to show. Add one above!
      </p>
    );
  }

  return (
    <div>
      {todos.map(function (todo) {
        return (
          <TodoItem
            key={todo.id}
            todo={todo}
            onToggle={onToggle}
            onDelete={onDelete}
          />
        );
      })}
    </div>
  );
}

export default TodoList;
```

src/components/TodoForm.jsx

```jsx
import React from 'react';

function TodoForm({ onAdd }) {
  const [text, setText] = React.useState('');
```

```
  function handleSubmit(event) {
    event.preventDefault();

    const trimmedText = text.trim();

    if (trimmedText === '') {
      return;
    }

    onAdd(trimmedText);
    setText('');
  }

  const formStyle = {
    display: 'flex',
    gap: '10px',
    padding: '20px',
    borderBottom: '2px solid #eee'
  };

  const inputStyle = {
    flex: 1,
    padding: '10px',
    fontSize: '16px',
    border: '1px solid #ddd',
    borderRadius: '4px'
  };

  const buttonStyle = {
    padding: '10px 20px',
    fontSize: '16px',
    backgroundColor: '#3498db',
    color: 'white',
    border: 'none',
    borderRadius: '4px',
    cursor: 'pointer'
  };

  return (
    <form onSubmit={handleSubmit} style={formStyle}>
      <input
        type="text"
        placeholder="What do you need to do?"
        value={text}
        onChange={function (event) {
          setText(event.target.value);
        }}
        style={inputStyle}
      />
      <button type="submit" style={buttonStyle}>
        Add
      </button>
    </form>
  );
}
```

```
export default TodoForm;
```

src/components/TodoFilter.jsx

```jsx
function TodoFilter({ filter, onFilterChange, remainingCount }) {
  const containerStyle = {
    display: 'flex',
    justifyContent: 'space-between',
    alignItems: 'center',
    padding: '12px 16px',
    backgroundColor: '#f8f9fa',
    borderTop: '2px solid #eee'
  };

  function getButtonStyle(buttonFilter) {
    const isActive = filter === buttonFilter;

    return {
      padding: '6px 14px',
      border: isActive ? '2px solid #3498db' : '2px solid transparent',
      backgroundColor: isActive ? '#3498db' : 'transparent',
      color: isActive ? 'white' : '#555',
      borderRadius: '4px',
      cursor: 'pointer',
      fontSize: '14px'
    };
  }

  return (
    <div style={containerStyle}>
      <span style={{ color: '#888', fontSize: '14px' }}>
        {remainingCount} {remainingCount === 1 ? 'item' : 'items'} left
      </span>
      <div style={{ display: 'flex', gap: '5px' }}>
        <button
          style={getButtonStyle('all')}
          onClick={function () { onFilterChange('all'); }}
        >
          All
        </button>
        <button
          style={getButtonStyle('active')}
          onClick={function () { onFilterChange('active'); }}
        >
          Active
        </button>
        <button
          style={getButtonStyle('completed')}
          onClick={function () { onFilterChange('completed'); }}
```

```
        >
          Completed
        </button>
      </div>
    </div>
  );
}

export default TodoFilter;
```

src/App.jsx

```jsx
import React from 'react';
import TodoForm from './components/TodoForm';
import TodoList from './components/TodoList';
import TodoFilter from './components/TodoFilter';

function App() {
  const [todos, setTodos] = React.useState([
    { id: 1, text: 'Learn React basics', completed: true },
    { id: 2, text: 'Build a todo app', completed: false },
    { id: 3, text: 'Practice every day', completed: false }
  ]);

  const [filter, setFilter] = React.useState('all');

  function handleAddTodo(text) {
    const newTodo = {
      id: Date.now(),
      text: text,
      completed: false
    };

    setTodos(function (currentTodos) {
      return [...currentTodos, newTodo];
    });
  }

  function handleToggleTodo(id) {
    setTodos(function (currentTodos) {
      return currentTodos.map(function (todo) {
        if (todo.id === id) {
          return { ...todo, completed: !todo.completed };
        }
        return todo;
      });
    });
  }

  function handleDeleteTodo(id) {
```

```
    setTodos(function (currentTodos) {
      return currentTodos.filter(function (todo) {
        return todo.id !== id;
      });
    });
  }

  let filteredTodos;

  if (filter === 'active') {
    filteredTodos = todos.filter(function (todo) {
      return !todo.completed;
    });
  } else if (filter === 'completed') {
    filteredTodos = todos.filter(function (todo) {
      return todo.completed;
    });
  } else {
    filteredTodos = todos;
  }

  const remainingCount = todos.filter(function (todo) {
    return !todo.completed;
  }).length;

  const appStyle = {
    maxWidth: '500px',
    margin: '40px auto',
    backgroundColor: 'white',
    borderRadius: '8px',
    boxShadow: '0 2px 10px rgba(0,0,0,0.1)',
    overflow: 'hidden'
  };

  const headerStyle = {
    textAlign: 'center',
    padding: '20px',
    backgroundColor: '#3498db',
    color: 'white',
    margin: 0
  };

  return (
    <div style={{ backgroundColor: '#ecf0f1', minHeight: '100vh', padding: '20px' }}>
      <div style={appStyle}>
        <h1 style={headerStyle}>My Todo List</h1>
        <TodoForm onAdd={handleAddTodo} />
        <TodoList
          todos={filteredTodos}
          onToggle={handleToggleTodo}
          onDelete={handleDeleteTodo}
        />
        <TodoFilter
          filter={filter}
          onFilterChange={setFilter}
```

```
          remainingCount={remainingCount}
        />
      </div>
    </div>
  );
}

export default App;
```

Quick Summary

- Plan before you code. Ask: What should it do? What components do I need? What state do I need?
- Build bottom-up. Start with the smallest components and work your way up.
- Keep state where it is needed. The `App` component holds shared state (todos, filter). The `TodoForm` keeps its own local state (input text).
- Pass data down with props, events up with functions. This is the pattern from Chapter 18 (lifting state up) in action.
- Use `.map()` to render lists, `.filter()` to filter data, and the spread operator (`...`) to create copies.
- Simple CSS can be added in a separate file for cleaner code and extra features like hover effects.

Key Points to Remember

1. Planning saves time. Even five minutes of thinking before coding prevents hours of confusion.
2. Break your app into small components. Each component should do one thing well.
3. State that multiple components need should live in their shared parent.
4. State that only one component needs (like form input text) can stay local.
5. Never modify state directly. Always create new arrays and objects using spread (`...`), `map`, and `filter`.
6. Error messages are your friends. Read them carefully — they tell you what went wrong and where.

7. Use `console.log` liberally when debugging. It is the simplest and most effective debugging tool.

Practice Questions

1. Why did we put the `todos` state in the `App` component instead of in `TodoList`?
2. Why does `TodoForm` keep its own local state for the input text instead of lifting it up to `App`?
3. What does `Date.now()` do, and why do we use it as an ID?
4. What is the difference between `todos` and `filteredTodos` in the `App` component?
5. Why do we use `event.preventDefault()` in the form submit handler?

Exercises

Exercise 1: Edit a Todo

Add the ability to edit an existing todo. When the user double-clicks on a todo's text, it should turn into an input field. The user can type a new text and press Enter to save. (Hint: You will need a new piece of state in `TodoItem` to track whether it is being edited.)

Exercise 2: Clear All Completed

Add a "Clear Completed" button to the filter bar. When clicked, it should remove all completed todos at once. (Hint: Use `filter` to keep only todos where `completed` is `false`.)

Exercise 3: Save to Local Storage

Make the todos persist even after the browser is closed. Use `localStorage` to save the todos whenever they change, and load them when the app starts. (Hint: `localStorage.setItem('todos', JSON.stringify(todos))` saves data, and `JSON.parse(localStorage.getItem('todos'))` loads it.)

What Is Next?

Congratulations! You have built your first complete React application. You planned it, broke it into components, managed state, handled user interactions, and styled it. These are real skills that professional developers use every day.

This Todo App is just the beginning. Here are some ideas for what to explore next:

- Add routing (from Chapter 19) to give your app multiple pages.
- Learn about `useEffect` to fetch data from the internet or save data automatically.
- Explore Context API to share data across many components without passing props through every level.
- Try a UI library like Material UI or Tailwind CSS to make your apps look polished.
- Build more projects. The best way to learn is by building. Try a weather app, a note-taking app, or a simple game.

Remember: every expert was once a beginner. You have come a long way, and the most important step is the next one. Keep building, keep learning, and keep having fun with React.

Chapter 21: Common Mistakes Beginners Make

Every programmer makes mistakes. Even experienced developers make mistakes every day. The difference is that experienced developers know how to recognize mistakes quickly and fix them.

In this chapter, we will look at the most common mistakes that beginners make when learning React. For each mistake, you will see what the mistake looks like, why it happens, what error message you might see, and how to fix it.

Think of this chapter like a troubleshooting guide. When something goes wrong in your code, come back here. There is a good chance the answer is in this chapter.

What You Will Learn

- 15 common mistakes that React beginners make
- Why each mistake happens
- What error messages to look for
- How to fix each mistake quickly

Why This Chapter Matters

When you are learning, mistakes can be frustrating. You might spend hours trying to figure out why your code is not working. This chapter will save you time. It will help you recognize problems quickly. It will also help you understand React better. Every mistake teaches you something about how React works.

Mistake 1: Forgetting to Return JSX

What the Mistake Looks Like

```jsx
function Greeting() {
  const name = "Alice";
  // Oops! No return statement
  <h1>Hello, {name}!</h1>
}
```

Why It Happens

In JavaScript, a function does not give anything back unless you use the `return` keyword. The word "return" means "give this back to whoever asked." Without it, your component gives back `undefined`. That means "nothing." React does not know what to show on the screen.

What You See

Your component shows nothing on the screen. No error message appears. This makes it hard to find the problem.

The Fix

Always use the `return` keyword before your JSX.

```jsx
function Greeting() {
  const name = "Alice";
  return <h1>Hello, {name}!</h1>;
}
```

Another Version of This Mistake

Sometimes this happens with curly braces `{}` instead of parentheses `()` after an arrow function.

```jsx
// WRONG – curly braces create a function body, so you need return
const Greeting = () => {
  <h1>Hello!</h1>
};
```

```
// RIGHT – option 1: add return
const Greeting = () => {
  return <h1>Hello!</h1>;
};

// RIGHT – option 2: use parentheses for implicit return
const Greeting = () => (
  <h1>Hello!</h1>
);
```

The word "implicit" means "without saying it directly." When you use parentheses with an arrow function, JavaScript automatically returns what is inside them. When you use curly braces, you must write `return` yourself.

Mistake 2: Using `class` Instead of `className`

What the Mistake Looks Like

```jsx
function Button() {
  return <button class="primary-btn">Click Me</button>;
}
```

Why It Happens

In regular HTML, you use `class` to add CSS styles to an element. But in React, you write JSX, not HTML. JSX is JavaScript. In JavaScript, `class` is a reserved word. A "reserved word" is a word that JavaScript already uses for something else. So React uses `className` instead.

What You See

React shows a warning in the browser console:

```
Warning: Invalid DOM property `class`. Did you mean `className`?
```

The Fix

Use `className` instead of `class`.

```jsx
function Button() {
  return <button className="primary-btn">Click Me</button>;
}
```

Quick Tip

This is easy to forget. If your CSS styles are not working, check if you used `class` instead of `className`.

Mistake 3: Not Wrapping Multiple Elements in a Parent

What the Mistake Looks Like

```jsx
function UserInfo() {
  return (
    <h1>Alice</h1>
    <p>Age: 25</p>
  );
}
```

Why It Happens

A React component can only return one element. Think of it like a gift box. You can put many things inside one box, but you can only hand someone one box at a time. If you try to return two elements side by side, React does not know how to handle it.

What You See

You see an error message like this:

```
SyntaxError: Adjacent JSX elements must be wrapped in an enclosing tag.
```

The word "adjacent" means "next to each other." The word "enclosing" means "surrounding."

The Fix

Wrap your elements in a parent element. You have three options.

Option 1: Use a `<div>`

```jsx
function UserInfo() {
  return (
    <div>
      <h1>Alice</h1>
      <p>Age: 25</p>
    </div>
  );
}
```

Option 2: Use a Fragment `<> </>`

A Fragment is an invisible wrapper. It groups elements together without adding an extra HTML element to your page.

```jsx
function UserInfo() {
  return (
    <>
      <h1>Alice</h1>
      <p>Age: 25</p>
    </>
  );
}
```

Option 3: Use `<React.Fragment>`

This does the same thing as `<>`, but you can add a `key` prop to it. We will explain keys later in this chapter.

```jsx
function UserInfo() {
  return (
    <React.Fragment>
      <h1>Alice</h1>
      <p>Age: 25</p>
    </React.Fragment>
  );
}
```

Mistake 4: Mutating State Directly

What the Mistake Looks Like

```jsx
function ShoppingList() {
  const [items, setItems] = useState(["Milk", "Bread"]);

  function addItem() {
    items.push("Eggs"); // WRONG! This changes the array directly
    setItems(items);
  }

  return (
    <div>
      <button onClick={addItem}>Add Eggs</button>
      <ul>
        {items.map((item, index) => (
          <li key={index}>{item}</li>
        ))}
      </ul>
    </div>
  );
}
```

Why It Happens

The word "mutate" means "to change something directly." When you use `push()`, you are changing the original array. React does not notice this change. React only re-renders (shows updates on the screen) when it sees a completely new value. Since you pushed to the same array and then passed the same array to `setItems`, React thinks nothing changed.

Think of it like this. Imagine you have a whiteboard with a grocery list. If you erase an item and write a new one on the same whiteboard, someone looking from far away might not notice the change. But if you throw away the old whiteboard and hold up a brand new one, everyone will notice.

What You See

The screen does not update. No error message appears. The button seems to do nothing.

The Fix

Create a new array with the spread operator `...` and add the new item to it.

```jsx
function ShoppingList() {
  const [items, setItems] = useState(["Milk", "Bread"]);

  function addItem() {
    setItems([...items, "Eggs"]); // RIGHT! Creates a new array
  }

  return (
    <div>
      <button onClick={addItem}>Add Eggs</button>
      <ul>
        {items.map((item, index) => (
          <li key={index}>{item}</li>
        ))}
      </ul>
    </div>
  );
}
```

The `...items` part means "copy everything from the old array." Then `"Eggs"` is added at the end. This creates a brand new array. React sees the new array and updates the screen.

The Same Rule Applies to Objects

```jsx
// WRONG
const [user, setUser] = useState({ name: "Alice", age: 25 });
user.age = 26; // Mutating directly
setUser(user);

// RIGHT
setUser({ ...user, age: 26 }); // Create a new object
```

Mistake 5: Infinite Loops in useEffect

What the Mistake Looks Like

```jsx
function Counter() {
  const [count, setCount] = useState(0);

  useEffect(() => {
    setCount(count + 1); // This runs, updates state, which triggers useEffect again...
  }); // No dependency array!
```

```
  return <p>Count: {count}</p>;
}
```

Why It Happens

`useEffect` runs after your component renders. When there is no dependency array (the square brackets `[]` at the end), `useEffect` runs after every single render. Inside the effect, `setCount` updates the state. Updating state causes the component to render again. After the render, `useEffect` runs again. This creates an endless loop.

Think of it like a dog chasing its own tail. The effect updates the state. The state update causes a re-render. The re-render triggers the effect. The effect updates the state again. It never stops.

What You See

Your browser becomes very slow or freezes. You might see this error in the console:

```
Error: Too many re-renders. React limits the number of renders to prevent an infinite l...
```

The Fix

Add a dependency array to control when `useEffect` runs.

```jsx
// Run only once, when the component first appears
useEffect(() => {
  setCount(count + 1);
}, []); // Empty array = run once

// Run only when "count" changes
useEffect(() => {
  document.title = `Count: ${count}`;
}, [count]); // Runs when count changes
```

The dependency array tells React: "Only run this effect when these values change." An empty array `[]` means "run this effect only once, when the component first appears on the screen."

Mistake 6: Forgetting Keys in Lists

What the Mistake Looks Like

```jsx
function FruitList() {
  const fruits = ["Apple", "Banana", "Cherry"];

  return (
    <ul>
      {fruits.map((fruit) => (
        <li>{fruit}</li>
      ))}
    </ul>
  );
}
```

Why It Happens

When you create a list in React, each item needs a unique `key`. A "key" is like a name tag. It helps React keep track of each item. Without keys, React cannot tell which items changed, which were added, or which were removed.

Think of a classroom. If students do not have name tags, the teacher cannot tell who is who. If a new student joins, the teacher does not know where to seat them.

What You See

React shows a warning in the console:

```
Warning: Each child in a list should have a unique "key" prop.
```

The Fix

Add a `key` prop to each item in the list. Use a unique value, like an `id` from your data.

```jsx
function FruitList() {
  const fruits = ["Apple", "Banana", "Cherry"];

  return (
    <ul>
      {fruits.map((fruit) => (
```

```
        <li key={fruit}>{fruit}</li>
      ))}
    </ul>
  );
}
```

Important: Do Not Use Array Index as Key (When Possible)

Using the index (position number) as a key can cause problems if the list order changes.

```jsx
// Not ideal - using index as key
{fruits.map((fruit, index) => (
  <li key={index}>{fruit}</li>
))}

// Better - using a unique value
{users.map((user) => (
  <li key={user.id}>{user.name}</li>
))}
```

Use the array index only as a last resort, when your items do not have a unique identifier and the list never changes order.

Mistake 7: Calling Hooks Inside if Statements or Loops

What the Mistake Looks Like

```jsx
function Profile({ isLoggedIn }) {
  if (isLoggedIn) {
    const [name, setName] = useState("Alice"); // WRONG!
  }

  return <p>Welcome!</p>;
}
```

Or inside a loop:

```jsx
function UserList({ users }) {
  for (let user of users) {
    const [selected, setSelected] = useState(false); // WRONG!
  }

  return <p>Users</p>;
}
```

Why It Happens

React needs hooks to run in the same order every time your component renders. If a hook is inside an `if` statement, it might run sometimes and not run other times. This confuses React. React keeps track of hooks by their position (first hook, second hook, third hook...). If the order changes, React gets mixed up.

Think of it like a row of numbered lockers. React puts each hook's value in a specific locker. Hook 1 goes in locker 1. Hook 2 goes in locker 2. If you skip a hook sometimes, the values end up in the wrong lockers.

What You See

```
Error: React Hook "useState" is called conditionally. React Hooks must be
called in the exact same order in every component render.
```

The Fix

Always call hooks at the top level of your component. Put the `if` statement inside the hook, not around it.

```jsx
function Profile({ isLoggedIn }) {
  const [name, setName] = useState("Alice"); // Always call the hook

  if (!isLoggedIn) {
    return <p>Please log in.</p>; // Conditional rendering is fine
  }

  return <p>Welcome, {name}!</p>;
}
```

The Rules of Hooks

1. Only call hooks at the top of your component function.

2. Do not put hooks inside `if` statements.
3. Do not put hooks inside loops.
4. Do not put hooks inside nested functions.

Mistake 8: Calling a Function Instead of Passing a Reference

What the Mistake Looks Like

```jsx
function App() {
  function handleClick() {
    alert("Button clicked!");
  }

  return <button onClick={handleClick()}>Click Me</button>;
}
```

Why It Happens

There is a big difference between `handleClick` and `handleClick()`.

- `handleClick` is a reference to the function. It means "here is the function, call it later when the button is clicked."
- `handleClick()` calls the function immediately. The parentheses `()` mean "run this right now."

Think of it like giving someone a phone number versus calling the phone number. `handleClick` is like writing down the number. `handleClick()` is like dialing the number right away.

What You See

The function runs immediately when the page loads, not when you click the button. If the function updates state, you might get an infinite loop.

The Fix

Remove the parentheses. Pass the function reference.

```jsx
// WRONG - calls the function immediately
<button onClick={handleClick()}>Click Me</button>

// RIGHT - passes a reference to the function
<button onClick={handleClick}>Click Me</button>
```

What If You Need to Pass an Argument?

If you need to pass a value to the function, wrap it in an arrow function.

```jsx
// WRONG
<button onClick={deleteItem(5)}>Delete</button>

// RIGHT
<button onClick={() => deleteItem(5)}>Delete</button>
```

The arrow function `() => deleteItem(5)` creates a new function that says "when the button is clicked, call `deleteItem` with the number 5."

Mistake 9: Not Handling Loading and Error States for API Calls

What the Mistake Looks Like

```jsx
function UserProfile() {
  const [user, setUser] = useState(null);

  useEffect(() => {
    fetch("https://api.example.com/user/1")
      .then((response) => response.json())
      .then((data) => setUser(data));
  }, []);

  return (
    <div>
      <h1>{user.name}</h1>  {/* CRASH! user is null at first */}
      <p>{user.email}</p>
    </div>
  );
}
```

Why It Happens

When your component first appears, `user` is `null` because the data has not arrived yet. The API call takes time. But your component tries to show `user.name` right away. You cannot read `.name` from `null`. This causes a crash.

Think of it like opening the oven before the food is cooked. You need to wait for the food to be ready.

What You See

```
TypeError: Cannot read properties of null (reading 'name')
```

The Fix

Add loading and error states. Check if the data is ready before showing it.

```jsx
function UserProfile() {
  const [user, setUser] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    fetch("https://api.example.com/user/1")
      .then((response) => response.json())
      .then((data) => {
        setUser(data);
        setLoading(false);
      })
      .catch((err) => {
        setError("Something went wrong.");
        setLoading(false);
      });
  }, []);

  if (loading) return <p>Loading...</p>;
  if (error) return <p>{error}</p>;

  return (
    <div>
      <h1>{user.name}</h1>
      <p>{user.email}</p>
    </div>
  );
}
```

Line-by-Line Explanation

- `useState(true)` for loading: The data is loading at first, so we start with `true`.
- `useState(null)` for error: There is no error at first, so we start with `null`.
- `setLoading(false)`: When data arrives (or an error happens), we are no longer loading.
- `if (loading)`: Show a loading message while we wait.
- `if (error)`: Show the error message if something went wrong.
- Only after both checks pass do we show the actual data.

Mistake 10: Typos in State Variable Names

What the Mistake Looks Like

```jsx
function Counter() {
  const [count, setCount] = useState(0);

  function increment() {
    setCont(count + 1); // Typo! "setCont" instead of "setCount"
  }

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={increment}>Add 1</button>
    </div>
  );
}
```

Why It Happens

It is easy to make spelling mistakes, especially with longer variable names. JavaScript does not warn you about undefined variables until you try to use them.

What You See

```
ReferenceError: setCont is not defined
```

Or sometimes your code just does nothing, and you spend a long time trying to figure out why.

The Fix

Double-check your variable names. Make sure the setter function name matches what you declared.

```jsx
const [count, setCount] = useState(0);
//             ^^^^^^^^
// Make sure you use this exact name everywhere

function increment() {
  setCount(count + 1); // Correct spelling
}
```

Tip

Many code editors highlight undefined variables. If you see a variable name with a red or yellow underline, check the spelling.

Mistake 11: Importing Components with Lowercase Names

What the Mistake Looks Like

```jsx
// In the file header.js
function header() {  // lowercase "h"
  return <h1>Welcome to My App</h1>;
}

export default header;

// In App.js
import header from "./header";

function App() {
  return <header />;  // React thinks this is an HTML <header> tag!
}
```

Why It Happens

React uses a simple rule to tell the difference between HTML tags and your custom components. If the name starts with a lowercase letter, React treats it as an HTML element. If it starts with an uppercase letter, React treats it as a component.

The name `header` starts with a lowercase "h", so React thinks you want the HTML `<header>` tag, not your custom component.

What You See

Your component does not show the expected content. Instead, React renders an HTML `<header>` element.

The Fix

Always start component names with an uppercase letter.

```jsx
// In Header.js
function Header() {  // Uppercase "H"
  return <h1>Welcome to My App</h1>;
}

export default Header;

// In App.js
import Header from "./Header";

function App() {
  return <Header />;  // Now React knows this is a component
}
```

Mistake 12: Not Installing Packages Before Importing Them

What the Mistake Looks Like

```jsx
import axios from "axios"; // You never installed axios!

function App() {
  // ...
}
```

Why It Happens

When you follow a tutorial, you might copy the import statement but forget to install the package first. A "package" is a piece of code that someone else wrote and shared. You need to download it to your computer before you can use it.

What You See

```
Module not found: Error: Can't resolve 'axios'
```

Or:

```
Module not found: Can't resolve 'react-router-dom'
```

The Fix

Install the package using npm (Node Package Manager) in your terminal before importing it.

```bash
npm install axios
```

Then you can import it in your code:

```jsx
import axios from "axios";
```

How to Know If You Need to Install Something

If your import path does NOT start with `./` or `../`, it is probably an external package that needs to be installed.

```jsx
import Header from "./Header";     // Local file - no install needed
import axios from "axios";          // External package - must install
import { BrowserRouter } from "react-router-dom"; // External - must install
```

Mistake 13: Forgetting to Export Components

What the Mistake Looks Like

```jsx
// In Greeting.js
function Greeting() {
  return <h1>Hello!</h1>;
}

// Oops! Forgot to export
```

```jsx
// In App.js
import Greeting from "./Greeting"; // This will not work!
```

Why It Happens

In JavaScript, each file is like a private room. Nothing can get in or out unless you open the door. The `export` keyword opens the door so other files can use your component.

What You See

```
Attempted import error: './Greeting' does not contain a default export.
```

Or your component shows as `undefined` or does not appear at all.

The Fix

Add `export default` to your component.

```jsx
// Option 1: Export at the bottom
function Greeting() {
  return <h1>Hello!</h1>;
}

export default Greeting;

// Option 2: Export on the same line
export default function Greeting() {
  return <h1>Hello!</h1>;
}
```

Mistake 14: Using `=` Instead of `===` in Conditions

What the Mistake Looks Like

```jsx
function StatusMessage({ status }) {
  if (status = "active") {  // WRONG! Single = is assignment, not comparison
    return <p>User is active</p>;
  }
  return <p>User is not active</p>;
}
```

Why It Happens

In JavaScript, `=` and `===` do very different things:

- `=` means "assign this value" (put a value into a variable)
- `===` means "is this equal to?" (compare two values)

Using `=` in an `if` statement assigns the value instead of comparing it. This means the condition is always true (because the assignment is successful), so your code always takes the first path.

Think of it like this. `=` is like writing on a name tag. `===` is like reading a name tag and checking if it matches.

What You See

No error message. But your code always behaves as if the condition is true, even when it should be false.

The Fix

Use `===` for comparisons.

```jsx
function StatusMessage({ status }) {
  if (status === "active") {  // RIGHT! Triple = compares values
    return <p>User is active</p>;
  }
  return <p>User is not active</p>;
}
```

Quick Reference

Symbol	Meaning	Example
`=`	Assign a value	`let x = 5`
`==`	Loose comparison (not recommended)	`5 == "5"` is `true`
`===`	Strict comparison (recommended)	`5 === "5"` is `false`

Always use `===` in React. It is safer and more predictable.

Mistake 15: Not Using the Key Prop Correctly

What the Mistake Looks Like

```jsx
// Using the same key for different items
function TaskList({ tasks }) {
  return (
    <ul>
      {tasks.map((task) => (
        <li key="task">{task.name}</li>  // Same key for every item!
      ))}
    </ul>
  );
}
```

Or using an index when items can be reordered:

```jsx
function TaskList({ tasks }) {
  return (
    <ul>
      {tasks.map((task, index) => (
        <li key={index}>
          <input type="checkbox" />
          {task.name}
        </li>
      ))}
    </ul>
  );
}
```

Why It Happens

Beginners sometimes set every key to the same value, or always use the array index. Keys must be unique among siblings. If two items have the same key, React cannot tell them apart.

Using the index as a key seems harmless, but it causes problems when items are added, removed, or reordered. React uses keys to match old elements with new elements. If the order changes, the indexes change, and React gets confused about which item is which.

What You See

With duplicate keys, you might see unexpected behavior. Items might not update correctly. Checkboxes might check the wrong items. Input fields might show the wrong values.

The Fix

Use a unique identifier from your data as the key.

```jsx
function TaskList({ tasks }) {
  return (
    <ul>
      {tasks.map((task) => (
        <li key={task.id}>
          <input type="checkbox" />
          {task.name}
        </li>
      ))}
    </ul>
  );
}
```

What Makes a Good Key?

- It should be unique among siblings (no two items in the same list should have the same key).
- It should be stable (the same item should always have the same key, even if the list is reordered).
- It should come from your data (like an `id` field).

Quick Summary

Here is a quick reference table of all 15 mistakes:

#	Mistake	Fix
1	Forgetting to return JSX	Add `return` before your JSX
2	Using `class` instead of `className`	Use `className` in JSX
3	Not wrapping elements in a parent	Use `<div>` or `<> </>`
4	Mutating state directly	Use spread `...` to create new arrays/objects
5	Infinite loops in useEffect	Add a dependency array `[]`
6	Forgetting keys in lists	Add unique `key` prop to list items
7	Hooks inside if/loops	Call hooks at the top level only
8	Calling function instead of passing reference	Use `onClick={fn}` not `onClick={fn()}`
9	Not handling loading/error states	Add loading and error state variables
10	Typos in state variable names	Double-check spelling
11	Lowercase component names	Start component names with uppercase
12	Not installing packages	Run `npm install` first
13	Forgetting to export	Add `export default`
14	Using `=` instead of `===`	Use `===` for comparisons
15	Wrong key prop usage	Use unique, stable identifiers

Key Points to Remember

1. Always return JSX from your components. No return means nothing on the screen.
2. Use `className` instead of `class` in JSX.
3. Wrap multiple elements in a parent element or Fragment.
4. Never mutate state directly. Always create new arrays and objects.

5. Always add a dependency array to `useEffect` to avoid infinite loops.
6. Give each list item a unique key from your data.
7. Call hooks at the top level of your component, never inside conditions or loops.
8. Pass function references to event handlers. Do not call the function.
9. Handle loading and error states when fetching data.
10. Read your error messages carefully. They often tell you exactly what is wrong.

Practice Questions

1. What happens when you forget to add `return` before your JSX?
2. Why does React use `className` instead of `class`?
3. What is the difference between `onClick={handleClick}` and `onClick={handleClick()}`?
4. Why do you need to create a new array (with the spread operator) instead of using `push()` to update state?
5. What does the dependency array in `useEffect` do?

Exercises

Exercise 1: Find the Bugs

Look at this code. There are 4 mistakes. Can you find and fix all of them?

```jsx
function todoList() {
  const [todos, setTodos] = useState(["Buy milk", "Walk dog"]);

  function addTodo() {
    todos.push("Read book");
    setTodos(todos);
  }

  return (
    <h1>My Todos</h1>
    <button onClick={addTodo()}>Add Todo</button>
    <ul>
      {todos.map((todo) => (
        <li class="todo-item">{todo}</li>
```

```
      ))}
    </ul>
  );
}
```

Hints: Look for issues with the component name, state mutation, the onClick handler, JSX wrapping, className, and missing keys.

Exercise 2: Fix the useEffect

This code causes an infinite loop. Fix it so it only runs once.

```jsx
function WelcomeMessage() {
  const [message, setMessage] = useState("");

  useEffect(() => {
    setMessage("Welcome to our app!");
  });

  return <h1>{message}</h1>;
}
```

Exercise 3: Handle Loading State

This code crashes when the page first loads. Add loading and error handling.

```jsx
function ProductList() {
  const [products, setProducts] = useState(null);

  useEffect(() => {
    fetch("https://api.example.com/products")
      .then((res) => res.json())
      .then((data) => setProducts(data));
  }, []);

  return (
    <ul>
      {products.map((product) => (
        <li key={product.id}>{product.name}</li>
      ))}
    </ul>
  );
}
```

What Is Next?

Now that you know the most common mistakes and how to fix them, let us look at the positive side. In the next chapter, we will learn best practices for writing clean, organized React code. These are the habits that will make your code easier to read, easier to maintain, and more professional. Think of this chapter as learning what NOT to do. The next chapter is about learning what TO do.

Chapter 22: Best Practices for Clean React Code

In the last chapter, we learned what NOT to do. Now let us learn what TO do. This chapter is about habits that will make your React code clean, readable, and professional.

"Clean code" does not mean fancy code. It means code that is easy to read, easy to understand, and easy to change. Clean code is like a well-organized kitchen. Everything has a place. You can find what you need quickly. Someone new can walk in and understand where things are.

These best practices come from years of experience from thousands of React developers. You do not need to follow every single one from day one. Start with a few that make sense to you. Over time, they will become natural habits.

What You Will Learn

- How to keep your components small and focused
- How to name things clearly
- How to organize your files and folders
- How to write code that other people (and future you) can understand
- Habits that separate beginners from professionals

Why This Chapter Matters

Writing code that works is the first step. Writing code that is clean and organized is the next step. Clean code is easier to fix when bugs appear. Clean code is easier to add new features to. And if you ever work on a team, clean code is a gift to your teammates. It says, "I care about making this easy for everyone."

Best Practice 1: Keep Components Small

The Problem

Beginners often put everything into one big component. The component grows and grows until it is hundreds of lines long. It becomes hard to read, hard to understand, and hard to fix.

The Rule

If your component is longer than about 50 to 100 lines, it is probably doing too much. Split it into smaller components.

Example: A Big Component

```jsx
// This component is doing too many things
function UserDashboard() {
  const [user, setUser] = useState(null);
  const [posts, setPosts] = useState([]);
  const [notifications, setNotifications] = useState([]);

  // ... lots of useEffect calls, handlers, and logic ...

  return (
    <div>
      <div className="header">
        <img src={user.avatar} alt="Avatar" />
        <h1>{user.name}</h1>
        <p>{user.bio}</p>
      </div>
      <div className="posts">
        <h2>Recent Posts</h2>
        {posts.map((post) => (
          <div key={post.id}>
            <h3>{post.title}</h3>
            <p>{post.content}</p>
          </div>
        ))}
      </div>
      <div className="notifications">
        <h2>Notifications</h2>
        {notifications.map((notif) => (
          <div key={notif.id}>
            <p>{notif.message}</p>
          </div>
```

```
        ))}
      </div>
    </div>
  );
}
```

Better: Split Into Smaller Components

```jsx
function UserDashboard() {
  const [user, setUser] = useState(null);
  const [posts, setPosts] = useState([]);
  const [notifications, setNotifications] = useState([]);

  return (
    <div>
      <UserHeader user={user} />
      <PostList posts={posts} />
      <NotificationList notifications={notifications} />
    </div>
  );
}

function UserHeader({ user }) {
  return (
    <div className="header">
      <img src={user.avatar} alt="Avatar" />
      <h1>{user.name}</h1>
      <p>{user.bio}</p>
    </div>
  );
}

function PostList({ posts }) {
  return (
    <div className="posts">
      <h2>Recent Posts</h2>
      {posts.map((post) => (
        <PostItem key={post.id} post={post} />
      ))}
    </div>
  );
}

function PostItem({ post }) {
  return (
    <div>
      <h3>{post.title}</h3>
      <p>{post.content}</p>
    </div>
  );
}
```

Each component now has one job. `UserHeader` shows the user's information. `PostList` shows the list of posts. `PostItem` shows a single post. This is much easier to read and maintain.

How to Know When to Split

Ask yourself: "Can I describe what this component does in one sentence?" If you need the word "and" in your description, the component might be doing too much.

- "This component shows the user profile." (Good -- one job.)
- "This component shows the user profile AND lists their posts AND shows notifications." (Too many jobs. Split it.)

Best Practice 2: Name Components Clearly

The Rule

Name your components based on what they are, not what they do. Use nouns, not verbs.

Examples

Bad Name	Good Name	Why
`ShowUser`	`UserCard`	It is a card that shows user info
`DoSearch`	`SearchBar`	It is a search bar
`HandleLogin`	`LoginForm`	It is a login form
`FetchData`	`ProductList`	It is a list of products
`RenderItem`	`TodoItem`	It is a single todo item

Why This Matters

When you read JSX that uses well-named components, it reads almost like a description of the page:

```jsx
<div>
  <NavigationBar />
  <UserProfile />
  <RecentOrders />
  <Footer />
</div>
```

You can immediately understand what this page shows, even without looking at the component code.

Best Practice 3: One Component Per File

The Rule

Put each component in its own file. Name the file the same as the component.

Example

```
src/
  components/
    Header.js        (contains the Header component)
    Footer.js        (contains the Footer component)
    UserCard.js      (contains the UserCard component)
    SearchBar.js     (contains the SearchBar component)
```

Why This Matters

When you need to find a component, you know exactly which file to open. If `UserCard` has a bug, you open `UserCard.js`. Simple.

Exception

Very small, closely related components can share a file. For example, an `Icon` component that is only used inside a `Button` component could live in the same file as `Button`. But when in doubt, use separate files.

Best Practice 4: Destructure Props

The Rule

Instead of using `props.name` and `props.age`, pull out the values at the top of your component. This is called "destructuring." It means taking things out of a container and giving them their own names.

Example

```jsx
// Without destructuring - harder to read
function UserCard(props) {
  return (
    <div>
      <h2>{props.name}</h2>
      <p>Age: {props.age}</p>
      <p>Email: {props.email}</p>
    </div>
  );
}

// With destructuring - cleaner
function UserCard({ name, age, email }) {
  return (
    <div>
      <h2>{name}</h2>
      <p>Age: {age}</p>
      <p>Email: {email}</p>
    </div>
  );
}
```

Why This Matters

Destructuring gives you two benefits:

1. Less typing. You write `name` instead of `props.name` throughout the component.
2. Quick understanding. By looking at the function signature `{ name, age, email }`, you immediately know what props this component needs.

Best Practice 5: Keep State as Simple as Possible

The Rule

Store the minimum amount of information in state. Do not store things that can be calculated from other state values.

Example

```jsx
// BAD - storing things that can be calculated
function ShoppingCart() {
  const [items, setItems] = useState([]);
  const [itemCount, setItemCount] = useState(0);   // Can be calculated!
  const [totalPrice, setTotalPrice] = useState(0);  // Can be calculated!

  function addItem(item) {
    const newItems = [...items, item];
    setItems(newItems);
    setItemCount(newItems.length);              // Extra work
    setTotalPrice(newItems.reduce((sum, i) => sum + i.price, 0)); // Extra work
  }
}

// GOOD - calculate values from state
function ShoppingCart() {
  const [items, setItems] = useState([]);

  const itemCount = items.length;                            // Calculated
  const totalPrice = items.reduce((sum, i) => sum + i.price, 0); // Calculated

  function addItem(item) {
    setItems([...items, item]); // Only update one thing
  }
}
```

Why This Matters

When you store values that can be calculated, you need to keep them in sync. If you update `items` but forget to update `itemCount`, your app shows wrong information. By calculating values from state, they are always correct.

Think of it like a clock. You only need to store the current time. You do not need to store "is it morning?" separately. You can always figure that out from the time.

Best Practice 6: Lift State Only When Needed

The Rule

Keep state in the lowest component that needs it. Only move state up to a parent component when two or more children need to share it.

Example

If only the `SearchBar` needs to know what the user typed, keep the state in `SearchBar`.

```jsx
// Good - state stays where it is needed
function SearchBar() {
  const [query, setQuery] = useState("");

  return (
    <input
      value={query}
      onChange={(e) => setQuery(e.target.value)}
      placeholder="Search..."
    />
  );
}
```

If both `SearchBar` and `ResultsList` need the search query, then lift the state up to the parent.

```jsx
// Lift state to parent because two children need it
function SearchPage() {
  const [query, setQuery] = useState("");

  return (
    <div>
      <SearchBar query={query} onQueryChange={setQuery} />
      <ResultsList query={query} />
    </div>
  );
}
```

Why This Matters

If you put all your state at the top of your app, every state change causes the entire app to re-render. This is slow and unnecessary. Keep state close to where it is used.

Best Practice 7: Use Meaningful Variable Names

The Rule

Variable names should describe what the variable holds. Someone reading your code should understand it without guessing.

Examples

```jsx
// BAD – unclear names
const [d, setD] = useState([]);
const [flag, setFlag] = useState(false);
const [val, setVal] = useState("");
const [x, setX] = useState(0);

// GOOD – descriptive names
const [users, setUsers] = useState([]);
const [isMenuOpen, setIsMenuOpen] = useState(false);
const [searchQuery, setSearchQuery] = useState("");
const [cartItemCount, setCartItemCount] = useState(0);
```

Naming Conventions

Type	Convention	Examples
Boolean state	Start with `is`, `has`, or `should`	`isLoading`, `hasError`, `shouldRefresh`
Arrays	Use plural nouns	`users`, `products`, `messages`
Single items	Use singular nouns	`user`, `product`, `message`
Event handlers	Start with `handle`	`handleClick`, `handleSubmit`, `handleChange`
Functions passed as props	Start with `on`	`onClick`, `onSubmit`, `onChange`

Best Practice 8: Keep JSX Clean

The Rule

If your JSX has complex logic, extract it into variables above the return statement.

Example

```jsx
// MESSY - logic mixed into JSX
function UserGreeting({ user, notifications }) {
  return (
    <div>
      <h1>
        Hello, {user.firstName} {user.lastName}
        {user.role === "admin" ? " (Administrator)" : ""}!
      </h1>
      <p>
        You have {notifications.filter((n) => !n.read).length} unread
        {notifications.filter((n) => !n.read).length === 1
          ? " notification"
          : " notifications"}
      </p>
    </div>
  );
}

// CLEAN - logic extracted into variables
function UserGreeting({ user, notifications }) {
  const fullName = `${user.firstName} ${user.lastName}`;
  const roleLabel = user.role === "admin" ? " (Administrator)" : "";
  const unreadCount = notifications.filter((n) => !n.read).length;
  const notificationWord = unreadCount === 1 ? "notification" : "notifications";

  return (
    <div>
      <h1>Hello, {fullName}{roleLabel}!</h1>
      <p>You have {unreadCount} unread {notificationWord}</p>
    </div>
  );
}
```

Why This Matters

The second version is much easier to read. Each variable has a clear name. The JSX is simple and focused on structure, not logic.

Best Practice 9: Use Constants for Magic Numbers and Strings

The Rule

A "magic number" is a number in your code with no explanation. A "magic string" is the same idea but with text. Replace them with named constants.

Example

```jsx
// BAD - what does 5 mean? What does "admin" mean here?
function UserList({ users }) {
  const displayedUsers = users.slice(0, 5);
  const isAdmin = user.role === "admin";
}

// GOOD - constants explain the meaning
const MAX_DISPLAYED_USERS = 5;
const ROLE_ADMIN = "admin";

function UserList({ users }) {
  const displayedUsers = users.slice(0, MAX_DISPLAYED_USERS);
  const isAdmin = user.role === ROLE_ADMIN;
}
```

Why This Matters

If you need to change the maximum number of displayed users, you change it in one place. You do not have to search your entire codebase for the number 5. The constant name also tells you WHY the number 5 is there.

Best Practice 10: Write Comments for Tricky Parts

The Rule

You do not need to comment every line. But when you do something that might confuse someone (or future you), add a short comment explaining WHY.

Example

```jsx
function PriceDisplay({ price, discount }) {
  // We round to 2 decimal places to avoid floating-point issues
  // like 19.99 * 0.1 = 1.9990000000000002
```

```
  const discountAmount = Math.round(price * discount * 100) / 100;

  const finalPrice = price - discountAmount;

  return <p>Final price: ${finalPrice.toFixed(2)}</p>;
}
```

Comments to Avoid

Do not write comments that just repeat what the code says.

```jsx
// BAD comments - they add no value
const [count, setCount] = useState(0); // Set count to 0
setCount(count + 1); // Add 1 to count
return <p>Hello</p>; // Return a paragraph
```

Good Comments Explain "Why," Not "What"

```jsx
// GOOD comments
// Using index as key here because items never reorder or delete
{staticItems.map((item, index) => (
  <li key={index}>{item}</li>
))}

// Debounce search to avoid too many API calls while user types
useEffect(() => {
  const timer = setTimeout(() => searchAPI(query), 300);
  return () => clearTimeout(timer);
}, [query]);
```

Best Practice 11: Organize Your Folder Structure

The Rule

As your project grows, organize your files into folders that make sense. There is no single "correct" structure, but here is a simple one that works well for beginners.

Recommended Structure

```
src/
  components/          (Reusable UI pieces)
    Header.js
    Footer.js
    Button.js
  pages/               (Full pages of your app)
    HomePage.js
    AboutPage.js
    ProfilePage.js
  hooks/               (Custom hooks)
    useLocalStorage.js
  utils/               (Helper functions)
    formatDate.js
    calculateTotal.js
  constants/           (Constants and config)
    apiUrls.js
    roles.js
  App.js
  index.js
```

What Each Folder Means

- components/: Small, reusable pieces. A `Button` can be used on many pages.
- pages/: Bigger components that represent entire pages. A `HomePage` is only used once.
- hooks/: Custom hooks that you create to reuse logic.
- utils/: Helper functions that are not React components. Things like formatting dates or doing math.
- constants/: Values that never change, like API URLs or role names.

Why This Matters

When your project has 5 files, organization does not matter much. When your project has 50 or 100 files, good organization saves you a lot of time. You always know where to find things.

Best Practice 12: Follow the DRY Principle

The Rule

DRY stands for "Don't Repeat Yourself." If you find yourself writing the same code in multiple places, extract it into a reusable component or function.

Example

```jsx
// REPETITIVE - the same card structure appears three times
function Dashboard() {
  return (
    <div>
      <div className="card">
        <h2>Users</h2>
        <p className="stat">1,234</p>
      </div>
      <div className="card">
        <h2>Orders</h2>
        <p className="stat">567</p>
      </div>
      <div className="card">
        <h2>Revenue</h2>
        <p className="stat">$89,012</p>
      </div>
    </div>
  );
}

// DRY - extract the repeated pattern into a component
function StatCard({ title, value }) {
  return (
    <div className="card">
      <h2>{title}</h2>
      <p className="stat">{value}</p>
    </div>
  );
}

function Dashboard() {
  return (
    <div>
      <StatCard title="Users" value="1,234" />
      <StatCard title="Orders" value="567" />
      <StatCard title="Revenue" value="$89,012" />
    </div>
  );
}
```

Why This Matters

If you need to change the card design, you change it in one place (`StatCard`), and every card updates. Without DRY, you would have to change every single card by hand.

Best Practice 13: Format Your Code Consistently

The Rule

Use the same formatting style everywhere. This includes indentation (spaces or tabs), where you put curly braces, and how you write your imports.

Use a Code Formatter

The easiest way to format code consistently is to use a tool called Prettier. Prettier automatically formats your code when you save a file.

How to Set Up Prettier

1. Install it in your project:

```bash
npm install --save-dev prettier
```

2. Create a `.prettierrc` file in your project root:

```json
{
  "semi": true,
  "singleQuote": false,
  "tabWidth": 2,
  "trailingComma": "es5"
}
```

This tells Prettier:

- `semi: true` -- Add semicolons at the end of lines.
- `singleQuote: false` -- Use double quotes for strings.
- `tabWidth: 2` -- Use 2 spaces for indentation.
- `trailingComma: "es5"` -- Add commas after the last item in lists.

3. Most code editors (like VS Code) can run Prettier automatically when you save a file. Search for "Prettier" in your editor's extensions and install it.

Why This Matters

Consistent formatting makes code easier to read. When everyone on a team uses the same formatting, you spend time thinking about logic, not about where to put spaces.

Best Practice 14: Learn to Read Error Messages

The Rule

Error messages are your friends, not your enemies. They tell you exactly what went wrong and where. Learning to read them is one of the most valuable skills you can develop.

How to Read a React Error Message

Most error messages have three parts:

1. The type of error (what kind of problem)
2. The description (what happened)
3. The stack trace (where it happened)

Example

```
TypeError: Cannot read properties of undefined (reading 'name')
    at UserCard (UserCard.js:8:22)
    at App (App.js:12:5)
```

Let us break this down:

- TypeError: The type of error. A `TypeError` means you tried to use a value in a way that does not work for its type.
- Cannot read properties of undefined (reading 'name'): You tried to read `.name` from something that is `undefined`. In other words, the variable does not have a value yet.
- at UserCard (UserCard.js:8:22): The problem is in the `UserCard` component, in the file `UserCard.js`, on line 8, at position 22.

Common Error Messages and What They Mean

Error Message	What It Means
`X is not defined`	You used a variable name that does not exist. Check spelling.
`X is not a function`	You tried to call something that is not a function. Check the variable type.
`Cannot read properties of null`	You tried to read a property from `null`. The value is empty.

`Cannot read properties of undefined`	You tried to read a property from something that was never set.
`Invalid hook call`	You called a hook in the wrong place (not inside a component).
`Each child in a list should have a unique "key" prop`	You forgot to add keys to your list items.

What to Do When You See an Error

1. Read the first line. It tells you the type and description.
2. Look at the file name and line number. Go to that exact line in your code.
3. Read the code on that line. Think about what could be wrong.
4. If you do not understand, copy the error message and search for it online. Someone else has probably had the same problem.

Quick Summary

#	Best Practice	Key Idea
1	Keep components small	Split when a component does too much
2	Name clearly	Use nouns that describe what it IS
3	One component per file	Easy to find, easy to manage
4	Destructure props	Cleaner code, clear expectations
5	Simple state	Do not store what you can calculate
6	Lift state only when needed	Keep state close to where it is used
7	Meaningful names	Names should describe the value
8	Clean JSX	Extract complex logic into variables
9	Named constants	Replace magic numbers and strings

10	Helpful comments	Explain why, not what
11	Organized folders	Group files by purpose
12	DRY	Do not repeat yourself
13	Consistent formatting	Use Prettier or similar tools
14	Read error messages	They are helpers, not enemies

Key Points to Remember

1. Small components are easier to understand, test, and fix. If your component needs the word "and" to describe it, split it.
2. Good names save you time. You will spend more time reading code than writing it. Clear names make reading faster.
3. State should be minimal. Store only what you must. Calculate everything else.
4. Consistency matters more than perfection. Pick a style and stick with it.
5. Error messages are helpful clues. Read them carefully. They usually point you to the exact problem.

Practice Questions

1. How do you decide when a component is too big and needs to be split?
2. What is the difference between naming a component `ShowUser` and naming it `UserCard`? Which is better and why?
3. Why is it better to calculate `itemCount` from `items.length` instead of storing it as a separate state variable?
4. What is a "magic number" and why should you avoid using them?
5. What are the three parts of a typical error message?

Exercises

Exercise 1: Refactor a Big Component

Take this big component and split it into smaller ones.

```jsx
function ProductPage({ product }) {
  return (
    <div>
      <div className="product-image">
        <img src={product.image} alt={product.name} />
      </div>
      <div className="product-info">
        <h1>{product.name}</h1>
        <p className="price">${product.price}</p>
        <p className="description">{product.description}</p>
      </div>
      <div className="reviews">
        <h2>Reviews</h2>
        {product.reviews.map((review) => (
          <div key={review.id} className="review">
            <strong>{review.author}</strong>
            <p>{review.text}</p>
            <p>Rating: {review.rating}/5</p>
          </div>
        ))}
      </div>
    </div>
  );
}
```

Create `ProductImage`, `ProductInfo`, `ReviewList`, and `ReviewItem` components.

Exercise 2: Clean Up Messy Code

Rewrite this code using the best practices from this chapter.

```jsx
function x({ d }) {
  const [f, setF] = useState(false);
  const [l, setL] = useState([]);
  const [c, setC] = useState(0);

  function z() {
    setF(!f);
  }

  return (
    <div>
      <h1>{d.n}</h1>
      <p>{d.items.length > 0 ? d.items.length + " items" : "No items"}</p>
      <button onClick={z}>{f ? "Hide" : "Show"} Details</button>
```

```
    </div>
  );
}
```

Rename the component and all variables to have clear, meaningful names.

Exercise 3: Add Constants and Comments

Take this code and replace the magic numbers with named constants. Add helpful comments where appropriate.

```jsx
function PasswordValidator({ password }) {
  const isLongEnough = password.length >= 8;
  const hasNumber = /[0-9]/.test(password);

  if (password.length > 128) {
    return <p>Password is too long</p>;
  }

  return (
    <div>
      <p style={{ color: isLongEnough ? "green" : "red" }}>
        {isLongEnough ? "Length OK" : "Too short"}
      </p>
      <p style={{ color: hasNumber ? "green" : "red" }}>
        {hasNumber ? "Has a number" : "Needs a number"}
      </p>
    </div>
  );
}
```

What Is Next?

You now know the common mistakes to avoid and the best practices to follow. In the next chapter, we will test your knowledge with interview-style questions. These are the kinds of questions that come up when you are applying for a job as a React developer. Even if you are not looking for a job, they are a great way to make sure you truly understand React.

Chapter 23: Interview-Style Beginner Questions

Whether you are preparing for a job interview or just want to check your own understanding, this chapter will help. We have collected 30 common questions that interviewers ask about React. Each question has a clear, simple answer.

Do not worry about memorizing these word for word. The goal is to understand the ideas behind each answer. If you understand the "why," you can explain it in your own words. That is what interviewers want to see.

What You Will Learn

- Answers to 30 common React interview questions
- How to explain React concepts in simple language
- The core ideas that every React developer should know

Why This Chapter Matters

Interview questions test whether you truly understand something, or whether you only memorized code. Going through these questions is a great way to find gaps in your knowledge. If you cannot answer a question, that is a sign that you need to review that topic. Think of this chapter as a self-test.

The Questions

Question 1: What is React?

Answer: React is a JavaScript library for building user interfaces. A "user interface" is what people see and interact with on a website or app -- buttons, text, forms, images, and so on. React was created by Facebook (now called Meta) and is free to use. It helps you build web pages by breaking them into small, reusable pieces called "components."

Question 2: What is JSX?

Answer: JSX stands for JavaScript XML. It is a special way of writing code that looks like HTML but lives inside JavaScript. JSX lets you describe what your page should look like. For example, `<h1>Hello</h1>` looks like HTML, but it is actually JSX. The browser cannot understand JSX directly. A tool called a "compiler" (like Babel) converts JSX into regular JavaScript that the browser can run.

Question 3: What are components?

Answer: Components are the building blocks of a React app. A component is a piece of your user interface that you can reuse. Think of components like LEGO bricks. Each brick is a small piece. You put them together to build something bigger. A `Button` component can be used in many places. A `Header` component appears at the top of every page. Components can be as small as a button or as large as an entire page.

Question 4: What is the difference between props and state?

Answer: Props and state are both ways to store data in React, but they work differently.

Props (short for "properties") are data passed from a parent component to a child component. They flow in one direction: down. A child cannot change its own props. Think of props like a gift -- you receive it, but you do not get to choose what is inside.

State is data that a component manages itself. The component can read it and change it. Think of state like your personal notebook -- you can write in it and erase things whenever you want.

	Props	State

Who controls it?	Parent component	The component itself
Can the component change it?	No	Yes
Direction	Parent to child (down)	Internal

Question 5: What is the Virtual DOM?

Answer: The DOM (Document Object Model) is the browser's version of your web page. It is like a tree of all the elements on the page. Changing the real DOM is slow. React solves this by keeping a "virtual" copy of the DOM in memory. When something changes, React first updates the virtual copy. Then it compares the virtual copy with the real DOM and only updates the parts that actually changed. This is faster than replacing everything.

Think of it like editing a document. Instead of reprinting the whole document every time you fix a typo, you just correct the one word that changed.

Question 6: What are hooks?

Answer: Hooks are special functions that let you use React features (like state and side effects) inside function components. Before hooks existed, you had to use "class components" to use state. Hooks made React simpler. The two most common hooks are `useState` (for managing state) and `useEffect` (for side effects like fetching data). Hooks always start with the word "use."

Question 7: What does useState do?

Answer: `useState` is a hook that lets you add state to a function component. It takes one argument: the initial value. It gives you back two things: the current value and a function to update it.

```jsx
const [count, setCount] = useState(0);
```

Here, `count` starts at 0. When you call `setCount(1)`, React updates `count` to 1 and re-renders the component to show the new value.

Question 8: What does useEffect do?

Answer: `useEffect` is a hook that lets you run code after your component renders. It is used for "side effects" -- things that happen outside of just showing content on the screen. Common side effects include fetching data from an API, setting up a timer, or updating the page title. The dependency array (second argument) controls when the effect runs.

```jsx
useEffect(() => {
  document.title = "My App";
}, []); // Empty array = runs once after the first render
```

Question 9: What is conditional rendering?

Answer: Conditional rendering means showing different content based on a condition. Just like an `if` statement in regular programming, you can choose what to display depending on the situation.

```jsx
function Greeting({ isLoggedIn }) {
  if (isLoggedIn) {
    return <h1>Welcome back!</h1>;
  }
  return <h1>Please sign in.</h1>;
}
```

You can also use the ternary operator for shorter conditions:

```jsx
return <h1>{isLoggedIn ? "Welcome back!" : "Please sign in."}</h1>;
```

Question 10: Why do we need keys in lists?

Answer: Keys help React identify which items in a list have changed, been added, or been removed. Each key should be unique among siblings. Without keys, React has to guess which items changed, and it might make mistakes. This can cause bugs, especially with inputs or checkboxes.

Good keys come from your data (like an `id` field). Avoid using the array index as a key if the list can be reordered.

Question 11: What is lifting state up?

Answer: Lifting state up means moving state from a child component to a parent component so that multiple children can share it. If two components need the same data, you put that data in their closest common parent. The parent passes the data down as props.

For example, if a `SearchBar` and a `ResultsList` both need the search query, you put the `query` state in their parent component and pass it down to both children.

Question 12: What is a controlled component?

Answer: A controlled component is a form element (like an input or select) whose value is controlled by React state. The input does not manage its own value. Instead, React tells the input what to show, and when the user types, React updates the state.

```jsx
function NameInput() {
  const [name, setName] = useState("");

  return (
    <input
      value={name}
      onChange={(e) => setName(e.target.value)}
    />
  );
}
```

The opposite is an "uncontrolled component," where the input manages its own value and you read it only when needed (using a ref).

Question 13: How do you fetch data in React?

Answer: You fetch data inside a `useEffect` hook. The most common way is to use the `fetch` function built into browsers. You call the API, wait for the response, convert it to JSON, and save it in state. You should also handle loading and error states.

```jsx
useEffect(() => {
  fetch("https://api.example.com/data")
    .then((response) => response.json())
    .then((data) => setData(data))
    .catch((error) => setError(error.message));
}, []);
```

Question 14: What is React Router?

Answer: React Router is a library that lets you add navigation to your React app. It allows you to have different "pages" without actually reloading the browser. When you click a link, React Router shows a different component, but the page never fully refreshes. This creates a smooth experience, similar to a mobile app.

Question 15: What is the difference between a library and a framework?

Answer: A library gives you tools that you choose when to use. You are in control. React is a library. You decide how to structure your app and which parts of React to use.

A framework gives you a structure and tells you where to put your code. The framework is in control. Angular is a framework.

Think of it this way: a library is like a set of tools. You pick the tool you need and use it. A framework is like a blueprint. You build inside its structure.

Question 16: What is a single-page application (SPA)?

Answer: A single-page application is a website that loads one HTML page and then updates the content dynamically without full page reloads. When you click a link, instead of loading an entirely new page from the server, JavaScript updates what you see on the screen. React apps are typically SPAs. This makes them feel faster and smoother because there is no "flash" between page changes.

Question 17: What is the difference between a function component and a class component?

Answer: Both are ways to create React components. Function components are simpler. They are just JavaScript functions that return JSX. Class components use the `class` keyword and have more complex syntax.

Modern React uses function components almost exclusively. Class components are older and rarely used in new code. Hooks (like `useState` and `useEffect`) made function components just as powerful as class components, but easier to write.

Question 18: What is the children prop?

Answer: The `children` prop is a special prop that contains whatever you put between the opening and closing tags of a component. It lets you create "wrapper" components.

```jsx
function Card({ children }) {
  return <div className="card">{children}</div>;
}

// Using it:
<Card>
  <h2>Title</h2>
  <p>Some content here.</p>
</Card>
```

Everything between `<Card>` and `</Card>` becomes the `children` prop. This makes components flexible because you can put anything inside them.

Question 19: What happens when state changes in React?

Answer: When state changes, React re-renders the component. "Re-render" means React runs the component function again to see what the new output should look like. React then compares the new output with the old output (using the Virtual DOM) and updates only the parts of the screen that changed. This process is fast because React only touches what is necessary.

Question 20: What is prop drilling?

Answer: Prop drilling is when you pass data through many layers of components just to get it to a deeply nested child. For example, if the `App` component has data that a deeply nested `Button` component needs, you might pass it through 5 or 6 components in between, even though those middle components do not use the data themselves.

Prop drilling makes code harder to maintain. Solutions include React Context or state management libraries.

Question 21: What is React Context?

Answer: React Context is a way to share data across many components without passing props through every level. You create a "context" and wrap your app (or part of it) with a "Provider." Any component inside that Provider can access the shared data directly, no matter how deep it is.

Common uses include sharing the current theme (dark mode or light mode), the logged-in user, or the selected language.

Question 22: What is the useRef hook?

Answer: `useRef` creates a "reference" that persists across renders without causing re-renders when it changes. It is commonly used for two things:

1. Accessing DOM elements directly (like focusing an input field).
2. Storing values that should not trigger re-renders (like a timer ID).

```jsx
const inputRef = useRef(null);

// Later, to focus the input:
inputRef.current.focus();
```

The key difference from state: changing a ref does NOT cause the component to re-render.

Question 23: What is the difference between `==` and `===` in JavaScript?

Answer: `==` is "loose equality." It converts the values to the same type before comparing. This can cause surprising results. For example, `5 == "5"` is `true` because JavaScript converts the string `"5"` to the number `5` before comparing.

`===` is "strict equality." It does NOT convert types. `5 === "5"` is `false` because one is a number and the other is a string. They are different types.

In React (and in most modern JavaScript), always use `===`. It is safer and more predictable.

Question 24: What is the spread operator?

Answer: The spread operator (`...`) copies all the elements from an array or all the properties from an object into a new array or object. It is widely used in React for updating state without mutating the original value.

```jsx
// Copying an array and adding a new item
const newItems = [...oldItems, "New Item"];

// Copying an object and changing one property
const newUser = { ...oldUser, name: "Alice" };
```

Think of it like photocopying a document and then writing on the copy. The original stays the same.

Question 25: What is destructuring?

Answer: Destructuring is a way to pull values out of an array or object and assign them to variables in one step.

```jsx
// Array destructuring
const [count, setCount] = useState(0);

// Object destructuring
const { name, age, email } = user;
```

Without destructuring, you would write `user.name`, `user.age`, and `user.email` every time. Destructuring saves typing and makes code cleaner.

Question 26: What is an event handler in React?

Answer: An event handler is a function that runs when something happens, like a button click, a key press, or a form submission. In React, you attach event handlers to elements using camelCase props like `onClick`, `onChange`, and `onSubmit`.

```jsx
function handleClick() {
  alert("Button was clicked!");
}
```

```
<button onClick={handleClick}>Click Me</button>
```

Important: pass the function reference (`handleClick`), not the function call (`handleClick()`).

Question 27: What is a fragment in React?

Answer: A fragment is an invisible wrapper that lets you group multiple elements without adding an extra HTML element to the page. You write it as `<> </>` (short syntax) or `<React.Fragment>`.

```jsx
return (
  <>
    <h1>Title</h1>
    <p>Paragraph</p>
  </>
);
```

This renders just the `<h1>` and `<p>` in the HTML, with no extra `<div>` around them. Fragments are useful when you want to return multiple elements from a component without adding unnecessary wrappers.

Question 28: What is the difference between map and forEach in React?

Answer: Both `map` and `forEach` loop through an array. The difference is that `map` returns a new array, while `forEach` does not return anything.

In React, you use `map` to render lists because you need the result (a new array of JSX elements) to show on the screen. `forEach` does not return anything, so React has nothing to display.

```jsx
// Works - map returns an array of elements
{items.map((item) => <li key={item.id}>{item.name}</li>)}

// Does not work - forEach returns nothing
{items.forEach((item) => <li key={item.id}>{item.name}</li>)}
```

Question 29: What is a custom hook?

Answer: A custom hook is a regular JavaScript function that uses React hooks inside it. Custom hooks let you extract and reuse logic that would otherwise be repeated in multiple components. By convention, custom hook names start with the word "use."

```jsx
function useWindowWidth() {
  const [width, setWidth] = useState(window.innerWidth);

  useEffect(() => {
    function handleResize() {
      setWidth(window.innerWidth);
    }
    window.addEventListener("resize", handleResize);
    return () => window.removeEventListener("resize", handleResize);
  }, []);

  return width;
}

// Using it in any component:
const width = useWindowWidth();
```

Now any component can know the window width without duplicating the resize logic.

Question 30: What should you learn after React basics?

Answer: After you are comfortable with React basics, here are some recommended next steps:

1. TypeScript -- Adds type safety to your JavaScript code. It helps catch errors before they happen.
2. React Router -- Adds navigation and multiple pages to your app.
3. State management (like Redux or Zustand) -- Helps manage complex state across large apps.
4. Next.js -- A framework built on top of React that adds server-side rendering, routing, and more.
5. Testing (like React Testing Library and Jest) -- Helps you write tests to make sure your code works correctly.
6. CSS-in-JS (like styled-components or Tailwind CSS) -- Different ways to style your React apps.

You do not need to learn all of these at once. Pick one that interests you and go from there.

Quick Summary

These 30 questions cover the core concepts of React:

- The basics: What React is, JSX, components, Virtual DOM
- Data flow: Props, state, lifting state, prop drilling, Context
- Hooks: useState, useEffect, useRef, custom hooks
- Patterns: Controlled components, conditional rendering, lists and keys
- JavaScript essentials: Spread operator, destructuring, `===`, map vs forEach
- Architecture: SPAs, library vs framework, function vs class components
- Next steps: TypeScript, Next.js, state management, testing

Key Points to Remember

1. React is a library, not a framework. You are in control.
2. Props flow down, state lives within. Props are read-only. State is read-write.
3. The Virtual DOM makes React fast by updating only what changed.
4. Hooks let you use state and effects in function components.
5. Keys in lists help React track items efficiently.

Practice Questions

1. Explain in your own words what the Virtual DOM is and why it is useful.
2. A friend asks you: "What is the difference between props and state?" How would you explain it to them using a real-life analogy?
3. Why do we need to use `useEffect` for fetching data instead of just calling `fetch` at the top of our component?
4. What would happen if you used `forEach` instead of `map` to render a list? Why?
5. When would you use React Context instead of just passing props?

Exercises

Exercise 1: Explain It Simply

Pick any 5 questions from this chapter and write the answers in your own words. Do not look at the answers while writing. Then compare your answers with the ones in this chapter.

Exercise 2: Teach a Friend

Find a friend (or a rubber duck -- seriously, programmers often explain code to rubber ducks to think more clearly!) and try to explain these three concepts:

- What a React component is
- The difference between props and state
- What `useState` does

If you can explain something simply, you understand it well.

Exercise 3: Build a Mini Quiz App

Create a simple React app that shows one question at a time from this chapter. Add a "Show Answer" button that reveals the answer. Add "Next" and "Previous" buttons to navigate between questions.

This is a great way to practice React while also studying the material.

What Is Next?

You have reviewed 30 key React questions. You understand the concepts. Now it is time to put everything together. In the next and final chapter, we will build a complete Weather App from scratch. This project will use components, state, effects, API calls, loading states, error handling, and styling. It is the perfect way to end this book -- by building something real.

Chapter 24: Final Mini Project -- Weather App

This is it. The final chapter. Everything you have learned in this book comes together here. We are going to build a real weather app from scratch. You will type in a city name, and the app will show you the current weather for that city.

This project uses components, state, effects, API calls, loading states, error handling, and styling. By the end of this chapter, you will have a working app that you can show to friends, family, or even put on your portfolio.

Let us get started.

What You Will Learn

- How to plan a React project before writing code
- How to get a free API key from OpenWeatherMap
- How to build multiple components that work together
- How to fetch real weather data from an API
- How to handle loading and error states
- How to add simple CSS styling
- How to add a Celsius/Fahrenheit toggle

Why This Chapter Matters

Building a complete project is the best way to solidify your learning. Reading about React is like reading about swimming. You understand the theory, but you need to jump in the water. This project is your pool.

What We Will Build

Our weather app will have these features:

1. A search bar where the user can type a city name
2. A button to search for the weather
3. A card that shows the city name, temperature, weather description, and an icon
4. A loading message while data is being fetched
5. An error message if something goes wrong (like an invalid city name)
6. A button to switch between Celsius and Fahrenheit

The Components We Will Create

Component	Purpose
`App`	The main component that holds everything together
`SearchBar`	The input field and search button
`WeatherCard`	Shows the weather data
`ErrorMessage`	Shows error messages
`LoadingSpinner`	Shows a loading message

Getting a Free API Key from OpenWeatherMap

Before we write any code, we need a way to get weather data. We will use a free service called OpenWeatherMap. An API (Application Programming Interface) is a way for your app to ask another service for data.

How to Get Your Free API Key

1. Go to https://openweathermap.org in your browser.
2. Click "Sign Up" and create a free account.
3. After signing up and verifying your email, go to your account page.

4. Look for a section called "API keys."
5. You will see a key that looks like a long string of letters and numbers, like `abc123def456ghi789`.
6. Copy this key. You will need it soon.

Important: Your API key is private. Do not share it publicly. It is free, but it is connected to your account.

Note: New API keys can take up to 2 hours to become active. If your first attempt does not work, wait a bit and try again.

Step 1: Set Up the Project

Open your terminal and create a new React project.

```bash
npx create-react-app weather-app
cd weather-app
```

This creates a new folder called `weather-app` with all the files you need.

Now start the development server to make sure everything works.

```bash
npm start
```

Your browser should open and show the default React page. Great. We are ready to build.

Clean Up the Default Files

Open the `src` folder. You will see several files. We only need a few of them. Open `App.js` and replace everything inside with this:

```jsx
function App() {
  return (
    <div className="app">
      <h1>Weather App</h1>
    </div>
  );
}

export default App;
```

Open `App.css` and delete everything inside it. We will add our own styles later.

Open `index.css` and replace everything with this basic reset:

```
css
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

body {
  font-family: Arial, sans-serif;
  background-color: #f0f4f8;
  color: #333;
}
```

Line-by-Line Explanation of index.css

- `*` selects every element on the page.
- `margin: 0` and `padding: 0` remove the default spacing that browsers add.
- `box-sizing: border-box` makes width and height calculations easier. It includes padding and borders in the total size.
- `font-family: Arial, sans-serif` sets the text font for the entire page.
- `background-color: #f0f4f8` gives the page a light gray-blue background.
- `color: #333` makes the text dark gray (easier on the eyes than pure black).

Step 2: Create the SearchBar Component

Create a new file at `src/SearchBar.js`.

```
jsx
import { useState } from "react";

function SearchBar({ onSearch }) {
  const [city, setCity] = useState("");

  function handleSubmit(event) {
    event.preventDefault();

    if (city.trim() === "") {
      return;
    }
```

```
    onSearch(city);
  }

  return (
    <form className="search-bar" onSubmit={handleSubmit}>
      <input
        type="text"
        placeholder="Enter city name..."
        value={city}
        onChange={(e) => setCity(e.target.value)}
      />
      <button type="submit">Search</button>
    </form>
  );
}

export default SearchBar;
```

Line-by-Line Explanation

- `import { useState } from "react"` -- We need `useState` to track what the user types.
- `function SearchBar({ onSearch })` -- This component receives a function called `onSearch` from its parent. We destructure it from props.
- `const [city, setCity] = useState("")` -- We create a state variable called `city` that starts as an empty string.
- `function handleSubmit(event)` -- This function runs when the user submits the form (clicks the button or presses Enter).
- `event.preventDefault()` -- By default, a form submission refreshes the entire page. This line stops that from happening.
- `if (city.trim() === "")` -- `trim()` removes spaces from both ends of the text. If the user typed only spaces, we do nothing.
- `onSearch(city)` -- We call the function that the parent gave us, passing the city name up.
- `value={city}` -- This makes the input a "controlled component." React controls what the input shows.
- `onChange={(e) => setCity(e.target.value)}` -- When the user types, we update the `city` state with whatever they typed.

Expected Behavior

The user types a city name. When they click "Search" or press Enter, the `onSearch` function is called with the city name. The search bar itself does not fetch data. It just tells the parent what city to search for.

Step 3: Fetch Weather Data from the API

Now let us update `App.js` to use the `SearchBar` and fetch weather data.

```jsx
import { useState } from "react";
import SearchBar from "./SearchBar";

const API_KEY = "YOUR_API_KEY_HERE";
const API_URL = "https://api.openweathermap.org/data/2.5/weather";

function App() {
  const [weather, setWeather] = useState(null);
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState(null);

  function handleSearch(city) {
    setLoading(true);
    setError(null);
    setWeather(null);

    fetch(`${API_URL}?q=${city}&appid=${API_KEY}&units=metric`)
      .then((response) => {
        if (!response.ok) {
          throw new Error("City not found");
        }
        return response.json();
      })
      .then((data) => {
        setWeather(data);
        setLoading(false);
      })
      .catch((err) => {
        setError(err.message);
        setLoading(false);
      });
  }

  return (
    <div className="app">
      <h1>Weather App</h1>
      <SearchBar onSearch={handleSearch} />

      {loading && <p>Loading...</p>}
      {error && <p className="error">{error}</p>}
      {weather && <p>Temperature: {weather.main.temp} C</p>}
    </div>
  );
}
```

```
export default App;
```

Important: Replace `YOUR_API_KEY_HERE` with the API key you got from OpenWeatherMap.

Line-by-Line Explanation

- `const API_KEY = "YOUR_API_KEY_HERE"` -- A constant that stores your API key. Using a constant makes it easy to find and change.
- `const API_URL = "..."` -- The base URL for the OpenWeatherMap API.
- `const [weather, setWeather] = useState(null)` -- Stores the weather data. Starts as `null` because we have no data yet.
- `const [loading, setLoading] = useState(false)` -- Tracks whether we are waiting for data.
- `const [error, setError] = useState(null)` -- Stores any error message.
- `setLoading(true)` -- When a search starts, we set loading to true.
- `setError(null)` -- Clear any previous error.
- `setWeather(null)` -- Clear any previous weather data.
- ` fetch(${API_URL}?q=${city}&appid=${API_KEY}&units=metric) -- This sends a request to the weather API. q=${city} tells it which city. units=metric` gives us temperatures in Celsius.
- `if (!response.ok)` -- If the response is not OK (like a 404 error for a city that does not exist), we throw an error.
- `throw new Error("City not found")` -- Creates an error that will be caught by the `.catch()` block.
- `.catch((err) => {...})` -- If anything goes wrong, we save the error message and stop loading.
- `{loading && <p>Loading...</p>}` -- Show "Loading..." only when `loading` is true.
- `{error && <p>{error}</p>}` -- Show the error only when there is one.
- `{weather && <p>...</p>}` -- Show the temperature only when we have weather data.

Expected Output

When you type "London" and click Search, you should see something like:

```
Temperature: 15.2 C
```

If you type a city that does not exist (like "Xyzabc"), you should see:

```
City not found
```

Step 4: Display the Weather Data in a WeatherCard

Let us create a proper component to show the weather data. Create a new file at `src/WeatherCard.js`.

```jsx
function WeatherCard({ weather, unit }) {
  const temperature = unit === "celsius"
    ? Math.round(weather.main.temp)
    : Math.round(weather.main.temp * 9 / 5 + 32);

  const unitSymbol = unit === "celsius" ? "C" : "F";

  const iconCode = weather.weather[0].icon;
  const iconUrl = `https://openweathermap.org/img/wn/${iconCode}@2x.png`;

  return (
    <div className="weather-card">
      <h2>{weather.name}, {weather.sys.country}</h2>
      <img src={iconUrl} alt={weather.weather[0].description} />
      <p className="temperature">
        {temperature}&deg;{unitSymbol}
      </p>
      <p className="description">{weather.weather[0].description}</p>
      <div className="details">
        <p>Humidity: {weather.main.humidity}%</p>
        <p>Wind: {weather.wind.speed} m/s</p>
      </div>
    </div>
  );
}

export default WeatherCard;
```

Line-by-Line Explanation

- `function WeatherCard({ weather, unit })` -- This component receives the weather data and the current temperature unit (Celsius or Fahrenheit).
- `const temperature = unit === "celsius" ? ... : ...` -- If the unit is Celsius, we use the temperature as is. If it is Fahrenheit, we convert it using the formula: `(Celsius * 9/5)`

`+ 32`.

- `Math.round(...)` -- Rounds the number to the nearest whole number. Nobody needs to see `15.237` degrees.
- `const unitSymbol` -- Shows "C" or "F" depending on the selected unit.
- `const iconCode = weather.weather[0].icon` -- The API gives us a code for a weather icon (like "01d" for sunny).
- `const iconUrl = ...` -- OpenWeatherMap provides free weather icons at this URL pattern.
- `°` -- This is an HTML code that shows the degree symbol (the little circle: 15°C).
- `weather.name` -- The city name from the API.
- `weather.sys.country` -- The country code (like "GB" for Great Britain).
- `weather.weather[0].description` -- A text description like "clear sky" or "light rain."
- `weather.main.humidity` -- The humidity percentage.
- `weather.wind.speed` -- The wind speed in meters per second.

Step 5: Handle Loading and Error States

Let us create proper components for loading and error states.

Create `src/LoadingSpinner.js`

```jsx
function LoadingSpinner() {
  return (
    <div className="loading">
      <p>Loading weather data...</p>
    </div>
  );
}

export default LoadingSpinner;
```

Create `src/ErrorMessage.js`

```jsx
function ErrorMessage({ message }) {
  return (
    <div className="error-message">
```

```
      <p>Oops! {message}</p>
      <p>Please check the city name and try again.</p>
    </div>
  );
}

export default ErrorMessage;
```

Update `App.js` to Use These Components

```jsx
import { useState } from "react";
import SearchBar from "./SearchBar";
import WeatherCard from "./WeatherCard";
import LoadingSpinner from "./LoadingSpinner";
import ErrorMessage from "./ErrorMessage";

const API_KEY = "YOUR_API_KEY_HERE";
const API_URL = "https://api.openweathermap.org/data/2.5/weather";

function App() {
  const [weather, setWeather] = useState(null);
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState(null);
  const [unit, setUnit] = useState("celsius");

  function handleSearch(city) {
    setLoading(true);
    setError(null);
    setWeather(null);

    fetch(`${API_URL}?q=${city}&appid=${API_KEY}&units=metric`)
      .then((response) => {
        if (!response.ok) {
          throw new Error("City not found");
        }
        return response.json();
      })
      .then((data) => {
        setWeather(data);
        setLoading(false);
      })
      .catch((err) => {
        setError(err.message);
        setLoading(false);
      });
  }

  function toggleUnit() {
    setUnit(unit === "celsius" ? "fahrenheit" : "celsius");
  }
```

```
  return (
    <div className="app">
      <h1>Weather App</h1>
      <SearchBar onSearch={handleSearch} />

      {loading && <LoadingSpinner />}
      {error && <ErrorMessage message={error} />}
      {weather && (
        <>
          <WeatherCard weather={weather} unit={unit} />
          <button className="unit-toggle" onClick={toggleUnit}>
            Show in {unit === "celsius" ? "Fahrenheit" : "Celsius"}
          </button>
        </>
      )}
    </div>
  );
}

export default App;
```

What Changed

- We imported `WeatherCard`, `LoadingSpinner`, and `ErrorMessage`.
- We added a `unit` state that tracks whether to show Celsius or Fahrenheit.
- We added a `toggleUnit` function that switches between the two units.
- We replaced the simple `<p>` tags with our proper components.
- We added a toggle button that only appears when weather data is available.

Step 6: Add Simple CSS Styling

Open `src/App.css` and add these styles:

```
.app {
  max-width: 500px;
  margin: 50px auto;
  padding: 20px;
  text-align: center;
}

.app h1 {
```

```
  margin-bottom: 20px;
  color: #2c3e50;
  font-size: 28px;
}

/* Search Bar */
.search-bar {
  display: flex;
  gap: 10px;
  margin-bottom: 20px;
}

.search-bar input {
  flex: 1;
  padding: 12px 16px;
  font-size: 16px;
  border: 2px solid #ddd;
  border-radius: 8px;
  outline: none;
}

.search-bar input:focus {
  border-color: #3498db;
}

.search-bar button {
  padding: 12px 24px;
  font-size: 16px;
  background-color: #3498db;
  color: white;
  border: none;
  border-radius: 8px;
  cursor: pointer;
}

.search-bar button:hover {
  background-color: #2980b9;
}

/* Weather Card */
.weather-card {
  background: white;
  border-radius: 12px;
  padding: 30px;
  box-shadow: 0 4px 6px rgba(0, 0, 0, 0.1);
  margin-bottom: 15px;
}

.weather-card h2 {
  color: #2c3e50;
  margin-bottom: 5px;
}

.weather-card img {
  width: 100px;
```

```
  height: 100px;
}

.weather-card .temperature {
  font-size: 48px;
  font-weight: bold;
  color: #e74c3c;
  margin: 5px 0;
}

.weather-card .description {
  font-size: 18px;
  color: #7f8c8d;
  text-transform: capitalize;
  margin-bottom: 15px;
}

.weather-card .details {
  display: flex;
  justify-content: center;
  gap: 30px;
  color: #555;
}

/* Unit Toggle */
.unit-toggle {
  padding: 8px 20px;
  font-size: 14px;
  background-color: #ecf0f1;
  border: 2px solid #bdc3c7;
  border-radius: 8px;
  cursor: pointer;
  color: #333;
}

.unit-toggle:hover {
  background-color: #bdc3c7;
}

/* Loading */
.loading {
  margin: 30px 0;
  color: #3498db;
  font-size: 18px;
}

/* Error */
.error-message {
  background-color: #fee;
  border: 1px solid #e74c3c;
  border-radius: 8px;
  padding: 15px;
  margin: 20px 0;
  color: #c0392b;
}
```

Now import the CSS in `App.js`. Add this line at the top of `App.js`:

```jsx
import "./App.css";
```

CSS Explanation for Key Parts

- `max-width: 500px` and `margin: 50px auto` -- Centers the app and limits its width.
- `display: flex` on the search bar -- Places the input and button side by side.
- `gap: 10px` -- Adds space between the input and the button.
- `flex: 1` on the input -- Makes the input take up all remaining space.
- `border-radius: 8px` -- Rounds the corners of elements.
- `box-shadow` -- Adds a subtle shadow under the weather card to make it look like it floats.
- `text-transform: capitalize` -- Makes the first letter of the weather description uppercase.
- `cursor: pointer` -- Changes the mouse cursor to a hand when hovering over buttons.

Step 7: Final Touches

Let us finalize our `App.js` with the import for styles included and everything polished.

Complete Final Code

Here is every file in the project, complete and ready to use.

`src/index.js`

```jsx
import React from "react";
import ReactDOM from "react-dom/client";
import "./index.css";
import App from "./App";

const root = ReactDOM.createRoot(document.getElementById("root"));
root.render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
```

This is the entry point. It renders the `App` component inside the HTML page.

src/index.css

```css
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

body {
  font-family: Arial, sans-serif;
  background-color: #f0f4f8;
  color: #333;
  min-height: 100vh;
}
```

src/App.js

```jsx
import { useState } from "react";
import SearchBar from "./SearchBar";
import WeatherCard from "./WeatherCard";
import LoadingSpinner from "./LoadingSpinner";
import ErrorMessage from "./ErrorMessage";
import "./App.css";

const API_KEY = "YOUR_API_KEY_HERE";
const API_URL = "https://api.openweathermap.org/data/2.5/weather";

function App() {
  const [weather, setWeather] = useState(null);
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState(null);
  const [unit, setUnit] = useState("celsius");

  function handleSearch(city) {
    setLoading(true);
    setError(null);
    setWeather(null);

    fetch(`${API_URL}?q=${city}&appid=${API_KEY}&units=metric`)
      .then((response) => {
        if (!response.ok) {
          throw new Error("City not found");
        }
        return response.json();
      })
      .then((data) => {
```

```
        setWeather(data);
        setLoading(false);
      })
      .catch((err) => {
        setError(err.message);
        setLoading(false);
      });
  }

  function toggleUnit() {
    setUnit(unit === "celsius" ? "fahrenheit" : "celsius");
  }

  return (
    <div className="app">
      <h1>Weather App</h1>
      <SearchBar onSearch={handleSearch} />

      {loading && <LoadingSpinner />}
      {error && <ErrorMessage message={error} />}
      {weather && (
        <>
          <WeatherCard weather={weather} unit={unit} />
          <button className="unit-toggle" onClick={toggleUnit}>
            Show in {unit === "celsius" ? "Fahrenheit" : "Celsius"}
          </button>
        </>
      )}
    </div>
  );
}

export default App;
```

src/SearchBar.js

```jsx
import { useState } from "react";

function SearchBar({ onSearch }) {
  const [city, setCity] = useState("");

  function handleSubmit(event) {
    event.preventDefault();

    if (city.trim() === "") {
      return;
    }

    onSearch(city);
  }
```

```
  return (
    <form className="search-bar" onSubmit={handleSubmit}>
      <input
        type="text"
        placeholder="Enter city name..."
        value={city}
        onChange={(e) => setCity(e.target.value)}
      />
      <button type="submit">Search</button>
    </form>
  );
}

export default SearchBar;
```

src/WeatherCard.js

```jsx
function WeatherCard({ weather, unit }) {
  const temperature = unit === "celsius"
    ? Math.round(weather.main.temp)
    : Math.round(weather.main.temp * 9 / 5 + 32);

  const unitSymbol = unit === "celsius" ? "C" : "F";

  const iconCode = weather.weather[0].icon;
  const iconUrl = `https://openweathermap.org/img/wn/${iconCode}@2x.png`;

  return (
    <div className="weather-card">
      <h2>{weather.name}, {weather.sys.country}</h2>
      <img src={iconUrl} alt={weather.weather[0].description} />
      <p className="temperature">
        {temperature}&deg;{unitSymbol}
      </p>
      <p className="description">{weather.weather[0].description}</p>
      <div className="details">
        <p>Humidity: {weather.main.humidity}%</p>
        <p>Wind: {weather.wind.speed} m/s</p>
      </div>
    </div>
  );
}

export default WeatherCard;
```

src/LoadingSpinner.js

```jsx
function LoadingSpinner() {
  return (
    <div className="loading">
      <p>Loading weather data...</p>
    </div>
  );
}

export default LoadingSpinner;
```

src/ErrorMessage.js

```jsx
function ErrorMessage({ message }) {
  return (
    <div className="error-message">
      <p>Oops! {message}</p>
      <p>Please check the city name and try again.</p>
    </div>
  );
}

export default ErrorMessage;
```

src/App.css

```css
.app {
  max-width: 500px;
  margin: 50px auto;
  padding: 20px;
  text-align: center;
}

.app h1 {
  margin-bottom: 20px;
  color: #2c3e50;
  font-size: 28px;
}

/* Search Bar */
.search-bar {
  display: flex;
  gap: 10px;
  margin-bottom: 20px;
}

.search-bar input {
```

```
  flex: 1;
  padding: 12px 16px;
  font-size: 16px;
  border: 2px solid #ddd;
  border-radius: 8px;
  outline: none;
}

.search-bar input:focus {
  border-color: #3498db;
}

.search-bar button {
  padding: 12px 24px;
  font-size: 16px;
  background-color: #3498db;
  color: white;
  border: none;
  border-radius: 8px;
  cursor: pointer;
}

.search-bar button:hover {
  background-color: #2980b9;
}

/* Weather Card */
.weather-card {
  background: white;
  border-radius: 12px;
  padding: 30px;
  box-shadow: 0 4px 6px rgba(0, 0, 0, 0.1);
  margin-bottom: 15px;
}

.weather-card h2 {
  color: #2c3e50;
  margin-bottom: 5px;
}

.weather-card img {
  width: 100px;
  height: 100px;
}

.weather-card .temperature {
  font-size: 48px;
  font-weight: bold;
  color: #e74c3c;
  margin: 5px 0;
}

.weather-card .description {
  font-size: 18px;
  color: #7f8c8d;
```

```
  text-transform: capitalize;
  margin-bottom: 15px;
}

.weather-card .details {
  display: flex;
  justify-content: center;
  gap: 30px;
  color: #555;
}

/* Unit Toggle */
.unit-toggle {
  padding: 8px 20px;
  font-size: 14px;
  background-color: #ecf0f1;
  border: 2px solid #bdc3c7;
  border-radius: 8px;
  cursor: pointer;
  color: #333;
}

.unit-toggle:hover {
  background-color: #bdc3c7;
}

/* Loading */
.loading {
  margin: 30px 0;
  color: #3498db;
  font-size: 18px;
}

/* Error */
.error-message {
  background-color: #fee;
  border: 1px solid #e74c3c;
  border-radius: 8px;
  padding: 15px;
  margin: 20px 0;
  color: #c0392b;
}
```

How the App Works: A Complete Walkthrough

Here is what happens when a user searches for a city:

1. The user types "Tokyo" into the search bar and clicks "Search."
2. The `SearchBar` component calls `onSearch("Tokyo")`.

3. The `App` component receives this and calls `handleSearch("Tokyo")`.
4. `handleSearch` sets `loading` to `true`, clears any previous error and weather data.
5. React re-renders. The `LoadingSpinner` appears because `loading` is `true`.
6. `fetch` sends a request to the OpenWeatherMap API.
7. The API sends back weather data for Tokyo.
8. `handleSearch` saves the data in `weather` state and sets `loading` to `false`.
9. React re-renders. The `LoadingSpinner` disappears. The `WeatherCard` appears with the weather data.
10. The user clicks "Show in Fahrenheit."
11. `toggleUnit` changes `unit` from `"celsius"` to `"fahrenheit"`.
12. React re-renders. `WeatherCard` recalculates the temperature and shows it in Fahrenheit.

If the user types a city that does not exist:

1. The API sends back an error response.
2. `response.ok` is `false`, so we throw an error.
3. The `.catch` block saves the error message and sets `loading` to `false`.
4. React re-renders. The `ErrorMessage` component appears.

Quick Summary

Step	What We Did
1	Set up the project and cleaned up default files
2	Created the SearchBar component with a controlled input
3	Added data fetching with fetch, loading, and error handling
4	Created the WeatherCard component to display weather data
5	Created LoadingSpinner and ErrorMessage components
6	Added CSS styling for a clean, modern look
7	Added the Celsius/Fahrenheit toggle and finalized all code

Key Points to Remember

1. Plan before you code. Decide what components you need and what each one does.
2. Handle all three states: loading, error, and success. Real apps must deal with all of them.
3. Keep components focused. Each component has one job.
4. Use constants for API URLs and keys. Do not scatter them throughout your code.
5. Test as you go. After each step, make sure your app works before moving on.

Congratulations!

You did it. You built a complete, working React application from scratch.

Take a moment to appreciate what you have accomplished. At the beginning of this book, you might not have known what a component was. Now you have built an app with multiple components, state management, API calls, error handling, and styling. That is a real achievement.

What You Have Learned in This Book

- What React is and why it is useful
- How to create components
- How to use JSX
- How to manage state with `useState`
- How to handle side effects with `useEffect`
- How to pass data with props
- How to handle user events
- How to render lists and use keys
- How to do conditional rendering
- How to fetch data from APIs
- How to handle forms
- How to use React Router for navigation
- Common mistakes and how to avoid them

- Best practices for clean code

What to Learn Next

Here are recommended topics to continue your React journey, in order:

1. TypeScript -- TypeScript adds type checking to JavaScript. It catches many bugs before your code even runs. Most professional React projects use TypeScript. Start by learning the basics: types for strings, numbers, arrays, and objects. Then learn how to type your React components and props.
2. Next.js -- Next.js is a framework built on top of React. It adds features like server-side rendering (pages are prepared on the server before being sent to the browser), file-based routing (create a file and it becomes a page), and automatic code splitting (loading only the JavaScript needed for each page). Many companies use Next.js for production React apps.
3. State Management -- As your apps grow bigger, you might need more powerful ways to manage state. Look into:
    - Zustand -- A simple, lightweight state management library. Great for beginners who have outgrown `useState` and Context.
    - Redux Toolkit -- The most popular state management library for large applications. It has a learning curve but is used by many companies.
4. Testing -- Learn how to write tests for your React components. Start with:
    - React Testing Library -- Tests your components from the user's perspective.
    - Jest -- A test runner that works well with React.
5. CSS Solutions -- Explore different ways to style React apps:
    - Tailwind CSS -- A utility-first CSS framework. You add styles directly in your JSX using class names.
    - CSS Modules -- Scoped CSS that prevents style conflicts between components.
    - styled-components -- Write CSS inside your JavaScript files.

Recommended Resources for Continued Learning

- React Official Documentation (react.dev) -- The official React docs are excellent. They include interactive tutorials and detailed explanations.
- freeCodeCamp (freecodecamp.org) -- Free courses on JavaScript, React, and web development.
- JavaScript.info -- The best resource for learning JavaScript deeply. Strong JavaScript skills make you a better React developer.
- Build Projects -- The best way to learn is to build. Here are some project ideas:

- A to-do app with categories and due dates
- A recipe finder that uses a food API
- A quiz app with multiple choice questions
- A note-taking app with local storage
- A simple e-commerce product page

A Final Word

Learning to code is a journey, not a destination. There will always be something new to learn. That is what makes it exciting. Do not rush. Do not compare yourself to others. Focus on understanding one thing at a time.

When you get stuck (and you will), remember:

- Read the error message carefully. It is trying to help you.
- Search for the error message online. Someone else has had the same problem.
- Take a break. Sometimes the answer comes when you step away.
- Come back to this book and review the relevant chapter.

You have the foundation now. Everything else builds on what you have learned here. Keep building. Keep learning. Keep going.

You are a React developer now.

Practice Questions

1. Why do we use `event.preventDefault()` in the form submit handler?
2. What happens if we do not check `response.ok` after a fetch call?
3. Why do we set `loading` to `true` and `error` to `null` at the beginning of `handleSearch`?
4. How does the Celsius to Fahrenheit conversion work in the `WeatherCard` component?
5. Why did we create separate components (SearchBar, WeatherCard, ErrorMessage, LoadingSpinner) instead of putting everything in App?

Exercises

Exercise 1: Add More Weather Details

Add the following information to the `WeatherCard`:

- "Feels like" temperature (available at `weather.main.feels_like`)
- Minimum and maximum temperature for the day (`weather.main.temp_min` and `weather.main.temp_max`)

Make sure these also respect the Celsius/Fahrenheit toggle.

Exercise 2: Save the Last Searched City

Use `localStorage` to save the last city the user searched for. When the app loads, automatically search for that city. Here is a hint:

```jsx
// Save to localStorage
localStorage.setItem("lastCity", city);

// Read from localStorage
const lastCity = localStorage.getItem("lastCity");
```

Exercise 3: Add a Search History

Keep a list of the last 5 cities the user searched for. Show them as clickable buttons below the search bar. When the user clicks a city name, search for that city's weather.

What Is Next?

There is no next chapter. This is the end of the book. But it is not the end of your learning. It is just the beginning. Go build something. The best way to learn React is to use React. Pick a project that excites you and start building.

Good luck, and happy coding!

Chapter 25: Thinking in React

What You Will Learn

- How to look at a design and break it into React components
- The five steps to building any React application
- How to decide what should be state and what should be props
- How to figure out where state should live
- How data flows in a React app (down through props, up through events)

Why This Chapter Matters

So far, you have learned how to write components, use props, manage state, and handle events. You know the individual tools. But knowing the tools is not the same as knowing how to build something with them.

Imagine someone gives you a hammer, nails, wood, and a saw. You know what each tool does. But if someone shows you a picture of a bookshelf and says "build this," you might not know where to start.

This chapter teaches you the thinking process. It gives you a step-by-step plan for turning any design into a working React app. This is one of the most important skills a React developer can have.

Building with Lego: A Helpful Analogy

Think about building something with Lego blocks. You do not just grab random pieces and start clicking them together. You follow a process:

1. Look at the picture on the box. What are you building?
2. Find the main sections. The house has a roof, walls, a door, and windows.
3. Build each section with simple blocks first. No moving parts yet.
4. Figure out which parts move. The door opens. The window slides.
5. Connect everything so the parts work together.

Building a React app follows a very similar process. Let us walk through it with a real example.

Our Example: A Simple Product List

Imagine your friend has a small online shop. They show you this design and say, "Can you build this in React?"

Here is what the page should look like:

```
+-----------------------------------------------------+
|  Fruit Shop                                         |
|                                                     |
|  Search: [_______________]                          |
|                                                     |
|  [x] Only show fruits in stock                      |
|                                                     |
|  Name                           Price               |
|  ---------------------------------------            |
|  Fruits                                             |
|  ---------------------------------------            |
|  Apple                          $1.00               |
|  Banana                         $0.50               |
|  ~~Passion Fruit~~              ~~$2.00~~           |
|  ---------------------------------------            |
|  Vegetables                                         |
|  ---------------------------------------            |
|  Spinach                        $1.50               |
|  ~~Pumpkin~~                    ~~$3.00~~           |
|  Peas                           $0.80               |
+-----------------------------------------------------+
```

Items with a line through them (like ~~Passion Fruit~~) are out of stock. The search box filters the list. The checkbox hides items that are out of stock.

Here is the data we will work with:

```jsx
const PRODUCTS = [
  { category: "Fruits", name: "Apple", price: "$1.00", inStock: true },
  { category: "Fruits", name: "Banana", price: "$0.50", inStock: true },
  { category: "Fruits", name: "Passion Fruit", price: "$2.00", inStock: false },
  { category: "Vegetables", name: "Spinach", price: "$1.50", inStock: true },
  { category: "Vegetables", name: "Pumpkin", price: "$3.00", inStock: false },
  { category: "Vegetables", name: "Peas", price: "$0.80", inStock: true },
];
```

Now, let us apply the five steps.

Step 1: Break the UI into a Component Hierarchy

The first step is to look at the design and draw boxes around every piece. Each box becomes a component.

Think of it like looking at the Lego bookshelf and identifying the separate sections.

Let us draw boxes around our product list:

```
+-----------------------------------------------------+
| [A] FilterableProductTable                          |
|                                                     |
|  +-----------------------------------------------+  |
|  | [B] SearchBar                                 |  |
|  |  Search: [_______________]                    |  |
|  |  [x] Only show fruits in stock                |  |
|  +-----------------------------------------------+  |
|                                                     |
|  +-----------------------------------------------+  |
|  | [C] ProductTable                              |  |
|  |                                               |  |
|  |  Name                        Price            |  |
|  |  ----------------------------------------     |  |
|  |  +---------------------------------------+ |  |
|  |  | [D] ProductCategoryRow                | |  |
|  |  |  Fruits                               | |  |
|  |  +---------------------------------------+ |  |
|  |  +---------------------------------------+ |  |
|  |  | [E] ProductRow                        | |  |
|  |  |  Apple                    $1.00       | |  |
|  |  +---------------------------------------+ |  |
|  |  +---------------------------------------+ |  |
|  |  | [E] ProductRow                        | |  |
|  |  |  Banana                   $0.50       | |  |
|  |  +---------------------------------------+ |  |
|  +-----------------------------------------------+  |
```

```
+-----------------------------------------------------+
```

Here are our components:

Letter	Component Name	What It Does
A	FilterableProductTable	The whole thing. Holds everything together.
B	SearchBar	The search input and the checkbox.
C	ProductTable	The table that shows products.
D	ProductCategoryRow	The category heading (like "Fruits").
E	ProductRow	One row showing a single product.

And here is how they fit together (this is called a hierarchy, which means "which component is inside which"):

```
FilterableProductTable
  |-- SearchBar
  |-- ProductTable
        |-- ProductCategoryRow
        |-- ProductRow
```

How Do You Decide What Should Be a Component?

A good rule is: each component should do one thing. If a component does too much, break it into smaller components.

Think about it like organizing your desk. You do not put everything into one giant drawer. You put pens in one cup, papers in one tray, and books on a shelf. Each container has one purpose.

Step 2: Build a Static Version First

Now we build the components, but with no interactivity yet. No state. No event handlers. Just show the data on the screen using props.

Why? Because it is easier to build something that looks right first, and then add behavior later. Like drawing a sketch before painting.

Let us build each component from the bottom up.

ProductRow Component

This shows one product:

```jsx
function ProductRow({ product }) {
  const name = product.inStock ? (
    product.name
  ) : (
    <span style={{ color: "red" }}>{product.name}</span>
  );

  return (
    <tr>
      <td>{name}</td>
      <td>{product.price}</td>
    </tr>
  );
}
```

Line-by-line explanation:

- Line 1: We create a component that receives a `product` object as a prop.
- Lines 2-6: If the product is in stock, we show the name normally. If it is not in stock, we show the name in red.
- Lines 8-12: We return a table row with two cells: the name and the price.

ProductCategoryRow Component

This shows a category heading:

```jsx
function ProductCategoryRow({ category }) {
  return (
    <tr>
      <th colSpan="2">{category}</th>
    </tr>
  );
}
```

Line-by-line explanation:

- Line 1: This component receives a `category` string as a prop.
- Line 4: It shows the category name in a header cell that spans both columns. The word `colSpan` means "stretch across this many columns."

ProductTable Component

This shows the full table:

```jsx
function ProductTable({ products }) {
  const rows = [];
  let lastCategory = null;

  products.forEach((product) => {
    if (product.category !== lastCategory) {
      rows.push(
        <ProductCategoryRow
          category={product.category}
          key={product.category}
        />
      );
    }
    rows.push(<ProductRow product={product} key={product.name} />);
    lastCategory = product.category;
  });

  return (
    <table>
      <thead>
        <tr>
          <th>Name</th>
          <th>Price</th>
        </tr>
      </thead>
      <tbody>{rows}</tbody>
    </table>
  );
}
```

Line-by-line explanation:

- Line 1: This component receives the full list of products.
- Line 2: We create an empty array to hold the rows we will build.
- Line 3: We track the last category we saw. This helps us know when to add a new category heading.
- Lines 5-16: We loop through each product. If the category changed, we add a category heading row. Then we add the product row.
- Lines 18-28: We return a table with a header row (Name, Price) and the body rows we built.

SearchBar Component

For the static version, this just shows the form without any behavior:

```jsx
function SearchBar() {
  return (
    <form>
      <input type="text" placeholder="Search..." />
      <label>
        <input type="checkbox" />
        {" "}Only show products in stock
      </label>
    </form>
  );
}
```

FilterableProductTable Component

This is the top-level component that puts everything together:

```jsx
function FilterableProductTable({ products }) {
  return (
    <div>
      <h1>Fruit Shop</h1>
      <SearchBar />
      <ProductTable products={products} />
    </div>
  );
}
```

Running the Static Version

```jsx
function App() {
  return <FilterableProductTable products={PRODUCTS} />;
}
```

Expected output: You will see the full product table on screen with the search box and checkbox. But nothing happens when you type or click. That is fine. We will add that next.

At this point, data flows in one direction only: from the top (FilterableProductTable) down to the children through props. This is called one-way data flow.

```
FilterableProductTable (has the data)
    |
    |--- passes products ---> ProductTable
    |                             |
    |                             |--- passes product ---> ProductRow
    |                             |--- passes category --> ProductCategoryRow
    |
    |--- (nothing yet) --------> SearchBar
```

Step 3: Figure Out the Minimal State

Now we need to add interactivity. To do that, we need state (data that changes over time).

But we do not want too much state. More state means more things to keep track of, and more chances for bugs. We want the minimum amount of state.

Think of it like packing for a trip. You do not pack everything you own. You pack only what you need. If you can wear the same shoes with three outfits, you bring one pair of shoes, not three.

Let us look at all the data in our app:

1. The original list of products
2. The search text the user typed
3. The value of the checkbox (checked or not)
4. The filtered list of products

Now, let us ask three questions about each piece of data:

Question	If yes, it is NOT state
Does it stay the same over time? (Never changes?)	Not state. Just a constant.
Is it passed from a parent through props?	Not state. It is a prop.
Can you calculate it from other state or props?	Not state. Compute it instead.

Let us check our data:

Data	Changes?	Passed as prop?	Can compute?	Is it state?
Product list	No	Yes (from outside)	No	No

Search text	Yes	No	No	Yes
Checkbox value	Yes	No	No	Yes
Filtered list	Yes	No	Yes (from the other three)	No

We only need two pieces of state:

1. The search text
2. Whether the checkbox is checked

The filtered list is NOT state. We can compute it from the product list, the search text, and the checkbox value. Do not store something in state if you can calculate it.

Step 4: Decide Where State Should Live

Now we know we need two pieces of state: `searchText` and `inStockOnly`. But which component should hold them?

Here is how you decide. For each piece of state:

1. Find every component that uses that state to render something.
2. Find their closest common parent (the nearest component that is above all of them in the hierarchy).
3. That common parent (or something above it) should hold the state.

Let us figure this out:

- SearchBar needs `searchText` and `inStockOnly` (to show them in the form).
- ProductTable needs `searchText` and `inStockOnly` (to filter the list).
- Their common parent is FilterableProductTable.

So `FilterableProductTable` should hold both pieces of state.

```
FilterableProductTable  <-- state lives here
    |                       (searchText, inStockOnly)
    |
    |--- SearchBar          <-- uses state to show values
    |
    |--- ProductTable       <-- uses state to filter products
```

Step 5: Add Data Flow

This is the last step. We need to:

1. Pass state down as props (parent to children).
2. Pass events up so children can change the state (children to parent).

Think of it like a restaurant:

- The menu goes from the kitchen down to the customers (data flows down).
- The order goes from the customers up to the kitchen (events flow up).

Adding State to FilterableProductTable

```jsx
import { useState } from "react";

function FilterableProductTable({ products }) {
  const [searchText, setSearchText] = useState("");
  const [inStockOnly, setInStockOnly] = useState(false);

  return (
    <div>
      <h1>Fruit Shop</h1>
      <SearchBar
        searchText={searchText}
        inStockOnly={inStockOnly}
        onSearchTextChange={setSearchText}
        onInStockOnlyChange={setInStockOnly}
      />
      <ProductTable
        products={products}
        searchText={searchText}
        inStockOnly={inStockOnly}
      />
    </div>
  );
}
```

Line-by-line explanation:

- Line 1: We import `useState` so we can create state.
- Line 4: We create state for the search text. It starts as an empty string.
- Line 5: We create state for the checkbox. It starts as `false` (unchecked).
- Lines 11-15: We pass the state values AND the setter functions to SearchBar. The setter functions let SearchBar change the state.
- Lines 17-20: We pass the products, search text, and checkbox value to ProductTable so it can filter the list.

Updating SearchBar to Use Props and Events

```jsx
function SearchBar({
  searchText,
  inStockOnly,
  onSearchTextChange,
  onInStockOnlyChange,
}) {
  return (
    <form>
      <input
        type="text"
        placeholder="Search..."
        value={searchText}
        onChange={(e) => onSearchTextChange(e.target.value)}
      />
      <label>
        <input
          type="checkbox"
          checked={inStockOnly}
          onChange={(e) => onInStockOnlyChange(e.target.checked)}
        />
        {" "}Only show products in stock
      </label>
    </form>
  );
}
```

Line-by-line explanation:

- Lines 1-6: SearchBar now receives the state values and the functions to update them.
- Line 12: The input shows the current search text.
- Line 13: When the user types, we call `onSearchTextChange` with the new text. This goes UP to the parent and changes the state.
- Line 18: The checkbox shows whether it is checked.
- Line 19: When the user clicks the checkbox, we call `onInStockOnlyChange`. This goes UP to the parent.

Updating ProductTable to Filter

```jsx
function ProductTable({ products, searchText, inStockOnly }) {
  const rows = [];
  let lastCategory = null;

  products.forEach((product) => {
    if (product.name.toLowerCase().indexOf(
```

```
      searchText.toLowerCase()) === -1) {
      return;
    }
    if (inStockOnly && !product.inStock) {
      return;
    }

    if (product.category !== lastCategory) {
      rows.push(
        <ProductCategoryRow
          category={product.category}
          key={product.category}
        />
      );
    }
    rows.push(<ProductRow product={product} key={product.name} />);
    lastCategory = product.category;
  });

  return (
    <table>
      <thead>
        <tr>
          <th>Name</th>
          <th>Price</th>
        </tr>
      </thead>
      <tbody>{rows}</tbody>
    </table>
  );
}
```

Line-by-line explanation:

- Line 1: Now we also receive `searchText` and `inStockOnly` as props.
- Lines 6-8: If the product name does not contain the search text, we skip it. The `toLowerCase()` makes the search case-insensitive (so "apple" matches "Apple").
- Lines 10-12: If the checkbox is checked and the product is out of stock, we skip it.

The Complete Data Flow

Here is how data flows in our finished app:

```
FilterableProductTable
  |  (holds searchText and inStockOnly state)
  |
  |--- state values flow DOWN as props ----+
  |                                        |
  |--- SearchBar                           |
  |     |  Shows the current values        |
```

```
|     |  When user types or clicks:      |
|     |  Events flow UP via callbacks    |
|     +--- calls onSearchTextChange() -->+-- updates state
|     +--- calls onInStockOnlyChange() ->+-- updates state
|                                        |
|--- ProductTable                        |
      |  Receives products + filters     |
      |  Filters the list                |
      |  Shows matching products         |
```

Expected output: Now when you type in the search box, the product list filters in real time. When you check the checkbox, out-of-stock items disappear. Everything works together.

The Five Steps Summarized

Here is a quick reference you can use for any React project:

```
+-----------------------------------------------------+
|  Step 1: Break UI into components                  |
|          Draw boxes around every piece              |
|                                                     |
|  Step 2: Build a static version                    |
|          Props only. No state. Just display data.   |
|                                                     |
|  Step 3: Find the minimal state                    |
|          What changes? What can be computed?         |
|                                                     |
|  Step 4: Decide where state lives                  |
|          Find the common parent of all users.       |
|                                                     |
|  Step 5: Add data flow                             |
|          State down via props. Events up via        |
|          callback functions.                        |
+-----------------------------------------------------+
```

Quick Summary

- Building a React app starts with a plan, just like building with Lego.
- Step 1: Look at the design and break it into components. Each component should do one thing.

- Step 2: Build a static version with no state. Just pass data down through props.
- Step 3: Figure out the smallest amount of state you need. If you can compute it, do not store it.
- Step 4: Decide which component should hold each piece of state. Find the common parent of all components that need it.
- Step 5: Add interactivity. Pass state down as props. Let children send events up through callback functions.

Key Points to Remember

1. Always start by breaking the design into components before writing code.
2. Build a static version first. Get it looking right before adding behavior.
3. Keep state minimal. Do not store what you can calculate.
4. State should live in the closest common parent of the components that need it.
5. Data flows down (through props). Events flow up (through callback functions).

Practice Questions

1. What are the five steps to building a React app?
2. How do you decide if a piece of data should be state or not?
3. If two sibling components both need the same state, where should that state live?
4. What does "one-way data flow" mean?
5. Why should you build a static version before adding state?

Exercises

1. Look at any website you use daily (like a to-do app, a weather app, or a shopping site). Draw boxes around the parts you see. Write down what components you would create. Think about which one is inside which.
2. Using the five-step process, plan out a simple "Contact List" app. It should show a list of names and phone numbers, and have a search box to filter them. Write down: (a) what components you need, (b) what data is state, and (c) where the state should live.
3. Take the product list example from this chapter and add a new feature: a dropdown that lets the user sort products by name or by price. Think through where the sort state

should live and how the data flows before writing any code.

What Is Coming Next?

Now that you know how to think about building a React app, we need to learn some important rules. In the next chapter, we will learn about keeping components pure. You will discover why React components should behave like math formulas: the same input should always give the same output. This will help you avoid confusing bugs and write more reliable code.

Chapter 26: Keeping Components Pure

What You Will Learn

- What a "pure" function is, in simple everyday words
- Why React components should be pure
- What side effects are and where they belong
- How to spot and fix impure components
- What StrictMode does and why React renders twice in development
- The rules for keeping your components pure

Why This Chapter Matters

Imagine you go to a bakery and order a chocolate cake. You expect a chocolate cake every time you order one. But what if sometimes you got vanilla, sometimes strawberry, and sometimes the baker ate half of it before giving it to you? That bakery would be chaos.

React components are like recipes. When you give them the same ingredients (props), they should always produce the same result. This is called being pure. When your components are pure, your app is predictable, easy to understand, and has fewer bugs.

This chapter teaches you the rules for writing pure components and shows you what happens when you break those rules.

What Does "Pure" Mean?

Pure Functions in Everyday Life

A pure function is like a math formula.

Think about the formula: double(x) = x times 2

- If you give it 3, you always get 6.
- If you give it 5, you always get 10.
- If you give it 3 again, you still get 6. Always.

It does not matter what day it is. It does not matter how many times you call it. It does not matter what other formulas you used before. Same input, same output. Every single time.

A pure function follows two rules:

1. Same input, same output. Given the same inputs, it always returns the same result.
2. No side effects. It does not change anything outside of itself. It minds its own business.

Pure Functions in Code

Here is a pure function:

```jsx
function double(number) {
  return number * 2;
}
```

Line-by-line explanation:

- Line 1: The function takes a number as input.
- Line 2: It returns that number multiplied by 2. Nothing else happens.

If you call `double(3)`, you get 6. Always. It does not change any other variable. It does not modify anything outside of itself. It just takes input and returns output.

What Are Side Effects?

A side effect is when a function changes something outside of itself. Think of it as a "side action" that happens in addition to returning a result.

Here are examples of side effects in everyday life:

- You go to a restaurant to eat (main action), but you also rearrange the furniture (side effect).
- You borrow a book from a friend (main action), but you write notes in it (side effect).

- You ask someone what time it is (main action), but you also change their watch (side effect).

In code, side effects include:

- Changing a variable that exists outside the function
- Changing the content on the screen directly
- Writing to a file
- Sending a network request
- Setting a timer

Side effects are not bad. They are necessary. Your app needs to do things like show data, save information, and talk to servers. But there is a right place and a wrong place for them.

React Components Should Be Pure

React expects your components to behave like pure functions. Given the same props and state, a component should always return the same JSX.

A Pure Component

```jsx
function Greeting({ name }) {
  return <h1>Hello, {name}!</h1>;
}
```

Line-by-line explanation:

- Line 1: The component receives a `name` prop.
- Line 2: It returns a heading that says "Hello" followed by the name.

If you pass `name="Sara"`, you always get `<h1>Hello, Sara!</h1>`. Every time. No surprises.

Expected output:

```
Hello, Sara!
```

A Pure Component Is Like a Recipe

Think of a component as a recipe:

```
+---------------------------------------------+
|  Recipe: Greeting                           |
|                                             |
|  Ingredients (props):                       |
|    - name                                   |
|                                             |
|  Instructions:                              |
|    1. Take the name                         |
|    2. Put "Hello, " before it               |
|    3. Put "!" after it                      |
|    4. Serve in an <h1> tag                  |
|                                             |
|  Same ingredients = Same dish. Always.      |
+---------------------------------------------+
```

Examples of Impure Components (And How to Fix Them)

Problem 1: Changing a Variable Outside the Component

Here is an impure component:

```jsx
let guestCount = 0;

function GuestGreeting() {
  guestCount = guestCount + 1;
  return <h2>Guest number {guestCount}</h2>;
}
```

Why this is impure: Every time this component renders, it changes `guestCount`. That variable lives outside the component. If React renders this component three times, you get "Guest number 1", "Guest number 2", "Guest number 3". Same component, different results. That is not pure.

Even worse, if another component also uses `guestCount`, it will get unexpected values. One component is secretly changing a value that others depend on.

The fix: Use a prop or compute the value inside:

```jsx
function GuestGreeting({ guestNumber }) {
  return <h2>Guest number {guestNumber}</h2>;
}
```

Line-by-line explanation:

- Line 1: Now the guest number comes in as a prop.
- Line 2: We just display it. We do not change anything outside.

Now you use it like this:

```jsx
<GuestGreeting guestNumber={1} />
<GuestGreeting guestNumber={2} />
<GuestGreeting guestNumber={3} />
```

Expected output:

```
Guest number 1
Guest number 2
Guest number 3
```

Same input, same output. Every time. And nothing outside the component changes.

Problem 2: Changing Props Directly

Here is another impure component:

```jsx
function ShoppingItem({ item }) {
  item.quantity = item.quantity + 1;  // DO NOT do this!
  return <p>{item.name}: {item.quantity}</p>;
}
```

Why this is impure: This component reaches into the `item` object and changes it. The `item` was passed in as a prop. Props are like a package someone gives you. You should not open it and rearrange the contents. The sender might not expect that.

The fix: Do not change props. If you need a different value, create a new variable:

```jsx
function ShoppingItem({ item }) {
  const displayQuantity = item.quantity + 1;
  return <p>{item.name}: {displayQuantity}</p>;
}
```

Line-by-line explanation:

- Line 2: We create a new variable `displayQuantity` instead of changing the original. The original `item` stays the same.
- Line 3: We use our new variable for display.

Problem 3: Using Random Values in Render

```jsx
function Dice() {
  const roll = Math.floor(Math.random() * 6) + 1;
  return <p>You rolled: {roll}</p>;
}
```

Why this is a problem: Every time this component renders, it gives a different result. Same input (no props), different output. This is technically impure because it is unpredictable.

If you need random values, generate them outside the component and pass them in as props, or generate them in an event handler (like when the user clicks a "Roll" button).

The fix:

```jsx
function Dice({ value }) {
  return <p>You rolled: {value}</p>;
}

// In a parent component:
function DiceGame() {
  const [roll, setRoll] = useState(1);

  function handleRoll() {
    setRoll(Math.floor(Math.random() * 6) + 1);
  }

  return (
    <div>
      <Dice value={roll} />
      <button onClick={handleRoll}>Roll dice</button>
    </div>
  );
}
```

Expected output (after clicking the button):

```
You rolled: 4
[Roll dice]
```

Now the randomness happens in an event handler (when the user clicks), not during rendering.

The Rules for Pure Components

Here are the rules, stated simply:

```
+--------------------------------------------------+
|  Rules for Pure Components                       |
|                                                  |
|  1. Do NOT change variables that existed         |
|     before your function was called.             |
|                                                  |
|  2. Do NOT change your props.                    |
|                                                  |
|  3. Do NOT change another component's state.     |
|                                                  |
|  4. Same inputs should ALWAYS produce            |
|     the same output.                             |
+--------------------------------------------------+
```

What You CAN Do Inside a Component

You can create and change local variables. Variables you create inside the component are fine to change because they are brand new each time the component renders:

```jsx
function ShoppingList({ items }) {
  const rows = [];  // Created fresh every render

  items.forEach((item) => {
    rows.push(<li key={item.id}>{item.name}</li>);
  });

  return <ul>{rows}</ul>;
}
```

Line-by-line explanation:

- Line 2: We create a new empty array. This is fine because we created it right here, right now. We are not changing something that existed before.
- Lines 4-6: We add items to our new array. Still fine. It is our array. We just made it.
- Line 8: We return the list. Everything that happened stayed inside this function.

This is called local mutation (mutation means "change"). Changing something you just created inside your function is perfectly fine.

StrictMode: React's Helper for Finding Impure Components

React has a tool called StrictMode that helps you find impure components during development.

When StrictMode is on, React renders every component twice. Why? Because if a component is pure, rendering it twice should give the same result. If rendering it twice gives different results, you have a bug.

Remember our impure example?

```jsx
let guestCount = 0;

function GuestGreeting() {
  guestCount = guestCount + 1;
  return <h2>Guest number {guestCount}</h2>;
}
```

With StrictMode, React renders this component twice:

- First render: `guestCount` becomes 1. Shows "Guest number 1".
- Second render: `guestCount` becomes 2. Shows "Guest number 2".

Different results! StrictMode helps you notice this problem.

StrictMode is usually already set up in your app:

```jsx
import { StrictMode } from "react";

function App() {
  return (
    <StrictMode>
      <MyApp />
    </StrictMode>
  );
}
```

Important: StrictMode only runs in development. When you build your app for real users (production), it does not render twice. It does not slow down your live app.

Do Not Panic About Double Rendering

When you see console.log messages appearing twice during development, it is usually StrictMode doing its job. This is normal and expected. It helps you catch bugs early.

Where Do Side Effects Belong?

If components should be pure, where do you put code that changes the outside world? Code that:

- Saves data to a server
- Changes the page title
- Starts a timer
- Focuses an input field

These side effects belong in two places:

1. Event Handlers

An event handler is a function that runs when the user does something (clicks a button, types in a field, submits a form).

```jsx
function ContactForm() {
  const [name, setName] = useState("");

  function handleSubmit() {
    // Side effect: sending data to a server
    fetch("/api/contacts", {
      method: "POST",
      body: JSON.stringify({ name }),
    });
    alert("Saved!");  // Side effect: showing an alert
  }

  return (
    <form onSubmit={handleSubmit}>
      <input value={name} onChange={(e) => setName(e.target.value)} />
      <button type="submit">Save</button>
    </form>
```

```
  );
}
```

The side effects (sending data and showing an alert) happen inside `handleSubmit`. This function runs only when the user clicks Submit, not during rendering. That is the right place.

2. useEffect (for effects that happen after rendering)

Sometimes you need a side effect that is not triggered by the user. For example, you want to fetch data when the component first appears. That is what `useEffect` is for:

```jsx
import { useState, useEffect } from "react";

function UserProfile({ userId }) {
  const [user, setUser] = useState(null);

  useEffect(() => {
    fetch(`/api/users/${userId}`)
      .then((response) => response.json())
      .then((data) => setUser(data));
  }, [userId]);

  if (!user) return <p>Loading...</p>;
  return <h1>{user.name}</h1>;
}
```

The `useEffect` runs AFTER the component renders. It is separate from the render process. The rendering itself stays pure.

Think of it this way:

```
+----------------------------------------------+
|  During rendering:                           |
|    - Calculate what to show (pure!)          |
|    - Return JSX                              |
|                                              |
|  After rendering (event handlers, useEffect):|
|    - Talk to servers                         |
|    - Update the page title                   |
|    - Start timers                            |
|    - Focus input fields                      |
+----------------------------------------------+
```

A Before-and-After Example

Let us fix one more impure component to really cement this concept.

Before (Impure)

```jsx
let totalOrders = 0;

function OrderSummary({ items }) {
  totalOrders = totalOrders + 1;
  document.title = `Order #${totalOrders}`;

  return (
    <div>
      <h2>Order #{totalOrders}</h2>
      <p>Items: {items.length}</p>
    </div>
  );
}
```

Problems:

- It changes `totalOrders`, which lives outside the component.
- It changes `document.title` directly during rendering (side effect).
- Calling it twice gives different results.

After (Pure)

```jsx
import { useEffect } from "react";

function OrderSummary({ orderNumber, items }) {
  useEffect(() => {
    document.title = `Order #${orderNumber}`;
  }, [orderNumber]);

  return (
    <div>
      <h2>Order #{orderNumber}</h2>
      <p>Items: {items.length}</p>
    </div>
  );
}
```

What changed:

- The order number now comes in as a prop. The component does not track it.

- Changing `document.title` moved into `useEffect`, so it happens after rendering, not during.
- Same props will always produce the same JSX. The component is now pure.

Expected output (with orderNumber={5} and 3 items):

```
Order #5
Items: 3
```

And the browser tab title will say "Order #5".

Quick Summary

- A pure function always gives the same output for the same input and does not change anything outside itself.
- React components should be pure. Same props and state should always produce the same JSX.
- Do not change variables outside your component, change props, or change other components' state during rendering.
- You can change variables you create inside your component during that render.
- Side effects (like saving data, changing the page title, or setting timers) belong in event handlers or `useEffect`, not during rendering.
- StrictMode renders components twice in development to help you find impure code.

Key Points to Remember

1. Pure component = same input, same output. Like a math formula.
2. Never change props. They belong to the parent.
3. Create new variables instead of changing existing ones.
4. Event handlers are the right place for most side effects.
5. StrictMode double-renders only in development, not in production.

Practice Questions

1. In your own words, what does "pure" mean for a React component?
2. What is a side effect? Give two examples.
3. Why does React render components twice in development when StrictMode is on?
4. A component reads the current time with `new Date()` and displays it. Is this component pure? Why or why not?
5. Where should you put code that sends data to a server: during rendering or in an event handler?

Exercises

1. Look at this component and explain what makes it impure. Then fix it:

```jsx
let clickCount = 0;

function ClickTracker() {
  clickCount = clickCount + 1;
  return <p>This component rendered {clickCount} times</p>;
}
```

2. Write a pure component called `PriceTag` that receives `price` and `currency` as props and displays them together (like "$25.00" or "EUR 15.00"). Make sure it follows all the purity rules.
3. Take this impure component and move the side effect to the correct place:

```jsx
function PageTitle({ title }) {
  document.title = title;
  return <h1>{title}</h1>;
}
```

What Is Coming Next?

Now that you understand purity, the next chapter dives deeper into how React actually renders your components. You will learn about the three steps React follows (trigger, render, commit), why state behaves like a snapshot, and why `console.log` after `setState` shows the old value. Understanding this will clear up many confusing moments you might encounter.

Chapter 27: How React Renders — State as a Snapshot

What You Will Learn

- The three steps React follows to update the screen: Trigger, Render, Commit
- What causes a render to happen
- Why state behaves like a snapshot (a frozen photo of that moment)
- Why console.log after setState shows the old value
- What batching is and why React does it
- How to queue multiple state updates using the updater function

Why This Chapter Matters

Have you ever changed state in your code and then immediately tried to use the new value, only to find it is still the old value? Have you ever called `setCount` three times in a row and expected the count to go up by three, but it only went up by one?

These are some of the most common confusions beginners face in React. They happen because of how React renders. Once you understand the three steps and the snapshot concept, these confusing moments will make perfect sense.

The Three Steps: Trigger, Render, Commit

Think of React like a restaurant:

```
+-----------------------------------------------------+
|  The Restaurant Analogy                              |
|                                                       |
|  Step 1: TRIGGER  =  The customer places an order  |
|  Step 2: RENDER   =  The kitchen cooks the food    |
|  Step 3: COMMIT   =  The waiter serves the food    |
+-----------------------------------------------------+
```

Let us look at each step.

Step 1: Trigger (Placing the Order)

A render is triggered in two situations:

1. Initial render: When your app first starts, React renders all your components for the first time. This is like a new customer sitting down and placing their first order.
2. State update: When you call a state setter function (like `setCount`), React knows something changed and it needs to update the screen. This is like the customer saying, "Actually, I would like to change my order."

```jsx
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);

  function handleClick() {
    setCount(count + 1);  // This TRIGGERS a re-render
  }

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={handleClick}>Add one</button>
    </div>
  );
}
```

Line-by-line explanation:

- Line 4: We create a state variable `count` starting at 0.
- Line 7: When the button is clicked, we call `setCount`. This is the trigger. React says, "The state changed. I need to re-render this component."

Step 2: Render (Cooking the Food)

After a trigger, React calls your component function to figure out what should be on the screen.

This is the "cooking" step. React runs your function, and the function returns JSX. That JSX tells React what the screen should look like.

- On the initial render, React calls your component for the first time.
- On a re-render, React calls your component again with the updated state.

Important: "Rendering" in React means calling your function. It does NOT mean updating the screen yet. The screen update happens in the next step.

```
Render 1 (count = 0):
  Your function runs.
  It returns: <p>Count: 0</p> <button>Add one</button>

  User clicks the button. setCount(1) is called. Trigger!

Render 2 (count = 1):
  Your function runs AGAIN.
  It returns: <p>Count: 1</p> <button>Add one</button>
```

Step 3: Commit (Serving the Food)

After rendering, React compares the new result with what is currently on the screen. If anything is different, React updates the actual screen (the DOM).

- On the initial render, React adds all the elements to the screen for the first time.
- On a re-render, React only changes the parts that are different. If only the count changed, React only updates that one number on screen. It does not rebuild the whole page.

This is like a waiter who only brings you the new dish. They do not take away all your food and re-serve everything. They just swap the one thing that changed.

```
+-----------------------------------------------------+
|  The Three Steps in Action                          |
|                                                      |
|  1. TRIGGER: setCount(1) is called                  |
|     "New order: count should be 1"                  |
|                                                      |
|  2. RENDER: React calls Counter()                   |
|     Function returns <p>Count: 1</p>                |
|     "The food is cooked"                             |
|                                                      |
|  3. COMMIT: React updates the <p> on screen         |
|     Changes "0" to "1"                               |
|     "The food is served"                             |
+-----------------------------------------------------+
```

State as a Snapshot

This is the most important concept in this chapter. Read it carefully.

When you call a state setter function, React does not change the state variable immediately. Instead, React schedules a new render. And each render gets its own "snapshot" of the state.

What Is a Snapshot?

A snapshot is like a photograph. When you take a photo, it captures that exact moment in time. If things change after you take the photo, the photo stays the same. It shows what things looked like at that moment.

State works the same way. When your component renders, it takes a "snapshot" of the state at that moment. All the code inside that render uses that snapshot. Even if you call setState, the current render still uses the old snapshot.

A Diagram to Make This Clear

```
+-----------------------------------------+
|  RENDER 1 (snapshot: count = 0)         |
|                                          |
|  Everything in this render sees          |
|  count as 0. Like a photo taken          |
|  when count was 0.                       |
|                                          |
|  setCount(1) is called.                  |
|  But count is STILL 0 in this render.    |
|  The new value (1) will be used in       |
|  the NEXT render.                        |
+-----------------------------------------+
         |
         | React schedules a new render
         v
+-----------------------------------------+
|  RENDER 2 (snapshot: count = 1)         |
|                                          |
|  Everything in this render sees          |
|  count as 1. Like a new photo taken      |
|  when count was 1.                       |
+-----------------------------------------+
```

Why console.log Shows the Old Value

This is one of the most asked questions by beginners. Look at this code:

```jsx
function Counter() {
  const [count, setCount] = useState(0);

  function handleClick() {
    setCount(count + 1);
    console.log(count);  // What does this print?
  }

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={handleClick}>Add one</button>
    </div>
  );
}
```

When you click the button, `console.log(count)` prints 0, not 1.

Why? Because this entire function is running inside Render 1, where the snapshot says `count = 0`. Calling `setCount(1)` does not change `count` right now. It schedules a new render where `count` will be 1.

Think of it like sending a letter. When you put a letter in the mailbox, the recipient does not immediately get it. They will get it later. Similarly, `setCount(1)` tells React "next render should have count = 1." But the current render still has the old value.

```
+-----------------------------------------------------+
|  Inside handleClick (during Render 1):              |
|                                                      |
|  count = 0  (this is the snapshot)                  |
|                                                      |
|  setCount(0 + 1)  --> schedules count = 1           |
|                       for the NEXT render            |
|                                                      |
|  console.log(count)  --> prints 0                   |
|  Because count is still 0 in THIS render.            |
+-----------------------------------------------------+
```

Another Example to Drive It Home

```jsx
function Counter() {
  const [count, setCount] = useState(0);
```

```
  function handleClick() {
    setCount(count + 5);
    alert(count);
  }

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={handleClick}>Add five</button>
    </div>
  );
}
```

Expected behavior:

1. You click the button.
2. The alert shows 0 (the snapshot value).
3. After you close the alert, the screen shows Count: 5 (the new render).

The alert uses the snapshot from when the click happened. The screen update happens in the next render.

Batching: React Groups Updates Together

What happens when you call setState multiple times in the same event handler?

```jsx
function Counter() {
  const [count, setCount] = useState(0);

  function handleClick() {
    setCount(count + 1);
    setCount(count + 1);
    setCount(count + 1);
  }

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={handleClick}>Add three?</button>
    </div>
  );
}
```

You might expect `count` to go from 0 to 3. But it only goes to 1.

Why? Remember the snapshot. All three `setCount` calls happen inside the same render where `count = 0`.

```
All three calls use the same snapshot (count = 0):

  setCount(0 + 1)  --> schedules count = 1
  setCount(0 + 1)  --> schedules count = 1
  setCount(0 + 1)  --> schedules count = 1

All three say "set count to 1." So count becomes 1.
```

It is like telling a waiter three times, "I want a coffee." You do not get three coffees. You get one coffee.

Why Does React Batch?

Batching means React waits until all the code in an event handler finishes before processing the state updates. React does not re-render after each `setCount` call. It waits, collects all the updates, and then does one single re-render.

This is like a waiter who takes your complete order before going to the kitchen. They do not run to the kitchen after every item you say. They write down everything, then go once. This is more efficient.

```
+-----------------------------------------------------+
|  Without batching (bad):                            |
|                                                      |
|  setCount(1) --> render --> update screen            |
|  setCount(1) --> render --> update screen            |
|  setCount(1) --> render --> update screen            |
|                                                      |
|  Three renders! Slow and wasteful.                  |
+-----------------------------------------------------+
|  With batching (what React does):                   |
|                                                      |
|  setCount(1)                                         |
|  setCount(1)                                         |
|  setCount(1)                                         |
|  --> ONE render --> ONE screen update                |
|                                                      |
|  One render! Fast and efficient.                    |
+-----------------------------------------------------+
```

Queuing State Updates: The Updater Function

So how do you actually add 3 to the count? You use an updater function.

Instead of passing a value to `setCount`, you pass a function. This function receives the previous state and returns the new state.

```jsx
function Counter() {
  const [count, setCount] = useState(0);

  function handleClick() {
    setCount((prev) => prev + 1);
    setCount((prev) => prev + 1);
    setCount((prev) => prev + 1);
  }

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={handleClick}>Add three</button>
    </div>
  );
}
```

Line-by-line explanation:

- Line 5: Instead of `setCount(count + 1)`, we write `setCount((prev) => prev + 1)`. The `prev` is the most recent state value.
- Lines 6-7: Each call gets the latest value from the previous call.

Now the count goes from 0 to 3. Here is why:

```
React processes the updater functions in order:

  Call 1: setCount((prev) => prev + 1)
          prev = 0, returns 1

  Call 2: setCount((prev) => prev + 1)
          prev = 1, returns 2

  Call 3: setCount((prev) => prev + 1)
          prev = 2, returns 3

Final state: count = 3
```

Expected output (after clicking):

```
Count: 3
```

When to Use Each Style

```
+-----------------------------------------------------+
|  setCount(count + 1)                                |
|  Use when: You want to set state to a specific     |
|  value based on the current snapshot.               |
|  Example: "Set the count to whatever it is now     |
|  plus one."                                         |
|                                                     |
|  setCount(prev => prev + 1)                         |
|  Use when: You want to update state based on       |
|  the PREVIOUS state, especially when doing          |
|  multiple updates in a row.                         |
|  Example: "Take whatever the latest count is       |
|  and add one."                                      |
+-----------------------------------------------------+
```

Naming the Updater Function Parameter

The parameter name `prev` is just a convention. You can name it anything. Some people use the first letter of the state variable:

```jsx
setCount((c) => c + 1);      // c for count
setAge((a) => a + 1);        // a for age
setItems((items) => [...items, newItem]);
```

Pick whatever makes the code clear.

Mixing Regular Updates and Updater Functions

You can mix both styles. React processes them in order:

```jsx
function Counter() {
  const [count, setCount] = useState(0);

  function handleClick() {
```

```
    setCount(count + 5);       // Replace: set to 0 + 5 = 5
    setCount((prev) => prev + 1); // Update: 5 + 1 = 6
  }

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={handleClick}>Update</button>
    </div>
  );
}
```

```
React processes:

  Call 1: setCount(0 + 5)        --> queue says "replace with 5"
  Call 2: setCount(prev => prev + 1) --> queue says "add 1 to previous"

  Step 1: Replace with 5 --> state is 5
  Step 2: 5 + 1 = 6      --> state is 6

Final state: count = 6
```

Expected output (after clicking):

```
Count: 6
```

A Complete Example: Understanding Render Flow

Let us trace through a full example to see everything working together.

```jsx
import { useState } from "react";

function TrafficLight() {
  const [color, setColor] = useState("red");

  function handleClick() {
    if (color === "red") {
      setColor("green");
    } else {
      setColor("red");
    }
    console.log("Color in this render:", color);
```

```
  }

  return (
    <div>
      <h2>The light is {color}</h2>
      <button onClick={handleClick}>Change</button>
    </div>
  );
}
```

What happens when you click the button the first time:

```
+-----------------------------------------------------+
|  RENDER 1 (snapshot: color = "red")                 |
|                                                      |
|  Screen shows: "The light is red"                   |
|                                                      |
|  User clicks the button.                            |
|  handleClick runs with color = "red" (snapshot).    |
|                                                      |
|  color === "red" is true.                           |
|  setColor("green") --> schedules next render.       |
|                                                      |
|  console.log prints: "Color in this render: red"    |
|  (Still the snapshot! Not "green" yet.)              |
+-----------------------------------------------------+
         |
         | React triggers a new render
         v
+-----------------------------------------------------+
|  RENDER 2 (snapshot: color = "green")               |
|                                                      |
|  Screen shows: "The light is green"                 |
|                                                      |
|  The console.log already ran in Render 1.            |
|  Nothing else happens until the next click.          |
+-----------------------------------------------------+
```

State Updates with Objects

The snapshot concept also applies to objects. When you update state that is an object, you must create a new object instead of changing the old one.

```jsx
function UserProfile() {
  const [user, setUser] = useState({ name: "Sara", age: 25 });

  function handleBirthday() {
    setUser({
      ...user,        // Copy all existing properties
      age: user.age + 1,  // Override the age
    });
    console.log(user.age);  // Still prints 25 (snapshot!)
  }

  return (
    <div>
      <p>{user.name} is {user.age} years old</p>
      <button onClick={handleBirthday}>Happy Birthday!</button>
    </div>
  );
}
```

Line-by-line explanation:

- Line 2: The state is an object with `name` and `age`.
- Lines 5-8: We create a NEW object with the spread operator (`...user`). We do not change the old object.
- Line 9: The console still shows the old age (25) because of the snapshot.

Expected output (after one click):

Screen: `Sara is 26 years old` Console: `25`

Quick Summary

- React updates the screen in three steps: Trigger (something changed), Render (React calls your component), Commit (React updates the screen).
- A render is triggered by the initial load or by calling a state setter function.
- Each render gets its own snapshot of the state. All code in that render uses that snapshot.
- `console.log` after `setState` shows the old value because it uses the current snapshot. The new value appears in the next render.
- React batches state updates. It collects all updates in an event handler and then does one re-render. This is efficient.
- To update state based on the previous value (especially multiple times in a row), use an updater function: `setState(prev => prev + 1)`.

Key Points to Remember

1. Trigger, Render, Commit. Like ordering food: order, cook, serve.
2. State is a snapshot. Each render has its own frozen copy of the state.
3. setState does not change state instantly. It schedules a new render.
4. React batches updates for efficiency. Multiple setStates result in one re-render.
5. Use updater functions (`prev => prev + 1`) when you need the latest value.

Practice Questions

1. What are the three steps React follows to update the screen?
2. Why does `console.log` after `setCount(count + 1)` show the old value of count?
3. If `count` is 0 and you call `setCount(count + 1)` three times in a row, what will `count` be after the re-render? Why?
4. How would you fix the above to make `count` go from 0 to 3?
5. What is batching, and why does React do it?

Exercises

1. Create a component with a `score` state that starts at 0. Add a button that calls `setScore(score + 10)` twice. Click the button and observe the result. Then change it to use updater functions (`prev => prev + 10`) and observe the difference.
2. Create a `TrafficLight` component that cycles through three colors: red, yellow, green. Use state and a button. After calling `setColor`, add a `console.log` to verify that the snapshot concept works (the log should show the old color, not the new one).
3. Build a simple form with a `name` and `email` stored as an object in state. Add a button that updates the name. Add a `console.log` after the update. Verify that the log shows the old name (the snapshot), while the screen shows the new name.

What Is Coming Next?

Now you understand how state and rendering work. But sometimes you need to remember a value WITHOUT causing a re-render. In the next chapter, you will learn about refs -- a special tool that lets your component remember things without re-rendering. This is useful for things like storing timer IDs, tracking previous values, and directly accessing DOM elements like input fields.

Chapter 28: Refs — Remembering Without Re-rendering

What You Will Learn

- What `useRef` is and how it works
- The key difference between state and refs
- When to use refs instead of state
- How to access DOM elements (like input fields) using refs
- Practical examples: auto-focusing an input and building a stopwatch
- The rules for using refs safely

Why This Chapter Matters

In the last chapter, you learned that every time state changes, React re-renders your component. That is usually what you want. You change data, and the screen updates.

But sometimes you need to remember a value without updating the screen. For example:

- You start a timer and need to remember its ID so you can stop it later.
- You want to focus an input field when the page loads.
- You want to track how many times a component rendered, without causing extra renders.

Using state for these would cause unnecessary re-renders. That is where refs come in. A ref is like a sticky note your component keeps in its pocket. It can read and update the note anytime, but changing the note does not cause the screen to re-render.

What Is useRef?

`useRef` is a React Hook that gives you a box to store a value. This box has one special property: changing what is inside the box does not cause a re-render.

The Sticky Note Analogy

Imagine two ways to write something down:

1. A whiteboard in a classroom (this is state). When you change what is on the whiteboard, everyone looks up and reads the new content. The "screen" updates.
2. A sticky note in your pocket (this is a ref). You can write anything on it, change it, erase it, and write again. Nobody else notices because it is in your pocket. No "screen update."

Creating a Ref

```jsx
import { useRef } from "react";

function MyComponent() {
  const myRef = useRef(0);

  console.log(myRef.current); // 0
}
```

Line-by-line explanation:

- Line 1: We import `useRef` from React.
- Line 4: We create a ref with an initial value of 0. The ref is an object that looks like this: `{ current: 0 }`.
- Line 6: To read the value, you use `myRef.current`.

The ref is an object with a single property called `current`. You read and write using `.current`:

```
+-------------------------------+
|  What a ref looks like:       |
|                               |
|  myRef = {                    |
|    current: 0    <-- your value|
|  }                            |
|                               |
|  Read:  myRef.current         |
|  Write: myRef.current = 5     |
+-------------------------------+
```

Ref vs State: A Side-by-Side Comparison

Here is the most important difference:

Feature	State (`useState`)	Ref (`useRef`)
Causes re-render when changed?	Yes	No
Returns	`[value, setValue]`	`{ current: value }`
How to read	`value`	`ref.current`
How to update	`setValue(newValue)`	`ref.current = newValue`
When to use	Data that should be displayed on screen	Data you need to remember but not display

Seeing the Difference in Action

Here is a component that uses state to count clicks:

```jsx
import { useState } from "react";

function StateCounter() {
  const [count, setCount] = useState(0);

  function handleClick() {
    setCount(count + 1);
  }

  return (
    <div>
      <p>You clicked {count} times</p>
      <button onClick={handleClick}>Click me</button>
    </div>
  );
}
```

Expected output (after 3 clicks):

```
You clicked 3 times
[Click me]
```

Every click updates the screen. The number changes visibly.

Now here is a component that uses a ref to count clicks:

```jsx
import { useRef } from "react";

function RefCounter() {
  const countRef = useRef(0);

  function handleClick() {
    countRef.current = countRef.current + 1;
    alert("You clicked " + countRef.current + " times");
  }

  return (
    <div>
      <p>Check the alert for the count</p>
      <button onClick={handleClick}>Click me</button>
    </div>
  );
}
```

Line-by-line explanation:

- Line 4: We create a ref starting at 0.
- Line 7: We update the ref directly. No setter function needed. Just assign to `.current`.
- Line 8: We show the count in an alert. The ref has the latest value.

Expected output (after 3 clicks):

The screen always says "Check the alert for the count." It never changes. But each alert shows the updated count: 1, then 2, then 3.

The ref remembers the value. But the screen does not update because refs do not trigger re-renders.

When to Use Refs

Use refs when you need to remember something but do not need to show it on the screen. Common situations:

```
+---------------------------------------------------+
|  Good reasons to use a ref:                       |
|                                                    |
```

```
|  1. Storing a timer ID (setTimeout, setInterval)  |
|  2. Accessing a DOM element (input, div, etc.)    |
|  3. Storing a value from the previous render      |
|  4. Storing values that do not affect the display |
+---------------------------------------------------+
```

If the value should appear on the screen, use state. If it is behind-the-scenes bookkeeping, use a ref.

Accessing DOM Elements with Refs

One of the most common uses of refs is to access an actual HTML element on the page. For example, you might want to focus an input field, scroll to an element, or measure an element's size.

How It Works

1. Create a ref.
2. Attach it to a JSX element using the `ref` attribute.
3. React fills in `ref.current` with the actual DOM element.

```
+---------------------------------------------------+
|  How DOM refs work:                               |
|                                                    |
|  1. const inputRef = useRef(null);                |
|     (Create an empty box)                          |
|                                                    |
|  2. <input ref={inputRef} />                      |
|     (Tell React: "Put the input element           |
|      in this box")                                 |
|                                                    |
|  3. inputRef.current                              |
|     (Now this IS the <input> element.             |
|      You can call .focus(), .scrollIntoView(),     |
|      etc.)                                         |
+---------------------------------------------------+
```

Example: Auto-Focus an Input Field

When a search page loads, you often want the search box to be focused automatically so the user can start typing right away.

```jsx
import { useRef, useEffect } from "react";

function SearchPage() {
  const inputRef = useRef(null);

  useEffect(() => {
    inputRef.current.focus();
  }, []);

  return (
    <div>
      <h1>Search</h1>
      <input ref={inputRef} type="text" placeholder="Type to search..." />
    </div>
  );
}
```

Line-by-line explanation:

- Line 4: We create a ref starting as `null` (empty). We use `null` because the input element does not exist yet when the component first starts running.
- Lines 6-8: After the component appears on screen, `useEffect` runs. By this time, React has put the actual input element into `inputRef.current`. We call `.focus()` on it to make the cursor appear in the input.
- Line 13: We attach the ref to the input using `ref={inputRef}`. This tells React, "When you create this input element, store it in `inputRef.current`."

Expected output:

When the page loads, you see a search box and the cursor is already blinking inside it. You can start typing immediately without clicking on it first.

Example: Scrolling to an Element

```jsx
import { useRef } from "react";

function LongPage() {
  const bottomRef = useRef(null);

  function scrollToBottom() {
    bottomRef.current.scrollIntoView({ behavior: "smooth" });
  }

  return (
    <div>
      <button onClick={scrollToBottom}>Go to bottom</button>
      <div style={{ height: "2000px", background: "#f0f0f0" }}>
```

```
        <p>Lots of content here...</p>
      </div>
      <p ref={bottomRef}>You reached the bottom!</p>
    </div>
  );
}
```

Line-by-line explanation:

- Line 4: We create a ref for the bottom element.
- Line 7: We call `scrollIntoView()` to smoothly scroll to that element.
- Line 16: We attach the ref to the paragraph at the bottom.

Expected output: When you click the button, the page smoothly scrolls down to the bottom paragraph.

Example: A Stopwatch

This is a classic example that shows why refs are important. A stopwatch needs a timer (using `setInterval`), and you need to remember the timer ID so you can stop it later. But the timer ID should not be shown on screen, so it does not need to be state.

```jsx
import { useState, useRef } from "react";

function Stopwatch() {
  const [seconds, setSeconds] = useState(0);
  const [isRunning, setIsRunning] = useState(false);
  const intervalRef = useRef(null);

  function handleStart() {
    if (isRunning) return;
    setIsRunning(true);

    intervalRef.current = setInterval(() => {
      setSeconds((prev) => prev + 1);
    }, 1000);
  }

  function handleStop() {
    setIsRunning(false);
    clearInterval(intervalRef.current);
    intervalRef.current = null;
  }
```

```
  function handleReset() {
    setIsRunning(false);
    clearInterval(intervalRef.current);
    intervalRef.current = null;
    setSeconds(0);
  }

  return (
    <div>
      <h2>{seconds} seconds</h2>
      <button onClick={handleStart}>Start</button>
      <button onClick={handleStop}>Stop</button>
      <button onClick={handleReset}>Reset</button>
    </div>
  );
}
```

Line-by-line explanation:

- Line 4: `seconds` is state because it is displayed on screen. When it changes, the screen needs to update.
- Line 5: `isRunning` is state because we use it to show/hide buttons or change behavior.
- Line 6: `intervalRef` is a ref. It stores the timer ID. The timer ID is never shown on screen. It is just bookkeeping so we can stop the timer later.
- Lines 8-15: When the user clicks Start, we begin a timer that adds 1 to `seconds` every 1000 milliseconds (1 second). We store the timer ID in `intervalRef.current`. Notice we use the updater function `(prev) => prev + 1` so each update gets the latest value.
- Lines 17-21: When the user clicks Stop, we clear the timer using the ID stored in the ref. We set the ref back to `null` to clean up.
- Lines 23-28: Reset stops the timer and sets seconds back to 0.

Expected output:

```
0 seconds
[Start] [Stop] [Reset]
```

After clicking Start, the number counts up: 1, 2, 3, 4... Click Stop and it freezes. Click Reset and it goes back to 0.

Why We Use a Ref for the Timer ID

What if we used state instead?

```jsx
const [intervalId, setIntervalId] = useState(null);
```

This would work, but every time we set the interval ID, it would cause a re-render. That is wasteful. The interval ID is not shown on screen. Nobody sees it. There is no reason to re-render the component just because we stored a timer ID.

Using a ref is more efficient. We remember the value without causing any extra work for React.

Rules for Using Refs

Refs are powerful, but they come with an important rule:

Do Not Read or Write Refs During Rendering

```
+-----------------------------------------------------+
|  THE REF RULE                                       |
|                                                      |
|  Do NOT read or write ref.current while your        |
|  component is rendering (returning JSX).             |
|                                                      |
|  OK in: event handlers, useEffect                   |
|  NOT OK in: the main body of the component          |
|  that runs during rendering                          |
+-----------------------------------------------------+
```

Bad example (reading ref during render):

```jsx
function BadComponent() {
  const countRef = useRef(0);
  countRef.current = countRef.current + 1;  // BAD! This runs during render.

  return <p>Rendered {countRef.current} times</p>;  // BAD! Reading ref during render.
}
```

Why this is bad: The component is not pure anymore. It changes a value and reads a changing value during rendering. React might call this function multiple times (like in StrictMode), and you would get unexpected results.

Good example (reading ref in event handler):

```jsx
function GoodComponent() {
  const countRef = useRef(0);

  function handleClick() {
    countRef.current = countRef.current + 1;  // OK! This is in an event handler.
    alert("Clicks: " + countRef.current);      // OK! Reading ref in event handler.
  }

  return <button onClick={handleClick}>Click me</button>;
}
```

This is fine because the ref is read and written inside an event handler, not during rendering.

Exception: It is OK to set a ref during the first render if you only do it once:

```jsx
function MyComponent() {
  const startTime = useRef(null);

  if (startTime.current === null) {
    startTime.current = new Date();  // OK: only happens once
  }

  // ...
}
```

Ref vs State: A Quick Decision Guide

When you need to remember a value in a component, ask yourself:

```
+-----------------------------------------------------+
|  "Does this value need to show on the screen?"    |
|                                                     |
|  YES --> Use useState                             |
|          (changes trigger re-renders)              |
|                                                     |
|  NO  --> Use useRef                               |
|          (changes are silent)                      |
+-----------------------------------------------------+
```

Some examples:

What you are storing	Use state or ref?	Why

A counter shown on screen	State	It is displayed
The text in an input field	State	It is displayed
A timer ID from setInterval	Ref	Not displayed
A reference to an input element	Ref	Not displayed
Whether a form has been submitted	State	Affects what is shown
The previous value of a prop	Ref	Not displayed

Quick Summary

- `useRef` creates a box (`{ current: value }`) that persists across renders but does not cause re-renders when changed.
- State is for data that should appear on screen. Refs are for behind-the-scenes bookkeeping.
- You read and write refs using `.current`. There is no setter function.
- To access a DOM element, create a ref and attach it with `ref={myRef}`. React fills in `.current` with the actual element.
- Common uses: storing timer IDs, focusing input fields, scrolling to elements, and remembering values that do not need to trigger re-renders.
- Do not read or write refs during rendering. Use them in event handlers and `useEffect`.

Key Points to Remember

1. `useRef` returns `{ current: initialValue }`. Change it with `ref.current = newValue`.
2. Changing a ref does NOT cause a re-render. That is the key difference from state.
3. Use refs for values that do not appear on screen (timer IDs, DOM elements, previous values).
4. Attach a ref to a JSX element with `ref={myRef}` to get the actual DOM element.
5. Do not read or write `ref.current` during rendering. Use event handlers or useEffect instead.

Practice Questions

1. What is the main difference between `useState` and `useRef`?
2. You need to store a timer ID from `setInterval`. Should you use state or a ref? Why?
3. How do you attach a ref to an HTML element in JSX?
4. Why should you not read or write `ref.current` during rendering?
5. A ref starts as `useRef(null)`. After you attach it to an `<input>` element, what does `ref.current` contain?

Exercises

1. Create a component with an input field and a button that says "Focus." When the user clicks the button, the cursor should jump into the input field. Use `useRef` to access the input element and call `.focus()` on it.
2. Build a stopwatch component with Start, Stop, and Reset buttons. Display the elapsed time in seconds on the screen. Use state for the displayed time and a ref for the interval ID. Make sure the Reset button stops the timer and sets the time back to 0.
3. Create a component that tracks how many times a button has been clicked, but does NOT show the count on screen. Instead, when the user clicks a separate "Show Count" button, display the count in an alert. Use a ref to store the count silently, without causing re-renders on each click.

What Is Coming Next?

You now know how to remember values with both state and refs. In the next chapter, we will learn about `useReducer`, a different way to manage state that works especially well when your state logic gets more complex. Think of it as upgrading from a simple calculator to a more organized system for handling state changes.

Chapter 29: useReducer — Managing Complex State

What You Will Learn

- When `useState` is not enough for managing state
- What a reducer is and how it works
- The three parts of useReducer: state, action, and the reducer function
- How `dispatch` works (sending messages to your reducer)
- How to build a counter with `useReducer` (and compare it to `useState`)
- How to build a to-do list with `useReducer`
- When to use `useReducer` vs `useState`

Why This Chapter Matters

You have been using `useState` for all your state management so far. It works great for simple things like a counter, a text input, or a boolean toggle.

But what happens when your state gets more complex? Imagine a shopping cart. You need to add items, remove items, change quantities, apply discounts, and clear the cart. That is five different ways to update state, and they all work on the same data.

With `useState`, you end up with many setter functions and scattered update logic. It becomes hard to read and easy to make mistakes.

`useReducer` gives you a better way. It puts all your update logic in one place. It makes complex state changes predictable and organized.

Think of it this way. If `useState` is like a light switch (on or off), `useReducer` is like a remote control with many buttons. Each button does something different, but they all control the same

TV.

When useState Is Not Enough

Let us look at a problem. Imagine you are building a simple form that tracks a user's profile.

```jsx
function ProfileForm() {
  const [name, setName] = useState('');
  const [email, setEmail] = useState('');
  const [age, setAge] = useState(0);
  const [isEditing, setIsEditing] = useState(false);
  const [error, setError] = useState('');

  function handleSave() {
    if (name === '') {
      setError('Name is required');
      return;
    }
    setIsEditing(false);
    setError('');
  }

  function handleReset() {
    setName('');
    setEmail('');
    setAge(0);
    setIsEditing(false);
    setError('');
  }

  // ... more code
}
```

Do you see the problem? We have five pieces of state. They are all related to the same thing (the profile form). When we reset, we have to call five setter functions. When we save, we update two pieces of state. It is easy to forget one.

This is where `useReducer` helps. It lets you manage all these related pieces of state together, in one place.

What Is a Reducer?

A reducer is a function that takes two things:

1. The current state (what things look like right now)
2. An action (what the user wants to do)

It returns the new state (what things should look like after the action).

The Cashier Analogy

Think of a reducer like a cashier at a restaurant.

```
You (the customer) say:       "I want to add a burger."
The cashier (the reducer):     Looks at your current order.
                               Adds a burger to it.
                               Gives you the updated order.

You say:                       "Remove the fries."
The cashier:                   Looks at your current order.
                               Removes the fries.
                               Gives you the updated order.

You say:                       "Clear everything."
The cashier:                   Gives you an empty order.
```

You do not reach behind the counter and change your order yourself. You tell the cashier what you want. The cashier handles the details.

In React:

- You are the component
- The cashier is the reducer function
- What you say is the action
- The order is the state

The Three Parts of useReducer

Every `useReducer` setup has three parts. Let us learn each one.

Part 1: State

This is your data. It can be a single value, or an object with many fields.

```jsx
// Simple state: just a number
const initialState = 0;

// Complex state: an object with multiple fields
const initialState = {
  name: '',
  email: '',
  age: 0,
  isEditing: false,
  error: ''
};
```

Part 2: Actions

An action is a plain object that describes what happened. It has a `type` property that tells the reducer what to do.

```jsx
// "The user clicked the add button"
{ type: 'add' }

// "The user wants to set the name to 'Alice'"
{ type: 'set_name', value: 'Alice' }

// "The user clicked reset"
{ type: 'reset' }
```

Think of the `type` as a label. It is like a sticky note that says "do this thing."

The action can also carry extra data. In the example above, the `set_name` action carries a `value` of `'Alice'`. This extra data is often called the payload (the package of information the action carries with it).

Part 3: The Reducer Function

The reducer function reads the action type and decides what the new state should be.

```jsx
function reducer(state, action) {
  switch (action.type) {
    case 'add':
      return state + 1;
    case 'subtract':
      return state - 1;
    case 'reset':
      return 0;
    default:
```

```
      return state;
  }
}
```

Here is what is happening:

- The function receives the current `state` and the `action`.
- It uses a `switch` statement to check `action.type`.
- For each type, it returns the new state.
- The `default` case returns the current state unchanged (in case of an unknown action).

Important rule: The reducer must return a new state value. It should never change (mutate) the existing state directly. This is the same rule you learned with `useState`.

How They Work Together

Here is a picture of how the three parts connect:

```
                        +-------------------+
Component calls         |                   |
dispatch({ type })  -->|  Reducer Function |
                        |                   |
Current State -------->|  Reads action     |
                        |  Returns new      |
                        |  state            |
                        +---------+---------+
                                  |
                                  v
                          New State
                          (Component re-renders)
```

Using useReducer: A Simple Counter

Let us start with the simplest possible example: a counter. We will build it with `useState` first, then rebuild it with `useReducer` so you can compare.

Counter with useState

```jsx
import { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>Add</button>
      <button onClick={() => setCount(count - 1)}>Subtract</button>
      <button onClick={() => setCount(0)}>Reset</button>
    </div>
  );
}

export default Counter;
```

This works fine. It is simple and clear.

Counter with useReducer

Now let us build the same counter with `useReducer`.

```jsx
import { useReducer } from 'react';

// Step 1: Define the reducer function
function counterReducer(state, action) {
  switch (action.type) {
    case 'add':
      return state + 1;
    case 'subtract':
      return state - 1;
    case 'reset':
      return 0;
    default:
      return state;
  }
}

// Step 2: Use it in the component
function Counter() {
  const [count, dispatch] = useReducer(counterReducer, 0);

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => dispatch({ type: 'add' })}>Add</button>
      <button onClick={() => dispatch({ type: 'subtract' })}>Subtract</button>
      <button onClick={() => dispatch({ type: 'reset' })}>Reset</button>
    </div>
```

```
  );
}

export default Counter;
```

Expected output: The same counter as before. Three buttons. Clicking "Add" increases the count by 1. Clicking "Subtract" decreases it by 1. Clicking "Reset" sets it back to 0.

Line-by-Line Explanation

```jsx
import { useReducer } from 'react';
```

We import `useReducer` from React, just like we import `useState`.

```jsx
function counterReducer(state, action) {
```

We define the reducer function. It takes two parameters: the current `state` and the `action` that was dispatched.

```jsx
  switch (action.type) {
```

We use a `switch` statement to check what type of action was sent. A `switch` is like a series of `if/else` checks, but cleaner when you have many cases.

```jsx
    case 'add':
      return state + 1;
```

If the action type is `'add'`, return the current state plus 1.

```jsx
    case 'subtract':
      return state - 1;
```

If the action type is `'subtract'`, return the current state minus 1.

```jsx
    case 'reset':
      return 0;
```

If the action type is `'reset'`, return 0 (ignore the current state entirely).

```jsx
    default:
      return state;
```

If the action type is something we do not recognize, return the state unchanged.

```jsx
  const [count, dispatch] = useReducer(counterReducer, 0);
```

This is how we use `useReducer`. It takes two arguments:

1. The reducer function (`counterReducer`)
2. The initial state (`0`)

It gives us back two things:

1. `count` — the current state value
2. `dispatch` — a function to send actions to the reducer

```jsx
  <button onClick={() => dispatch({ type: 'add' })}>Add</button>
```

When the user clicks the button, we call `dispatch` with an action object. The action has a `type` of `'add'`. React will call our reducer with the current state and this action.

What Is Dispatch?

Dispatch means "to send." When you call `dispatch`, you are sending a message to the reducer.

Think of it like sending a text message:

```
You send:     dispatch({ type: 'add' })
Message:      "Hey reducer, the user clicked Add."
Reducer:      "Got it. Current count is 3. Adding 1. New count is 4."
React:        Re-renders the component with count = 4.
```

You can also send extra information with the message:

```jsx
dispatch({ type: 'add_amount', amount: 5 });
```

The reducer can then read `action.amount`:

```jsx
case 'add_amount':
  return state + action.amount;
```

This is like telling the cashier: "Add 5 burgers" instead of just "Add a burger."

A Real Example: To-Do List with useReducer

Now let us build something more practical. A to-do list where you can add tasks, mark them as done, and delete them.

This is where `useReducer` really shines. We have three different actions, all working on the same list of tasks.

```jsx
import { useReducer, useState } from 'react';

// The initial state: an empty list of todos
const initialTodos = [];

// The reducer: handles all the ways we can change our todos
function todoReducer(state, action) {
  switch (action.type) {
    case 'add':
      return [
        ...state,
        {
          id: Date.now(),
          text: action.text,
          done: false
        }
      ];

    case 'toggle':
      return state.map(todo => {
        if (todo.id === action.id) {
          return { ...todo, done: !todo.done };
        }
        return todo;
      });

    case 'delete':
      return state.filter(todo => todo.id !== action.id);

    default:
      return state;
```

```
  }
}

function TodoApp() {
  const [todos, dispatch] = useReducer(todoReducer, initialTodos);
  const [text, setText] = useState('');

  function handleAdd() {
    if (text.trim() === '') return;
    dispatch({ type: 'add', text: text });
    setText('');
  }

  return (
    <div>
      <h1>My To-Do List</h1>

      <input
        value={text}
        onChange={(e) => setText(e.target.value)}
        placeholder="Add a task..."
      />
      <button onClick={handleAdd}>Add</button>

      <ul>
        {todos.map(todo => (
          <li key={todo.id}>
            <span
              style={{
                textDecoration: todo.done ? 'line-through' : 'none'
              }}
            >
              {todo.text}
            </span>
            <button onClick={() => dispatch({ type: 'toggle', id: todo.id })}>
              {todo.done ? 'Undo' : 'Done'}
            </button>
            <button onClick={() => dispatch({ type: 'delete', id: todo.id })}>
              Delete
            </button>
          </li>
        ))}
      </ul>

      <p>{todos.filter(t => !t.done).length} tasks remaining</p>
    </div>
  );
}

export default TodoApp;
```

Expected output:

```
My To-Do List

[Add a task...    ] [Add]

- Buy groceries    [Done] [Delete]
- Walk the dog     [Done] [Delete]
- Read a book      [Done] [Delete]

3 tasks remaining
```

After clicking "Done" on "Walk the dog":

```
- Buy groceries        [Done] [Delete]
- ~~Walk the dog~~     [Undo] [Delete]
- Read a book          [Done] [Delete]

2 tasks remaining
```

Line-by-Line Explanation of the Reducer

```jsx
case 'add':
  return [
    ...state,
    {
      id: Date.now(),
      text: action.text,
      done: false
    }
  ];
```

When we add a to-do, we create a new array. It has all the existing items (`...state`) plus a new item at the end. The new item has:

- `id`: A unique number (we use the current time in milliseconds)
- `text`: The text from the action (`action.text`)
- `done`: Starts as `false` (not completed yet)

```jsx
case 'toggle':
  return state.map(todo => {
    if (todo.id === action.id) {
      return { ...todo, done: !todo.done };
    }
    return todo;
  });
```

When we toggle a to-do, we go through every item. If we find the one with the matching `id`, we create a copy with `done` flipped. All other items stay the same.

```jsx
case 'delete':
  return state.filter(todo => todo.id !== action.id);
```

When we delete a to-do, we filter out the item with the matching `id`. This creates a new array without that item.

Why This Is Better Than useState

If we used `useState`, our add, toggle, and delete logic would be spread across three different functions in the component. With `useReducer`, all the state update logic lives in one place: the reducer function.

This makes it easier to:

- Find bugs: All your logic is in one function
- Add new features: Just add a new case to the switch
- Test: You can test the reducer function separately from the component

---

When to Use useReducer vs useState

Here is a simple rule:

Situation	Use
One or two simple state values	`useState`
A boolean, string, or number	`useState`
Three or more related state values	`useReducer`
Multiple actions on the same data	`useReducer`
Complex update logic	`useReducer`

The Simple Rule

> If you have three or more pieces of state that are related to each other, consider using `useReducer`.

Here are some real-life examples:

Use useState:

- A counter (one number)
- A text input (one string)
- A show/hide toggle (one boolean)
- Dark mode on/off (one boolean)

Use useReducer:

- A to-do list (add, toggle, delete, filter)
- A shopping cart (add item, remove item, change quantity, apply coupon)
- A form with validation (set field, validate, submit, reset)
- A multi-step wizard (next step, previous step, set data, reset)

You Can Always Start with useState

Here is some good news. You can always start with `useState`. If your component gets complex and you have too many setter functions, you can refactor to `useReducer` later. They are interchangeable.

Do not feel pressure to use `useReducer` everywhere. It is a tool for when things get complex. Use `useState` for simple things. That is perfectly fine.

Quick Summary

A reducer is a function that takes the current state and an action, and returns the new state. It puts all your update logic in one place.

`useReducer` gives you two things: the current state and a `dispatch` function. You call `dispatch` with an action object to tell the reducer what to do.

An action is a simple object with a `type` property (and optionally extra data).

`useReducer` is most useful when you have multiple related pieces of state or several different ways to update the same state.

Key Points to Remember

1. A reducer takes state and an action, and returns new state. That is its entire job.
2. Actions are plain objects with a `type` property. The type tells the reducer which update to perform.
3. `dispatch` sends actions to the reducer. Think of it as sending a message: "Hey, this happened."
4. The reducer must return a new state value. Never change (mutate) the existing state directly.
5. Define the reducer function outside the component. This keeps your component clean and makes the reducer easy to test.
6. Use `useState` for simple state, `useReducer` for complex state. A good rule of thumb: if you have three or more related state variables, consider `useReducer`.
7. You can use `useState` and `useReducer` together. In our to-do example, we used `useReducer` for the to-do list and `useState` for the input text. Use the right tool for each job.

Practice Questions

1. What are the two arguments that a reducer function receives?
2. What does the `dispatch` function do?
3. Look at this reducer. What will the state be after dispatching `{ type: 'double' }`?

```jsx
function reducer(state, action) {
  switch (action.type) {
    case 'add':
      return state + 1;
    case 'double':
      return state * 2;
    case 'reset':
      return 0;
    default:
      return state;
  }
}

// Initial state is 5
```

```
const [count, dispatch] = useReducer(reducer, 5);
dispatch({ type: 'double' });
```

4. Why should you define the reducer function outside the component?
5. Name two situations where `useReducer` is a better choice than `useState`.

Exercises

Exercise 1: Add a "Set To" Feature

Take the counter example from this chapter. Add a new action type called `'set'` that sets the counter to a specific number. You will need to pass the number in the action.

Hint: The dispatch call would look like `dispatch({ type: 'set', value: 10 })`.

Exercise 2: Add a "Clear Completed" Button

Take the to-do list example. Add a "Clear Completed" button that removes all to-do items where `done` is `true`.

Hint: Add a new case in the reducer that uses `.filter()` to keep only items where `done` is `false`.

Exercise 3: Build a Simple Score Tracker

Build a game score tracker using `useReducer`. It should support these actions:

- `'score'`: Add points (the action carries a `points` value)
- `'penalty'`: Subtract points (the action carries a `points` value)
- `'reset'`: Reset the score to 0

The state should be an object: `{ score: 0, highScore: 0 }`. When the score goes above the high score, update the high score too.

What Is Next?

You now know how to manage complex state with `useReducer`. But what happens when multiple components need to share that state?

Right now, if you want to share state between components, you have to pass it through props. Sometimes you need to pass props through five levels of components just to get data where it needs to go. This is called "prop drilling," and it is not fun.

In the next chapter, you will learn about Context — React's built-in way to share data with any component in your tree, without passing props through every level. Combined with `useReducer`, it gives you a powerful state management system built right into React.

Chapter 30: Context — Sharing Data Without Passing Props

What You Will Learn

- What the prop drilling problem is and why it is painful
- What Context is and how it solves prop drilling
- The three steps to using Context: `createContext`, `Provider`, and `useContext`
- How to share a theme (light/dark mode) across components
- How to share the current user across components
- When to use Context and when NOT to use it
- How to combine Context with `useReducer` for mini state management

Why This Chapter Matters

Imagine you live in an apartment building. You receive a package, but it is for someone on the 5th floor. You cannot just teleport it there. You hand it to the person on the 2nd floor, who hands it to the 3rd floor, who hands it to the 4th floor, who finally gives it to the 5th floor.

That is what passing props through many levels of components feels like. It is tedious. Everyone in the chain has to handle the package, even though they do not need it.

React Context solves this. It is like an elevator that takes the package directly to the 5th floor. No middlemen needed.

This is one of the most useful features in React. It will change how you structure your applications.

The Prop Drilling Problem

Prop drilling is when you pass data through many levels of components, even though only the deepest component needs it.

Let us see an example. Imagine you have a user's name that you want to display in a deeply nested component.

```
App (has the user's name)
  └── Layout
        └── Sidebar
              └── UserPanel
                    └── UserGreeting (needs the user's name)
```

Without Context, you would have to pass the name through every level:

```jsx
function App() {
  const userName = 'Alice';
  return <Layout userName={userName} />;
}

function Layout({ userName }) {
  // Layout does not use userName. It just passes it along.
  return <Sidebar userName={userName} />;
}

function Sidebar({ userName }) {
  // Sidebar does not use userName either. Just passing it along.
  return <UserPanel userName={userName} />;
}

function UserPanel({ userName }) {
  // Still just passing it along...
  return <UserGreeting userName={userName} />;
}

function UserGreeting({ userName }) {
  // Finally! This is the component that actually uses it.
  return <p>Hello, {userName}!</p>;
}
```

This is prop drilling. Four components receive `userName` as a prop, but only one actually uses it. The other three are just middlemen.

Why Prop Drilling Is a Problem

1. It is tedious. You have to add the prop to every component in the chain.

2. It is fragile. If you rename the prop, you have to rename it everywhere.
3. It clutters your code. Components receive props they do not use.
4. It makes refactoring hard. Moving a component means rewiring all the props.

Here is what prop drilling looks like as a diagram:

```
Prop Drilling (without Context):

+--------+
|  App   |  userName="Alice"
+---+----+
    |
    | passes userName
    v
+--------+
| Layout |  userName="Alice"  (does not use it)
+---+----+
    |
    | passes userName
    v
+----------+
| Sidebar  |  userName="Alice"  (does not use it)
+---+------+
    |
    | passes userName
    v
+-----------+
| UserPanel |  userName="Alice"  (does not use it)
+---+-------+
    |
    | passes userName
    v
+--------------+
| UserGreeting |  userName="Alice"  (USES it!)
+--------------+
```

What Is Context?

Context is like a radio station. One component broadcasts data, and any component anywhere in the tree can tune in and receive it. No passing through middlemen.

```
Context (like a radio broadcast):

+--------+
|  App   |  Broadcasts: userName="Alice"
+---+----+        |
```

```
      |              |  (broadcast signal)
      v              |
  +--------+         |
  | Layout |         |  (does NOT need to receive userName)
  +---+----+         |
      |              |
      v              |
  +----------+       |
  | Sidebar  |       |  (does NOT need to receive userName)
  +---+------+       |
      |              |
      v              |
  +-----------+      |
  | UserPanel |      |  (does NOT need to receive userName)
  +---+-------+      |
      |              |
      v              |
  +--------------+   |
  | UserGreeting |<-+  Tunes in: receives userName="Alice"
  +--------------+
```

The middle components (Layout, Sidebar, UserPanel) do not need to know about `userName` at all. Only the component that actually needs the data tunes in.

The Three Steps to Using Context

Using Context involves three steps. Think of it as setting up a radio system.

Step 1: Create the Context (Build the Radio Station)

```jsx
import { createContext } from 'react';

const UserContext = createContext('Guest');
```

`createContext` creates a new Context. The argument (`'Guest'`) is the default value. This is used only if a component tries to read the Context but there is no Provider above it.

You usually put this in its own file so multiple components can import it.

Step 2: Provide the Context (Start Broadcasting)

```jsx
import { UserContext } from './UserContext';

function App() {
  const userName = 'Alice';

  return (
    <UserContext.Provider value={userName}>
      <Layout />
    </UserContext.Provider>
  );
}
```

The `Provider` wraps your component tree. It broadcasts the `value` to every component inside it. Any component inside `<UserContext.Provider>` can access `userName`.

Step 3: Use the Context (Tune In)

```jsx
import { useContext } from 'react';
import { UserContext } from './UserContext';

function UserGreeting() {
  const userName = useContext(UserContext);

  return <p>Hello, {userName}!</p>;
}
```

`useContext` reads the value from the nearest Provider above it. That is it. No props needed. The component gets the data directly.

Complete Example: Theme Switching

Let us build a real example. We will create a light/dark theme that any component can access.

Step 1: Create the Context

```jsx
// ThemeContext.js
import { createContext } from 'react';

export const ThemeContext = createContext('light');
```

Step 2: Provide the Context in App

```jsx
// App.jsx
import { useState } from 'react';
import { ThemeContext } from './ThemeContext';
import Page from './Page';

function App() {
  const [theme, setTheme] = useState('light');

  function toggleTheme() {
    setTheme(theme === 'light' ? 'dark' : 'light');
  }

  return (
    <ThemeContext.Provider value={theme}>
      <div>
        <button onClick={toggleTheme}>
          Switch to {theme === 'light' ? 'Dark' : 'Light'} Mode
        </button>
        <Page />
      </div>
    </ThemeContext.Provider>
  );
}

export default App;
```

Step 3: Use the Context in Any Component

```jsx
// Page.jsx
import Header from './Header';
import Content from './Content';

function Page() {
  // Page does not need to know about the theme!
  return (
    <div>
      <Header />
      <Content />
    </div>
  );
}

export default Page;
```

```jsx
// Header.jsx
import { useContext } from 'react';
import { ThemeContext } from './ThemeContext';

function Header() {
  const theme = useContext(ThemeContext);

  const style = {
    backgroundColor: theme === 'dark' ? '#333' : '#fff',
    color: theme === 'dark' ? '#fff' : '#333',
    padding: '20px'
  };

  return (
    <header style={style}>
      <h1>My Website</h1>
    </header>
  );
}

export default Header;
```

```jsx
// Content.jsx
import { useContext } from 'react';
import { ThemeContext } from './ThemeContext';

function Content() {
  const theme = useContext(ThemeContext);

  const style = {
    backgroundColor: theme === 'dark' ? '#222' : '#f9f9f9',
    color: theme === 'dark' ? '#ddd' : '#333',
    padding: '20px'
  };

  return (
    <main style={style}>
      <p>This content changes with the theme.</p>
      <p>Current theme: {theme}</p>
    </main>
  );
}

export default Content;
```

Expected output (light mode):

```
[Switch to Dark Mode]

My Website
-----------------------
```

```
This content changes with the theme.
Current theme: light
```

Expected output (after clicking the button, dark mode):

```
[Switch to Light Mode]

My Website                    (white text on dark background)
-----------------------
This content changes           (light gray text on very dark background)
with the theme.
Current theme: dark
```

Notice how `Page` does not receive or pass the theme. `Header` and `Content` get the theme directly from Context. No prop drilling.

Complete Example: Current User

Here is another common use case: sharing the current user's information across your app.

```jsx
// UserContext.js
import { createContext } from 'react';

export const UserContext = createContext(null);
```

```jsx
// App.jsx
import { useState } from 'react';
import { UserContext } from './UserContext';
import Dashboard from './Dashboard';

function App() {
  const [user, setUser] = useState({
    name: 'Alice',
    email: 'alice@example.com',
    role: 'admin'
  });

  return (
    <UserContext.Provider value={user}>
      <Dashboard />
    </UserContext.Provider>
```

```
  );
}

export default App;
```

```jsx
// Dashboard.jsx
import Navbar from './Navbar';
import MainContent from './MainContent';

function Dashboard() {
  // Dashboard does not need to know about the user
  return (
    <div>
      <Navbar />
      <MainContent />
    </div>
  );
}

export default Dashboard;
```

```jsx
// Navbar.jsx
import { useContext } from 'react';
import { UserContext } from './UserContext';

function Navbar() {
  const user = useContext(UserContext);

  return (
    <nav>
      <span>Welcome, {user.name}</span>
      <span>Role: {user.role}</span>
    </nav>
  );
}

export default Navbar;
```

```jsx
// MainContent.jsx
import { useContext } from 'react';
import { UserContext } from './UserContext';

function MainContent() {
  const user = useContext(UserContext);

  return (
    <main>
      <h2>Dashboard</h2>
```

```
      <p>Logged in as: {user.email}</p>
    </main>
  );
}

export default MainContent;
```

Expected output:

```
Welcome, Alice    Role: admin
--------------------------------
Dashboard
Logged in as: alice@example.com
```

Both `Navbar` and `MainContent` access the user directly. `Dashboard` does not need to pass it along.

When to Use Context

Context is best for data that many components need at different levels of your tree.

Good uses for Context:

- Theme (light/dark mode)
- Current user information
- Language/locale setting
- UI preferences (font size, sidebar open/closed)

When NOT to use Context:

- Data only needed by one component (just use local state)
- Data only needed by a parent and its direct child (just pass it as a prop)
- Data that changes very frequently (Context re-renders all consumers)

The Simple Rule

> If you are passing a prop through more than two levels of components, and the middle components do not use it, consider Context.

If only a parent and child need to share data, a prop is simpler and better.

Context + useReducer: Mini State Management

Here is something powerful. You can combine Context and `useReducer` to create a mini state management system. This is like having a simpler version of libraries like Redux, built right into React.

The idea is:

1. Use `useReducer` to manage complex state
2. Use Context to share that state (and the dispatch function) with all components

```jsx
// TodoContext.js
import { createContext } from 'react';

export const TodoContext = createContext(null);
```

```jsx
// App.jsx
import { useReducer } from 'react';
import { TodoContext } from './TodoContext';
import TodoList from './TodoList';
import AddTodo from './AddTodo';
import TodoStats from './TodoStats';

function todoReducer(state, action) {
  switch (action.type) {
    case 'add':
      return [
        ...state,
        { id: Date.now(), text: action.text, done: false }
      ];
    case 'toggle':
      return state.map(todo =>
        todo.id === action.id
          ? { ...todo, done: !todo.done }
          : todo
      );
    case 'delete':
      return state.filter(todo => todo.id !== action.id);
    default:
      return state;
  }
}

function App() {
  const [todos, dispatch] = useReducer(todoReducer, []);
```

```
  return (
    <TodoContext.Provider value={{ todos, dispatch }}>
      <h1>My Todos</h1>
      <AddTodo />
      <TodoList />
      <TodoStats />
    </TodoContext.Provider>
  );
}

export default App;
```

Notice that we pass both `todos` and `dispatch` in the Provider value. This way, any component can read the to-do list and also send actions to update it.

```jsx
// AddTodo.jsx
import { useState, useContext } from 'react';
import { TodoContext } from './TodoContext';

function AddTodo() {
  const [text, setText] = useState('');
  const { dispatch } = useContext(TodoContext);

  function handleAdd() {
    if (text.trim() === '') return;
    dispatch({ type: 'add', text });
    setText('');
  }

  return (
    <div>
      <input
        value={text}
        onChange={(e) => setText(e.target.value)}
        placeholder="Add a task..."
      />
      <button onClick={handleAdd}>Add</button>
    </div>
  );
}

export default AddTodo;
```

```jsx
// TodoList.jsx
import { useContext } from 'react';
import { TodoContext } from './TodoContext';

function TodoList() {
  const { todos, dispatch } = useContext(TodoContext);
```

```
  return (
    <ul>
      {todos.map(todo => (
        <li key={todo.id}>
          <span
            style={{
              textDecoration: todo.done ? 'line-through' : 'none'
            }}
          >
            {todo.text}
          </span>
          <button onClick={() => dispatch({ type: 'toggle', id: todo.id })}>
            {todo.done ? 'Undo' : 'Done'}
          </button>
          <button onClick={() => dispatch({ type: 'delete', id: todo.id })}>
            Delete
          </button>
        </li>
      ))}
    </ul>
  );
}

export default TodoList;
```

```jsx
// TodoStats.jsx
import { useContext } from 'react';
import { TodoContext } from './TodoContext';

function TodoStats() {
  const { todos } = useContext(TodoContext);

  const total = todos.length;
  const completed = todos.filter(t => t.done).length;
  const remaining = total - completed;

  return (
    <p>
      Total: {total} | Completed: {completed} | Remaining: {remaining}
    </p>
  );
}

export default TodoStats;
```

Expected output:

```
My Todos

[Add a task...    ] [Add]
```

```
- Buy groceries     [Done] [Delete]
- Walk the dog      [Done] [Delete]

Total: 2 | Completed: 0 | Remaining: 2
```

Why This Is Powerful

Look at the component tree:

```
App (has the reducer)
  ├── AddTodo      (uses dispatch to add)
  ├── TodoList     (uses todos and dispatch)
  └── TodoStats    (uses todos)
```

Every component gets exactly what it needs directly from Context. No prop drilling. All state update logic is in one reducer function. This is a clean, organized pattern that works well for medium-sized applications.

Quick Summary

Prop drilling is passing data through many component levels. It is tedious and messy.

Context lets you broadcast data so any component can access it directly. No middlemen needed.

Context has three steps: create the context, provide it (wrap your tree), and consume it (use `useContext`).

Combining Context + useReducer gives you a mini state management system. Share both the state and the dispatch function through Context.

Use Context for data that many components need. Use regular props for data shared between a parent and its direct children.

Key Points to Remember

1. Context solves prop drilling. It lets you share data without passing props through every level.
2. Three steps: create, provide, consume. Use `createContext`, wrap with `Provider`, read with `useContext`.
3. The Provider sets the value. Every component inside the Provider can access that value.
4. `useContext` reads from the nearest Provider above. If there are multiple Providers for the same Context, the component uses the closest one.
5. Context is not for everything. Use it for data that many components need at different levels. For simple parent-to-child data, use props.
6. You can put objects in Context. Pass `value={{ todos, dispatch }}` to share both data and functions.
7. Context + useReducer is a powerful combo. It gives you centralized state management without installing extra libraries.

Practice Questions

1. What is prop drilling? Why is it a problem?
2. What are the three steps to use Context in React?
3. What is the default value in `createContext` used for?
4. Look at this code. What will `UserGreeting` display?

```jsx
const NameContext = createContext('Guest');

function App() {
  return (
    <NameContext.Provider value="Bob">
      <UserGreeting />
    </NameContext.Provider>
  );
}

function UserGreeting() {
  const name = useContext(NameContext);
  return <p>Hello, {name}!</p>;
}
```

5. When should you use Context instead of props? When should you stick with props?

Exercises

Exercise 1: Language Switcher

Create a `LanguageContext` that stores the current language (`'en'` or `'es'`). Build:

- A `LanguageSwitcher` component with a button to toggle between English and Spanish
- A `Greeting` component that displays "Hello" (in English) or "Hola" (in Spanish)
- Both components should use Context to get and change the language

Exercise 2: Theme with Multiple Properties

Expand the theme example from this chapter. Instead of just `'light'` or `'dark'`, make the theme an object:

```jsx
const lightTheme = {
  background: '#ffffff',
  text: '#333333',
  primary: '#0066cc'
};
```

Create a `Card` component and a `Button` component that both read from this theme Context.

Exercise 3: Counter with Context + useReducer

Build a counter that uses `useReducer` for the logic and Context to share it. Create three separate components:

- `CounterDisplay` — shows the current count
- `CounterButtons` — has Add, Subtract, and Reset buttons
- `CounterHistory` — shows the highest count reached

All three components should access the same state through Context.

What Is Next?

You now know how to share data across your app with Context and manage complex state with `useReducer`. These are built-in React tools that cover many real-world needs.

But have you noticed something? In our examples, we keep writing similar patterns. The `useFetch` logic (loading, error, data) appears in many components. A toggle (true/false) with a flip function appears in many places. The local storage read/write pattern keeps repeating.

In the next chapter, you will learn how to create custom hooks — your own reusable hooks that package up common logic. Instead of copying and pasting the same `useState` + `useEffect` pattern, you will wrap it in a custom hook and reuse it everywhere.

Chapter 31: Custom Hooks — Making Your Own Hooks

What You Will Learn

- What a custom hook is and why you would create one
- The one rule: custom hooks must start with "use"
- How to build `useToggle` — a simple boolean toggle hook
- How to build `useLocalStorage` — a hook that saves state to the browser
- How to build `useFetch` — a reusable data fetching hook
- When to extract a custom hook from your component
- That custom hooks share logic, not state

Why This Chapter Matters

Have you ever written the same code in two different components? Maybe both components need to fetch data from an API. Or both need to toggle something on and off. Or both need to save something to local storage.

Copying and pasting code is bad. If you find a bug, you have to fix it in every place you copied it. If you want to improve the logic, you have to update it everywhere.

Custom hooks solve this problem. They let you take a piece of logic — any combination of `useState`, `useEffect`, or other hooks — and wrap it up in a reusable function. Then any component can use that logic by calling your hook.

Think of it this way. React gives you built-in tools like `useState` and `useEffect`. Custom hooks let you build your own tools from those built-in ones. It is like combining a hammer and nails into a "hang a picture" tool.

What Is a Custom Hook?

A custom hook is a regular JavaScript function that:

1. Starts with the word `use` (like `useToggle`, `useFetch`, `useWindowSize`)
2. Uses other hooks inside it (like `useState`, `useEffect`, `useContext`)

That is it. There is nothing magical about it. It is just a function.

Here is the simplest possible custom hook:

```jsx
function useMyName() {
  const [name, setName] = useState('');
  return [name, setName];
}
```

This is not very useful, but it shows the idea. It is a function that starts with `use` and uses `useState` inside.

Why the "use" Prefix?

React has one important rule: hooks must start with "use."

This is not just a suggestion. React uses this naming convention to check that you follow the rules of hooks (like not calling hooks inside loops or conditions). If your function starts with "use", React knows it is a hook and will check it.

Good names:

- `useToggle`
- `useFetch`
- `useLocalStorage`
- `useWindowSize`

Bad names (React will not treat these as hooks):

- `toggle`
- `fetchData`
- `getFromStorage`

Example 1: useToggle

Let us start with something simple. Many components need a boolean value that flips between `true` and `false`. Think of: showing/hiding a modal, opening/closing a menu, expanding/collapsing a section.

The Problem: Repeated Code

Without a custom hook, every component writes the same pattern:

```jsx
function Modal() {
  const [isOpen, setIsOpen] = useState(false);
  const toggle = () => setIsOpen(!isOpen);
  // ...
}

function Menu() {
  const [isOpen, setIsOpen] = useState(false);
  const toggle = () => setIsOpen(!isOpen);
  // ...
}

function Accordion() {
  const [isExpanded, setIsExpanded] = useState(false);
  const toggle = () => setIsExpanded(!isExpanded);
  // ...
}
```

The same two lines are repeated three times. Let us fix that.

The Solution: useToggle

```jsx
// useToggle.js
import { useState } from 'react';

function useToggle(initialValue = false) {
  const [value, setValue] = useState(initialValue);

  function toggle() {
    setValue(!value);
  }

  return [value, toggle];
}

export default useToggle;
```

Line-by-Line Explanation

```jsx
function useToggle(initialValue = false) {
```

Our custom hook is a function. It starts with `use`. It accepts an optional initial value (defaults to `false`).

```jsx
  const [value, setValue] = useState(initialValue);
```

Inside, it uses `useState` just like you would in a component.

```jsx
  function toggle() {
    setValue(!value);
  }
```

It creates a `toggle` function that flips the value.

```jsx
  return [value, toggle];
```

It returns the current value and the toggle function as an array. This follows the same pattern as `useState` returning `[value, setter]`.

Using useToggle

Now any component can use it:

```jsx
import useToggle from './useToggle';

function Navbar() {
  const [menuOpen, toggleMenu] = useToggle(false);

  return (
    <nav>
      <button onClick={toggleMenu}>
        {menuOpen ? 'Close Menu' : 'Open Menu'}
      </button>
      {menuOpen && (
        <ul>
          <li>Home</li>
          <li>About</li>
          <li>Contact</li>
        </ul>
```

```
      )}
    </nav>
  );
}
```

```jsx
import useToggle from './useToggle';

function FAQ() {
  const [showAnswer, toggleAnswer] = useToggle(false);

  return (
    <div>
      <h3 onClick={toggleAnswer}>What is React?</h3>
      {showAnswer && <p>React is a JavaScript library for building UIs.</p>}
    </div>
  );
}
```

Expected output (Navbar, menu closed):

```
[Open Menu]
```

Expected output (Navbar, after clicking):

```
[Close Menu]
- Home
- About
- Contact
```

Both components use the same toggle logic, but each has its own independent state. We will talk more about this important point later in the chapter.

Example 2: useLocalStorage

Local storage is a feature built into every web browser. It lets you save small amounts of data that persist even when the user closes the browser and comes back later. Think of it like a tiny notebook the browser keeps.

Many components need to save their state to local storage. Let us build a hook for that.

The Hook

```jsx
// useLocalStorage.js
import { useState, useEffect } from 'react';

function useLocalStorage(key, initialValue) {
  // Step 1: Get the saved value, or use the initial value
  const [value, setValue] = useState(() => {
    const saved = localStorage.getItem(key);
    if (saved !== null) {
      return JSON.parse(saved);
    }
    return initialValue;
  });

  // Step 2: Save to local storage whenever the value changes
  useEffect(() => {
    localStorage.setItem(key, JSON.stringify(value));
  }, [key, value]);

  return [value, setValue];
}

export default useLocalStorage;
```

Line-by-Line Explanation

```jsx
function useLocalStorage(key, initialValue) {
```

The hook takes two arguments: a `key` (the name to save under, like `'theme'` or `'username'`) and an `initialValue` (what to use if nothing is saved yet).

```jsx
  const [value, setValue] = useState(() => {
    const saved = localStorage.getItem(key);
    if (saved !== null) {
      return JSON.parse(saved);
    }
    return initialValue;
  });
```

We use the function form of `useState` (lazy initialization). This runs only once when the component first renders. It checks local storage for a saved value. If one exists, it parses it from a string back into JavaScript data. If not, it uses the initial value.

`JSON.parse` converts a string like `'"dark"'` back to a JavaScript value like `'dark'`. `JSON.stringify` does the opposite — it converts JavaScript values to strings for storage.

```jsx
  useEffect(() => {
    localStorage.setItem(key, JSON.stringify(value));
  }, [key, value]);
```

Whenever the value changes, this effect saves it to local storage. `JSON.stringify` converts the value to a string (local storage only stores strings).

```jsx
  return [value, setValue];
```

It returns the same thing as `useState`: the current value and a function to update it. The component using this hook does not even know that local storage is involved.

Using useLocalStorage

```jsx
import useLocalStorage from './useLocalStorage';

function Settings() {
  const [theme, setTheme] = useLocalStorage('theme', 'light');
  const [fontSize, setFontSize] = useLocalStorage('fontSize', 16);

  return (
    <div>
      <h2>Settings</h2>

      <label>
        Theme:
        <select
          value={theme}
          onChange={(e) => setTheme(e.target.value)}
        >
          <option value="light">Light</option>
          <option value="dark">Dark</option>
        </select>
      </label>

      <label>
        Font Size:
        <input
          type="number"
          value={fontSize}
          onChange={(e) => setFontSize(Number(e.target.value))}
        />
      </label>

      <p>Current theme: {theme}</p>
      <p>Current font size: {fontSize}px</p>
    </div>
```

```
  );
}

export default Settings;
```

Expected output:

```
Settings

Theme:     [Light  v]
Font Size: [16]

Current theme: light
Current font size: 16px
```

The magic: if the user picks "dark" and closes the browser, when they come back, the theme will still be "dark". The `useLocalStorage` hook handles all the saving and loading automatically.

Example 3: useFetch

Fetching data from an API is one of the most common things you do in React. And the pattern is always the same:

1. Set up loading, error, and data states
2. Fetch the data in `useEffect`
3. Handle loading and error states
4. Display the data

Let us wrap this pattern in a reusable hook.

The Hook

```jsx
// useFetch.js
import { useState, useEffect } from 'react';

function useFetch(url) {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);
```

```
  useEffect(() => {
    setLoading(true);
    setError(null);

    fetch(url)
      .then(response => {
        if (!response.ok) {
          throw new Error('Something went wrong');
        }
        return response.json();
      })
      .then(data => {
        setData(data);
        setLoading(false);
      })
      .catch(error => {
        setError(error.message);
        setLoading(false);
      });
  }, [url]);

  return { data, loading, error };
}

export default useFetch;
```

Line-by-Line Explanation

```jsx
function useFetch(url) {
```

The hook takes a URL as its argument. This is the API endpoint (web address) to fetch data from.

```jsx
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);
```

Three pieces of state: the fetched `data`, a `loading` flag, and any `error` message. Loading starts as `true` because we start fetching right away.

```jsx
  useEffect(() => {
    setLoading(true);
    setError(null);
```

When the URL changes, we start a new fetch. We set loading to `true` and clear any previous error.

```jsx
    fetch(url)
      .then(response => {
        if (!response.ok) {
          throw new Error('Something went wrong');
        }
        return response.json();
      })
```

We call `fetch` with the URL. If the response is not OK (for example, a 404 error), we throw an error. Otherwise, we convert the response to JSON data.

```jsx
      .then(data => {
        setData(data);
        setLoading(false);
      })
      .catch(error => {
        setError(error.message);
        setLoading(false);
      });
```

If the fetch succeeds, we save the data and stop loading. If it fails, we save the error message and stop loading.

```jsx
  return { data, loading, error };
```

We return an object with all three values. The component can use whichever ones it needs.

Using useFetch

```jsx
import useFetch from './useFetch';

function UserList() {
  const { data, loading, error } = useFetch(
    'https://jsonplaceholder.typicode.com/users'
  );

  if (loading) return <p>Loading...</p>;
  if (error) return <p>Error: {error}</p>;

  return (
    <ul>
```

```
      {data.map(user => (
        <li key={user.id}>{user.name}</li>
      ))}
    </ul>
  );
}
```

```jsx
import useFetch from './useFetch';

function PostList() {
  const { data, loading, error } = useFetch(
    'https://jsonplaceholder.typicode.com/posts'
  );

  if (loading) return <p>Loading posts...</p>;
  if (error) return <p>Error: {error}</p>;

  return (
    <ul>
      {data.slice(0, 5).map(post => (
        <li key={post.id}>{post.title}</li>
      ))}
    </ul>
  );
}
```

Expected output (UserList, while loading):

```
Loading...
```

Expected output (UserList, after loading):

```
- Leanne Graham
- Ervin Howell
- Clementine Bauch
- ...
```

Both `UserList` and `PostList` fetch data from different URLs, but they use the exact same hook. No duplicated loading/error logic. If you want to improve error handling, you change it in one place.

When to Extract a Custom Hook

Here is a simple guideline:

> If you copy and paste the same `useState` + `useEffect` (or similar hook combination) pattern into more than one component, it is time to make a custom hook.

Signs that you should create a custom hook:

1. Two or more components have the same hook pattern. They use `useState` and `useEffect` in the same way.
2. A component has complex hook logic that distracts from its main purpose. Moving the logic to a hook makes the component easier to read.
3. You want to test the logic separately. Custom hooks can be tested independently from any component.

Signs that you should NOT create a custom hook:

1. The logic is only used in one component. Wait until you need it in a second place.
2. The "hook" would just wrap a single `useState`. That does not add value.
3. You are doing it just to reduce lines of code. Hooks should improve clarity, not just reduce line count.

Custom Hooks Share Logic, Not State

This is a very important concept that trips up many beginners.

When two components use the same custom hook, they share the logic (the code). They do NOT share the state (the data). Each component gets its own independent copy of the state.

Let us see this clearly:

```jsx
function ComponentA() {
  const [isOn, toggle] = useToggle(false);
  return <button onClick={toggle}>{isOn ? 'ON' : 'OFF'}</button>;
}

function ComponentB() {
  const [isOn, toggle] = useToggle(false);
  return <button onClick={toggle}>{isOn ? 'ON' : 'OFF'}</button>;
}
```

If you click the button in `ComponentA`, only `ComponentA` changes. `ComponentB` is not affected. They each have their own `isOn` state.

Think of it like a cookie recipe. If you and your friend both use the same recipe, you each bake your own batch of cookies. Eating one of your cookies does not affect your friend's cookies.

```
Custom Hook: useToggle

ComponentA                    ComponentB
+--------------+              +--------------+
| isOn = false |              | isOn = false |
| toggle()     |              | toggle()     |
+--------------+              +--------------+
     |                             |
     | (click toggle)              |
     v                             |
+--------------+              +--------------+
| isOn = true  |              | isOn = false |  (unchanged!)
| toggle()     |              | toggle()     |
+--------------+              +--------------+

Same logic. Different state.
```

If you want components to share the same state, you need Context (which you learned in the previous chapter).

Quick Summary

A custom hook is a function that starts with `use` and uses other hooks inside. It lets you package reusable logic into a single function that any component can call.

Custom hooks share logic, not state. Each component that calls a hook gets its own independent copy of the state.

Create a custom hook when you see the same hook pattern repeated in multiple components.

Key Points to Remember

1. Custom hooks must start with "use". This is a rule, not a suggestion. React uses this to enforce hook rules.
2. A custom hook is just a function. There is nothing special about it. It is a regular function that happens to use hooks inside.

3. Custom hooks can use any other hooks. You can use `useState`, `useEffect`, `useContext`, `useReducer`, or even other custom hooks inside your custom hook.
4. Each call to a custom hook creates independent state. Two components using the same hook do not share state.
5. Custom hooks can return anything. Return an array (`[value, setter]`), an object (`{ data, loading, error }`), or even a single value. Choose whatever makes sense for your hook.
6. Extract a hook when you see a pattern repeated. If you copy-paste the same hooks logic into a second component, it is time to create a custom hook.
7. Name your hooks clearly. The name should describe what the hook does: `useToggle`, `useLocalStorage`, `useFetch`, `useWindowSize`.

Practice Questions

1. What two things make a function a "custom hook"?
2. If two components both use `useToggle`, do they share the same state? Why or why not?
3. What is wrong with this custom hook name?

```jsx
function getWindowSize() {
  const [width, setWidth] = useState(window.innerWidth);
  // ...
  return width;
}
```

4. When should you create a custom hook? When should you NOT?
5. What does `useLocalStorage` do differently from `useState`?

Exercises

Exercise 1: useCounter

Create a custom hook called `useCounter` that manages a counter. It should accept an `initialValue` (default `0`) and return:

- `count` — the current count
- `increment` — a function to add 1

- `decrement` — a function to subtract 1
- `reset` — a function to reset to the initial value

Then use it in two separate components to verify that they have independent state.

Exercise 2: useDocumentTitle

Create a custom hook called `useDocumentTitle` that updates the browser tab title. It should accept a string and set `document.title` to that string using `useEffect`.

Use it in a component where the user can type in an input, and the browser tab title updates as they type.

Exercise 3: useOnlineStatus

Create a custom hook called `useOnlineStatus` that tracks whether the user is online or offline. Use `window.addEventListener` to listen for `'online'` and `'offline'` events. Return a boolean: `true` if online, `false` if offline.

Hint: Use `navigator.onLine` for the initial value, and `useEffect` to add and remove event listeners.

What Is Next?

You now know how to create your own reusable hooks. This is a skill that separates beginners from intermediate React developers. As you build more projects, you will create hooks that are specific to your application, like `useAuth`, `useCart`, or `useTheme`.

In the next and final chapter, you will learn about TypeScript — a tool that adds type safety to your React code. TypeScript helps you catch bugs before you even run your code, like a spell-checker for your programming. It is used by most professional React projects today, and knowing the basics will make you a stronger developer.

Chapter 32: TypeScript Basics with React

What You Will Learn

- What TypeScript is and why it exists
- How TypeScript helps you catch mistakes early
- How to set up a React project with TypeScript
- Basic types: string, number, boolean, and arrays
- How to type component props
- How to type `useState`
- How to type events
- How to type function return values
- Common TypeScript errors and how to fix them

Why This Chapter Matters

Imagine you have a box in your kitchen. Sometimes you put flour in it. Sometimes you put sugar in it. They look the same. One day, you grab what you think is sugar and put it in your coffee. It is flour. Your coffee is ruined.

Now imagine every box has a clear label: "FLOUR", "SUGAR", "SALT". You never mix them up again.

That is what TypeScript does for your code. JavaScript lets you put anything anywhere, and sometimes things go wrong in surprising ways. TypeScript adds labels (called types) to your data, so you always know what is what. And if you try to put flour in the sugar box, TypeScript tells you before you even run your code.

TypeScript is used in most professional React projects today. Learning the basics now will prepare you for real-world jobs and larger projects.

What Is TypeScript?

TypeScript is JavaScript with types. That is really all it is.

Every piece of TypeScript is valid JavaScript with extra information added. TypeScript files end in `.ts` (for regular files) or `.tsx` (for files with JSX, which is what you use in React).

Here is the difference:

```javascript
// JavaScript — no types
let name = 'Alice';
let age = 25;
let isStudent = true;
```

```typescript
// TypeScript — same code, but with type labels
let name: string = 'Alice';
let age: number = 25;
let isStudent: boolean = true;
```

The `: string`, `: number`, and `: boolean` are type annotations (labels). They tell TypeScript what kind of data each variable holds.

What Happens If You Make a Mistake?

```typescript
let age: number = 25;
age = 'twenty-five'; // ERROR! TypeScript says: "You cannot put a string in a number box."
```

TypeScript catches this mistake before you run your code. It underlines the error in your editor with a red squiggly line, just like a spell checker.

Without TypeScript, this mistake would silently break your app at runtime (when the code is running). With TypeScript, you see it immediately.

Setting Up a React + TypeScript Project

Creating a React project with TypeScript is simple. You use the same tool (Vite) but choose a TypeScript template.

Open your terminal and run:

```bash
npm create vite@latest my-app -- --template react-ts
```

Then:

```bash
cd my-app
npm install
npm run dev
```

That is it. The project is set up with TypeScript. You will notice the files end in `.tsx` instead of `.jsx`.

What Is Different?

Your project now has a file called `tsconfig.json`. This is the TypeScript configuration file. You do not need to change it. The defaults work great.

Your component files are `.tsx` instead of `.jsx`. Inside them, you write the same React code you already know, but with type annotations added.

Basic Types

TypeScript has a few basic types that you will use all the time. Let us learn each one.

string

For text.

```typescript
let firstName: string = 'Alice';
let greeting: string = "Hello, world!";
let message: string = `Welcome, ${firstName}`;
```

number

For all numbers (whole numbers, decimals, negatives).

```typescript
let age: number = 25;
let price: number = 9.99;
let temperature: number = -5;
```

boolean

For true or false values.

```typescript
let isLoggedIn: boolean = true;
let hasError: boolean = false;
```

Arrays

For lists of items. You write the type of the items, then add `[]`.

```typescript
let names: string[] = ['Alice', 'Bob', 'Charlie'];
let scores: number[] = [95, 87, 73, 100];
let flags: boolean[] = [true, false, true];
```

If you try to put the wrong type in an array, TypeScript catches it:

```typescript
let names: string[] = ['Alice', 'Bob'];
names.push(42); // ERROR! 42 is a number, not a string.
```

Objects

For data with named fields. You define the shape with an interface (a description of what the object looks like).

```typescript
interface User {
  name: string;
  age: number;
  email: string;
}
```

```
let user: User = {
  name: 'Alice',
  age: 25,
  email: 'alice@example.com'
};
```

An interface is like a blueprint. It says: "A User must have a `name` that is a string, an `age` that is a number, and an `email` that is a string."

If you forget a field or use the wrong type, TypeScript tells you:

```typescript
let user: User = {
  name: 'Alice',
  age: 'twenty-five' // ERROR! age should be a number.
};
```

Optional Properties

Sometimes a field might or might not exist. Add a `?` after the name:

```typescript
interface User {
  name: string;
  age: number;
  email?: string; // email is optional
}

let user1: User = { name: 'Alice', age: 25 }; // OK, no email
let user2: User = { name: 'Bob', age: 30, email: 'bob@example.com' }; // Also OK
```

Typing Component Props

This is where TypeScript really shines in React. You can define exactly what props a component accepts.

Without TypeScript

```jsx
// JavaScript — no type checking on props
function Greeting({ name, age }) {
  return <p>Hello, {name}! You are {age} years old.</p>;
}

// Oops, we passed a string for age. No error until something breaks.
<Greeting name="Alice" age="twenty-five" />
```

With TypeScript

```tsx
// TypeScript — props are type-checked
interface GreetingProps {
  name: string;
  age: number;
}

function Greeting({ name, age }: GreetingProps) {
  return <p>Hello, {name}! You are {age} years old.</p>;
}

// TypeScript catches the mistake immediately
<Greeting name="Alice" age="twenty-five" /> // ERROR! age should be a number.
<Greeting name="Alice" age={25} /> // OK!
```

Line-by-Line Explanation

```tsx
interface GreetingProps {
  name: string;
  age: number;
}
```

We define an interface called `GreetingProps`. It describes the shape of the props object. The component expects a `name` (string) and an `age` (number).

```tsx
function Greeting({ name, age }: GreetingProps) {
```

We add `: GreetingProps` after the props destructuring. This tells TypeScript that the props must match the `GreetingProps` interface.

Props with Optional Values

```tsx
interface CardProps {
  title: string;
  subtitle?: string; // optional
  color?: string;    // optional
}

function Card({ title, subtitle, color = 'blue' }: CardProps) {
  return (
    <div style={{ borderColor: color }}>
      <h2>{title}</h2>
      {subtitle && <p>{subtitle}</p>}
    </div>
  );
}

// All of these work:
<Card title="Hello" />
<Card title="Hello" subtitle="World" />
<Card title="Hello" subtitle="World" color="red" />
```

Props with Functions

```tsx
interface ButtonProps {
  label: string;
  onClick: () => void;
}

function Button({ label, onClick }: ButtonProps) {
  return <button onClick={onClick}>{label}</button>;
}
```

The type `() => void` means "a function that takes no arguments and returns nothing." The word void means "no return value."

Props with Children

If your component accepts children (content placed between opening and closing tags), type it like this:

```tsx
interface ContainerProps {
  children: React.ReactNode;
}

function Container({ children }: ContainerProps) {
  return <div className="container">{children}</div>;
}
```

```
// Usage:
<Container>
  <h1>Hello!</h1>
  <p>This is inside the container.</p>
</Container>
```

`React.ReactNode` is a special type that means "anything that React can render." This includes strings, numbers, JSX elements, arrays, and more.

Typing useState

When you use `useState`, TypeScript can usually figure out the type automatically:

```tsx
const [count, setCount] = useState(0); // TypeScript knows count is a number
const [name, setName] = useState('');   // TypeScript knows name is a string
```

But sometimes you need to tell TypeScript the type explicitly. Use angle brackets `<>` after `useState`:

```tsx
// When the initial value does not tell the full story
const [user, setUser] = useState<User | null>(null);
```

Here, `User | null` means "the value is either a User object or null." The `|` symbol means "or." This is common when you fetch data: the value starts as `null` and becomes a `User` after loading.

More Examples

```tsx
// An array that starts empty
const [items, setItems] = useState<string[]>([]);

// A number that could be undefined
const [selectedId, setSelectedId] = useState<number | undefined>(undefined);

// An object with a specific shape
interface Todo {
  id: number;
  text: string;
```

```
  done: boolean;
}

const [todos, setTodos] = useState<Todo[]>([]);
```

Why is this needed? When you write `useState([])`, TypeScript sees an empty array and thinks the type is `never[]` (an array that can never have anything in it). By writing `useState<string[]>([])`, you tell TypeScript that this array will hold strings.

Typing Events

When you handle events like clicks, form submissions, or input changes, TypeScript needs to know the type of the event.

Typing Input Change Events

```tsx
function SearchBox() {
  const [query, setQuery] = useState('');

  function handleChange(event: React.ChangeEvent<HTMLInputElement>) {
    setQuery(event.target.value);
  }

  return <input value={query} onChange={handleChange} />;
}
```

`React.ChangeEvent<HTMLInputElement>` means "a change event from an HTML input element." It is a long type, but here is how to read it:

- `React.ChangeEvent` — this is a change event (the value changed)
- `<HTMLInputElement>` — it happened on an input element

Typing Form Submit Events

```tsx
function LoginForm() {
  function handleSubmit(event: React.FormEvent<HTMLFormElement>) {
    event.preventDefault();
    // handle the form submission
  }
```

```
  return (
    <form onSubmit={handleSubmit}>
      <button type="submit">Log In</button>
    </form>
  );
}
```

Typing Click Events

```tsx
function MyButton() {
  function handleClick(event: React.MouseEvent<HTMLButtonElement>) {
    console.log('Button clicked!');
  }

  return <button onClick={handleClick}>Click me</button>;
}
```

The Easy Way: Inline Handlers

Here is a tip. When you write the event handler inline (directly in the JSX), TypeScript figures out the type automatically:

```tsx
function SearchBox() {
  const [query, setQuery] = useState('');

  return (
    <input
      value={query}
      onChange={(e) => setQuery(e.target.value)} // TypeScript knows e is a ChangeEvent!
    />
  );
}
```

You only need to type events manually when you define the handler as a separate function.

Typing Function Return Values

You can tell TypeScript what a function returns:

```tsx
function add(a: number, b: number): number {
  return a + b;
}

function greet(name: string): string {
  return `Hello, ${name}!`;
}

function logMessage(message: string): void {
  console.log(message);
  // returns nothing, so the return type is void
}
```

For React components, the return type is usually `JSX.Element`:

```tsx
function Greeting({ name }: { name: string }): JSX.Element {
  return <p>Hello, {name}!</p>;
}
```

However, you usually do not need to write the return type for components. TypeScript figures it out automatically. It is fine to leave it off.

A Complete TypeScript React Component

Let us put everything together. Here is a complete component with full TypeScript types:

```tsx
import { useState } from 'react';

interface Todo {
  id: number;
  text: string;
  done: boolean;
}

interface TodoItemProps {
  todo: Todo;
  onToggle: (id: number) => void;
  onDelete: (id: number) => void;
}

function TodoItem({ todo, onToggle, onDelete }: TodoItemProps) {
  return (
    <li>
```

```
      <span
        style={{ textDecoration: todo.done ? 'line-through' : 'none' }}
        onClick={() => onToggle(todo.id)}
      >
        {todo.text}
      </span>
      <button onClick={() => onDelete(todo.id)}>Delete</button>
    </li>
  );
}

function TodoApp() {
  const [todos, setTodos] = useState<Todo[]>([]);
  const [text, setText] = useState('');

  function handleAdd() {
    if (text.trim() === '') return;
    const newTodo: Todo = {
      id: Date.now(),
      text: text,
      done: false
    };
    setTodos([...todos, newTodo]);
    setText('');
  }

  function handleToggle(id: number) {
    setTodos(todos.map(todo =>
      todo.id === id ? { ...todo, done: !todo.done } : todo
    ));
  }

  function handleDelete(id: number) {
    setTodos(todos.filter(todo => todo.id !== id));
  }

  return (
    <div>
      <h1>My Todos</h1>
      <input
        value={text}
        onChange={(e) => setText(e.target.value)}
        placeholder="Add a task..."
      />
      <button onClick={handleAdd}>Add</button>
      <ul>
        {todos.map(todo => (
          <TodoItem
            key={todo.id}
            todo={todo}
            onToggle={handleToggle}
            onDelete={handleDelete}
          />
        ))}
      </ul>
```

```
    </div>
  );
}

export default TodoApp;
```

Expected output:

```
My Todos

[Add a task...    ] [Add]

- Buy groceries    [Delete]
- Walk the dog     [Delete]
```

This is the same to-do list you have built before. The only difference is the type annotations. TypeScript will now catch mistakes like passing a string where a number is expected, forgetting a required prop, or misspelling a property name.

Common TypeScript Errors and How to Fix Them

Here are errors you will see often as a beginner. Do not worry — they are all easy to fix.

Error 1: Type 'string' is not assignable to type 'number'

```tsx
let age: number = 'twenty-five';
// Fix: use a number
let age: number = 25;
```

What it means: You tried to put a string where a number belongs.

Error 2: Property 'X' does not exist on type 'Y'

```tsx
interface User {
  name: string;
}

const user: User = { name: 'Alice' };
console.log(user.age); // ERROR: Property 'age' does not exist on type 'User'
```

Fix: Either add `age` to the interface, or remove the line that uses it.

Error 3: Argument of type 'X' is not assignable to parameter of type 'Y'

```tsx
function greet(name: string) {
  return `Hello, ${name}`;
}

greet(42); // ERROR: Argument of type 'number' is not assignable to parameter of type '...
```

Fix: Pass the correct type: `greet('Alice')`.

Error 4: Object is possibly 'null'

```tsx
const [user, setUser] = useState<User | null>(null);

return <p>{user.name}</p>; // ERROR: user might be null!
```

Fix: Check for null before using the value:

```tsx
if (user === null) return <p>Loading...</p>;
return <p>{user.name}</p>; // TypeScript now knows user is not null
```

Error 5: Property 'X' is missing in type 'Y'

```tsx
interface ButtonProps {
  label: string;
  onClick: () => void;
}

<Button label="Click me" /> // ERROR: Property 'onClick' is missing
```

Fix: Either provide the missing prop or make it optional with `?`:

```tsx
interface ButtonProps {
  label: string;
  onClick?: () => void; // now optional
}
```

Quick Summary

TypeScript is JavaScript with type labels. It catches mistakes before you run your code, like a spell-checker for programming.

You add types using the `: type` syntax. For component props, you define an interface that describes the shape of the props object.

`useState` usually infers the type automatically, but you can specify it with `useState<Type>()` when needed.

Event types like `React.ChangeEvent<HTMLInputElement>` tell TypeScript what kind of event you are handling. Inline handlers get their types automatically.

Key Points to Remember

1. TypeScript is JavaScript with types. Every valid JavaScript is valid TypeScript. You are adding type information, not learning a new language.
2. Use interfaces to define prop shapes. Write `interface Props { ... }` and apply it to your component parameters.
3. TypeScript often infers types automatically. You do not need to type everything. Let TypeScript figure it out when it can.
4. Use `useState<Type>()` for complex initial values. Especially when the initial value is `null`, `undefined`, or an empty array.
5. Inline event handlers get types for free. If you write the handler directly in JSX, TypeScript knows the event type.
6. Read error messages carefully. TypeScript errors are helpful. They tell you exactly what is wrong and where.
7. Start simple. You do not need to type everything perfectly on day one. Add types gradually as you learn.

Practice Questions

1. What is the difference between JavaScript and TypeScript?
2. What is an interface in TypeScript? Why is it useful for React props?
3. What does `useState<string[]>([])` mean?
4. What type would you use for an `onChange` event on an `<input>` element?
5. What does the error "Object is possibly null" mean, and how do you fix it?

Exercises

Exercise 1: Type a Profile Card

Create a `ProfileCard` component with these typed props:

- `name` (required string)
- `bio` (required string)
- `age` (required number)
- `website` (optional string)

Display all the information. If no website is provided, show "No website."

Exercise 2: Type a Counter Component

Build a counter component using TypeScript. The state should be typed. Create a `CounterProps` interface with:

- `initialValue` (required number)
- `step` (optional number, defaults to 1)

The counter should increment and decrement by the step value.

Exercise 3: Type a User List

Create a `User` interface with `id` (number), `name` (string), and `email` (string). Create a component that:

- Uses `useState<User[]>([])` for the users list
- Has an input form to add new users
- Displays the list of users
- Types all event handlers properly

Congratulations and What to Learn Next

You have made it to the end of this book. Take a moment to appreciate how far you have come.

When you started, you did not know what React was. Now you can:

- Build components with JSX
- Manage state with `useState` and `useReducer`
- Handle side effects with `useEffect`
- Fetch data from APIs
- Route between pages
- Share data with Context
- Create your own custom hooks
- Add type safety with TypeScript

That is a lot. You should be proud.

But learning never stops. Here are the most important things to explore next, in order of priority.

Next.js

Next.js is a framework built on top of React. While React handles the user interface, Next.js adds features like:

- Server-side rendering (pages load faster)
- File-based routing (create a file, get a route — no React Router needed)
- API routes (build your backend in the same project)
- Built-in optimizations for images, fonts, and more

Most new React projects today use Next.js. It is the natural next step after learning React.

State Management Libraries

For large applications, Context + `useReducer` might not be enough. Libraries like Zustand, Jotai, and Redux Toolkit provide more powerful state management. Zustand is the easiest to learn and is a great place to start.

Testing

Testing ensures your code works correctly and keeps working as you make changes. Learn:

- Vitest — a fast test runner for Vite projects
- React Testing Library — for testing React components the way users interact with them

CSS Frameworks

Styling large applications by hand is slow. Popular options include:

- Tailwind CSS — utility classes directly in your JSX
- CSS Modules — scoped CSS that does not leak between components
- styled-components — CSS written inside your JavaScript

Where to Practice

The best way to learn is to build things. Here are some project ideas:

- A personal blog
- A weather app that fetches real data
- A simple e-commerce store
- A task management app with authentication

Final Words

Programming is a journey, not a destination. Every expert was once a beginner. Every complex application started as a simple component.

You now have a solid foundation. Build things. Break things. Fix things. That is how you grow.

Good luck, and happy coding.